1

Introduction

Subfiles can be viewed as an incredibly powerful programming technique. They simplify programming by minimizing the amount of code required. This, in turn, reduces the time you spend maintaining your applications. In fact, subfiles are so powerful, you can actually change program capabilities through the DDS specifications, without changing a single line of program code.

So why isn't every programmer fluent in subfile programming? Probably it is because subfiles can be overwhelming if you start from scratch without examples. Subfiles are inherent to DDS specifications, and are less influenced by a high-level language. There are several types of subfiles, and each one has differing requirements of specifications. If you don't understand these requirements up front, attempting to write a subfile program can be frustrating at best. Combine this with the fact that DDS compile time errors can be rather vague, and it is no wonder that a majority of programmers are not actively using subfiles.

However, once you grasp the concepts and requirements of subfiles, you will wonder how you ever programmed without them. The first thing you will say when you see a subfile program is "Where's the code?" You will also find yourself doing what just about every other subfile programmer does—copying and reusing subfile programs instead of writing new ones. The actual program code for different subfile programs can be nearly identical. You may use different files, with different field names, but the structure and code remain the same.

For this reason, it is very possible to write an inquiry program in just a few minutes by copying an existing subfile and making a few minor changes. However, just because subfiles can be easily copied from one program to another does not mean that you should avoid understanding the concepts of subfiles. All to often, this situation is the case. A programmer will copy a subfile program that works, and make slight modifications to "fit" the application they are designing without actually understanding what is really going on inside the subfile program.

And don't think that subfiles are just glamorous ways of writing inquiry screens; that does them a big injustice. They are invaluable in designing data entry screens for fast keypunch operators, since subfiles outperform standard programs in this area. They can be used to satisfy some very difficult screen requirements, especially in the area of data input.

Subfiles can also be used for nondisplay processing. For instance, you can use them instead of using an array. In an array, you must define the exact size and number of entries up front. By using a subfile, you can define a certain number of entries, but have the entries extend themselves automatically if more are needed. This technique keeps your program size small, while allowing the flexibility to accommodate a growing number of entries if needed.

Although this book is written to stand alone, it can be a great benefit to study the concepts in front of a terminal with the actual programs running in front of you. This method allows you to get the look and feel for how a subfile program processes in real time. Also, by using this book in conjunction with the interaction of a terminal, you can run the programs, make a quick change to the source, and then run them again to see how different concepts work. If you don't want to make the actual changes and recompile the programs at your terminal, you will be able to see the changes made for you in the book. This book is designed for those of you on the run. If you only have a few minutes to go over a concept, you will see the differences without having to make changes to source code and wait for compiles to finish.

Organization

This book is designed to have multiple entry points, depending on your current level of subfile knowledge. Chapter 2 covers subfile basics. It is designed for those who have little or no knowledge of subfile programming or those who need to refresh their skills. It is subdivided so that the material is covered on a very high level, then covered again in more detail, followed by a very thorough explanation. Using this approach, the novice subfile programmer will not be overwhelmed and will understand more fully the concepts behind subfile programming.

For those who have a solid knowledge of subfiles, but want to explore "what's new" in subfile programming, this book is designed for you as well. There are chapters that cover all of the various subfile functionalities including message subfiles, using the Open Query File (OPNQRYF) command with subfiles, and displaying multiple subfiles.

All source code for the programs is provided. Upon purchasing this book, you can freely copy the source code for your own use. Copy the provided source code, make a few quick modifications, and you will have a subfile program usable for your company. As stated earlier, since most subfile programs are inherently similar, subfile programs can be copied to create new subfile programs. But do not just look at the program examples and move on from there. It is vitally important that you have a good understanding of why the concepts work, not just how to get them to work. This understanding will allow you to make more intelligent decisions when designing a subfile application. Many programmers who have copied subfiles and made them work still don't understand why they work.

For those of you who have been coding and copying subfile programs for years, this book includes some recent enhancements to subfiles. These enhancements include the ability to easily present a subfile in a window, as well as some techniques that use the Open Query File (OPNQRYF) command to optimize subfile performance even further. There are also related topics that make subfile programming easier, such as the ability to retrieve the cursor position from the screen. The book is up to date through V2R3 of OS/400.

Hopefully, you will apply the knowledge that you gain from reading this book. The use of subfiles can boost your productivity and that is probably the reason why most technical interviews include a question such as "What is your experience with subfiles?" For your programming skills to be taken seriously, you need to know how and when to implement a subfile program, and why it works.

Program Overview

To help you learn about subfiles, the programs used throughout this book are as structurally identical as possible. For example, you will see the following in every program:

Comments

It may seem like there are too many comments inside the programs, especially since the code is explained in the book. However, it is better to include too many comments than not enough. You may not want to retain all of the comments in the programs as you copy them for your own use. Too many comments inside a program, especially the ones that tell you something you already know, tend to make a program look a lot larger and more cluttered than it really is. For instance, you may see a program that has 75 lines, but only 29 of them are actually code statements. A lot of these comments exist to take you through a guided tour of the program and can be reduced or eliminated if you find them unnecessary.

The @CMD Subroutine

Anytime a display format is read, either by a READ or EXFMT opcode, the subroutine @CMD is executed immediately afterwards. For example:

```
C               EXFMTDISPLAY1
C               EXSR @CMD
```

The purpose of the @CMD subroutine is to process any function keys that the user may have pressed on the display. In this case, the programs do not follow a "top down" design. But, the use of the @CMD subroutine makes the program much more readable, since it moves the command key processing code out of the mainline process, and groups it together in its own subroutine. This design does not condemn nor condone any particular coding practice. This method was chosen because it makes the programs more readable and thus more understandable.

The *INZSR Subroutine

The *INZSR subroutine contains the descriptions for all program defined fields, parameter lists, and key lists. Occasionally, field names will be defined outside the *INZSR subroutine in places where seeing the field definition is beneficial in understanding a concept. In other words, an attempt has been made to define all fields in *INZSR, but they may be defined anywhere in the program.

Naming Conventions

RPG programs follow the naming convention SFRxxx. The prefix SF stands for subfile. The "R" means it is an RPG program. "xxx" is a numeric value. For example SFR100.

CL Programs follow the same standard, except that the “R” is replaced by a “C”. For example SFC100.

Display Files follow the same standard, with a “D”. For example SFD100.

Field Names

Any field name that is described inside a program will usually be prefixed with an “@”. For example @CUSNO.

If a field is defined from a file, the field name will not begin with an “@”.

All subroutines start with “@”, except the *INZSR subroutine.

These standards are not recommended as any kind of coding practice. The programs have been coded for understandability and readability. They have not been coded for performance or speed unless specifically indicated. For instance, the programs that use the READC (read next changed record) operation are designed to boost performance. But, in general the code is designed for readability only.

2

Subfile Basics

What Are Subfiles?

Subfiles are used in display file programming to allow programmers to handle lists of columnar data more easily. You can use subfiles to greatly simplify the programming required for a workstation program in which you need to display multiple records on the display that have the same record layout. An example of a program that would display a group of records with the same layout could be a customer search inquiry program, like the one below.

```
 xx/xx/xx              Customer Master Search                    QPGMR
 xx:xx:xx                                                        SFR001

Number  Customer Name
00001   David's Computers and Bait Shop
00002   AS/400s Are Us
00003   Tennessee Minicomputer Sales
00004   PCs For Less
00005   Mid-South Computer Sales
00010   Mini Computing Clearance









F3-Exit
```

As you can see, the program calls for displaying six customer master records on the display at a time. Since these six records are identical, each record displays a customer number and customer name. The program required for this application is an ideal subfile candidate. Subfiles greatly minimize the programming effort needed to handle displaying repetitious data.

Although this book mainly covers using subfiles to display lists of data in a workstation program, subfiles are not limited to this purpose. You can use subfiles to simulate arrays in order to perform array processing.

Most programmers find that as they start learning subfiles, the structure and methods to using subfiles are somewhat foreign. Indeed, they are foreign. With subfiles, you use display file functions, and even RPG operation codes, that were designed specifically for subfile programming. Don't let the initial overwhelming complexity of subfiles keep you from mastering them. Subfiles are not complex, but they are different. However, they are different for a reason. Subfiles are a programming tool that will greatly increase your productivity. Once you learn subfiles, you will wonder how you ever got along without them.

Why Use Subfiles?

The two major reasons for using subfiles are productivity and simplicity. Subfiles make your life easier by providing you with a tool to quickly and easily design programs which display records with an identical record layout. A complex-looking workstation program can be totally written in a matter of minutes and take only a few lines of code.

Modifying a subfile program is much easier than modifying a non-subfile program, especially since there is much less program code to handle. Adding capabilities to a subfile program can be done with relative ease, even if you did not write the original program. You may not even have to change a single line of RPG code to add additional capabilities, as a majority of subfile functionality is contained within the DDS.

The reason subfile programs are easy to write and maintain is because most of the functionality resides within the DDS specifications of the display file. On some subfiles, even the roll key capability is handled within the display file itself, and coding for the roll keys isn't even required inside the program or in the display file.

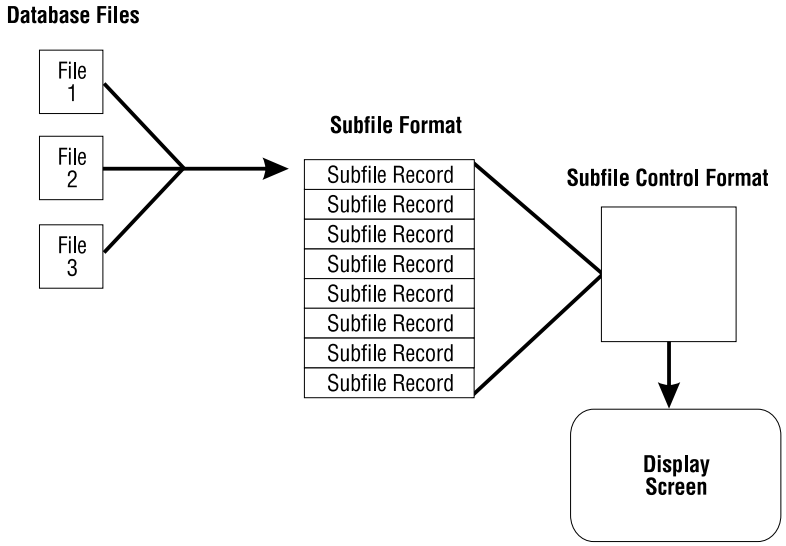
Because most of the functionality of subfile programs reside within the display file, new subfile programs are easily generated by copying existing ones. Most of the RPG code for different subfile applications are similar, if not identical.

One of the biggest reasons why most programmers aren't very fluent in subfile programming is that the RPG code required for different subfile programs is so similar. Some programmers copy one subfile program into another one and make it function, but they do not have a clue as to how the subfile actually works. Programming subfiles seems so complicated, that if they can copy a working subfile program into a new application and get it to work, they are happy. If you think about it, your programs can do a lot of work with a relatively small amount of code. It is almost like programs know what they need to do, even without you coding for it. Once you commit yourself to learning subfiles, you will be amazed at what they can do! So don't allow yourself to just copy subfile programs and make them work. Learn how to make subfiles work for you.

Basic Subfile Concepts

Subfiles are essentially a two step process—loading and displaying. First, you load whatever data you want to display into the subfile. Then you tell the subfile to display the data.

Let's get a feel for the "big picture" in the subfile process. The following diagram shows the steps required to load a subfile, and display it on a screen:



In the program, you load the subfile with the data that you want to display from one or more data files. After you have loaded the subfile, you can display the data inside the subfile.

Subfiles consist of two record formats—a *subfile* format and a *subfile control* format. Programmers frequently begin to feel overwhelmed at this point. Why would you need two display file formats?

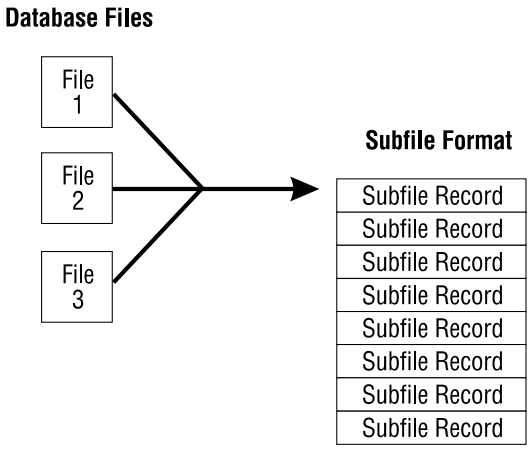
The subfile format is actually a file that resides in memory. In the load process, you write the data that you want to display to the subfile format. This format holds the subfile records which reside in main memory with your job. This format also defines the record layout for the subfile. A subfile is essentially a memory file and is similar to the way a physical file describes a record layout and contains data. The subfile format describes the subfile records, and also contains data. The DDS keyword to describe the subfile format is SFL.

Subfiles are also similar to physical files in other ways. For example, remember how you can change a physical file to hold a maximum of 100 records, or 10,000 records, or that there should be no maximum number of records? You can do the same thing with subfiles by using the subfile control format. The subfile control format defines the subfile as a whole, such as how many records it can contain, how many records to display on a screen and so on. It is also used to display the subfile, or to clear records out of a subfile. The DDS keyword used to define a subfile control format is SFLCTL.

So, in essence, the subfile format handles individual subfile records, whereas the subfile control format handles the subfile as a whole entity.

Loading a Subfile

You place records into the subfile format with a process known as loading as illustrated in the following diagram.



During the loading process, you read one or more database files to extract information. You then write this information in the form of subfile records to the subfile format.

Every subfile uses a value known as the *subfile page*. It is controlled by the subfile page keyword SFLPAG, which can be found in the subfile control format. A subfile page is the number of subfile records that can be displayed on the screen at one time. For example, the customer search program described earlier has a subfile page value of six. This value tells the subfile to display six records at a time. So when you display the subfile, you get six records displayed at a time, like this:

```
 xx/xx/xx              Customer Master Search                    QPGMR
 xx:xx:xx                                                        SFR001

Number  Customer Name
00001   David's Computers and Bait Shop
00002   AS/400s Are Us
00003   Tennessee Minicomputer Sales
00004   PCs For Less
00005   Mid-South Computer Sales
00010   Mini Computing Clearance









F3-Exit
```

By setting the value of SFLPAG to six, the subfile displays six records on the screen at a time. You can change SFLPAG to any value you want, as long as it is physically possible to display that many records on the screen at a time.

The subfile page actually has two meanings, which can complicate things a bit for the new subfile programmer. A subfile page also refers to a specific page number inside a subfile. A page number is the group of records that are displayed together on a screen. For instance, in the above example, subfile page one contains subfile records 1 through 6. Subfile page two contains records 7 through 12.

Displaying a Subfile

You can program subfiles to display in one of two ways. The first way is to define the subfile in full screen mode. The customer search inquiry program described earlier is an example of this type of subfile:

```
 xx/xx/xx              Customer Master Search                    QPGMR
 xx:xx:xx                                                        SFR001

Number  Customer Name
00001   David's Computers and Bait Shop
00002   AS/400s Are Us
00003   Tennessee Minicomputer Sales
00004   PCs For Less
00005   Mid-South Computer Sales
00010   Mini Computing Clearance









F3-Exit
```

The second way is to define the subfile as a window subfile. Defining a subfile this way causes the subfile to be placed inside a window on the display. The same customer search inquiry program displayed in a window would look like this:

```
.........................................
: Number  Customer Name                 :
: 00001   David's Computers and Bait Shop :
: 00002   AS/400s Are Us                 :
: 00003   Tennessee Minicomputer Sales   :
: 00004   PCs For Less                   :
: 00005   Mid-South Computer Sales       :
: 00010   Mini Computing Clearance       :
:                                        :
: F3-Exit                                :
:                                        :
:........................................:
```

Windowed subfiles will be covered in Chapter 3. This chapter will only discuss full screen subfiles.

Types of Subfiles

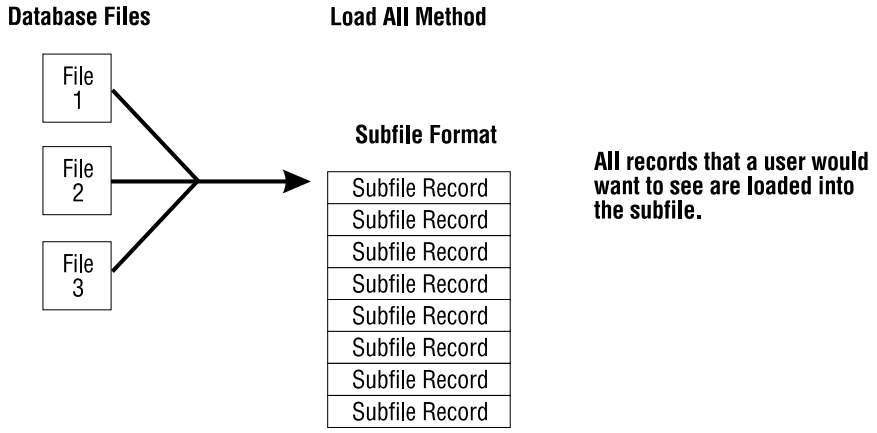
What makes a subfile possible is that each subfile record has the same record layout. In other words, in the above example, all six records contained the customer number

and the customer name. All records were identical, which makes this program an ideal subfile program candidate.

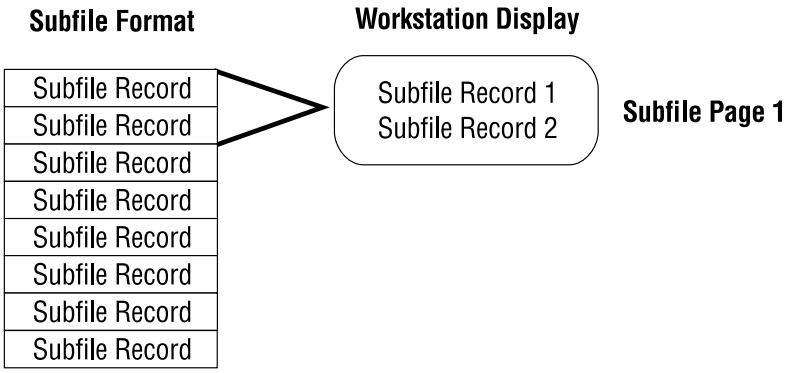
To get data records into a subfile, you have to write them to the subfile format. You actually write the records as if you were writing to a data file. The subfile in effect is a file, and it resides in memory. Writing records to a subfile is known as loading. There are three ways to load a subfile: the load-all method, the expanding subfile method, and the single-page load method.

Load-All Method

The first method of loading a subfile is to load all of the records that you want to display into the subfile at one time. After this load is complete, you can display the subfile, which gives the users access to all of the subfile records by allowing them to roll backwards and forwards. The diagram below shows how the load-all method works.



Every record that you want to display must be loaded into the subfile before displaying the subfile. Sometimes you may only load a few records into the subfile, other times you might load hundreds. But for the load-all method, you simply load every record in before you display the subfile.

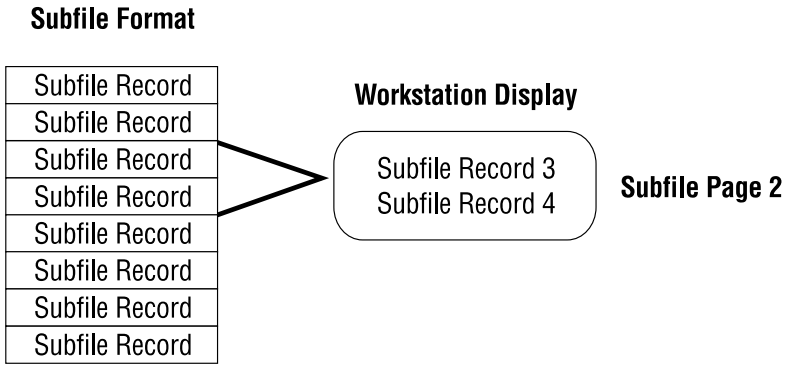


Loading the subfile is accomplished by writing the subfile records to the subfile format. Once every record has been loaded, you can display the subfile by writing the subfile control format.

In this example, two subfile records are being displayed on the screen at one time. You set the subfile page to 2, which means subfile page one consists of subfile records 1 and 2. Subfile page two consists of subfile records 3 and 4 and so on.

The user can roll forward and backward through the subfile. The subfile itself handles the roll keys, and you don't even have to make reference of them in either the RPG program or the DDS specifications for the display file.

If the user rolls forward, the subfile displays the second subfile page which contains subfile records 3 and 4.

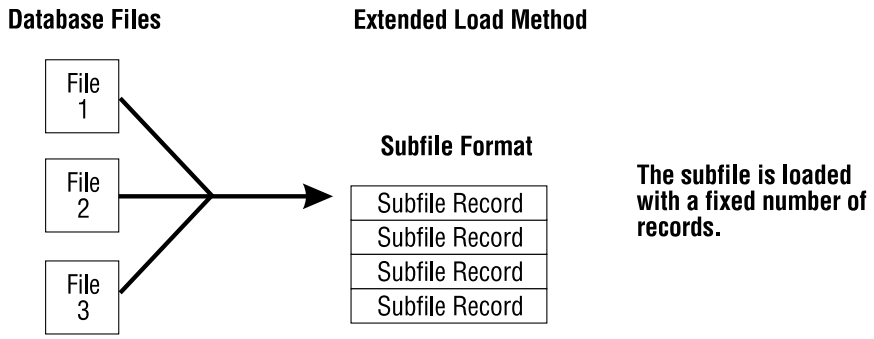


This figure shows the load-all subfile method. You first load all possible records into the subfile, then display the subfile.

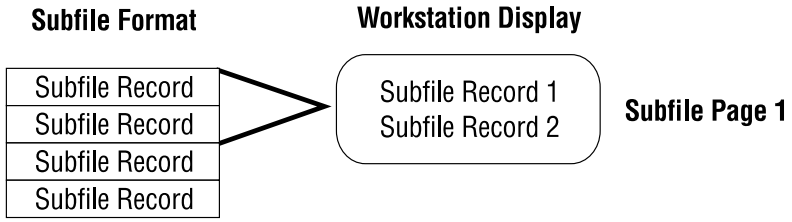
The Expanding Subfile Method

The second method of loading a subfile is to start by loading a specific number of records into the subfile. You can then display the subfile to the user. If the information the user needs is not currently loaded into the subfile, your program can load more subfile records behind those already loaded. This technique is known as an expanding subfile.

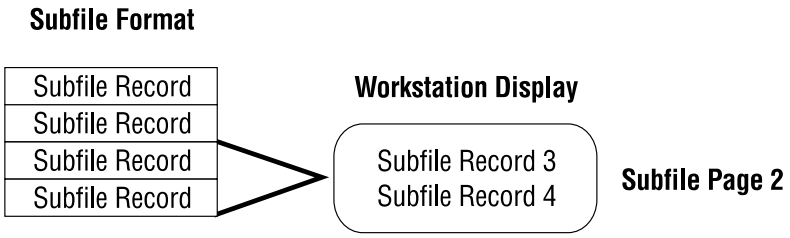
Below is a diagram of an expanding subfile. In an expanding subfile, your program loads a fixed number of records. In this example the program loads four records initially into the subfile.



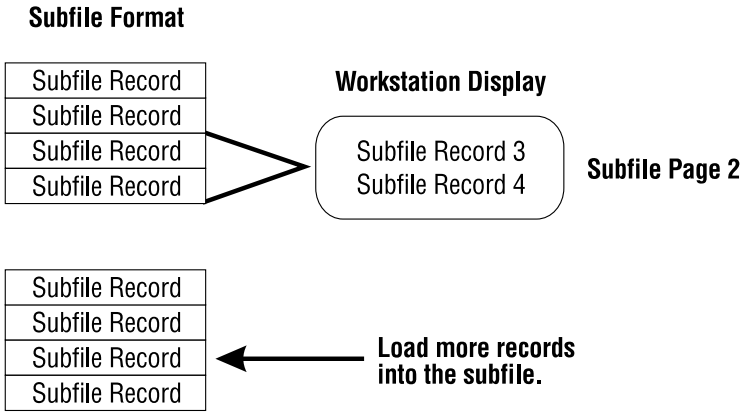
Now the program can display the subfile to the user. By default, the subfile displays the first subfile page.



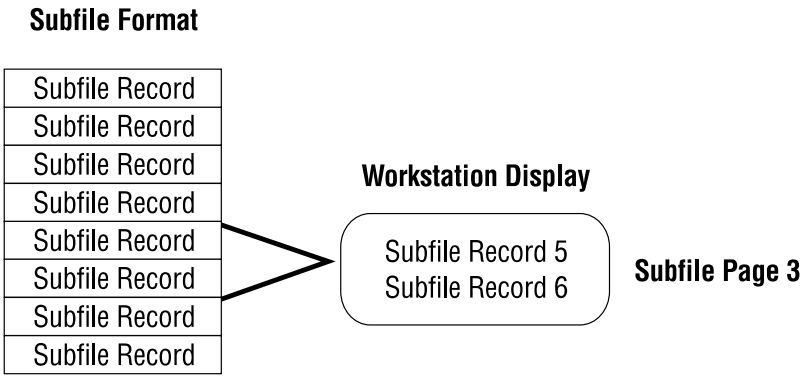
If the user rolls forward, then the subfile automatically switches the display to subfile page two for you.



If the user attempts to roll past the second page, then your program has to load more records into the subfile.



Once these records have been loaded, your program can tell the subfile to display the third page of records.

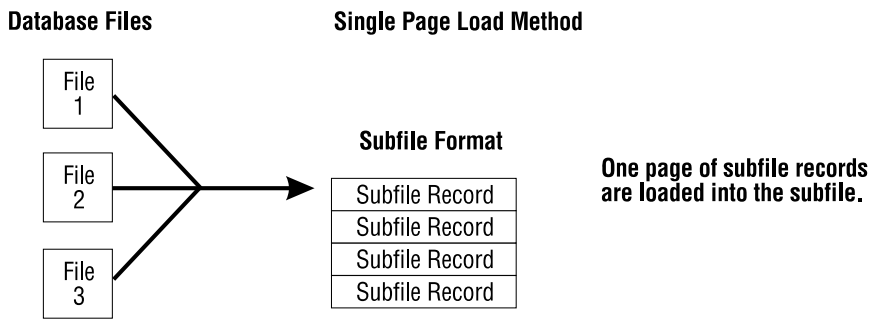


This is the concept behind the expanding load method. You load a fixed number of records into the subfile. If the information the user needs is not in the subfile, you load another fixed number of records. You continue this process until the user finds the information he is looking for, or there are no more records to load.

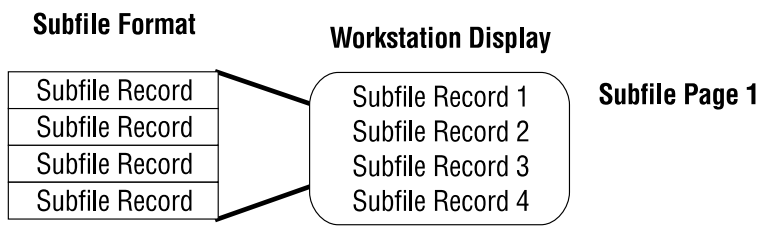
Single-Page Load Method

The third method of loading is known as a single-page load. In this process you only load the number of subfile records that can be displayed on the screen at one time. If the information the user needs is not in the subfile, then the existing subfile records are discarded, and another page worth of subfile records is loaded.

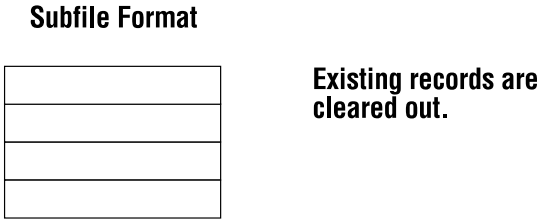
Below is a diagram of the single-page load method. In this example the subfile page value is set to four.



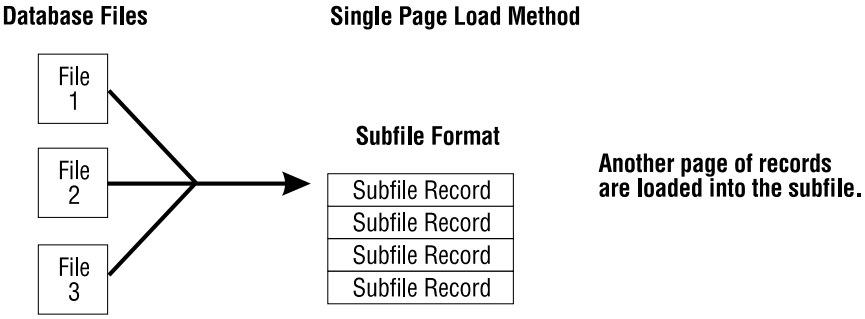
In the single-page load method, your program loads only one page worth of records into the subfile. In this case, it loaded four records. The program can then display the subfile records.



If the information the user wants to see is not on this page of subfile records, then you can't just add more records to the subfile like you do in an expanding subfile method. You must first clear the existing records out of the subfile. You clear a subfile by setting on an indicator located in the subfile control format that controls the clear operation. Then you write the subfile control format, which removes all data from the subfile.



Now your program can load another page worth of records into the subfile.



Now that you have seen how records are added to the subfile, let's see how to go about displaying those records.

Subfile Formats

The basic subfile concept is that a subfile is made up of two formats—the subfile format and the subfile control format. When you want to add subfile records, you write them to the subfile format. If you want to display the records, you write the subfile control format.

You display subfile records by using the subfile control format. The subfile control format controls the subfile as a whole by controlling the number of records to display on the screen at a time, clearing the subfile, and displaying the subfile. If you want to display the subfile, you would write the subfile control format.

When you design the display output screen, you want more than just your subfile records on the display. You may also want some heading information for the data, such as the headings on the customer master inquiry program:

```
 xx/xx/xx              Customer Master Search                    QPGMR
 xx:xx:xx                                                        SFR001

Number  Customer Name
00001   David's Computers and Bait Shop
00002   AS/400s Are Us
00003   Tennessee Minicomputer Sales
00004   PCs For Less
00005   Mid-South Computer Sales
00010   Mini Computing Clearance

F3-Exit
```

On this screen, the heading data includes a program heading, the date, time and user, as well as headings for the customer number and name fields.

The subfile control format allows you to decide if you want the headings and/or the subfile data to be displayed. The subfile control format has two keywords that allow you to control this information. The subfile display (SFLDSP) keyword allows you to control if you want the subfile records to be displayed. The subfile display control (SFLDSPCTL) keyword allows you to display the heading fields on the control format. By conditioning these two keywords when you write the control format, you can display either the subfile format, the subfile control format, both formats, or neither format.

A Subfile Example

Let's take a look at the code for a subfile program to see how you can do so much with a very small amount of RPG code.

Three source members are required for the customer master inquiry program. The first one is a physical file called CUSTOMER. The second one is the display file SFD001. The third is the RPG program SFR001. The three source members must be compiled in that order.

After you create the CUSTOMER file, you will need to enter some sample data into it before running the program. An easy way to do this is to use the Update Data (UPDDTA) command as shown below:

```
UPDDTA FILE(CUSTOMER)
```

The following is the code for CUSTOMER file, the SFD001 display file, and the SFR001 RPG program:

```
A*============================================================
A* To compile:
A*
A*      CRTPF      FILE(XXX/CUSTOMER) SRCFILE(XXX/QDDSSRC)
A*
A*============================================================
A                                       UNIQUE
A          R RCUST
A            CUSNO          5  0
A            CUSNM         30
A            CUSAD         30
A            CUSCT         30
A            CUSST          2
A            CUSCP          9
A            CUSPH         10  0
A          K CUSNO


A*============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD001) SRCFILE(XXX/QDDSSRC)
A*
A*============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03)
A          R SFD001A                    SFL
A            CUSNO     R        0  5  2REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                       COLOR(PNK)
A            CUSNM     R        0  5 10REFFLD(RCUST/CUSNM *LIBL/CUSTOMER)
A                                       COLOR(PNK)
A          R SFD001B                    SFLCTL(SFD001A)
A                                       SFLSIZ(0007)
A                                       SFLPAG(0006)
A                                       OVERLAY
A  30                                   SFLDSP
A                                       SFLDSPCTL
A                                   4  2'Number'
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A                                   4 10'Customer Name                  '
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A                                   1  3DATE
A                                       EDTCDE(Y)
```

```
A                                       COLOR(BLU)
A                                  2  3TIME
A                                       COLOR(BLU)
A                                  1 26'Customer Master Search'
A                                       COLOR(WHT)
A                                  1 71USER
A                                       COLOR(BLU)
A                                  2 71'SFR001'
A                                       COLOR(BLU)
A          R TRAILER
A                                 23  1'F3-Exit'
A                                       DSPATR(HI)


F*==============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR001) SRCFILE(XXX/QRPGSRC)
F*
F*==============================================================
FCUSTOMERIF  E           K        DISK
FSFD001  CF  E                    WORKSTN
F                                        @RRN  KSFILE SFD001A
C*
C                     WRITETRAILER
C                     EXFMTSFD001B
C                     EXSR @CMD
C*
C*********************************************************
C*    @ C M D   S U B R O U T I N E                      *
C*                                                       *
C*    Executed after every EXFMT or display read op code.  *
C*    Used to process command keys.                      *
C*********************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, seton LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C                     ENDSR
C*
C*********************************************************
C*    * I N Z S R   S U B R O U T I N E                  *
C*                                                       *
C*    This routine is automatically called by the        *
C*    program at startup.                                *
C*********************************************************
C*
C           *INZSR    BEGSR
C*
C*  Set the subfile's relative record number to 0, then
```

```
C*  load all of the records into the subfile.
C*
C                     Z-ADD0         @RRN    40

C                     EXSR @LOAD
C*
C                     ENDSR
C*
C*********************************************************
C*    @ L O A D     S U B R O U T I N E                  *
C*                                                       *
C*    This subroutine will read the customer file and    *
C*    load the subfile records. Since this is a load all *
C*    subfile program, all records in the customer file  *
C*    would be read and loaded, except that you have      *
C*    conditioned the subfile not to load past the subfile *
C*    size value.                                        *
C*********************************************************
C*
C           @LOAD     BEGSR
C*
C*  Perform this loop until you reach the end of
C*  the Customer file, or the maximum subfile records
C*  allowed. Both cases would turn on indicator 99.
C*
C           *IN99     DOWNE*ON
C                     READ CUSTOMER                 99
C*
C*  If indicator 99 is on, then leave this do loop.
C*
C           *IN99     IFEQ *ON
C                     LEAVE
C                     END
C*
C*  Add 1 to the subfile relative record number. If you
C*  reach the maximum allowable subfile records, indicator
C*  99 will turn on.
C*
C                     ADD  1         @RRN
C*
C* Write the record to the subfile.
C*
C                     WRITESFD001A
C           @RRN      IFEQ 9999
C                     MOVE *ON       *IN99
C                     END
C                     END
C*
C* If you have loaded at least one record into the subfile,
C* turn on indicator 30. This indicator conditions the

C* SFLDSP keyword. If you have loaded at least one record
C* into the subfile, it will be all right to display the subfile
C* format. If you did not load any records into the subfile, you
C* would not turn on indicator 30, because attempting to display
C* an empty subfile would result in an error being issued.
```

```
C*
C           @RRN      IFGT 0
C                     MOVE *ON       *IN30
C                     END
C*
C                     ENDSR
```

What a Subfile Looks Like to a User

Once you have the source members entered and compiled on your system, try calling the SFR001 program. Use the roll keys to roll back and forth. Notice that if you try to roll past the last record in the subfile, or attempt to roll past the first record in the subfile, a message is displayed.

What a Subfile Looks Like to a Programmer

Program SFR001 has only 29 lines of RPG code. That's all it took to write a program that displays, in this case, six customer master records on the screen. How many lines of code would it take to write this program without the use of subfiles? A whole lot more!

How many lines of RPG code do you think you would need if you wanted to display 15 customer master records at a time in a subfile program? The answer is only 29 lines because the number of records per subfile display (the subfile page) is controlled by the display file itself, not by the RPG program. You can (and will) change the number of subfile records to display by simply modifying one line of DDS and recompiling the display file and RPG program. How much added code, time, and trouble would it be to make this kind of modification to a non-subfile program? A lot!

And remember, this program allows the roll keys to be used, and even tells you if you are trying to roll beyond the first or last record. But the roll keys in this program aren't even referred to in the RPG program. They are not even referred to in the DDS! The subfile handles this process for you automatically. As you probably know, programming for roll keys can be very complicated in a non-subfile program.

For the most part, subfile functionality is controlled by DDS. Placing this functionality in the display files allows the code for both the RPG program and DDS to be greatly minimized. A complex program that uses very little code takes less time to program, test, and maintain.

The Subfile Record Layout

Notice on the subfile screen that there are two fields—a customer number and a customer name.

```
 xx/xx/xx              Customer Master Search                    QPGMR
 xx:xx:xx                                                        SFR001

Number  Customer Name
00001   David's Computers and Bait Shop
00002   AS/400s Are Us
00003   Tennessee Minicomputer Sales
00004   PCs For Less
00005   Mid-South Computer Sales
00010   Mini Computing Clearance

F3-Exit
```

These two fields make up the record layout for the subfile. Although these two fields are displayed on the screen, you can also use hidden fields that do not display on the screen as part of the record layout. In this example, the subfile record layout looks like this:

```
Customer Number||Customer Name
```

If you want to display other fields on the screen, you can also incorporate these into the subfile record layout. For example, if you add the phone number and zip code the subfile record layout looks like this:

```
Customer Number||Customer Name||Phone Number||Zip Code
```

The conditioning indicators for these fields are also kept as part of the subfile record. For instance, suppose you use indicator 80 to condition whether or not the customer number field is to be displayed in reverse image. This indicator is also stored in the subfile record layout.

```
Customer Number||Customer Name||Phone Number||Zip Code||*IN80
```

You can also store constant fields as part of the subfile record. For example, on the display screen, you could display the text "Zip Code:" in front of the zip code field for each subfile record. This text is also stored as part of the subfile record layout.

```
Customer Number||Customer Name||Phone Number||Zip Code||*IN80|| Zip Code:
```

A field does not have to be displayed on the screen in order to be stored in the subfile record layout. Let's look at the SFR001 display again.

```
 xx/xx/xx              Customer Master Search                      QPGMR
 xx:xx:xx                                                          SFR001

Number  Customer Name
00001   David's Computers and Bait Shop
00002   AS/400s Are Us
00003   Tennessee Minicomputer Sales
00004   PCs For Less
00005   Mid-South Computer Sales
00010   Mini Computing Clearance









F3-Exit
```

This display has the customer number and name fields. But suppose that if the user selects one of these records, you want to chain to a 1099 detail file that is keyed by social security number.You can take the customer number, chain back to the customer master file and get the social security number, then chain to the 1099 detail file, or you can save the customer's social security number in the subfile record layout without displaying the field. You do this by specifying that the social security number is a hidden field. This way, it does not display on the screen, but it does reside in the subfile record. Now if the user selects a subfile record, you already have the social security number for the customer, so there is no reason to chain back to the customer file. All you have to do is chain directly to the 1099 detail file. The subfile record layout would then look like this:

```
Customer Number||Customer Name||Phone Number||Zip Code||*IN80|| Zip
Code: ||Social Security Number
```

Don't confuse a hidden field with a nondisplay field. A non-display field actually takes up a position on the screen, whereas a hidden field does not.

In the display file, SFD001, the SFD001A format only has two fields described, CUSNO and CUSNM. How can this be? You see six lines of information on the screen when you call the subfile program. Why does the display file only describe two fields? Because only two fields are used by the display file—CUSNO and CUSNM. However, you want to see six of them every time you display the screen. You can do this by setting the SFLPAG value to six, which tells the subfile to display six records on the screen at a time. The beauty of subfiles is that repetitious data is described only once. The display file handles the repetitions for you.

Think of what you would have to do if you tried to write this program without using a subfile. You would have to have a display file record format with twelve fields described—six for CUSNO and six for CUSNM. You would have to clear these twelve fields, then you would have to read the customer file six times, loading each field with data. If the user pressed the roll-up or roll-down key, you would have to code for them in your program.

Here's another example. Suppose a user needs the customer's phone number and state added to the screen. You will see in an upcoming example that if you use a subfile, it requires only two more lines to be added to the display file, and none to the RPG program. If you use a non-subfile program, and need to add the customer's phone number and state, you must add another twelve fields to the display file. And think of all the additional RPG code that would be required. Now, which would you rather do—add two lines of DDS, recompile the display file, recompile the program, run a test and move this change into production, or write repetitious code that is a nightmare to maintain?

Subfile Screen Domains

The subfile screen is broken up into two domains, each domain is controlled by a display file format. The first domain is the subfile control format, which is also known as the "header" display. You can place heading data in the control format as was done in SFR001. The control domain looked like this:

```
 xx/xx/xx              Customer Master Search                    QPGMR
 xx:xx:xx                                                        SFR001

Number  Customer Name
```

This part of the domain is the subfile control format.

The second part of the subfile domain is the subfile format. This subfile format screen is made up of all of the subfile records that can be displayed on one screen, or page. For SFR001, the subfile format domain looks like this:

```
00001   David's Computers and Bait Shop
00002   AS/400s Are Us
00003   Tennessee Minicomputer Sales
00004   PCs For Less
00005   Mid-South Computer Sales
00010   Mini Computing Clearance
```

This part of the domain is the subfile format. You display both domains at the same time when you write the subfile control format. When you combine them both on the screen for SFR001, they looked like this:

```
 xx/xx/xx              Customer Master Search                    QPGMR
 xx:xx:xx                                                        SFR001

Number  Customer Name
00001   David's Computers and Bait Shop
00002   AS/400s Are Us
00003   Tennessee Minicomputer Sales
00004   PCs For Less
00005   Mid-South Computer Sales
00010   Mini Computing Clearance
```

It is important to know that the subfile control format domain and the subfile format domain cannot overlap. The subfile control domain must be on display lines which are above the subfile format.

So one question remains. How do you get the command key text on the bottom of the subfile? The answer is by using a third record format.

The third record format for a subfile screen is known as the "trailer" or "footer." This format is used to display the command keys that are allowed within the application and is not part of the subfile display.

In SFR001, the trailer format looked like this:

```
F3-Exit
```

When you place the three formats together as in SFR001, you get the following screen:

```
 xx/xx/xx              Customer Master Search                    QPGMR
 xx:xx:xx                                                        SFR001

Number  Customer Name
00001   David's Computers and Bait Shop
00002   AS/400s Are Us
00003   Tennessee Minicomputer Sales
00004   PCs For Less
00005   Mid-South Computer Sales
00010   Mini Computing Clearance

F3-Exit
```

You will always need at least two record formats for a subfile, the control format and the subfile format. If you wish to display the command keys or any other text below the subfile records, you need the third format. You cannot combine the trailer information into the subfile or control format. The control format screen information must be located on the screen prior to the subfile format data. It cannot reside beneath it. The subfile format must only contain subfile record information. Therefore, to get the command key line, you must write a trailer format, then overlay this format with the subfile control and subfile display formats.

The Subfile Display Format

As stated before, the subfile display format is actually a file, which resides in memory, that contains data records. This file is accessed by relative record number. You can actually chain, read, and write to this file. The subfile format contains what can be considered as the external description for each record in the subfile. Look again at SFD001. Notice in the subfile format record (SFD001A) that the CUSNM and CUSNO fields are listed. Each record contains CUSNO and CUSNM. The records reside in the file by relative record number. You can add records to a subfile by using the WRITE operation code, just like you would a physical file.

The subfile record format (SFL keyword) defines the layout of the records in the subfile. So when you deal with the subfile format, you are dealing with individual

records, just like you would be if it were a physical file. The program sends output operations to the subfile record format to load the subfile.

In summary, the subfile format contains the record layout for each subfile record, and this format is the vehicle in which you access or add records to the subfile.

Here is the subfile format for the SFD001 example:

```
A          R SFD001A                    SFL
A            CUSNO     R        O  5  2REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                      COLOR(PNK)
A            CUSNM     R        O  5 10REFFLD(RCUST/CUSNM *LIBL/CUSTOMER)
A                                      COLOR(PNK)
```

You can see in this example that the subfile format consists only of the SFL keyword, plus two field descriptions. This subfile is defined as a record containing two fields, CUSNO and CUSNM. This description defines a single subfile record, which is all you can do with the subfile format. The subfile control format, on the other hand, allows you to control the size of the subfile.

The SFL keyword is a mandatory keyword for the subfile format. This keyword specifies that this format is a subfile format.

The Subfile Control Format

The subfile control format is used to display the records in the subfile format. It is also used to determine how many of the subfile records to display at a time (page size), how many records to store (subfile size), and how to tailor the subfile to your specifications.

Keep in mind one of the differences between the subfile format and subfile control format. The subfile format contains the subfile data, whereas the subfile control format is used to display the data. The format of the subfile control keyword is:

```
SFLCTL(subfile-record-format-name)
```

You must specify the name of the subfile record format as the parameter value for this keyword. The subfile control record format can contain field descriptions as well as subfile control keywords. your program can only display subfile records by issuing an output operation to the subfile control record format. You can add many different functions to subfiles. These functions are also controlled by the subfile control format.

Think of the two formats this way.

- Subfile Format = The writing and reading of subfile records individually.

- Subfile Control Format = Acting upon the subfile as a whole (such as displaying the subfile).

Subfile Keywords

The following keywords must be included in the subfile control format.

- SFLCTL (Subfile Control) - This keyword is used to specify that this format is a subfile control format. You can see this on the SFD001B format in the SFD001 display file. The subfile control keyword specifies which subfile format this control format will control. In the example, this control format controls subfile format SFD001A.

- SFLDSPCTL (Subfile Display Control) - Use this record-level keyword on the subfile control record format so that OS/400 displays fields in the subfile control record when your program sends an output operation to the subfile control record format. If you do not use an option indicator with this keyword, the subfile control record is displayed on every output operation to the subfile control record format. SFLDSPPCTL is required if your program sends an input operation to the subfile control format. Since this is almost always the case, I have listed this keyword with the other mandatory keywords.

- SFLDSP (Subfile Display) - This keyword specifies whether or not to display the subfile format domain. The subfile format domain contains the subfile records to display. If there are no records in the subfile, you must not activate this keyword. Attempting to display an empty subfile will result in an error condition.

- SFLPAG (Subfile Page) - This keyword specifies how many records make up one subfile page. This number is usually, but not necessarily, the number of subfile records displayed on the screen at the same time. The SFD001 example specifies that six subfile records are to be displayed on each page.

- SFLSIZ (Subfile Size) - The meaning of this keyword depends on the value to which the Subfile Page (SFLPAG) keyword is set. If the SFLSIZ is equal to the SFLPAG, then the maximum number of records that can be loaded into the subfile at any one time is equal to the SFLSIZ. SFLSIZ cannot be less than the value specified in the SFLPAG keyword. If the value of SFLSIZ is greater than SFLPAG, then the subfile is initially sized to handle the number of records specified in the SFLSIZ keyword. However, the subfile will extend itself to handle more records if the program writes more records than the number specified in SFLSIZ. You will see later on what the proper setting of this keyword should be. In this example, it is set to seven. The subfile can initially handle seven

records, but if the program writes more than seven, the subfile will grow to accommodate them. The maximum number of records that can exist in a subfile at the same time is 9,999.

The following keywords are optional, and can be used to add functionality to the subfile. Their availability of use depends on certain factors.

- SFLCLR (Subfile Clear)
- SFLDLT (Subfile Delete)
- SFLDROP (Subfile Drop)
- SFLDSPCTL (Subfile Display Control) Note: The SFLDSPCTL keyword is required if your program sends an input operation to the subfile control record format. Otherwise, it is not required.
- SFLEND (Subfile End)
- SFLENTER (Subfile Enter)
- SFLFOLD (Subfile Fold)
- SFLINZ (Subfile Initialize)
- SFLLIN (Subfile Line)
- SFLMSG (Subfile Message)
- SFLMSGID (Subfile Message Identifier)
- SFLPGMQ (Subfile Program Message Queue)
- SFLRCDNBR (Subfile Record Number)
- SFLRNA (Subfile Records Not Active)
- SFLROLVAL (Subfile Roll Value)

Under certain circumstances, some keywords will be ignored. If the subfile size (SFLSIZ) equals subfile page (SFLPAG), then the following keywords are ignored:

- SFLDROP (Subfile Drop)
- SFLFOLD (Subfile Fold)
- SFLROLVAL (Subfile Roll Value)

If the subfile record format contains field selection, the following keywords are not valid on the subfile control record format:

- SFLDROP (Subfile Drop)
- SFLFOLD (Subfile Fold)
- SFLINZ (Subfile Initialize)
- SFLLIN (Subfile Line)
- SFLRCDNBR (Subfile Record Number)
- SFLRNA (Subfile Records Not Active)
- SFLROLVAL (Subfile Roll Value)

The following keywords are not valid for the subfile control record format:

- USRDFN (User-Defined)
- ASSUME (Assume)

We will see all of these keywords listed here in detail later in the book.

Controlling the Subfile

The subfile control format is responsible for displaying both the control format domain and the subfile format domain. It is controlled further by the keywords SFLDSP and SFLDSPCTL. If you wish to output the subfile control domain, you must ensure that the SFLDSPCTL keyword is activated. This keyword is activated if there is no conditioning indicator, or if the conditioning indicator is in a "true" state.

In the following examples, the subfile control format domain is written to the display:

```
A                                       SFLDSPCTL
```

For the next example, assume indicator 30 is on:

```
A  30                                   SFLDSPCTL
```

For the next example, assume indicator 30 is off:

```
A N30                                   SFLDSPCTL
```

In all of these cases, when you do an output operation to the subfile control format, the subfile control domain information is displayed. You perform an output to the

control format domain in program SFR001 by performing the following operation in the RPG program:

```
C                       EXFMTSFD001B
```

This operation causes the following to be displayed on the screen:

```
 xx/xx/xx              Customer Master Search                    QPGMR
 xx:xx:xx                                                        SFR001

Number  Customer Name
```

The subfile domain is controlled by the SFLDSP keyword. There is usually a conditioning indicator specified for this keyword. If the condition of the indicator is true, then the subfile domain is displayed. Otherwise, it is not displayed.

In the following examples, the subfile domain is written to the display. Assume indicator 30 is on:

```
A  30                                   SFLDSP
```

For the next example, assume indicator 30 is off:

```
A N30                                   SFLDSP
```

One of the most frequent causes of errors in subfiles is attempting to display an empty subfile. You cannot display the subfile format if the subfile does not contain any records. Indicator 30 controls whether or not you display the subfile format. You can control the indicator from within the RPG program. If you have loaded any records into the subfile, the program turns on indicator 30 so that the subfile will be displayed. If you have not loaded any records into the subfile, then the program turns off indicator 30. This switch prevents the program from displaying an empty subfile, which would result in an error.

The subfile domain is displayed when your program performs a write to the control format, just like the way the control domain is displayed.

Let's set up an example to see how the two domains are displayed at the same time. Assume that the SFLDSPCTL keyword has no conditioning indicator, so it is always activated. And assume that the SFLDSP keyword is conditioned to be active if indicator 30 is on. The DDS code would look like this:

```
A                                       SFLDSPCTL
A  30                                   SFLDSP
```

Now let's say that your program loads some subfile records into the subfile by performing output operations to the subfile format:

```
C                       WRITESFD001A
```

Once the program is finished loading records, it turns on indicator 30 to activate the SFLDSP keyword. This indicator tells the control format to display the subfile format because you are sure that there are records in the subfile. The program then performs a write operation to the trailer format to display the command key text:

```
C                       WRITETRAILER
```

Then the program performs a write and read operation (EXFMT) to the subfile control format:

```
C                       EXFMTSFD001B
```

The display screen would now look like this:

```
 xx/xx/xx              Customer Master Search                    QPGMR
 xx:xx:xx                                                        SFR001

Number  Customer Name
00001   David's Computers and Bait Shop
00002   AS/400s Are Us
00003   Tennessee Minicomputer Sales
00004   PCs For Less
00005   Mid-South Computer Sales
00010   Mini Computing Clearance









F3-Exit
```

But let's say that the program doesn't add any subfile records. If the program turns on indicator 30 anyway, it causes an error stating that you attempted to display an empty subfile.

If you don't add any subfile records, the program sets off indicator 30, which controls the subfile domain. Then it can write the trailer and control formats:

```
C                       WRITETRAILER
C                       EXFMTSFD001B
```

The display screen would now look like this:

```
 xx/xx/xx              Customer Master Search                    QPGMR
 xx:xx:xx                                                        SFR001

Number  Customer Name                    


F3-Exit
```

Now, look at the subfile control format for SFD001 again to see how it is designed.

```
A          R SFD001B                     SFLCTL(SFD001A)
A                                        SFLSIZ(0007)
A                                        SFLPAG(0006)
A                                        OVERLAY
A  30                                    SFLDSP
A                                        SFLDSPCTL
A                                   4  2'Number'
A                                        COLOR(YLW)
A                                        DSPATR(UL)
A                                   4 10'Customer Name                 '
A                                        COLOR(YLW)
A                                        DSPATR(UL)
A                                   1  3DATE
A                                        EDTCDE(Y)
A                                        COLOR(BLU)
A                                   2  3TIME
A                                        COLOR(BLU)
A                                   1 26'Customer Master Search'
A                                        COLOR(WHT)
A                                   1 71USER
A                                        COLOR(BLU)
A                                   2 71'SFR001'
A                                        COLOR(BLU)
```

You can see the SFLCTL keyword with its parameter SFD001A. The subfile control format SFD001B is the controlling format for subfile format SFD001A. Inside the display file, the subfile format comes first, followed by the control format.

Next you can see the keywords SFLSIZ and SFLPAG. The size of the subfile is set to seven. This number does not mean that only seven records can be contained in the subfile, but that the subfile is initially set up for seven records. The subfile can automatically extend itself to allow more than seven records.

The SFLPAG is set to six. You will see six records at a time on the display. Notice that the subfile page and size are not equal. You do not have to code for the roll keys in the display file. You don't code them in the RPG program either. The subfile itself handles the roll keys.

Next, you see the OVERLAY keyword. If you do not fully understand DDS keywords such as OVERLAY, examples and explanations are given in the next section.

Next, you see SFLDSP, which is a mandatory keyword. It is conditioned with indicator 30. If indicator 30 is on, the program displays the subfile records. If indicator 30 is off, it does not display the subfile records. This indicator is coded this way because if the subfile is empty, and the program tries to display the subfile, it is issued an error message. You allow the RPG program to turn on 30 as soon as it loads a record, or leave it off if no records are loaded. That way, when the program displays the subfile, it knows whether or not to display the records.

The next keyword is SFLDSPCTL. It is a mandatory keyword since the program is performing an input operation to the subfile. After these keywords, you will see the constant string for the customer and name headings.

Subfile Fact

A maximum of 24 subfiles can be active at any one time. A maximum of 12 subfiles can be displayed on the base display or in a single window at any one time. Under normal situations, this number is more than enough!

General Subfile Information

Reading and writing to and from the subfile record format is fast because a subfile stores its information in memory, as part of the job's process access group (PAG). Because disk accesses are not usually needed once records are loaded into a subfile, a subfile can process the roll keys very quickly.

Subfiles aren't just for *displaying* multiple lines of related data. They can also be used to *enter* multiple lines of data. Another use of subfiles is for non-display processing, such as simulating arrays or multiple occurrence data structures. Using a subfile

instead of an array or multiple occurrence data structure for the same purpose has some benefits. With an array or multiple occurrence data structure, you must hard code in the program the maximum number of entries. Your array or data structure must be large enough to handle the maximum number of occurrences. By using a subfile, you can set the initial number of occurrences low, and allow the subfile to extend itself if needed. Using this technique causes the program object size to be much smaller than using an array or multiple occurrence data structure.

Subfile Size and Page Parameters for the Single-Page Load Method

If the SFLPAG and SFLSIZ values are equal, as they would be for a single-page load method subfile, then the value of SFLSIZ is the total number of records that can be contained in the subfile at one time. For example, if you have the following defined in your subfile control format, then ten subfile records is the maximum number of records that can be loaded into the subfile at one time:

```
A                                       SFLPAG(0010)
A                                       SFLSIZ(0010)
```

Attempting to load more records results in an error. This type of subfile is known as a single-page subfile, or a page-at-a-time subfile, because the subfile can contain only one page worth of records.

Page and Size Parameters for the Load-All and Expanding Subfile

If the SFLPAG and SFLSIZ values are not equal, then you have either a load-all subfile or an expanding subfile. The load-all and expanding subfile size value must be at least one greater than the value of the subfile page. For instance:

```
A                                       SFLPAG(0016)
A                                       SFLSIZ(0017)
```

You should set the size of a load-all or expanding subfile equal to what would be a reasonable number of records that could exist in the subfile. For example, if a subfile normally gets loaded with approximately 80 records, then set the subfile size value somewhere around 80-100.

If you attempt to load more records into the subfile than the value specified in the subfile size, the subfile will extend itself to handle these additional records. A single-page subfile, however, can never be extended. Single-page subfiles can hold only the number of records specified in the value of the SFLSIZ keyword.

The maximum number of records that can be loaded in a subfile is 9,999. If you have the possibility of having more than 9,999 records loaded, then you need to trap for this error. The most common way to do this is to check the relative record number of the subfile record you are writing out. If it equals 9,999, then do not load any more records into the subfile. Display the subfile and notify the user with a message that the subfile is full, and that rolling forward will cause all previous records in the subfile to be removed. If the user rolls forward at this point, the subfile is cleared and the loading process continues with the relative record number reset to 1.

Load-All Subfiles

Now that you have studied the display file SFD001, let's take a closer look at the RPG program SFR001. This program uses the load-all technique. Any data that you want to display must reside in the subfile before the program displays it.

```
F*==============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR001) SRCFILE(XXX/QRPGSRC)
F*
F*==============================================================
FCUSTOMERIF  E           K        DISK
FSFD001  CF  E                    WORKSTN
F                                        @RRN  KSFILE SFD001A
C*
C                     WRITETRAILER
C                     EXFMTSFD001B
C                     EXSR @CMD
C*
C*************************************************************
C*    @ C M D   S U B R O U T I N E                          *
C*                                                           *
C*    Executed after every EXFMT or display read op code.    *
C*    Used to process command keys.                          *
C*************************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, seton LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C                     ENDSR
C*
C*************************************************************
C*    * I N Z S R   S U B R O U T I N E                      *
C*                                                           *
```

```
C*    This routine is automatically called by the           *
C*    program at startup.                                   *
C*********************************************************
C*
C           *INZSR    BEGSR
C*
C*  Set the subfile's relative record number to 0, then
C*  load all of the records into the subfile.
C*
C                     Z-ADD0         @RRN    40
C                     EXSR @LOAD
C*
C                     ENDSR
C*
C*********************************************************
C*    @ L O A D     S U B R O U T I N E                   *
C*                                                        *
C*    This subroutine will read the customer file and     *
C*    load the subfile records. Since this is a load all  *
C*    subfile program, all records in the customer file   *
C*    would be read and loaded, except that you have       *
C*    conditioned the subfile not to load past the subfile *
C*    size value.                                         *
C*********************************************************
C*
C           @LOAD     BEGSR
C*
C*  Perform this loop until you reach the end of
C*  the Customer file or the maximum subfile records
C*  allowed. Both cases would turn on indicator 99.
C*
C           *IN99     DOWNE*ON
C                     READ CUSTOMER                 99
C*
C*  If indicator 99 is on, then leave this do loop.
C*
C           *IN99     IFEQ *ON
C                     LEAVE
C                     END
C*
C*  Add 1 to the subfile relative record number. If you
C*  reach the maximum allowable subfile records, indicator
C*  99 will turn on.
C*
C                     ADD  1         @RRN
C*
C* Write the record to the subfile.
C*
C                     WRITESFD001A
C           @RRN      IFEQ 9999
C                     MOVE *ON       *IN99
C                     END
C                     END
C*
C* If you have loaded at least one record into the subfile,
C* turn on indicator 30. This indicator conditions the
```

```
C* SFLDSP keyword. If you have loaded at least one record
C* into the subfile, it will be all right to display the subfile
C* format. If you did not load any records into the subfile, you
C* would not turn on indicator 30, because attempting to display
C* an empty subfile would result in an error being issued.
C*
C           @RRN      IFGT 0
C                     MOVE *ON       *IN30
C                     END
C*
C                     ENDSR
```

Let's first take a look at the file specifications.

```
FCUSTOMERIF  E           K        DISK
FSFD001  CF  E                    WORKSTN
F                                        @RRN  KSFILE SFD001A
```

The database file from which the program is extracting data is the customer master file (CUSTOMER). The program reads this file in keyed sequence, which is by customer number.

The display file, SFD001, has a file extension that all subfiles must have. The file extension tells the RPG program what field to use as the relative record number when adding records to the subfile. As stated earlier, subfile records are written and read by relative record number. This file extension links the variable that contains the relative record number to the subfile format.

You can have more than one subfile in a display file. If you do, you will need to add a file extension for each subfile to link the subfile format to its corresponding relative record number field.

When the program is first called, the initialize subroutine is executed automatically. This routine looks like this:

```
C           *INZSR    BEGSR
C                     Z-ADD0         @RRN    40
C                     EXSR @LOAD
C                     ENDSR
```

This routine prepares the program to load the subfile records. First, the program zeros out the subfile's relative record number variable, @RRN. Then it executes the @LOAD subroutine, which loads the subfile records into the subfile. The @LOAD subroutine looks like this:

```
C           @LOAD     BEGSR
C           *IN99     DOWNE*ON
C                     READ CUSTOMER                 99
C           *IN99     IFEQ *ON
C                     LEAVE
C                     END
C                     ADD  1         @RRN
```

```
C                       WRITESFD001A
C             @RRN      IFEQ 9999
C                       MOVE *ON        *IN99
C                       END
C                       END
C             @RRN      IFGT 0
C                       MOVE *ON        *IN30
C                       END
C                       ENDSR
```

Essentially, the program reads the customer file until indicator 99 is set on, which means it has reached the end of the customer file. If indicator 99 is not on, the program adds one to the subfile relative record number field and writes the subfile record out. The first time it reads a record from customer file, the relative record number field @RRN contains the value of 1. Then the program writes the subfile format, which adds the subfile record to the subfile. Since the fields in the subfile (CUSNO and CUSNM) have the same name as the fields in the customer file, it can simply write the subfile record to the subfile format. Then the program reads the next customer record.

The next thing the program does is to check the relative record number field @RRN to see if it equals 9,999. This code is necessary because a subfile can contain only 9,999 records. If the subfile relative record number is equal to 9,999, then the program is not allowed to load any more subfile records into the subfile. Attempting to load more than 9,999 records results in an error. If the program checks @RRN and finds it equal to 9,999, then it sets on indicator 99, which causes the program to drop out of the loop.

This load routine loops until the program either loads 9,999 subfile records, or until it reaches the end of the CUSTOMER file. Right after loading the subfile, the program executes the following code:

```
C             @RRN      IFGT 0
C                       MOVE *ON        *IN30
C                       END
```

This code determines whether or not to display the subfile format. Indicator 30 conditions the SFLDSP keyword in the subfile control format.

```
A  30                                     SFLDSP
A                                         SFLDSPCTL
```

As long as the relative record number field @RRN is greater than the value 0, the program knows it has loaded records into the subfile. If this is the case, then it is able to display the subfile format, which causes the subfile records to display. If the value of @RRN is 0, it indicates that the program has not loaded any records into the subfile, and that if it attempts to display the subfile format, an error will occur. You can see this effect by commenting out the statements in this fashion:

```
C*          @RRN      IFGT 0
C*                    MOVE *ON       *IN30
C*                    END
```

Now copy the CUSTOMER file into library QTEMP without copying any records. Next make sure that library QTEMP is at the top of your library list. Now call SFR001. You will experience an error for attempting to display an empty subfile.

That concludes the code for the load routine. Essentially the program has read every record in the CUSTOMER file, loaded them into the subfile, and now it is ready to display the subfile. The next thing the program does is to display the subfile.

```
C                     WRITETRAILER
C                     EXFMTSFD001B
```

When the program writes the TRAILER format, it places the command key line on the screen. Then it displays the subfile by writing the subfile control format, SFD001B. The control format has two keywords (SFLDSP and SFLDSPCTL) which determine if the program displays the control format fields and the subfile records. That part of the control format looks like this:

```
A  30                                   SFLDSP
A                                       SFLDSPCTL
```

When the program writes the control format, it uses these keywords to determine if the control format fields and the subfile records are displayed. Since the display subfile control format keyword (SFLDSPCTL) is not conditioned, it displays the control format fields. The subfile format keyword (SFLDSP) is conditioned with indicator 30. The program sets indicator 30 on in the @LOAD subroutine if it loaded any records into the subfile. So if it loaded records into the subfile, it displays the subfile records. Otherwise, it doesn't.

That is the basic flow of a load-all subfile program, the program loads all the records first, then it displays the subfile. Before you actually run this program, let's examine the importance of the subfile page and subfile size keywords.

Looking at the subfile control format specifications, you can see that the subfile page is set to six, and the subfile size is set to seven. With the subfile page set to six, the program displays six records on the screen at one time. In other words, there are six records to a "page."

To use the load-all technique, the value of subfile size must be at least one more than the value of subfile page. In this case the subfile size has been set to seven. The subfile is initially sized to handle seven records, but it does not mean that seven is the maximum number of records that can be loaded into the subfile, just that the subfile is initially set up to handle seven. If your program attempts to load more than seven records, as in this program, the subfile extends itself to accommodate more

subfile records. The importance of the subfile page and size will be examined throughout this book, as well as what proper settings should be. If you call program SFR001, you get a screen that looks like this:

```
 xx/xx/xx              Customer Master Search                    QPGMR
 xx:xx:xx                                                        SFR001

Number  Customer Name_____________________

F3-Exit
```

You get the subfile display with six subfile records on it. Now here's the beauty of load-all subfiles—roll forward and backward. The subfile rolls back and forth through the subfile records for you, automatically. Neither the RPG program nor the DDS make a single reference to the roll-up or roll-down keys. Nor did you have to code an RPG routine to handle the scrolling of data. The load-all subfile handles all rolling for you automatically. This feature is one reason why subfile programs use so little code compared to conventional programming, the DDS removes this function from the high-level language.

It is important to know what is going on inside the RPG program at this point. Essentially, nothing is going on. Once the program displays the subfile, it turns control over to the display file. So the program sits on the following statement:

```
C                     EXFMTSFD001B
```

It remains on this statement until a function key or the Enter key is pressed. If the roll keys are pressed, the RPG program does nothing because the display file handles this process automatically. If the F3 key is pressed, then control is passed back to the program. The program executes the very next RPG statement.

```
C                     EXSR @CMD
```

This statement executes the @CMD subroutine. The @CMD subroutine is used to check if the F3 function key is pressed. If F3 is not pressed, then the program writes the trailer format, and then executes the subfile control format again. If F3 is pressed, the following code is processed:

```
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
```

So if F3 is on, the program sets on indicator LR and returns out of the program.

Now let's expand on this program a bit. As stated earlier, the subfile size is the initial number of subfile records that a subfile can hold. The subfile can automatically extend itself to handle more records. You saw this happen in SFR001. However, you can keep the subfile from loading more records than the subfile size. Display file SFD003 and RPG program SFR003 show how this can be done:

```
A*==============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD003) SRCFILE(XXX/QDDSSRC)
A*
A*==============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03)
A          R SFD003A                    SFL
A            CUSNO     R        O  5  2REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                       COLOR(PNK)
A            CUSNM     R        O  5 10REFFLD(RCUST/CUSNM *LIBL/CUSTOMER)
A                                       COLOR(PNK)
A          R SFD003B                    SFLCTL(SFD003A)
A                                       SFLSIZ(0007)
A                                       SFLPAG(0006)
A                                       OVERLAY
A  30                                   SFLDSP
A                                       SFLDSPCTL
A                                   4  2'Number'
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A                                   4 10'Customer Name                  '
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A                                   1  3DATE
A                                       EDTCDE(Y)
A                                       COLOR(BLU)
A                                   2  3TIME
A                                       COLOR(BLU)
A                                   1 26'Customer Master Search'
A                                       COLOR(WHT)
A                                   1 71USER
A                                       COLOR(BLU)
A                                   2 71'SFR003'
A                                       COLOR(BLU)
```

```
A          R TRAILER
A                                 23  1'F3-Exit'
A                                      DSPATR(HI)
F*=============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR003) SRCFILE(XXX/QRPGSRC)
F*
F*=============================================================
FCUSTOMERIF  E           K        DISK
FSFD003  CF  E                    WORKSTN
F                                        @RRN  KSFILE SFD003A
C*
C                     WRITETRAILER
C                     EXFMTSFD003B
C                     EXSR @CMD
C*
C*********************************************************
C*    @ C M D   S U B R O U T I N E                      *
C*                                                       *
C*    Executed after every EXFMT or display read op code.  *
C*    Used to process command keys.                      *
C*********************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, seton LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C                     ENDSR
C*
C*********************************************************
C*    * I N Z S R   S U B R O U T I N E                  *
C*                                                       *
C*    This routine is automatically called by the        *
C*    program at startup.                                *
C*********************************************************
C*
C           *INZSR    BEGSR
C*
C*  Set the subfile's relative record number to 0, then
C*  load all of the records into the subfile.
C*
C                     Z-ADD0         @RRN    40
C                     EXSR @LOAD
C*
C                     ENDSR
C*
C*********************************************************
C*    @ L O A D     S U B R O U T I N E                  *
C*                                                       *
C*    This subroutine will read the customer file and    *
C*    load the subfile records. Since this is a load all *
```

```
C*    subfile program, all records in the customer file     *
C*    would be read and loaded, except that you have         *
C*    conditioned the subfile not to load past the subfile  *
C*    size value.                                           *
C*************************************************************
C*
C           @LOAD     BEGSR
C*
C*  Perform this loop until you reach the end of
C*  the Customer file or the maximum subfile records
C*  allowed. Both cases would turn on indicator 99.
C*
C           *IN99     DOWNE*ON
C                     READ CUSTOMER                 99
C*
C*  If indicator 99 is on, then leave this do loop.
C*
C           *IN99     IFEQ *ON
C                     LEAVE
C                     END
C*
C*  Add 1 to the subfile relative record number. If you
C*  reach the maximum allowable subfile records, indicator
C*  99 will turn on.
C*
C                     ADD  1         @RRN
C*
C* Write the record to the subfile.
C*
C                     WRITESFD003A                  99
C           @RRN      IFEQ 9999
C                     MOVE *ON       *IN99
C                     END
C                     END
C*
C* If you have loaded at least one record into the subfile,
C* turn on indicator 30. This indicator conditions the
C* SFLDSP keyword. If you have loaded at least one record
C* into the subfile, it will be all right to display the subfile
C* format. If you did not load any records into the subfile, you
C* would not turn on indicator 30, because attempting to display
C* an empty subfile would result in an error being issued.
C*
C           @RRN      IFGT 0
C                     MOVE *ON       *IN30
C                     END
C*
C                     ENDSR
```

The only difference between SFR001 and SFR003 is that one line of code has been modified. It is the statement which contains the write operation to the subfile:

```
C                     WRITESFD001A                  99
```

Indicator 99 has been placed in the equal column (positions 58 and 59) of the WRITE operation code. Now when the program loads the subfile, indicator 99 turns on as soon as it writes the seventh record to the subfile, since the subfile size is set to 7. If indicator 99 is on, then the program falls out of the loop.

You could just as easily have changed the following line of code to get the same result:

From:

```
C           @RRN      IFEQ 9999
C                     MOVE *ON       *IN99
C                     END
```

To:

```
C           @RRN      IFEQ 7
C                     MOVE *ON       *IN99
C                     END
```

This change would have the same effect, except that it is not the best design method. By coding the indicator on the write statement, you can be assured that the program will load the correct number of records even if you change the subfile size. In other words, if you change the subfile size from seven to ten, then the program will automatically load ten records with the write operation conditioned. Otherwise, you would have to change the subfile size and change the RPG program.

The reason for coding an indicator on the write statement is to control the load time for a load-all subfile. The maximum number of records that can be loaded into a subfile is 9,999. For a load-all subfile, your program could theoretically load 9,999 records into the subfile, before it displays the screen to the user. This program would cause the user to wait for an extended amount of time while the it loads 9,999 subfile records. You could set the subfile size to 500, then code an indicator on the write statement. This change keeps the subfile from loading more than 500 records, which allows the screen to be displayed to the user in a reasonable amount of time.

In display file SFD004 and RPG program SFR004, you will increase the size of the subfile to allow 14 records to be displayed on the screen:

```
A*============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD004) SRCFILE(XXX/QDDSSRC)
A*
A*============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03)
A          R SFD004A                    SFL
A            CUSNO     R        O  5  2REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                       COLOR(PNK)
```

```
A            CUSNM     R        O  5 10REFFLD(RCUST/CUSNM *LIBL/CUSTOMER)
A                                      COLOR(PNK)
A          R SFD004B                   SFLCTL(SFD004A)
A                                      SFLSIZ(0015)
A                                      SFLPAG(0014)
A                                      OVERLAY
A  30                                  SFLDSP
A                                      SFLDSPCTL
A                                  4  2'Number'
A                                      COLOR(YLW)
A                                      DSPATR(UL)
A                                  4 10'Customer Name                  '
A                                      COLOR(YLW)
A                                      DSPATR(UL)
A                                  1  3DATE
A                                      EDTCDE(Y)
A                                      COLOR(BLU)
A                                  2  3TIME
A                                      COLOR(BLU)
A                                  1 26'Customer Master Search'
A                                      COLOR(WHT)
A                                  1 71USER
A                                      COLOR(BLU)
A                                  2 71'SFR004'
A                                      COLOR(BLU)
A          R TRAILER
A                                 23  1'F3-Exit'
A                                      DSPATR(HI)


F*============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR004) SRCFILE(XXX/QRPGSRC)
F*
F*============================================================
FCUSTOMERIF  E           K        DISK
FSFD004  CF  E                    WORKSTN
F                                       @RRN  KSFILE SFD004A
C*
C                     WRITETRAILER
C                     EXFMTSFD004B
C                     EXSR @CMD
C*
C*********************************************************
C*    @ C M D   S U B R O U T I N E                      *
C*                                                       *
C*    Executed after every EXFMT or display read op code.  *
C*    Used to process command keys.                      *
C*********************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, seton LR and exit.
C*
C           *IN03     IFEQ *ON
```

```
C                       MOVE *ON       *INLR
C                       RETRN
C                       END
C*
C                       ENDSR
C*
C*************************************************************
C*    * I N Z S R   S U B R O U T I N E                      *
C*                                                           *
C*    This routine is automatically called by the            *
C*    program at startup.                                    *
C*************************************************************
C*
C           *INZSR      BEGSR
C*
C*  Set the subfile's relative record number to 0, then
C*  load all of the records into the subfile.
C*
C                       Z-ADD0         @RRN    40
C                       EXSR @LOAD
C*
C                       ENDSR
C*
C*************************************************************
C*    @ L O A D     S U B R O U T I N E                      *
C*                                                           *
C*    This subroutine will read the customer file and        *
C*    load the subfile records. Since this is a load all     *
C*    subfile program, all records in the customer file      *
C*    would be read and loaded, except that you have          *
C*    conditioned the subfile not to load past the subfile   *
C*    size value.                                            *
C*************************************************************
C*
C           @LOAD       BEGSR
C*
C*  Perform this loop until you reach the end of
C*  the Customer file or the maximum subfile records
C*  allowed. Both cases would turn on indicator 99.
C*
C           *IN99       DOWNE*ON
C                       READ CUSTOMER                 99
C*
C*  If indicator 99 is on, then leave this do loop.
C*
C           *IN99       IFEQ *ON
C                       LEAVE
C                       END
C*
C*  Add 1 to the subfile relative record number. If you
C*  reach the maximum allowable subfile records, indicator
C*  99 will turn on.
C*
C                       ADD  1         @RRN
C*
C* Write the record to the subfile.
```

```
C*
C                       WRITESFD004A                    99
C            @RRN       IFEQ 9999
C                       MOVE *ON        *IN99
C                       END
C                       END
C*
C* If you have loaded at least one record into the subfile,
C* turn on indicator 30. This indicator conditions the
C* SFLDSP keyword. If we have loaded at least one record
C* into the subfile, it will be all right to display the subfile
C* format. If you did not load any records into the subfile, you
C* would not turn on indicator 30, because attempting to display
C* an empty subfile would result in an error being issued.
C*
C            @RRN       IFGT 0
C                       MOVE *ON        *IN30
C                       END
C*
C                       ENDSR
```

The difference between SFD003 and SFD004 is that the subfile page is changed to 14, which allows14 records to be displayed on the screen at a time. Since the subfile page is 14, the subfile size has to be at least 15, because the program is using the load-all subfile technique. These subfile size and subfile page statements in SFD004 now look like this:

```
A                                       SFLPAG(0014)
A                                       SFLSIZ(0015)
```

Now all you have to do is to compile the display file, compile the RPG program, and you are done. Now if you call the SFR004 program your display looks like this:

```
 xx/xx/xx              Customer Master Search                    QPGMR
 xx:xx:xx                                                        SFR004

Number  Customer Name
00001   David's Computers and Bait Shop
00002   AS/400s Are Us
00003   Tennessee Minicomputer Sales
00004   PCs For Less
00005   Mid-South Computer Sales
00010   Mini Computing Clearance
00011   Computer Mart
00025   Westmoreland Computers
00042   Midrange Supercenter
00045   Full Range Computing
00054   Open Systems Inc.
00087   SemiCondutor City
00099   Hard Drive Warehouse
00386   Memory Systems Inc.



F3-Exit
```

Since an indicator has been coded on the WRITE statement, the subfile loads a maximum of 15 records into the subfile, which is equal to the subfile size.

Expanding Subfiles

In a load-all subfile, your program loads every subfile record into the subfile before displaying the subfile. An expanding subfile utilizes a different load technique. In an expanding subfile, your program loads a fixed number of records into the subfile. Then it displays the subfile to the user. The user can roll through the records. If the user attempts to roll past the last loaded record in the subfile, control passes back to the RPG program, and it can load more records into the subfile. It can then display the subfile again, which contains the original subfile records, plus the newly added records. your program can continue this process as long as it does not attempt to load more than 9,999 records into the subfile.

One advantage of the expanding subfile over a load-all subfile is that the subfile is displayed to the user in a very stable amount of time, depending on how many records the program loads. For example, if the program loads 50 records with a load-all technique, the subfile is quickly displayed to the user. If the next time the user calls the subfile program it attempts to load 1,000 records, then the amount of time it takes the program to display the screen is very long. With an expanding subfile, you can divide the load process into a fixed number of records. For instance, suppose you

load six records at a time in an expanding subfile. It would not matter if there were a possibility of loading 5,000 records into the subfile, your program would only load six records before it displays the subfile to the user. This method displays the screen to the user very quickly.

Now let's take the original SFR001 program and modify it so that it becomes an expanding subfile. The changes are made in the display file SFD005 and the RPG program SFR005 as shown below:

```
A*============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD005) SRCFILE(XXX/QDDSSRC)
A*
A*============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03)
A          R SFD005A                    SFL
A            CUSNO     R        O  5  2REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                       COLOR(PNK)
A            CUSNM     R        O  5 10REFFLD(RCUST/CUSNM *LIBL/CUSTOMER)
A                                       COLOR(PNK)
A          R SFD005B                    SFLCTL(SFD005A)
A N99                                   ROLLUP(90)
A                                       SFLSIZ(0007)
A                                       SFLPAG(0006)
A                                       OVERLAY
A  30                                   SFLDSP
A                                       SFLDSPCTL
A                                    4  2'Number'
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A                                    4 10'Customer Name                  '
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A                                    1  3DATE
A                                       EDTCDE(Y)
A                                       COLOR(BLU)
A                                    2  3TIME
A                                       COLOR(BLU)
A                                    1 26'Customer Master Search'
A                                       COLOR(WHT)
A                                    1 71USER
A                                       COLOR(BLU)
A                                    2 71'SFR005'
A                                       COLOR(BLU)
A          R TRAILER
A                                   23  1'F3-Exit'
A                                       DSPATR(HI)
```

```
F*==============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR005) SRCFILE(XXX/QRPGSRC)
F*
F*==============================================================
FCUSTOMERIF  E           K        DISK
FSFD005  CF  E                    WORKSTN
F                                       @RRN  KSFILE SFD005A
C*
C                     WRITETRAILER
C                     EXFMTSFD005B
C                     EXSR @CMD
C*
C*********************************************************
C*    @ C M D   S U B R O U T I N E                      *
C*                                                       *
C*    Executed after every EXFMT or display read op code.  *
C*    Used to process command keys.                      *
C*********************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, seton LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C           *IN90     IFEQ *ON
C                     MOVE *OFF      *IN90
C                     EXSR @LOAD
C                     END
C*
C                     ENDSR
C*
C*********************************************************
C*    * I N Z S R   S U B R O U T I N E                  *
C*                                                       *
C*    This routine is automatically called by the        *
C*    program at startup.                                *
C*********************************************************
C*
C           *INZSR    BEGSR
C*
C*  Set the subfile's relative record number to 0, then
C*  load all of the records into the subfile.
C*
C                     Z-ADD0         @RRN    40
C                     EXSR @LOAD
C*
C                     ENDSR
C*
C*********************************************************
C*    @ L O A D     S U B R O U T I N E                  *
C*                                                       *
```

```
C*    This subroutine will read the customer file and       *
C*    load the subfile records. Since this is a load all    *
C*    subfile program, all records in the customer file     *
C*    would be read and loaded, except that you have         *
C*    conditioned the subfile not to load past the subfile  *
C*    size value.                                           *
C*************************************************************
C*
C           @LOAD     BEGSR
C*
C*  Perform this loop until you reach the end of
C*  the Customer file or the maximum subfile records
C*  allowed. Both cases would turn on indicator 99.
C*
C                     DO   6
C                     READ CUSTOMER                 99
C*
C*  If indicator 99 is on, then leave this do loop.
C*
C           *IN99     IFEQ *ON
C                     LEAVE
C                     END
C*
C*  Add 1 to the subfile relative record number. If you
C*  reach the maximum allowable subfile records, indicator
C*  99 will turn on.
C*
C                     ADD  1         @RRN
C*
C* Write the record to the subfile.
C*
C                     WRITESFD005A
C           @RRN      IFEQ 9999
C                     LEAVE
C                     END
C                     END
C*
C* If you have loaded at least one record into the subfile,
C* turn on indicator 30. This indicator conditions the
C* SFLDSP keyword. If you have loaded at least one record
C* into the subfile, it will be all right to display the subfile
C* format. If you did not load any records into the subfile, you
C* would not turn on indicator 30, because attempting to display
C* an empty subfile would result in an error being issued.
C*
C           @RRN      IFGT 0
C                     MOVE *ON       *IN30
C                     END
C*
C                     ENDSR
```

The subfile size of an expanding subfile must be one more than the subfile page, just like in a load-all subfile. The following line of code is the only one added to the subfile control format of the display file to reference the roll-up key.

```
A N99                                   ROLLUP(90)
```

The ROLLUP keyword is conditioned with indicator 99, which is the end of file indicator. You will see exactly how this conditioning works in just a minute. If the roll-up key is pressed, it sets on indicator 90. The program code needed is not extensive. The following code has been added to the @CMD subroutine:

```
C           *IN90     IFEQ *ON
C                     MOVE *OFF      *IN90
C                     EXSR @LOAD
C                     END
```

The do loop has been changed to:

```
C                     DO   6
```

Now let's see how these code revisions work. The first code revision that you will encounter is the load routine. When the @LOAD subroutine executes, it loads only six records into the subfile. After that, the program writes the subfile control format to the display screen with the following line of code:

```
C                     EXFMTSFD005B
```

With the load-all subfile technique, the program stayed on this statement until a function or Enter key was pressed. The program did not leave this statement if a roll key was pressed, because the display file handled the rolling for us. This technique is similar to the way the expanding subfiles work. The program does not leave this statement if a roll-down key is pressed. It also does not leave this statement if a roll-up key is pressed, and there are more records in the subfile that can be rolled to. However, if the last page in the subfile is currently being displayed, you run into a different situation. The display file passes control back to the program. For example, when you call SFR005, it loads six records and displays the subfile. Since there are only six records in the subfile, the program displays the last loaded subfile page. If you press the roll-up key, the subfile has no more records to automatically roll to, so it passes control back to the program. When the program does this, it turns on indicator 90. Inside the RPG program, SFR005 processes the roll-up key in the @CMD routine, where it executes the @LOAD subroutine again. So another six records are loaded into the subfile and the subfile is redisplayed.

Whenever the program displays the subfile, it displays the first subfile page by default. You can see this action in SFR005 if you press the roll-up key. The program loads six more records into the subfile, then redisplays the subfile, but the first subfile page is still shown. Now if you press the roll-up key, control is not passed back to the program. The subfile has more records it can roll to, so it rolls to the next page in the subfile automatically, and control is not passed back to the RPG program.

When SFR005 reaches end of the CUSTOMER file, indicator 99 turns on. Indicator 99 also controls the roll-up key, but not like it appears. If indicator 99 is on, it means that you cannot use the roll-up key if you are displaying the last page of the subfile. However, you are allowed to use the roll-up key, even when indicator 99 is on, if you are not displaying the last page of the subfile.

Expanding subfiles, and their associated functions, are covered in more detail later.

Single-Page Subfiles

The third way to load a subfile is known as the single-page load, or page-at-a-time load. This method of loading a subfile is different than the load-all or expanding subfile in that you must handle the roll keys inside your program. It is also different from the prior two methods in that a single-page subfile can only hold as many records as specified in the subfile size keyword. If the subfile size is set to six, then the single-page subfile can only hold six records. Attempting to load more records into the subfile than the value of subfile size results in an error. The single-page subfile is also different from the load-all and expanding subfile in that the subfile size and subfile page values must be equal.

Single-page subfiles are usually the least desirable method, because the program becomes much more involved. You must handle the roll keys inside the program because the display file will not perform this function for you. The display cannot automatically handle the roll keys because there is only one page of subfile records in the subfile. There are no additional pages in the subfile for it to automatically roll to. Single-page subfiles are often used when the maximum number of records that a user will attempt to roll through could exceed 9,999. If you use a load-all or expanding subfile, then the maximum number of records you can display is 9,999 because that is the maximum number of records that can exists in a subfile. Using a single-page load, the maximum number of subfile records is never exceeded because the subfile never contains more than one page worth of records at a time.

An expanding subfile can be converted into a single-page load subfile by making some modifications. These changes are reflected in the display file SFD006 and the RPG program SFR006 as shown below:

```
A*==============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD006) SRCFILE(XXX/QDDSSRC)
A*
A*==============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03)
```

```
A          R SFD006A                    SFL
A            CUSNO     R         O  5  2REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                      COLOR(PNK)
A            CUSNM     R         O  5 10REFFLD(RCUST/CUSNM *LIBL/CUSTOMER)
A                                      COLOR(PNK)
A          R SFD006B                    SFLCTL(SFD006A)
A N99                                   ROLLUP(90)
A N98                                   ROLLDOWN(91)
A  31                                   SFLCLR
A                                       SFLSIZ(0006)
A                                       SFLPAG(0006)
A                                       OVERLAY
A  30                                   SFLDSP
A                                       SFLDSPCTL
A                                   4  2'Number'
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A                                   4 10'Customer Name                  '
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A                                   1  3DATE
A                                       EDTCDE(Y)
A                                       COLOR(BLU)
A                                   2  3TIME
A                                       COLOR(BLU)
A                                   1 26'Customer Master Search'
A                                       COLOR(WHT)
A                                   1 71USER
A                                       COLOR(BLU)
A                                   2 71'SFR006'
A                                       COLOR(BLU)
A          R TRAILER
A                                  23  1'F3-Exit'
A                                       DSPATR(HI)


F*============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR006) SRCFILE(XXX/QRPGSRC)
F*
F*============================================================
FCUSTOMERIF  E           K        DISK
FSFD006  CF  E                    WORKSTN
F                                       @RRN  KSFILE SFD006A
C*
C                     WRITETRAILER
C                     EXFMTSFD006B
C                     EXSR @CMD
C*
C*********************************************************
C*    @ C M D   S U B R O U T I N E                      *
C*                                                       *
C*    Executed after every EXFMT or display read op code.  *
C*    Used to process command keys.                      *
C*********************************************************
```

```
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, seton LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C           *IN90     IFEQ *ON
C                     MOVE *ON       *IN31
C                     WRITESFD006B
C                     MOVE *OFF      *IN31
C                     Z-ADD0         @RRN
C                     MOVE *OFF      *IN90
C                     MOVE *OFF      *IN98
C                     EXSR @LOAD
C                     END
C*
C*
C           *IN91     IFEQ *ON
C                     MOVE *ON       *IN31
C                     WRITESFD006B
C                     MOVE *OFF      *IN31
C                     MOVE *OFF      *IN91
C                     Z-ADD7         @D0     30
C                     ADD  @RRN      @D0
C           *IN99     IFEQ '1'
C           *HIVAL    SETGTCUSTOMER
C                     END
C                     DO   @D0
C                     READPCUSTOMER                91
C           *IN91     IFEQ '1'
C                     MOVE *ON       *IN98
C           *LOVAL    SETLLCUSTOMER
C                     LEAVE
C                     END
C                     END
C                     Z-ADD0         @RRN
C                     EXSR @LOAD
C                     END
C*
C                     ENDSR
C*
C*************************************************************
C*    * I N Z S R   S U B R O U T I N E                      *
C*                                                           *
C*    This routine is automatically called by the            *
C*    program at startup.                                    *
C*************************************************************
C*
C           *INZSR    BEGSR
C*
C*  Set the subfile's relative record number to 0, then
C*  load all of the records into the subfile.
C*
```

```
C                     Z-ADD0         @RRN    40
C                     EXSR @LOAD
C*
C                     ENDSR
C*
C*********************************************************
C*    @ L O A D     S U B R O U T I N E                  *
C*                                                       *
C*    This subroutine will read the customer file and    *
C*    load the subfile records. Since this is a load all *
C*    subfile program, all records in the customer file  *
C*    would be read and loaded, except that you have      *
C*    conditioned the subfile not to load past the subfile *
C*    size value.                                        *
C*********************************************************
C*
C           @LOAD     BEGSR
C*
C*  Perform this loop until you reach the end of
C*  the Customer file or the maximum subfile records
C*  allowed. Both cases would turn on indicator 99.
C*
C                     DO   6
C                     READ CUSTOMER                 99
C*
C*  If indicator 99 is on, then leave this do loop.
C*
C           *IN99     IFEQ *ON
C                     LEAVE
C                     END
C*
C*  Add 1 to the subfile relative record number. If you
C*  reach the maximum allowable subfile records, indicator
C*  99 will turn on.
C*
C                     ADD  1         @RRN
C*
C* Write the record to the subfile.
C*
C                     WRITESFD006A
C           @RRN      IFEQ 9999
C                     LEAVE
C                     END
C                     END
C*
C* If you have loaded at least one record into the subfile,
C* turn on indicator 30. This indicator conditions the
C* SFLDSP keyword. If you have loaded at least one record
C* into the subfile, it will be all right to display the subfile
C* format. If you did not load any records into the subfile, you
C* would not turn on indicator 30, because attempting to display
C* an empty subfile would result in an error being issued.
C*
C           @RRN      IFGT 0
C                     MOVE *ON       *IN30
C                     END
```

```
C*
C                       ENDSR
```

There are several modifications to the display file. First, the ROLLDOWN keyword has been added:

```
A N98                                   ROLLDOWN(91)
```

Then the subfile page and size values are made equal each other. In this case, we want to see six records on the display. So we change the subfile size value to six.

```
A                                       SFLSIZ(0006)
```

Since you can only load one page of subfile records into a single-page load subfile, you need a way to clear the subfile of all records before reloading it. You can do so with the subfile clear (SFLCLR) keyword as shown below:

```
A  31                                   SFLCLR
```

Whenever you write the subfile control format with indicator 31 on, the subfile is cleared of all records, so you must handle this condition carefully. You must also write the control format whenever you display the subfile. When you want to display the subfile, you must ensure that indicator 31 is not on, or your subfile records will be cleared.

You must use the subfile clear keyword because a single-page subfile cannot have more records than the value of the subfile size. If you already have records in your subfile and the user presses the roll-up key, you can't just load more records into the subfile. You must clear the subfile of all records first, then load the subfile with the new records.

Now you can compile the display file. That is all of the changes needed to turn the subfile into a single-page load.

The changes needed for the RPG program are a bit more extensive. As stated before, you must handle the roll keys inside your program. The changes required for a single-page load are in the @CMD subroutine to check if the roll-up or roll-down keys were pressed. Notice that the roll-up routine from the expanding subfile has been totally revamped.

```
C           *IN90     IFEQ *ON
C                     MOVE *ON       *IN31
C                     WRITESFD006B
C                     MOVE *OFF      *IN31
C                     Z-ADD0         @RRN
C                     MOVE *OFF      *IN90
C                     MOVE *OFF      *IN98
C                     EXSR @LOAD
C                     END
```

Let's break the roll-up routine into its different parts. Before loading any records into a single-page subfile, you must ensure that the subfile is empty. So the subfile is cleared of any existing records with the following code:

```
C                     MOVE *ON       *IN31
C                     WRITESFD006B
C                     MOVE *OFF      *IN31
```

First, the program sets on the indicator that conditions the subfile clear keyword, then writes the subfile control format. This action clears the subfile. Then the program turns off the conditioning indicator for the subfile clear keyword, so that when it displays the subfile, the records won't be cleared again.

The second part of the routine looks like this:

```
C                     Z-ADD0         @RRN
C                     MOVE *OFF      *IN90
C                     MOVE *OFF      *IN98
C                     EXSR @LOAD
```

In this routine, the program sets the subfile relative record number field to 0. Since you just cleared the subfile, you want to ensure that this field is 0. Then the program turns off indicator 90, which was turned on when the roll-up key was pressed. Another way to turn off indicator 90 is to use the SETOFF keyword in the display file. The program also turns off indicator 98, which controls the ROLLDOWN keyword. Then the program executes the @LOAD routine, which places six records into the subfile.

The roll-down routine is more complicated than the roll-up routine. The roll-down routine looks like this:

```
C           *IN91     IFEQ *ON
C                     MOVE *ON       *IN31
C                     WRITESFD006B
C                     MOVE *OFF      *IN31
C                     MOVE *OFF      *IN91
C                     Z-ADD7         @D0     30
C                     ADD  @RRN      @D0
C           *IN99     IFEQ '1'
C           *HIVAL    SETGTCUSTOMER
C                     END
C                     DO   @D0
C                     READPCUSTOMER                 91
C           *IN91     IFEQ '1'
C                     MOVE *ON       *IN98
C           *LOVAL    SETLLCUSTOMER
C                     LEAVE
C                     END
C                     END
C                     Z-ADD0         @RRN
C                     EXSR @LOAD
C                     END
```

Rather than discuss the details of this routine here, they will be broken apart later. In general, the program clears the subfile, then positions the CUSTOMER file pointer back one screen of records.

As you can see, it is easier to use the load-all and expanding subfile techniques whenever possible, because you don't have to put all of this code into your programs.

Summary

This chapter covered how to code for the three basic ways of loading a subfile. Here are some important notes.

1. In the load-all and expanding subfile techniques, the subfile size must be at least one greater than the subfile page.
2. In the single-page load, the subfile size must equal the subfile page.
3. In the load-all and expanding subfile techniques, you can have a maximum number of 9,999 subfile records loaded in the subfile.
4. In the single-page load, you can load only a single subfile page worth of records into the subfile. For example, if the subfile page is set to 10, then you can load only 10 records into the subfile. Attempting to load more records causes an error.
5. In the load-all subfile technique, the roll-up and roll-down keys are processed by the display file.
6. In the expanding subfile technique, the roll-down key is handled by the display file. The roll-up key is handled both by the display file and the program. If there are records in the subfile that you can roll up to, then the display file will handle processing the roll-up key. If you are displaying the last subfile page, then the display file turns control over to your program, where you can load more subfile records into the subfile.
7. In a single-page load, your program must handle the roll-up and roll-down keys.
8. A subfile progressively gets more complex as you move from a load-all technique, to an expanding load, to a single-page load. The code required to handle the process is the least in a load-all subfile, and the most in a single-page lode. That is not to say that the single-page load is complex, just that it is more complex than a load-all subfile. A single-page load is still much easier to code than writing a non-subfile program to get the same effect.

3

DDS Windows

Before diving into all of the functionality of subfiles, let's first get some exposure to some DDS keywords and techniques. Although these keywords and techniques are not used solely for subfiles, you do make use of them in subfiles, so knowledge of how they work is important. This section is not meant to be an introduction to DDS; it is assumed you already have a working knowledge of DDS. This chapter covers some of the keywords used in this book, so if you have never used these keywords, or have just a vague idea of what they do, you will have a better understanding of how to use them and why you would want to. If you find that you already know the material presented here, you may want to skip ahead to the next section.

The Clear Line Keyword

The first keyword is Clear Line (CLRL). There are several options for this keyword, but only the CLRL(*NO) option will be addressed here. If you do not specify the CLRL keyword, the default is CLRL(*YES), which means the display is totally cleared before the format is written. Specifying CLRL(*NO) tells the system not to clear the display, but to write this display format over the current information displayed.

This keyword is used in the display file SFD100. The program SFR100 uses this display file. The code for these two source members is shown below:

```
A*============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD100) SRCFILE(XXX/QDDSSRC)
A*
A*============================================================
A                                     DSPSIZ(24 80 *DS3)
A                                     CF03(03)
A          R SFD100A
A                                     CLRL(*NO)
A                                 1  3DATE
A                                     EDTCDE(Y)
A                                 1 23'Test Screen For CLEAR Keyword'
A                                     DSPATR(HI)
A                                 2  3TIME
A                                 1 71USER
A                                 2 71'SFD100A'
A                                23  2'F3-Exit'
A                                     DSPATR(HI)
A          R SFD100B
A                                15 20'This is the format for SFD100B'
A                                     DSPATR(RI)
A                                     COLOR(RED)


F*============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR100) SRCFILE(XXX/QRPGSRC)
F*
F*============================================================
FSFD100  CF  E                    WORKSTN
C                     WRITESFD100B
C                     EXFMTSFD100A
C                     EXSR @CMD
C*
C*********************************************************
C*    @ C M D   S U B R O U T I N E                      *
C*                                                       *
C*    Executed after every EXFMT or display read op code.  *
C*    Used to process command keys.                      *
C*********************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, seton LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C                     ENDSR
```

To see a demonstration of this keyword, call program SFR100. You should get a display screen that looks like this:

```
xx/xx/xx            Test Screen For CLEAR Keyword                QPGMR
xx:xx:xx                                                         SFD100A



                 This is the format for SFD100B



F3-Exit
```

Although it doesn't look like much, it is a simple demonstration of how to write data to the screen and not erase the existing data. This technique is going to become important as you program subfiles.

The program writes format SFD100B and then performs an EXFMT operation to display format SFD100A. The EXFMT operation code is the equivalent of performing a write and a read. When the program writes the format SFD100B, it first clears the whole display. Since the CLRL keyword is not specified on SFD100B, it defaults to CLRL(*YES). So when SFD100B is written, the system first clears the display.

But SFD100A has the keyword CLRL(*NO) specified. Therefore, when the program writes SFD100A to the display, it does not clear the display first. It simply lays the information onto the display without erasing the information placed there by SFD100B.

Reading Multiple Formats

One of the features available—that has seen only limited overall use—is the ability to read multiple display formats at the same time. You can write three formats to the display screen, and read from all three at the same time.

The display file SFD105 and program SFR105 demonstrate this technique as shown below:

```
A*============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD105) SRCFILE(XXX/QDDSSRC)
A*
A*============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03)
A          R SFD105A
A                                       FRCDTA
A                                       CLRL(*NO)
A                                       PUTOVR
A                                       OVRDTA
A                                   1  3DATE
A                                       EDTCDE(Y)
A                                   1 23'Test Screen For Multiple Display R-
A                                       eads'
A                                       DSPATR(HI)
A                                   2  3TIME
A                                   1 71USER
A                                   2 71'SFD105A'
A                                   6  1'SFD105A Format...:'
A                                       COLOR(BLU)
A            @INPA          1A  I  6 21COLOR(WHT)
A          R SFD105B
A                                       FRCDTA
A                                       CLRL(*NO)
A                                       PUTOVR
A                                       OVRDTA
A                                   8  1'SFD105B Format...:'
A                                       COLOR(BLU)
A            @INPB          1A  I  8 21COLOR(WHT)
A          R SFD105C
A                                       FRCDTA
A                                       CLRL(*NO)
A                                       PUTOVR
A                                       OVRDTA
A                                  23  2'F3-Exit'
A                                       DSPATR(HI)
A                                  10  1'SFD105C Format...:'
A                                       COLOR(BLU)
A            @INPC          1A  I 10 21COLOR(WHT)
A          R SFD105D
A  41                              19  2'Field @INPA From Format SFD105A Wa-
A                                       s Read'
A                                       COLOR(RED)
A  42                              20  2'Field @INPB From Format SFD105B Wa-
A                                       s Read '
A                                       COLOR(RED)
A  43                              21  2'Field @INPC From Format SFD105C Wa-
A                                       s Read '
A                                       COLOR(RED)
```

```
F*==============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR105) SRCFILE(XXX/QRPGSRC)
F*
F*==============================================================
FSFD105  CF  E                    WORKSTN
C*
C* Set the input fields from the three formats to blanks
C*
C                     MOVE *BLANKS   @INPA
C                     MOVE *BLANKS   @INPB
C                     MOVE *BLANKS   @INPC
C*
C* Write the SFD105D format, which displays the messages, first.
C* Then write the SFD105A and SFD105B formats, which use the
C* Clear *no option. This will keep them from clearing the
C* message format screen.
C*
C                     WRITESFD105D
C                     WRITESFD105A
C                     WRITESFD105B
C*
C* Write the SFD105C format (it also has CLear *no option) and
C* read it. (EXFMT) Then read the SFD105B and SFD105A formats.
C* Since the reads are all contiguous, they will be processed
C* together.
C*
C                     EXFMTSFD105C
C                     READ SFD105B                 31
C                     READ SFD105A                 31
C                     EXSR @CMD
C*
C* Ensure that the message indicators are all off
C*
C                     MOVE *OFF      *IN41
C                     MOVE *OFF      *IN42
C                     MOVE *OFF      *IN43
C*
C* Set on the appropraite message indicator, based on
C* the fields that were keyed into.
C*
C           @INPA     IFNE *BLANKS
C                     MOVE *ON       *IN41
C                     END
C*
C           @INPB     IFNE *BLANKS
C                     MOVE *ON       *IN42
C                     END
C*
C           @INPC     IFNE *BLANKS
C                     MOVE *ON       *IN43
C                     END
C*
C*************************************************************
C*    @ C M D   S U B R O U T I N E                          *
C*                                                           *
```

```
C*    Executed after every EXFMT or display read op code.   *
C*    Used to process command keys.                         *
C*********************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, seton LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C                     ENDSR
```

To see a demonstration of this, call program SFR105. You should see a display like this:

```
  xx/xx/xx           Test Screen For Multiple Display Reads         QPGMR
  xx:xx:xx                                                          SFD105A



SFD105A Format...:  _

SFD105B Format...:  _

SFD105C Format...:  _









 F3-Exit
```

Place any character in the input field next to the SFD105A Format line. Press Enter. You will get a screen that looks like this:

```
  xx/xx/xx            Test Screen For Multiple Display Reads          QPGMR
  xx:xx:xx                                                            SFD105A


SFD105A Format...:  _

SFD105B Format...:  _

SFD105C Format...:  _









 Field @INPA From Format SFD105A Was Read


 F3-Exit
```

Now key a character into all the input fields. Press Enter. You should see a display that looks like this:

```
  xx/xx/xx            Test Screen For Multiple Display Reads          QPGMR
  xx:xx:xx                                                            SFD105A



SFD105A Format...:  _

SFD105B Format...:  _

SFD105C Format...:  _









 Field @INPA From Format SFD105A Was Read
 Field @INPB From Format SFD105B Was Read
 Field @INPC From Format SFD105C Was Read

 F3-Exit
```

You can try other combinations if you like. But what is so special about this procedure? If you look at the display file specifications, you can see that it contains four display formats .

Format SFD105D is only used to display the messages on the bottom of the screen, letting the user know which field was read. This format does not have a CLRL keyword specified, so it clears the screen before it is written to the display.

Format SFD105A contains all of the header information on the screen, as well as the input field @INPA. It also has the keyword CLRL(*NO) specified. Format SFD105B contains the input field @INPB, and has the keyword CLRL(*NO) specified. Format SFD105C contains the input field @INPC, has the command key line at the bottom of the display, and has CLRL(*NO) specified.

As you can see, this example makes use of CLRL(*NO) to lay multiple formats on top of each other, and to keep the existing information from being cleared. Now look at the RPG program which allows you to read several display formats at the same time. In program SFR105, there is a section of code which looks like this:

```
C                       WRITESFD105D
C                       WRITESFD105A
C                       WRITESFD105B
C                       EXFMTSFD105C
```

```
C                       READ SFD105B                  31
C                       READ SFD105A                  31
```

The first thing the program does is to write the messages out (format SFD105D), which clears the display. Then it writes out the formats SFD105A, SFD105B, and SFD105C, which do not clear the display. The EXFMT operation code performs a write and then a read. You could have also coded this section as:

```
C                       WRITESFD105D
C                       WRITESFD105A
C                       WRITESFD105B
C                       WRITESFD105C
C                       READ SFD105C                  31
C                       READ SFD105B                  31
C                       READ SFD105A                  31
```

When the user presses a function key, the program reads formats SFD105A, SFD105B, and SFD105C. But why would you want to do such a thing? There are going to be instances where you will display multiple formats of information at a time to the users. An example would be an inventory inquiry program. On the display you may show a list of all open purchase orders for an item. On the same display you may also show the open customer orders for an item. The user may want the ability to scroll up and down through the open purchase orders without affecting the data displayed in the open orders. You can write these two formats out to the same display, but maintain the functionality of each format separately.

The Assume Keyword

The ASSUME keyword acts somewhat similar to the CLRL(*NO) keyword. The difference is that the CLRL(*NO) cause a display format to not erase the format from the screen from the *same* display file. The assume keyword keeps the display from a *previous* program from being erased.

You can see an example of using the ASSUME keyword in the display file SFD110 and program SFR110 shown below:

```
A*============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD110) SRCFILE(XXX/QDDSSRC)
A*
A*============================================================
A                                       DSPSIZ(24 80 *DS3)
A          R SFD110A
A                                       CF04(04 'Call next program')
A                                       CF03(03 'Exit')
A                                       ASSUME
A                                  1  3DATE
```

```
A                                       EDTCDE(Y)
A                                   1 23'Test Screen For Assume Keyword '
A                                       DSPATR(HI)
A                                   2  3TIME
A                                   1 71USER
A                                   2 71'SFD110A'
A                                  23  2'F3-Exit   F4-Call Next Program'
A                                       DSPATR(HI)


F*============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR110) SRCFILE(XXX/QRPGSRC)
F*
F*============================================================
FSFD110  CF  E                     WORKSTN
C*
C                     EXFMTSFD110A
C                     EXSR @CMD
C*
C*********************************************************
C*    @ C M D   S U B R O U T I N E                      *
C*                                                       *
C*    Executed after every EXFMT or display read op code.  *
C*    Used to process command keys.                      *
C*********************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, seton LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C* If F4 was pressed, call program SFR111.
C*
C           *IN04     IFEQ *ON
C                     CALL 'SFR111'
C                     END
C*
C                     ENDSR
```

Two other source members are also required for this demonstration. These are the display file SFD111 and program SFR111 shown below:

```
A*============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD111) SRCFILE(XXX/QDDSSRC)
A*
A*============================================================
A                                       DSPSIZ(24 80 *DS3)
A          R SFD111A
```

```
A                                        ASSUME
A                                        OVERLAY
A                                   10 17'This Format Comes From The SFR111 -
A                                        Program'
A                                        COLOR(RED)
A                                        DSPATR(BL)


F*=============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR111) SRCFILE(XXX/QRPGSRC)
F*
F*=============================================================
FSFD111  CF  E                      WORKSTN
C                      EXFMTSFD111A
C                      MOVE *ON       *INLR
```

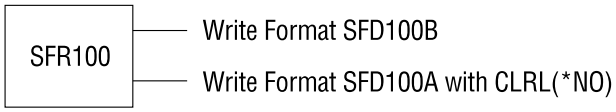
Now you can see a demonstration of this keyword by calling program SFR110. You should get a display that looks like this:

```
xx/xx/xx          Test Screen For Assume Keyword                QPGMR
xx:xx:xx                                                        SFD110A



F3-Exit   F4-Call Next Program
```

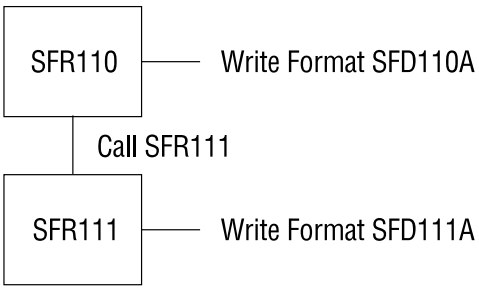
Now press F4. Your display should look like this:

```
xx/xx/xx          Test Screen For Assume Keyword                QPGMR
xx:xx:xx                                                        SFD110A



              This Format Comes From The SFR111 Program



F3-Exit   F4-Call Next Program
```

The program laid the information on top of the previous display just like it did with the CLRL(*NO) keyword. So what is different? In the previous example with CLRL(*NO), the formats were written to the display from within the same program.



But the current program is writing the formats from two different programs:



You can see the code required to keep the display from the previous program from being erased by looking at programs SFR110 and SFR111, which show the difference. But again, why would you want to do this? You would do this anytime you call a display program from another display program and want the original display to left on the screen. Then you can still see the contents of the original screen to be viewed while the called display is up.

For instance, this technique is how you can cause windows to "pop-up" onto an existing display, without erasing the current data on the display.

There are two important notes about the ASSUME keyword. First, you must specify at least one displayable field with this keyword. Second, if any format in a display file has the ASSUME keyword, it applies to the whole display file. You do not even have to reference the format with the ASSUME keyword in the program for the ASSUME feature to work. You will see a demonstration of this feature and why it is necessary in the next example program.

The Window Keyword

Now take a look at the WINDOW keyword. The window keyword is very useful. Not only will it draw a window border for you, but it will also clear the portion of the screen inside the window. This method is much easier than having to simulate a window prior to version 2 of the operating system.

You can see an example of using the WINDOW keyword by looking at the following source members:

```
A*==============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD120) SRCFILE(XXX/QDDSSRC)
A*
A*==============================================================
A                                       DSPSIZ(24 80 *DS3)
A          R SFD120A
A                                       CF04(04 'Call next program')
A                                       CF03(03 'Exit')
A                                       ASSUME
A                                   1  3DATE
A                                       EDTCDE(Y)
A                                   1 23'Test Screen For Assume Keyword '
A                                       DSPATR(HI)
A                                   2  3TIME
A                                   1 71USER
A                                   2 71'SFD120A'
A                                  23  2'F3-Exit   F4-Call Next Program'
A                                       DSPATR(HI)
A                                   5  1'This Line Came From Format SFD120A-
```

```
A                                       In Program SFR120'
A                                  6  1'This Line Came From Format SFD120A-
A                                       In Program SFR120'
A                                  7  1'This Line Came From Format SFD120A-
A                                       In Program SFR120'
A                                  8  1'This Line Came From Format SFD120A-
A                                       In Program SFR120'
A                                  9  1'This Line Came From Format SFD120A-
A                                       In Program SFR120'
A                                 10  1'This Line Came From Format SFD120A-
A                                       In Program SFR120'
A                                 11  1'This Line Came From Format SFD120A-
A                                       In Program SFR120'
A                                 12  1'This Line Came From Format SFD120A-
A                                       In Program SFR120'
A                                 13  1'This Line Came From Format SFD120A-
A                                       In Program SFR120'
A                                 14  1'This Line Came From Format SFD120A-
A                                       In Program SFR120'
A                                 15  1'This Line Came From Format SFD120A-
A                                       In Program SFR120'
A                                 16  1'This Line Came From Format SFD120A-
A                                       In Program SFR120'
A                                 17  1'This Line Came From Format SFD120A-
A                                       In Program SFR120'
A                                 18  1'This Line Came From Format SFD120A-
A                                       In Program SFR120'
A                                 19  1'This Line Came From Format SFD120A-
A                                       In Program SFR120'
A                                 20  1'This Line Came From Format SFD120A-
A                                       In Program SFR120'
A                                 21  1'This Line Came From Format SFD120A-
A                                       In Program SFR120'
A                                 22  1'This Line Came From Format SFD120A-
A                                       In Program SFR120'


F*==============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR120) SRCFILE(XXX/QRPGSRC)
F*
F*==============================================================
FSFD120  CF  E                    WORKSTN
C*
C                     EXFMTSFD120A
C                     EXSR @CMD
C*
C*********************************************************
C*    @ C M D   S U B R O U T I N E                      *
C*                                                       *
C*    Executed after every EXFMT or display read op code.  *
C*    Used to process command keys.                      *
C*********************************************************
C*
C           @CMD      BEGSR
```

```
C*
C* If F3 was pressed, seton LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C* If F4 was pressed, call program SFR121.
C*
C           *IN04     IFEQ *ON
C                     CALL 'SFR121'
C                     END
C*
C                     ENDSR


A*============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD121) SRCFILE(XXX/QDDSSRC)
A*
A*============================================================
A                                       DSPSIZ(24 80 *DS3)
A          R SFD121A
A                                       WINDOW(10 10 10 30)
A                                       WDWBORDER((*COLOR BLU) (*DSPATR RI)-
A                                        (*CHAR '        '))
A                                   4  2'This Window Was Displayed '
A                                       COLOR(RED)
A                                       DSPATR(BL)
A                                   6  2'From Format SFD121A In    '
A                                       COLOR(RED)
A                                       DSPATR(BL)
A                                   8  2'Program SFR121            '
A                                       COLOR(RED)
A                                       DSPATR(BL)
A          R ASSUME
A                                       ASSUME
A                                  24  2' '


F*============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR121) SRCFILE(XXX/QRPGSRC)
F*
F*============================================================
FSFD121  CF  E                     WORKSTN
C*
C* Write the assume record first, so that the display is not
C* erased, then display the window.
C*
C                     EXFMTSFD121A
C                     MOVE *ON       *INLR
```

To demonstrate a window, call program SFR120. You should get a display that looks like this:

```
  xx/xx/xx            Test Screen For Assume Keyword                  QPGMR
  xx:xx:xx                                                            SFD120A


This Line Came From Format SFD120A In Program SFR120
This Line Came From Format SFD120A In Program SFR120
This Line Came From Format SFD120A In Program SFR120
This Line Came From Format SFD120A In Program SFR120
This Line Came From Format SFD120A In Program SFR120
This Line Came From Format SFD120A In Program SFR120
This Line Came From Format SFD120A In Program SFR120
This Line Came From Format SFD120A In Program SFR120
This Line Came From Format SFD120A In Program SFR120
This Line Came From Format SFD120A In Program SFR120
This Line Came From Format SFD120A In Program SFR120
This Line Came From Format SFD120A In Program SFR120
This Line Came From Format SFD120A In Program SFR120
This Line Came From Format SFD120A In Program SFR120
This Line Came From Format SFD120A In Program SFR120
This Line Came From Format SFD120A In Program SFR120
This Line Came From Format SFD120A In Program SFR120
This Line Came From Format SFD120A In Program SFR120
 F3-Exit   F4-Call Next Program
```

Now press the F4 key. Your display should look like this:

```
  xx/xx/xx            Test Screen For Assume Keyword                  QPGMR
  xx:xx:xx                                                            SFD120A


This Line Came From Format SFD120A In Program SFR120
This Line Came From Format SFD120A In Program SFR120
This Line Came From Format SFD120A In Program SFR120
This Line Came From Format SFD120A In Program SFR120
This Line Came From Format SFD120A In Program SFR120
This Lin ................................ m SFR120
This Lin :                               : m SFR120
This Lin :                               : m SFR120
This Lin :                               : m SFR120
This Lin :  This Window Was Displayed    : m SFR120
This Lin :                               : m SFR120
This Lin :  From Format SFD121A In       : m SFR120
This Lin :                               : m SFR120
This Lin :  Program SFR121               : m SFR120
This Lin :                               : m SFR120
This Lin :                               : m SFR120
This Lin :................................: m SFR120
This Line Came From Format SFD120A In Program SFR120
 F3-Exit   F4-Call Next Program
```

See how the window popped up without clearing the display? It did so because the display file used the ASSUME keyword. But you cannot use the ASSUME keyword on the same format which contains the WINDOW keyword (SFD121A). If your program tried to do perform this action, it would be issued an error message. To compensate, you have to create a dummy display format that is not referred to in your program. In the example, the dummy format is called ASSUME. Now, since one of the formats in the display file has the ASSUME keyword, all formats operate under the ASSUME mode. Remember, this format is not referred to in the RPG program. Just the fact that one of the display file formats has the keyword ASSUME causes the entire file to inherit the ASSUME capabilities.

If you have ever tried to use windows before, you may have noticed all of the associated parameters and keywords associated with the WINDOW keyword.

Window Functionality

Now that you have seen some basic windowing techniques, you should now become familiar with different windowing options. This section covers windows in great detail.

Windows are very important to the implementation of subfiles. Both subfiles and windows are very easy to program. When a subfile is used in conjunction with a window, the subfile itself becomes much easier to program. While subfiles have been around since the S/38, windows are relatively new. The next section will bring you up to date on windows.

Windows allow complex functionality to be easily implemented. For example, you can use subfiles to display data in windows very easily. You can also have the AS/400 automatically save and restore the underlying display by using windows. In other words, the screen on the display when you call a window program is saved. When you remove the window from the display, the system restores the previous screen for you automatically.

You can position data in the window by referring to positions in the window itself instead of positions on the full screen. In other words, you can specify that a field should be displayed in position 1,1 of the window. If the window itself starts in position 10,10, the field will also start at position 10,10. So the fields inside the window will follow the window around on the display, wherever you decide to display it.

This feature is useful, because you can easily change the position of a window before you display it. A user can "point and click" where he would like a pop-up window to be placed. This feature is extremely useful if you have a single window subfile inquiry program that is used in multiple calling programs. Now you, the programmer, do not need to be concerned where on the display you cause the window to pop up. Just allow the user to point and click where the window should pop up. The user can keep a useful portion of the original screen in view, when he brings up the window, no matter where on the screen this portion may exist.

Some functions such as messages and help are displayed inside the window automatically. If you have multiple windows on the display, the meaning of messages is clearer, since the messages are displayed in the corresponding window. If you have three windows on your display, and you attempt to press an invalid function key, the invalid function key message is displayed within the window where the user was last.

☛ **Note:** Under V2R3 of OS/400, you can control whether or not the window should have a message line attached to it by using the MSGLIN keyword.

Window Terminology

The following is a list of common window terminologies:

- Window: an area of information that overlays part of the display. You can view information inside the window and the portion of the display that is not overlaid by the window. The window is the only part of the display that is active. You cannot work with the original display until you remove the window from the display. If more than one window is displayed, only one window can be active at a time. You can display a maximum of 12 windows at a time.
- Window Definition Record: a record format that contains the WINDOW keyword and defines the window border positions. If the format does not contain the WINDOW keyword and border positions, it is not a window definition record.
- Window Reference Record: a record format that contains the WINDOW keyword, but does not contain the window size and position attributes. Instead, it references another record format which is a window definition format.
- Window Border: an area that surrounds the window, marking off the window boundaries on the display. When a window is first displayed, all data contents within the window borders are cleared. The window borders can either be visible, or invisible.
- Active Window: the window area on the display in which the most recent input or output operation was performed. If you use displayable window borders, the active window appears to be the topmost window on the display, in which its borders overlay those of other windows. The active window is the only part of the display with which the user can interact.

Window Fact

A window remains on the display until your application, or the system, takes an action to remove it. Removing a window and overlaying a window are different operations. When a window is removed, it no longer exists on the display, and you can no longer perform an input or output operation to it. When a window is overlaid by another window, it might not be visible to the workstation user, and is not currently made available for the user to perform input or output to it, but it is still available on the screen. When the currently active window is removed, the overlaid window is again made the active window. To sum up, an overlaid window is accessible again if it is made active again, whereas a removed window cannot be accessed again until it is redisplayed.

There are four DDS keywords that allow your applications to create windows:

The Window Keyword

The WINDOW keyword is used whenever you want to create a window on the display. You can use this keyword to change the contents of an existing window. You can also use this keyword to cause a window that is displayed on the screen (but is currently inactive because it is overlaid) to become the active window.

The Window Border Keyword

The WDWBORDER keyword specifies the color, display attributes, and characters of a window border. You can use this keyword to make the window border invisible. You can also use it to make the border visible by using a variety of characters and attributes.

The Remove Window Keyword

The RMVWDW keyword removes other windows from the display when a new window is displayed or when an existing window is redisplayed as the active window. The windows that are removed are no longer active, and cannot be made active unless redisplayed.

The User Restore Display Keyword

The USRRSTDSP keyword prevents OS/400 from automatically saving and restoring the underlying display when windows are displayed and removed. Usually, you want the system to do this automatically, since it requires no additional code. There are times, however, that the saving and restoring of displays may not be needed, and can adversely affect performance, especially when running over a communications line. You can also use the USRRSTDSP keyword to cause a window that has been overlaid to pop up and overlay later windows, or to make two windows seem active at the same time.

Performance Issues

The time needed for the system to perform save and restore operations depends on your communications setup and on the window being displayed.

If the window operations are being performed on a workstation that is locally attached via twinax or twisted pair, through a local area network, or other high-speed communications line, the response time is very fast. However, if you are attached through a communications line running at 2400 baud, your response time may be unacceptable. The screen must be redisplayed for each display and window that is written to the display. The display could be rewritten multiple times just by pressing the Enter key. According to IBM, if you are connected via a 2400-baud communications line, it may take up to ten seconds for the system to automatically perform a save and restore, then display a window. A 9600-baud modem is four times as fast, therefore, the response would be four times as quick, or about 2.5 seconds. Keep this in mind when designing your displays. They may run just fine on your locally attached terminal, but if a user is using the same application on a 2400-baud communication line, they may be dozing off in between screen refreshes.

Using USRRSTDSP to Override the System

You can use the USRRSTDSP keyword to override the automatic system display file save and restore processing. Your programs can control the save and restore processing only when needed. This technique can improve system performance and response time for the user of the application, especially those connected through a communication line. Consider using it when you display only one window at a time and the windows are in a different display file, or when you display a series of windows in which the user will not return to earlier windows. In these cases, there are no reasons to allow the system to automatically save and restore after every window, because it is duplicating the work multiple times, or performing a save and restore process that will never be needed.

The following conditions normally perform two save operations:

1. Your application displays only one window at a time.
2. The display file is created with RSTDSP(*YES).
3. The first window record to overlay the display is located in a separate display file.

The first save operation is performed when the display file is suspended. The second save operation is performed because a window is being displayed. The USRRSTDSP keyword eliminates the second save operation, since it is not necessary to perform it, because it results in duplicate work.

To override the system save and restore processing, perform the following steps.

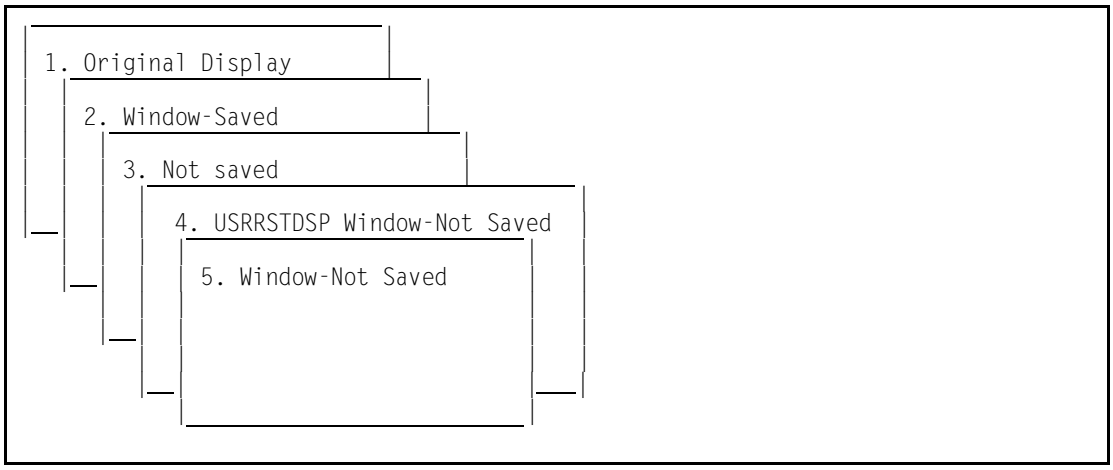
1. Create your own routine inside your program to rebuild the display after a window is removed. You can do this by performing a write to the original display.
2. Specify the record-level USRRSTDSP keyword on the window following the first window you do not want the system to save. The USRRSTDSP keyword keeps the system from performing save and restore operations.

The USRRSTDSP keyword is allowed only on records containing the WINDOW keyword and is ignored on the window reference record.

Once the USRRSTDSP keyword is specified, it remains in effect until one of the following occurs:

1. You read or write to the initial display that does not contain a window. An example is shown below.

Screen one is your original display. Screen two is a window without the USRRSTDSP keyword, and it is saved because screen three is also a window without the USRRSTDSP keyword. Screen three, however, is not saved because screen four is a window with the USRRSTDSP keyword activated. At this point, no other windows are saved, even if you set off the option indicator for USRRSTDSP. To allow the displays to start being saved, you would have to set off the option indicator for the USRRSTDSP keyword on screen four, then write to the original display. After this point, all windows are saved.



2. You write to the last window that was saved.

In the example above, you could also turn off the option indicator for the USRRSTDSP keyword in screen four, and allow the system to start saving all displays again. Then you could write screen two, because it was the last window saved. Now the system begins saving and restoring the screen automatically.

```
┌──────────────────────────┐
│ 1. Original Display      │
│  ┌──────────────────────┐│
│  │ 2. Window-Saved      │
│  │  ┌──────────────────┐│
│  │  │ 3. Not saved     │
│  │  │  ┌──────────────────────────┐
│  │  │  │ 4. USRRSTDSP Window Not Saved │
│  │  │  │  ┌──────────────────────┐│ │
│  │  │  │  │ 5. Window Not Saved  │ │
│  │  │  │  │                      │ │
│  │  │  │  │                      │ │
│  │  │  │  │                      
```

6. If a non-window record is written to the display and USRRSTDSP is specified on the first window, then the window is not removed, and the non-window record may overlay all or part of the window.

Displaying Windows

To create a window on the display, write a record format that specifies a WINDOW keyword. The first window record you write must be a window definition record, which specifies the window size and its location on the display. The window definition record places the window borders on the display. Then you can write the same window definition record again or use one or more window reference records to complete the specifications for the window. If you are not using referenced windows, then simply writing the window definition format displays your window. If you are using window references, then you must first write the window definition format, and then write the window reference formats to the display.

Window Definition Records

A window definition record is a display file record containing a WINDOW keyword that defines the size and location of the window.

Size and location attributes are defined by specifying the starting line on the display, the starting position on the display, the number of lines the window will need, and the number of positions the window will need. The location attributes can be hard coded, such as specifying that a window should be placed on the display starting on line 4, position 10. The positions can also be variable, such as specifying that the starting line is contained in variable &@LIN, and the starting position is contained in variable &@POS. Care must be used when using these variables. You must ensure that the starting line or starting position is not 0 or that the starting line and position are not both 1. You must also check to ensure that the window can totally fit on the display. For instance, if you have a 10-line window with 40 positions, and a 24-line by 80-position display, you must ensure that the starting line is not greater than 14, and that the position is not greater than 40, otherwise the window could not totally fit on the display.

The window definition record must be the first record written for each window you display. It is the record that actually creates the window and makes it visible on the display. The record can contain any type of fields or data found in any typical display record. It can also contain a WDWBORDER keyword that defines the window border.

The WDWBORDER keyword can be specified at the file level, which allows all windows, by default, to get the border attributes from the file level keyword. If you specify that the window border at the file level is to display a reverse image with a blue border, then all window formats in the display file inherit these characteristics. You can also specify the window border at the format level, and override certain attributes of the file-level specifications. For example, you can have a window border keyword on one window format, specifying that the border is white. The window inherits the reverse image blue border, and then overrides to a white border, which is displayed as a reverse image white border for that one window.

All of the information needed for a window can be stored in the window definition format. Alternatively, you can combine window definition formats and referenced windows to build up the window on the screen.

Window Reference Records

Window reference records give you the ability to display additional data in a window by letting you display more than one record format in a window.

Each window reference record contains a WINDOW keyword specifying the name of the window definition record to which it applies. When the window reference record is written, the window definition record being referred to must be on the display. If the referenced record is not on the display, then an error message stating that the window does not exist is returned to the program.

Window reference records do not contain size and position attributes, and any active WDWBORDER keywords are ignored.

If you specify the WDWBORDER keyword on a window reference format, a warning message is sent when the display file is compiled.

You can use as many window reference records as needed to complete the window. However, you do not need to write any window reference records to display a window. You can display a window using only a window definition record. In subfile programming, you almost always use one window definition format and one window reference format, so you have the ability to display a command key line on the bottom part of the window.

Window Size and Location

The size and location is defined on the window definition format, which contains the WINDOW keyword.

```
A                                       WINDOW(5 20 10 30)
```

This example tells the system to build a window starting on line 5, position 20, that has 10 lines and 30 positions. In reality, the window will actually be larger than 10 lines by 30 positions. It will actually be 12 lines long, because an extra line is needed for the top window border, and another extra line is needed for the bottom window border. Extra positions are also needed. One extra position is needed to the left of the left border for the border attribute byte, one position is needed for the left border itself, and one position is needed for the attribute of the data inside the window. This pattern is repeated for the right hand side of the border and adds up to six additional positions that are needed for the window.

In summary, the window usually takes up 2 more lines and six more positions than the number of lines specified on the WINDOW statement.

The last data line that you specify in your program is reserved for messages. This line acts like line 24 on a typical 24 x 80 or 24 x 132 display. Any messages that the program incurs are displayed on this line, so you need to make sure you have this line available for the system. The previous example specified that the window was to have 10 lines. The tenth line of the window is used by the system to display messages.

Variable Starting Line and Position

The start line and position of the window can be program defined or variable. This example is as follows.

```
A                                       WINDOW(&@SL &@SP 10 30)
```

The starting line and position fields are replaced with two program to system fields, that are preceded with an ampersand (&). These fields must be described on the same record format as a three position, system to program field, signed, with zero decimal places.

```
A                                       WINDOW(&@SL &@SP 10 30)
A             @SL             3S 0P
A             @SP             3S 0P
```

Notice that when you describe program to system fields, you leave off the preceding ampersand (&). In this example, the window will attempt to be located starting on

the line number specified by the value of @SL, and the starting position located in value @SP.

☛ **Note:** OS/400 V2R3 allows you to define a default (*DFT) starting position for the window. Using this method, the system determines where to position the window. However, the system usually tries to position the window near the center of the display, whereas when you define your own window start positions, the window is positioned as close to the cursor as possible.

Cursor Position

If the cursor is outside of the usable area of the active window, only the Print and Home command function (CF) keys are active. If the user presses any other CF key, the alarm sounds and the cursor is moved back to its position for the previous write operation. If you are running OS/400 V2R3 or later, the operating system does not allow the cursor to be positioned outside of the window.

You can see this within program SFR121, which is called by SFR120. If you move the cursor outside the window and press Enter, the alarm beeps and the cursor is placed back on the first displayable position inside the window. If the window had contained input-capable fields, the cursor would have been placed on either the first input-capable field, or the field which specified the DSPATR(PC) (display attribute position cursor) keyword.

Error Messages

When windows exist on the display, any error messages are displayed on the last usable line of the active window. The last usable line is reserved for error messages (i.e., no fields can be displayed there). If the error message is longer than the line, it is padded to the right of the message with "...", and truncated to fit the window. You can see this effect if you call the program SFR002 and press the roll-down key. The message would ordinarily read:

```
Roll up or down past the first or last subfile record
```

But since this is longer than the window, it reads:

```
Roll up or down past the first or last...
```

If a message is displayed reporting operational and keyboard errors, such as a function key not allowed, the keyboard is locked and the user must press the Error Reset key to continue. You can see this effect with program SFR120. Press F4 to call SFR121.

Press F3. You will get a message that the function key is not allowed. You must now press error reset before continuing.

Messages generated from the ERRMSG, ERRMSGID, SFLMSG, SFLMSGID, and DDS validity-checking keywords are displayed in the window, but do not lock the keyboard. Messages generated from these keywords do lock the keyboard when displayed in full screen mode but do not inside a window.

This example brings up a design requirement: you always need to allow one more line to your window for the system to display messages.

Subfiles and Windows

A maximum of 24 subfiles can be active at any one time. A maximum of 12 subfiles can be displayed on the base display or in a single window at any one time.

If a subfile is displayed in a window and the window is removed from the display, the subfile is not deleted. The subfile remains active until the display file is closed or you explicitly delete the subfile. The window that the subfile is in can be made active again by redisplaying it. The original subfile contents remain intact.

Defining Window Borders

You can use the system defaults for your window borders or define them using the WDWBORDER keyword. The keyword is optional. If you do not specify it, it will assume the system default.

This keyword allows us to customize three border components—border color , border attributes, and border characters.

The default window border is green, has no attributes, and is made up of periods and colons. More than one WDWBORDER keyword can be specified on a format, and can be conditioned with indicators. You can use the WDWBORDER keyword at the file level, where it sets up the defaults for the entire display file.

You can specify the WDWBORDER keyword at both the file and record format levels. You can set up defaults for the entire file, and override specific components at the record format level for each window. For example, you could specify at the file level:

```
A                          WDWBORDER((*COLOR BLU) (*DSPATR RI))
```

On one window format record you could specify another WDWBORDR keyword:

```
A                          WDWBORDER((*COLOR RED))
```

In this case, this window would be in red and in reverse image, because the default at the file level was set to reverse image.

The window format must be a window definition record to contain the WDWBORDER keyword. If you use it on window reference records, a warning message is issued when the file is compiled.

Now with this overall knowledge of windows at your disposal, let's see some more these functions in actual programs.

A very useful feature of the window keyword is the ability to specify the line and position on the screen to display the window by using variables. The following source members demonstrate this technique:

```
A*==============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD130) SRCFILE(XXX/QDDSSRC)
A*
A*==============================================================
A                                       DSPSIZ(24 80 *DS3)
A          R SFD130A
A                                       CF04(04 'Call next program')
A                                       CF03(03 'Exit')
A                                       ASSUME
A                                   1  3DATE
A                                       EDTCDE(Y)
A                                   1 23'Test Screen For Assume Keyword '
A                                       DSPATR(HI)
A                                   2  3TIME
A                                   1 71USER
A                                   2 71'SFD130A'
A                                  23  2'F3-Exit   F4-Call Next Program'
A                                       DSPATR(HI)
A                                   5  1'This Line Came From Format SFD130A-
A                                         In Program SFR130'
A                                   6  1'This Line Came From Format SFD130A-
A                                         In Program SFR130'
A                                   7  1'This Line Came From Format SFD130A-
A                                         In Program SFR130'
A                                   8  1'This Line Came From Format SFD130A-
A                                         In Program SFR130'
A                                   9  1'This Line Came From Format SFD130A-
A                                         In Program SFR130'
A                                  10  1'This Line Came From Format SFD130A-
A                                         In Program SFR130'
A                                  11  1'This Line Came From Format SFD130A-
A                                         In Program SFR130'
A                                  12  1'This Line Came From Format SFD130A-
A                                         In Program SFR130'
```

```
A                                   13  1'This Line Came From Format SFD130A-
A                                         In Program SFR130'
A                                   14  1'This Line Came From Format SFD130A-
A                                         In Program SFR130'
A                                   15  1'This Line Came From Format SFD130A-
A                                         In Program SFR130'
A                                   16  1'This Line Came From Format SFD130A-
A                                         In Program SFR130'
A                                   17  1'This Line Came From Format SFD130A-
A                                         In Program SFR130'
A                                   18  1'This Line Came From Format SFD130A-
A                                         In Program SFR130'
A                                   19  1'This Line Came From Format SFD130A-
A                                         In Program SFR130'
A                                   20  1'This Line Came From Format SFD130A-
A                                         In Program SFR130'
A                                   21  1'This Line Came From Format SFD130A-
A                                         In Program SFR130'
A                                   22  1'This Line Came From Format SFD130A-
A                                         In Program SFR130'


F*==============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR130) SRCFILE(XXX/QRPGSRC)
F*
F*==============================================================
FSFD130  CF  E                      WORKSTN
C*
C                      EXFMTSFD130A
C                      EXSR @CMD
C*
C*************************************************************
C*    @ C M D   S U B R O U T I N E                          *
C*                                                           *
C*    Executed after every EXFMT or display read op code.    *
C*    Used to process command keys.                          *
C*************************************************************
C*
C           @CMD       BEGSR
C*
C* If F3 was pressed, seton LR and exit.
C*
C           *IN03      IFEQ *ON
C                      MOVE *ON       *INLR
C                      RETRN
C                      END
C*
C* If F4 was pressed, call program SFR131.
C*
C           *IN04      IFEQ *ON
C                      CALL 'SFR131'
C                      END
C*
C                      ENDSR
```

```
A*============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD131) SRCFILE(XXX/QDDSSRC)
A*
A*============================================================
A                                     DSPSIZ(24 80 *DS3)
A                                     CF03(03)
A          R SFD131A
A                                     WINDOW(&@SL &@SP 10 30)
A                                     WDWBORDER((*COLOR BLU) (*DSPATR RI)-
A                                      (*CHAR '        '))
A                                 4  2'This Window Was Displayed '
A                                     COLOR(RED)
A                                     DSPATR(BL)
A                                 6  2'From Format SFD131A In    '
A                                     COLOR(RED)
A                                     DSPATR(BL)
A                                 8  2'Program SFR131            '
A                                     COLOR(RED)
A                                     DSPATR(BL)
A            @SL            3S OP
A            @SP            3S OP
A          R ASSUME
A                                     ASSUME
A                                24  2' '


F*============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR131) SRCFILE(XXX/QRPGSRC)
F*
F*============================================================
FSFD131  CF  E                    WORKSTN
C*
C* The @RANDM subroutine will generate a variable start
C* line and position.
C*
C                     EXSR @RANDM
C                     EXFMTSFD131A
C                     EXSR @CMD
C*
C*************************************************************
C*    @ C M D   S U B R O U T I N E                          *
C*                                                           *
C*    Executed after every EXFMT or display read op code.    *
C*    Used to process command keys.                          *
C*************************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, seton LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
```

```
C                     RETRN
C                     END
C*
C                     ENDSR
C*
C*********************************************************
C*    @ R A N D M   S U B R O U T I N E                  *
C*                                                       *
C*    Generate a variable start line and start position  *
C*    for the window.                                    *
C*********************************************************
C*
C           @RANDM    BEGSR
C*
C                     ADD  1         @CNT    30
C           @CNT      IFEQ 10
C                     MOVE *ON       *INLR
C                     END
C*
C                     ADD  3         @SL
C                     ADD  22        @SP
C*
C*  If the start line is greater than 12, our window will not
C*  fit on the display. Reset the line to 1.
C*
C           @SL       IFGT 12
C                     Z-ADD1         @SL
C                     END
C*
C*  If the start position is greater than 46, our window will not
C*  fit on the display. Reset the position to 1.
C*
C           @SP       IFGT 46
C                     Z-ADD1         @SP
C                     END
C*
C*   Nothing can be displayed from position 1,1 on the display.
C*   If the position is 1,1, reset it to 1,2. Also, check and
C*   make sure that the start line or start position is not zero.
C*
C           @SL       IFLE 1
C           @SP       ANDLE1
C                     Z-ADD1         @SL
C                     Z-ADD2         @SP
C                     END
C*
C                     ENDSR
```

When you call SFR130 your display should look like this:

```
  xx/xx/xx            Test Screen For Assume Keyword                  QPGMR
  xx:xx:xx                                                            SFD130A


This Line Came From Format SFD130A In Program SFR130
This Line Came From Format SFD130A In Program SFR130
This Line Came From Format SFD130A In Program SFR130
This Line Came From Format SFD130A In Program SFR130
This Line Came From Format SFD130A In Program SFR130
This Line Came From Format SFD130A In Program SFR130
This Line Came From Format SFD130A In Program SFR130
This Line Came From Format SFD130A In Program SFR130
This Line Came From Format SFD130A In Program SFR130
This Line Came From Format SFD130A In Program SFR130
This Line Came From Format SFD130A In Program SFR130
This Line Came From Format SFD130A In Program SFR130
This Line Came From Format SFD130A In Program SFR130
This Line Came From Format SFD130A In Program SFR130
This Line Came From Format SFD130A In Program SFR130
This Line Came From Format SFD130A In Program SFR130
This Line Came From Format SFD130A In Program SFR130
This Line Came From Format SFD130A In Program SFR130
 F3-Exit   F4-Call Next Program
```

Now press F4 to call SFR131. Your screen should look like this:

```
  xx/xx/xx            Test Screen For Assume Keyword                  QPGMR
  xx:xx:xx                                                            SFD130A
                     ..................................
                     :                                :
This Line Came From  :                                :
This Line Came From  :                                :
This Line Came From  :  This Window Was Displayed     :
This Line Came From  :                                :
This Line Came From  :  From Format SFD131A In        :
This Line Came From  :                                :
This Line Came From  :  Program SFR131                :
This Line Came From  :                                :
This Line Came From  :                                :
This Line Came From  :................................:
This Line Came From Format SFD130A In Program SFR130
This Line Came From Format SFD130A In Program SFR130
This Line Came From Format SFD130A In Program SFR130
This Line Came From Format SFD130A In Program SFR130
This Line Came From Format SFD130A In Program SFR130
This Line Came From Format SFD130A In Program SFR130
This Line Came From Format SFD130A In Program SFR130
This Line Came From Format SFD130A In Program SFR130
 F3-Exit   F4-Call Next Program
```

Press the Enter key a few times. Notice how the window bounces around the screen in an unpredictable fashion.

How was this done? Well, on the WINDOW keyword, you can specify the starting line and position using system to program fields. These are special fields that allow your program to communicate to your display file.

In this case, the window statement reads:

```
A                                        WINDOW(&@SL &@SP 10 30)
```

This line tells the file that the program will give the starting line and position, using the fields @SL and @SP. These field names must have an ampersand (&) prefix on the WINDOW keyword.

These variables must also be described on the format. They are described as three-byte signed fields, with no decimal positions, and as program to system fields. The three byte, signed, zero decimal positions are mandatory.

The program calculates the position of the window on the display by adding 3 to @SL and 22 to @POS each time the Enter key is pressed. This calculation is done in subroutine @RANDM. The program then verifies that the generated starting line and position is valid for this window. It makes sure that the starting line is not greater than 12, and that the position is not greater than 46. If these values are greater than

12 and 46, they will be reset to 1. If the program did not reset the values, it would receive an error as soon as it attempted to write the window at a starting line greater than 12 or a position greater than 46, because the window would "fall off" the display. After this check is done, the program makes sure that the starting line and position are not both one, which is an invalid position for any data to be positioned to in a display file. Therefore, if the starting line and position are equal to 01,01, the values are reset to 01,02, which means the window starts at line 1, position 2.

This subroutine contains another check. You cannot have more than 12 windows open at a time. Since this program continually writes new windows over the existing ones, it could attempt to display more, and thus get an error. The program traps for it by incrementing a counter called @CNT. As soon as @CNT reaches 10, the program sets on the LR indicator. Therefore, the program should never attempt to display more than the maximum number of 12 windows.

Hopefully, you have started to think of some real applications where variable start lines and positions would be nice. Here is one scenario.

There are new display stations on the market which have the ability to control the cursor with a mouse, such as the new Infowindow terminals from IBM. There are other products available that allow mouse control of the cursor, such as RUMBA/400. If a user has a mouse, they can easily point and click to where they would like their inquiry program to "pop-up" on the screen. Even if the user does not have mouse support, it would be nice to direct where a pop-up program will be placed on the screen. The user may want to view existing data in a certain area of the screen, and call an inquiry program that they could force to display in an area of the screen they did not need to view.

Let's change the program so that the variable start position can be useful. Take a look at the following source members:

```
A*==============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD140) SRCFILE(XXX/QDDSSRC)
A*
A*==============================================================
A                                       DSPSIZ(24 80 *DS3)
A          R SFD140A
A                                       CF04(04 'Call next program')
A                                       CF03(03 'Exit')
A                                       ASSUME
A                                   1  3DATE
A                                       EDTCDE(Y)
A                                   1 23'Test Screen For Assume Keyword '
A                                       DSPATR(HI)
A                                   2  3TIME
A                                   1 71USER
A                                   2 71'SFD140A'
```

```
A                                   23  2'F3-Exit   F4-Call Next Program'
A                                        DSPATR(HI)
A                                    5  1'This Line Came From Format SFD140A-
A                                         In Program SFR140'
A                                    6  1'This Line Came From Format SFD140A-
A                                         In Program SFR140'
A                                    7  1'This Line Came From Format SFD140A-
A                                         In Program SFR140'
A                                    8  1'This Line Came From Format SFD140A-
A                                         In Program SFR140'
A                                    9  1'This Line Came From Format SFD140A-
A                                         In Program SFR140'
A                                   10  1'This Line Came From Format SFD140A-
A                                         In Program SFR140'
A                                   11  1'This Line Came From Format SFD140A-
A                                         In Program SFR140'
A                                   12  1'This Line Came From Format SFD140A-
A                                         In Program SFR140'
A                                   13  1'This Line Came From Format SFD140A-
A                                         In Program SFR140'
A                                   14  1'This Line Came From Format SFD140A-
A                                         In Program SFR140'
A                                   15  1'This Line Came From Format SFD140A-
A                                         In Program SFR140'
A                                   16  1'This Line Came From Format SFD140A-
A                                         In Program SFR140'
A                                   17  1'This Line Came From Format SFD140A-
A                                         In Program SFR140'
A                                   18  1'This Line Came From Format SFD140A-
A                                         In Program SFR140'
A                                   19  1'This Line Came From Format SFD140A-
A                                         In Program SFR140'
A                                   20  1'This Line Came From Format SFD140A-
A                                         In Program SFR140'
A                                   21  1'This Line Came From Format SFD140A-
A                                         In Program SFR140'
A                                   22  1'This Line Came From Format SFD140A-
A                                         In Program SFR140'

F*==============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR140) SRCFILE(XXX/QRPGSRC)
F*
F*==============================================================
FSFD140  CF  E                      WORKSTN
F                                               KINFDS INFDS
IINFDS       DS
I                                     370 371 @LP
I            DS
I                                   B   1   20@LIN
I                                       2   2 @L
I            DS
I                                   B   1   20@POS
I                                       2   2 @P
C*
C                      EXFMTSFD140A
```

```
C                     EXSR @CMD
C*
C*************************************************************
C*    @ C M D   S U B R O U T I N E                          *
C*                                                           *
C*    Executed after every EXFMT or display read op code.    *
C*    Used to process command keys.                          *
C*************************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, seton LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C* If F4 was pressed, call program SFR141.
C*
C           *IN04     IFEQ *ON
C                     MOVE *ZEROS    @LIN
C                     MOVE *ZEROS    @POS
C                     MOVEL@LP       @L
C                     MOVE @LP       @P
C                     Z-ADD@LIN      @SL     30
C                     Z-ADD@POS      @SP     30
C                     CALL 'SFR141'
C                     PARM           @SL
C                     PARM           @SP
C                     END
C*
C                     ENDSR


A*==============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD141) SRCFILE(XXX/QDDSSRC)
A*
A*==============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03)
A          R SFD141A
A                                       WINDOW(&@SL &@SP 10 30)
A                                       WDWBORDER((*COLOR BLU) (*DSPATR
A                                        (*CHAR '        '))
A                                   4  2'This Window Was Displayed '
A                                       COLOR(RED)
A                                       DSPATR(BL)
A                                   6  2'From Format SFD141A In    '
A                                       COLOR(RED)
A                                       DSPATR(BL)
A                                   8  2'Program SFR141            '
A                                       COLOR(RED)
A                                       DSPATR(BL)
```

```
A            @SL            3  OP
A            @SP            3  OP
A          R ASSUME
A                                    ASSUME
A                                24  2' '


F*==============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR141) SRCFILE(XXX/QRPGSRC)
F*
F*==============================================================
FSFD141  CF  E                    WORKSTN
C*
C                      EXFMTSFD141A
C                      EXSR @CMD
C*
C*********************************************************
C*    @ C M D   S U B R O U T I N E                      *
C*                                                       *
C*    Executed after every EXFMT or display read op code.  *
C*    Used to process command keys.                      *
C*********************************************************
C*
C           @CMD       BEGSR
C*
C* If F3 was pressed, seton LR and exit.
C*
C           *IN03      IFEQ *ON
C                      MOVE *ON       *INLR
C                      RETRN
C                      END
C*
C                      ENDSR
C*
C*********************************************************
C*    * I N Z S R   S U B R O U T I N E                  *
C*                                                       *
C*    Retrieve the starting line and position for the    *
C*    window. Check to make sure that these values are   *
C*    valid. If not, put the closest valid number in.    *
C*********************************************************
C*
C           *INZSR     BEGSR
C           *ENTRY     PLIST
C                      PARM           @SL
C                      PARM           @SP
C*
C*  If the start line is greater than 12, our window will not
C*  fit on the display. Reset the line to 12.
C*
C           @SL        IFGT 12
C                      Z-ADD12        @SL
C                      END
C*
```

```
C*  If the start position is greater than 46, our window will not
C*  fit on the display. Reset the position to 46.
C*
C           @SP       IFGT 46
C                     Z-ADD46        @SP
C                     END
C*
C*   Nothing can be displayed from position 1,1 on the display.
C*   If the position is 1,1, reset it to 1,2. Also, check and
C*   make sure that the start line or start position is not zero.
C*
C           @SL       IFLE 1
C           @SP       ANDLE1
C                     Z-ADD1         @SL
C                     Z-ADD2         @SP
C                     END
C*
C                     ENDSR
```

Call program SFR140. Move the cursor to screen position 10, 20. Press the F4 key. Notice that the window popped up at position 10, 20. You can use the cursor to place the window where you want it to show up. Notice what happens if you attempt to place the window at position 01, 01. It is placed at 01, 02. See what happens if you attempt to place the window past line 12, or past position 40. The program places the window as far down or over as allowed.

This program can really be useful. First, look at the changes made in SFR140, which was the original calling program. It is in this program that you place the cursor where you want the window to pop up. Therefore, the program must extract the cursor line and position from this program. You can do so by reading the file information data structure for the display file SFD140. The name if the file information data structure is INFDS.

In the "I" specifications, you can see the INFDS data structure, with a field defined in the structure. The name of the field is @LP, and is a two-byte alpha field in position 370-371 of the file information data structure. This field contains the position of the cursor in hexadecimal. The first byte is the line, and the second is the position. The program converts this hexadecimal number to binary, then to decimal, in the @CMD subroutine. If F4 is pressed, this series of calculations is performed. You end up with the line position in field @SL and the position in field @SP.

Now the program calls the SFR141 program, passing @SL and @SP as parameters.

When SFR141 is first called, it executes subroutine *INZSR. Inside this subroutine the program performs the same validity checks as in program SFR131, with one small difference. Instead of changing the value of the start line or position to one if it is invalid, the program changes it to the maximum value allowed. A user can have the window placed as far down or as far to the right on the screen as possible. The

program also checks for the attempt to write the window to position 01, 01 and forces it to 01,02 if an occurs.

When you run program SFR140 your screen should look like this:

```
  xx/xx/xx            Test Screen For Assume Keyword                  QPGMR
  xx:xx:xx                                                            SFD140A


This Line Came From Format SFD140A In Program SFR140
This Line Came From Format SFD140A In Program SFR140
This Line Came From Format SFD140A In Program SFR140
This Line Came From Format SFD140A In Program SFR140
This Line Came From Format SFD140A In Program SFR140
This Line Came From Format SFD140A In Program SFR140
This Line Came From Format SFD140A In Program SFR140
This Line Came From Format SFD140A In Program SFR140
This Line Came From Format SFD140A In Program SFR140
This Line Came From Format SFD140A In Program SFR140
This Line Came From Format SFD140A In Program SFR140
This Line Came From Format SFD140A In Program SFR140
This Line Came From Format SFD140A In Program SFR140
This Line Came From Format SFD140A In Program SFR140
This Line Came From Format SFD140A In Program SFR140
This Line Came From Format SFD140A In Program SFR140
This Line Came From Format SFD140A In Program SFR140
 F3-Exit   F4-Call Next Program
```

Now place the cursor at position 10, 20. Press F4 to call SFR141. Your screen should look like this:

```
  xx/xx/xx            Test Screen For Assume Keyword                  QPGMR
  xx:xx:xx                                                            SFD140A


This Line Came From Format SFD140A In Program SFR140
This Line Came From Format SFD140A In Program SFR140
This Line Came From Format SFD140A In Program SFR140
This Line Came From Format SFD140A In Program SFR140
This Line Came From Format SFD140A In Program SFR140
This Line Came Fro ................................
This Line Came Fro :                              :
This Line Came Fro :                              :
This Line Came Fro :                              :
This Line Came Fro :  This Window Was Displayed    :
This Line Came Fro :                              :
This Line Came Fro :  From Format SFD141A In      :
This Line Came Fro :                              :
This Line Came Fro :  Program SFR141              :
This Line Came Fro :                              :
This Line Came Fro :                              :
This Line Came Fro :...............................:
This Line Came From Format SFD140A In Program SFR140
 F3-Exit   F4-Call Next Program
```

The program positions the window where your cursor had been. Try playing around with this program to see what would happen if you tried to position the window at 01, 01 or any other invalid position.

4

Subfile Functionality

You have already learned that there are three ways to load a subfile—the load-all method, the expanding method, and the single-page load method. This section will make use of all three subfile loading techniques and will explore all of the functions available for use in subfiles. Progressing through these examples, you will find out how to convert a full screen subfile to a window subfile. Then you will convert the load-all subfile to an expanding subfile, then to a single-page load. Throughout this section, you will be adding functionality to the subfile. You will also see how an outwardly complex looking program is inwardly simple to code.

Load-All Subfile

For a load-all subfile, you load all records that are to be displayed into the subfile before displaying the screen. The main reason to use a load-all subfile is for simplicity in programming. You do not need to program for the roll keys in a load-all subfile. The roll keys are handled automatically by the display file.

The basic steps of a load-all subfile program are:

1. Load all needed records into the subfile
2. Execute the subfile control format

The subfile is loaded one time in the program. Then the subfile control format is executed. If a roll key is pressed, the display file itself rolls the records on the screen, and the program does not leave the EXFMT statement. If a command key is pressed, the program leaves the EXFMT statement and processes the command key. After processing the command key routine, you can route the program back to the EXFMT statement.

Roll Key Consideration For A Load-All Subfile

You do not have to program for the roll keys on a load-all subfile. The roll keys are handled automatically by the display file itself. Do not include the roll keys in the RPG program or the DDS.

Subfile Functionality

This section allows you to see what functionality is available in subfiles, how easy it is to implement these functionalities, and the possible uses for the functions.

In this section, you will take the same maintenance and inquiry programs and modify them slightly to show where the functionality is added. You will start off with a simple full screen subfile, and grow it into a very functional pop-up window inquiry with select capabilities. Using the same program will help you see how the functions work, as you will already be familiar with the program.

Windowed Subfiles

First, you need to make changes to the SFR001 program so that the subfile will be displayed inside a window. These changes have been made in program SFR002.

```
A*============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD002) SRCFILE(XXX/QDDSSRC)
A*
A*============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03)
A          R SFD002A                    SFL
A            CUSNO     R        O  2  1REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                       COLOR(PNK)
A            CUSNM     R        O  2  9REFFLD(RCUST/CUSNM *LIBL/CUSTOMER)
A                                       COLOR(PNK)
```

```
A          R SFD002B                    SFLCTL(SFD002A)
A                                       SFLSIZ(0007)
A                                       SFLPAG(0006)
A                                       WINDOW(WINDOW)
A                                       OVERLAY
A  30                                   SFLDSP
A                                       SFLDSPCTL
A                                   1  1'Number'
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A                                   1  9'Customer Name                  '
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A          R WINDOW
A                                       WINDOW(10 10 10 40)
A                                       WDWBORDER((*COLOR BLU) (*DSPATR RI)-
A                                        (*CHAR '        '))
A                                   9  1'F3-Exit'
A                                       DSPATR(HI)
A          R ASSUME
A                                       ASSUME
A                                  24  2' '


F*==============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR002) SRCFILE(XXX/QRPGSRC)
F*
F*==============================================================
FCUSTOMERIF  E           K        DISK
FSFD002  CF  E                    WORKSTN
F                                       @RRN  KSFILE SFD002A
C*
C                     WRITEWINDOW
C                     EXFMTSFD002B
C                     EXSR @CMD
C*
C*********************************************************
C*    @ C M D   S U B R O U T I N E                      *
C*                                                       *
C*    Executed after every EXFMT or display read op code.  *
C*    Used to process command keys.                      *
C*********************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, seton LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C                     ENDSR
C*
```

```
C*************************************************************
C*    * I N Z S R   S U B R O U T I N E                      *
C*                                                           *
C*    This subroutine is called automatically when the       *
C*    program is started up.                                 *
C*************************************************************
C*
C           *INZSR    BEGSR
C*
C*  Load the subfile
C*
C                     Z-ADDO         @RRN    40
C                     EXSR @LOAD
C*
C                     ENDSR
C*
C*************************************************************
C*    @ L O A D     S U B R O U T I N E                      *
C*                                                           *
C*    This subroutine will read the customer file and        *
C*    load the subfile records. Since this is a load all     *
C*    subfile program, all records in the customer file      *
C*    would normally be written into the subfile. But you     *
C*    have conditioned the subfile write opcode not to load *
C*    more reocrds than the value of subfile size.           *
C*************************************************************
C*
C           @LOAD     BEGSR
C*
C*  Perform this loop until you reach the end of
C*  the Customer file or the maximum subfile records
C*  allowed. Both cases would turn on indicator 99.
C*
C           *IN99     DOWNE*ON
C                     READ CUSTOMER                 99
C*
C*  If indicator 99 is on, then leave this do loop.
C*
C           *IN99     IFEQ *ON
C                     LEAVE
C                     END
C*
C*  Add 1 to the subfile relative record number. If you
C*  reach the maximum allowable subfile records, indicator
C*  99 will turn on.
C*
C                     ADD  1         @RRN
C*
C*  Write the record into the subfile.
C*
C                     WRITESFDOO2A                  99
C           @RRN      IFEQ 9999
C                     MOVE *ON       *IN99
C                     END
C                     END
C*
```

```
C* If you have loaded at least one record into the subfile,
C* turn on indicator 30. This indicator conditions the
C* SFLDSP keyword. If you have loaded at least one record
C* into the subfile, it will be all right to display the subfile
C* format. If you did not load any records into the subfile, you
C* would not turn on indicator 30. Attempting to display
C* an empty subfile would result in an error being issued.
C*
C           @RRN      IFGT 0
C                     MOVE *ON       *IN30
C                     END
C*
C                     ENDSR
```

In the display file, the WINDOW keyword is added into the subfile control format. This addition changes the subfile from a full screen to a window subfile.

```
A                                        WINDOW(WINDOW)
```

The keyword has a parameter specified. The window definitions are found in the format WINDOW.

A new record format has been added in the display file which contains the ASSUME keyword. This keyword allows the subfile to "pop-up" on the display.

```
A          R ASSUME
A                                        ASSUME
A                                   24  2' '
```

The only change in the RPG program is to change the following line of code from

```
C                     WRITETRAILER
```

to:

```
C                     WRITEWINDOW
```

Now, you can compile the program and run it. The program will "pop-up" on the display without erasing the entire screen, like this:

```
.........................................
: Number  Customer Name                 :
: 00001   David's Computers and Bait Shop  :
: 00002   AS/400s Are Us                :
: 00003   Tennessee Minicomputer Sales  :
: 00004   PCs For Less                  :
: 00005   Mid-South Computer Sales      :
: 00010   Mini Computing Clearance      :
:                                       :
: F3-Exit                               :
:                                       :
:.......................................:
```

This pop-up adds a nice effect. But what use do you have for a pop-up inquiry subfile?

You can now convert this program into a pop-up window that allows cursor placement, which was demonstrated earlier in the book, but now you will utilize it with a subfile. You will combine the subfile inquiry program with a customer maintenance program. You will call the subfile maintenance program, and from there you can call the subfile inquiry program.

The code necessary to do this is shown in SFD210, SFR210, SFD211, and SFR211 below:

```
A*============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD210) SRCFILE(XXX/QDDSSRC)
A*
A*============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03 'Exit')
A          R SFD210A
A                                       CF04(04 'Call Search Program')
A                                  1  3DATE
A                                       EDTCDE(Y)
A                                  1 23'Customer Master Maintenance'
A                                       DSPATR(HI)
A                                  2  3TIME
A                                  1 71USER
A                                  2 71'SFD210A'
A                                 23  2'F3-Exit   F4-Customer Search'
A                                       DSPATR(HI)
A                                  5  1'Customer Number....:'
A                                       COLOR(BLU)
A                                  7  1'Customer Name......:'
A                                       COLOR(BLU)
A                                  9  1'Customer Address...:'
A                                       COLOR(BLU)
A                                 11  1'Customer City......:'
A                                       COLOR(BLU)
A                                 13  1'Customer State.....:'
A                                       COLOR(BLU)
A                                 15  1'Customer Zip.......:'
A                                       COLOR(BLU)
A                                 17  1'Phone Number.......:'
A                                       COLOR(BLU)
A            CUSNO     R     Y  B  5 23REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A                                       CHECK(ER)
A                                       CHECK(FE)
A                                       CHECK(RB)
A                                 24  1'Customer Number Not Found In Custo-
A                                       mer Master File-Key In A Valid Numb-
A                                       er'
A                                       COLOR(RED)
```

```
A                                       DSPATR(BL)
A N99                                   DSPATR(ND)
A            CUSPH     R     Y  B  8 46REFFLD(RCUST/CUSPH *LIBL/CUSTOMER)
A                                       EDTWRD('   -   -    ')
A                                    8 60'Previous'
A                                       COLOR(WHT)
A                                       DSPATR(HI)
A                                       DSPATR(BL)
A                                    8 69'Phone#'
A                                       DSPATR(HI)
A                                       DSPATR(BL)
A          R SFD210B
A                                       CLRL(*NO)
A                                    2 71'SFD210B'
A            CUSNO     R        O  5 23REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                       COLOR(PNK)
A            CUSNM     R        B  7 22REFFLD(RCUST/CUSNM *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A            CUSAD     R        B  9 22REFFLD(RCUST/CUSAD *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A            CUSCT     R        B 11 22REFFLD(RCUST/CUSCT *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A            CUSST     R        B 13 22REFFLD(RCUST/CUSST *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A            CUSCP     R        B 15 22REFFLD(RCUST/CUSCP *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A            CUSPH     R        B 17 22REFFLD(RCUST/CUSPH *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A                                       EDTWRD('   -   -    ')
A                                   23  1'F3-Exit                          -
A                                          '
A                                       COLOR(WHT)
A                                   24  1'                                  -
A                                                                          -
A                                                 '


F*============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR210) SRCFILE(XXX/QRPGSRC)
F*
F*============================================================
FSFD210  CF  E                     WORKSTN
F                                            KINFDS INFDS
FCUSTOMERIF  E           K         DISK
IINFDS       DS
I                                       370 371 LINPOS
I            DS
I                                     B   1   20@LIN
I                                         2   2 @L
I            DS
I                                     B   1   20@POS
I                                         2   2 @P
C*
C                      EXFMTSFD210A
```

```
C                     EXSR @CMD
C*
C           CUSNO     CHAINCUSTOMER             99
C*
C           *IN99     IFEQ *OFF
C                     EXFMTSFD210B
C                     EXSR @CMD
C                     END
C*
C*********************************************************
C*    @ C M D   S U B R O U T I N E                      *
C*                                                       *
C*    Executed after every EXFMT or display read op code.  *
C*    Used to process command keys.                      *
C*********************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, seton LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C* If F4 was pressed, call program SFR211.
C*
C           *IN04     IFEQ *ON
C*
C* Convert the hex values of the cursor line/position into
C* binary, then to decimal.
C*
C                     MOVE *OFF      *IN04
C                     MOVE *ZEROS    @LIN
C                     MOVE *ZEROS    @POS
C                     MOVELLINPOS    @L
C                     MOVE LINPOS    @P
C                     Z-ADD@LIN      @SL     30
C                     Z-ADD@POS      @SP     30
C*
C                     CALL 'SFR211'
C                     PARM           @SL
C                     PARM           @SP
C*
C                     END
C*
C                     ENDSR


A*==============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD211) SRCFILE(XXX/QDDSSRC)
A*
A*==============================================================
A                                       DSPSIZ(24 80 *DS3)
```

```
A                                       CF03(03)
A          R ASSUME
A                                       ASSUME
A                                 24  2' '
A          R SFD211A                    SFL
A            CUSNO     R        O  2  1REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                       COLOR(PNK)
A            CUSNM     R        O  2  9REFFLD(RCUST/CUSNM *LIBL/CUSTOMER)
A                                       COLOR(PNK)
A          R SFD211B                    SFLCTL(SFD211A)
A                                       SFLSIZ(0007)
A                                       SFLPAG(0006)
A                                       WINDOW(WINDOW)
A                                       OVERLAY
A  30                                   SFLDSP
A                                       SFLDSPCTL
A                                  1  1'Number'
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A                                  1  9'Customer Name                  '
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A          R WINDOW
A                                       WINDOW(&@SL &@SP 10 40)
A                                       WDWBORDER((*COLOR BLU) (*DSPATR RI)-
A                                        (*CHAR '        '))
A            @SL            3S OP
A            @SP            3S OP
A                                  9  1'F3-Exit'
A                                       DSPATR(HI)


F*=============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR211) SRCFILE(XXX/QRPGSRC)
F*
F*=============================================================
FCUSTOMERIF  E           K        DISK
FSFD211  CF  E                    WORKSTN
F                                       @RRN  KSFILE SFD211A
C*
C                     WRITEWINDOW
C                     EXFMTSFD211B
C                     EXSR @CMD
C*
C*********************************************************
C*    @ C M D   S U B R O U T I N E                      *
C*                                                       *
C*    Executed after every EXFMT or display read op code.  *
C*    Used to process command keys.                      *
C*********************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, seton LR and exit.
```

```
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C                     ENDSR
C*
C*********************************************************
C*    * I N Z S R   S U B R O U T I N E                  *
C*                                                       *
C*    Retrieve the starting line and position for the    *
C*    window. Check to make sure that these values are   *
C*    valid. If not, put the closest valid number in.    *
C*********************************************************
C*
C           *INZSR    BEGSR
C*
C           *ENTRY    PLIST
C                     PARM           @SL
C                     PARM           @SP
C*
C                     Z-ADD0         @RRN    50
C*
C*  If the start line is greater than 12, the window will not
C*  fit on the display. Reset the line to 12.
C*
C           @SL       IFGT 12
C                     Z-ADD12        @SL
C                     END
C*
C*  If the start position is greater than 36, the window will not
C*  fit on the display. Reset the position to 36.
C*
C           @SP       IFGT 36
C                     Z-ADD36        @SP
C                     END
C*
C*   Nothing can be displayed from position 1,1 on the display.
C*   If the position is 1,1, reset it to 1,2. Also, check and
C*   make sure that the start line or start position is not zero.
C*
C           @SL       IFLE 1
C                     Z-ADD1         @SL
C           @SP       IFLE 1
C                     Z-ADD2         @SP
C                     END
C                     END
C*
C           @SP       IFLT 1
C                     Z-ADD1         @SP
C                     END
C*
C*  Load the subfile
C*
C                     EXSR @LOAD
C*
```

```
C                     ENDSR
C*
C*********************************************************
C*    @ L O A D     S U B R O U T I N E                  *
C*                                                       *
C*    This subroutine will read the customer file and    *
C*    load the subfile records. Since this is a load all *
C*    subfile program, all records in the customer file  *
C*    will be read and loaded.                           *
C*********************************************************
C*
C           @LOAD     BEGSR
C*
C*  Perform this loop until you reach the end of
C*  the Customer file or the maximum subfile records
C*  allowed. Both cases would turn on indicator 99.
C*
C           *IN99     DOWNE*ON
C                     READ CUSTOMER                 99
C*
C*  If indicator 99 is on, then leave this do loop.
C*
C           *IN99     IFEQ *ON
C                     LEAVE
C                     END
C*
C*  Add 1 to the subfile relative record number. If you
C*  reach the maximum allowable subfile records, indicator
C*  99 will turn on.
C*
C                     ADD  1         @RRN
C                     WRITESFD211A                  99
C                     END
C*
C* If you have loaded at least one record into the subfile,
C* turn on indicator 30. This indicator conditions the

C* SFLDSP keyword. If you have loaded at least one record
C* into the subfile, it will be all right to display the subfile
C* format. If you did not load any records into the subfile, you
C* would not turn on indicator 30. Attempting to display
C* an empty subfile would result in an error being issued.
C*
C           @RRN      IFGT 0
C                     MOVE *ON       *IN30
C                     END
C*
C                     ENDSR
```

The customer maintenance program is SFR210. When you call it you should get a screen that looks like this:

```
  xx/xx/xx            Customer Master Maintenance                    QPGMR
  xx:xx:xx                                                           SFD210A


Customer Number....:  00000

Customer Name......:
                                              _____________  Previous Phone#
Customer Address...:

Customer City......:

Customer State.....:

Customer Zip.......:

Phone Number.......:



 F3-Exit   F4-Customer Search
```

Now place the cursor on position 20,70. Press the F4 key. You will get a screen that looks like this:

```
  xx/xx/xx            Customer Master Maintenance                    QPGMR
  xx:xx:xx                                                           SFD210A


Customer Number....:  00000

Customer Name......:
                                            _____________  Previous Phone#
Customer Address...:

Customer City......:
                                     ..........................................
Customer State.....:                 : Number  Customer Name                  :
                                     : 00001   David's Computers and Bait Shop :
Customer Zip.......:                 : 00002   AS/400s Are Us                 :
                                     : 00003   Tennessee Minicomputer Sales   :
Phone Number.......:                 : 00004   PCs For Less                   :
                                     : 00005   Mid-South Computer Sales       :
                                     : 00010   Mini Computing Clearance       :
                                     :                                        :
                                     : F3-Exit                                :
                                     :                                        :
 F3-Exit   F4-Customer Search        :........................................:
```

Now press the roll-up key. Your screen should look like this:

```
  xx/xx/xx            Customer Master Maintenance                    QPGMR
  xx:xx:xx                                                           SFD210A


Customer Number....:  00000

Customer Name......:
                                          _____________  Previous Phone#
Customer Address...:

Customer City......:
                                   ........................................
Customer State.....:               : Number  Customer Name                 :
                                   : 00011   Computer Mart                 :
Customer Zip.......:               :                                       :
                                   :                                       :
Phone Number.......:               :                                       :
                                   :                                       :
                                   :                                       :
                                   :                                       :
                                   : F3-Exit                               :
                                   :                                       :
 F3-Exit   F4-Customer Search      :.......................................:
```

Load-All Method

Program SFR211 is an example of a load-all subfile. All records are loaded into the subfile before the first screen comes up. By looking at some of the major sections of program SFR211, you can see how this is done and how the subfile works.

```
      **************************
      * File Section
      **************************
01    FCUSTOMERIF  E           K        DISK
02    FSFD211  CF  E                    WORKSTN
03    F                                        @RRN  KSFILE SFD211A
      **************************
      * Display Section-Step 3
      **************************
04    C                      WRITEWINDOW
05    C                      EXFMTSFD211B
06    C                      EXSR @CMD
      ***************************
      * Final Section-Step 4
      ***************************
07    C            @CMD      BEGSR
08    C            *IN03     IFEQ *ON
09    C                      MOVE *ON       *INLR
10    C                      RETRN
11    C                      END
```

```
12   C                      ENDSR
      *************************
      * Initial Section-Step 1
      *************************
13   C           *INZSR     BEGSR
14   C                      Z-ADD0          @RRN    50
15   C                      EXSR @LOAD
16   C                      ENDSR
      *************************
      * Load Section-Step 2
      *************************
17   C           @LOAD      BEGSR
18   C           *IN99      DOWNE*ON
19   C                      READ CUSTOMER                   99
20   C           *IN99      IFEQ *ON
21   C                      LEAVE
22   C                      END
23   C                      ADD  1          @RRN
24   C                      WRITESFD211A                    99
25.  C                      END
26   C           @RRN       IFGT 0
27   C                      MOVE *ON        *IN30
28   C                      END
29   C                      ENDSR
```

As you can see, the basic subfile code consists of only 29 lines. These basic lines deal with the subfile processing. Very little code is needed for this specific application that wouldn't have applied to another similar subfile. For instance, lines 1 and 19 contain a reference to the file CUSTOMER. These two lines are the only ones that would need to be changed if you wanted to retrofit this subfile code to a state file inquiry program.

At this point you can see how the subfile process actually works. Notice in the file section that the display file SFD211 has a continuation line assigned to it.

```
FSFD211  CF  E                    WORKSTN
F                                        @RRN  KSFILE SFD211A
```

The SFILE entry specifies that this file contains a subfile, and that the subfile format's name is SFD211A. The relative record number field for this subfile is called @RRN. Field @RRN is important because it allows you to add records to the subfile. Subfile records are accessed by relative record number. You are responsible for assigning the relative record number for each subfile record. This entry in the file specification allows the program to know which field you are using for the relative record number. Thus, when you perform a write to the subfile (when you add a subfile record), the program knows which relative record number to assign to it. You will see where this is done shortly.

If the program contained more than one subfile in this display file, the display file would have a continuation line for each subfile. Each subfile would have a corresponding field to be used as the relative record number field.

Step 1

```
 **************************
 * Initial Section-Step 1
 **************************
C           *INZSR    BEGSR
C                     Z-ADD0         @RRN    50
C                     EXSR @LOAD
C                     ENDSR
```

The first part of the program that gets executed is the *INZSR subroutine. A lot of code in this section had to do with positioning the window on the screen, but as far as the subfile is concerned, these are the only lines of code needed. You will seen how the window positioning works a little later.

First, you must set the relative record number for the subfile to 0. Then you specify that the @LOAD subroutine is to be executed, which is responsible for loading the subfile records.

Step 2

```
 **************************
 * Load Section-Step 2
 **************************
C           @LOAD     BEGSR
C           *IN99     DOWNE*ON
C                     READ CUSTOMER                 99
C           *IN99     IFEQ *ON
C                     LEAVE
C                     END
C                     ADD  1         @RRN
C                     WRITESFD211A                  99
C                     END
C           @RRN      IFGT 0
C                     MOVE *ON       *IN30
C                     END
C                     ENDSR
```

The next step is to physically load the subfile records, which is done inside the @LOAD subroutine. Execute a DO loop until indicator 99 gets set on. The indicator is turned on when you reach end of file or attempt to load more records than the subfile size. If you did not have the resulting indicator on the write statement, then the DO loop would be executed until it reached end of file, or until you attempted to load more

than 9,999 records into the subfile. If you attempted to load more than 9,999 records, you would receive an error message.

Notice that before each write, you must add one to the relative record number field. When you issue the write to the subfile format, the system writes the record and assigns it to the relative record number contained in the relative record number field specified in the "F" specifications for this subfile. You are responsible for ensuring that the relative record number is valid. It cannot be zero, nor can you attempt to write a duplicate relative record number to the file.

After you leave the DO loop, you must check the relative record number counter to see if it is greater than 0. If it is, set it on indicator 30. This indicator conditions the SFLDSP keyword. If indicator 30 is on, it means that you have at least one subfile record in the subfile, and that you can display the subfile. If you did not have any records in the subfile and attempted to display it, you would receive an error.

Step 3

```
 *************************
 * Display Section-Step 3
 *************************
C                       WRITEWINDOW
C                       EXFMTSFD211B
C                       EXSR @CMD
```

After you load all the records into the subfile (which in this case is seven, since the subfile size is seven and you have a resulting indicator on the subfile write statement), perform the statements in step 3.

First write the window definition format. Then perform a write and then a read (EXFMT) to the subfile control format SFD211B. This format also happens to be a window reference format. Since indicator 30 is on (because you loaded records into the subfile), both the control format domain and the subfile format domain are displayed. The headings and subfile records are displayed on the screen.

Now what happens? Well, until a valid function key or the Enter key is pressed, the program stays on the EXFMT statement. If the user presses the roll-up or roll-down key, the program does not leave this statement because the subfile is handling the roll keys.

Step 4

```
 ************************
 * Final Section-Step 4
 ************************
C           @CMD      BEGSR
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C                     ENDSR
```

If the user presses a function or Enter key, the @CMD subroutine is processed. It checks to see if F3 was pressed. If it was, the routine sets on LR, and returns out of the program. If any other function key besides F3 was pressed, the program falls out of the @CMD subroutine and redisplays the screen.

SFR211 is a classic example of a load-all subfile. Before the user sees a display screen, all records that could be displayed are first loaded. In the load-all subfile, the subfile is responsible for handling the roll keys. It is not necessary to reference them anywhere in the DDS or in the RPG program. Once the program reaches the EXFMT statement, where it writes and reads the subfile control format, the program does not leave this statement when a roll key is pressed.

Notice the responsibilities of the subfile format and the subfile control format. When the program writes the subfile records out, it uses the subfile format. When the program displays the entire subfile, it uses the subfile control format. If the program accesses or writes a single subfile record, it uses the subfile format. If the program accesses the subfile as a whole entity, it uses the subfile control format.

Positioning a Windowed Subfile

Now you can look at some of the functionality that is used in this program.

First, how do you get the inquiry window to pop up where the cursor is positioned? When the user presses F4 to call the subfile inquiry program, the program needs to retrieve the cursor position of the screen to pass the coordinates to the subfile program. This process is coded in the following lines in program SFR210:

```
FSFD210  CF  E                    WORKSTN
F                                            KINFDS INFDS
IINFDS       DS
I                                      370 371 LINPOS
I            DS
I                                   B   1   20@LIN
I                                       2   2 @L
I            DS
```

```
I                                       B   1   20@POS
I                                           2   2 @P

C                       MOVE *ZEROS     @LIN
C                       MOVE *ZEROS     @POS
C                       MOVELLINPOS     @L
C                       MOVE LINPOS     @P
C                       Z-ADD@LIN       @SL     30
C                       Z-ADD@POS       @SP     30
```

Notice that the program contains a file continuation specification. This statement specifies that you want to use the file informational data structure INFDS, which retrieves the position of the cursor. Next, you must define the INFDS, specifying that the data you want is in position 370 through 371, which is where the cursor position is located. The name of this field is LINPOS. It contains the position of the cursor in hexadecimal. Position 370 contains the hexadecimal value of the line number, and position 371 contains the hexadecimal value of the position.

The next two data structures define a two-byte binary field and a corresponding one-byte alpha field. These data structures are used to convert the hexadecimal values of the line and position to decimal. This conversion is performed in the calculation specifications. First, the program moves zeros to the binary fields to initialize the binary fields. You can also do this by placing an "I" in the data structure option field in the "I" specifications. Next the program moves the decimal values of the line and position into three-byte numeric fields, which are the same size as the program to system fields in program SFR211. SFR210 passes these two fields as parameters to program SFR211, which contains the starting line and position of the window.

Notice that you don't have to specify anything in the display file for SFD210 to retrieve the cursor position.

The following lines are the code necessary in the subfile program SFR211 to display the window where the cursor is positioned.

```
C           *ENTRY    PLIST
C                     PARM            @SL
C                     PARM            @SP
C*
C*  If the start line is greater than 12, the window will not
C*  fit on the display. Reset the line to 12.
C*
C           @SL       IFGT 12
C                     Z-ADD12         @SL
C                     END
C*
C*  If the start position is greater than 36, the window will not
C*  fit on the display. Reset the position to 36.
C*
C           @SP       IFGT 36
C                     Z-ADD36         @SP
C                     END
```

```
C*
C*   Nothing can be displayed from position 1,1 on the display.
C*   If the position is 1,1, reset it to 1,2. Also, check and
C*   make sure that the start line or start position is not zero.
C*
C           @SL       IFLE 1
C                     Z-ADD1         @SL
C           @SP       IFLE 1
C                     Z-ADD2         @SP
C                     END
C                     END
C*
C           @SP       IFLT 1
C                     Z-ADD1         @SP
C                     END
```

First, the program receives the start line and position parameters in the *ENTRY parameter list. Next, the program executes an edit routine to ensure that the starting line and position fields contain valid values. If the start line is greater than 12, or if the start position is greater than 36, the window will not have enough room to be fully displayed. Therefore, the program traps for these conditions. If the starting line is greater than 12, the program moves 12 to the starting line. If the position is greater than 36, then the program moves 36 into the starting position, which allows the user to place the cursor on the bottom right-hand side of the screen. In this case, the window subfile program places the window in as close to the bottom and as far to the right as possible without going outside of the screen boundaries.

The program also checks to be sure that neither the start line or the position are 0, and that the start line and start position are not both 1. Position 1, 1 is reserved. Attempting to display a field at this position results in an error. If any of these conditions are true, the program forces the start position to be 1, 2.

The following two lines are responsible for building the window and displaying it on the screen.

```
C                     WRITEWINDOW
C                     EXFMTSFD211B
```

This display file (SFD211) uses a window reference format. In this case, the program must write the window definition format first, which happens to be named WINDOW. This window definition format actually positions a window on the display with the correct border attributes. The window at this point is positioned where the cursor was positioned in program SFR210 and looks like this:

```
.........................................
:                                       :
:                                       :
:                                       :
:                                       :
:                                       :
:                                       :
:                                       :
: F3-Exit                               :
:                                       :
:.......................................:
```

Then the program executes format SFD211B, which is the subfile control format. This is also a window reference format. It completes the window and looks like this:

```
.........................................
: Number  Customer Name                 :
:                                       :
:                                       :
:                                       :
:                                       :
:                                       :
:                                       :
: F3-Exit                               :
:                                       :
:.......................................:
```

Why does the display file use a window reference format? Why doesn't it just contain the literal "F3-Exit" on the bottom of the subfile control format (SFD210B). It uses a window reference format because you cannot place any subfile control format data below the subfile format. Therefore, to place the "F3-Exit" literal on the display the program writes it out on the window definition, then overlays it with the subfile control format fields located in SFR210B.

The following keyword in the display file SFD211 is responsible for setting the window at the correct starting coordinates:

```
A          R WINDOW
A                                    WINDOW(&@SL &@SP 10 40)
A                                    WDWBORDER((*COLOR BLU) (*DSPATR RI)-
A                                     (*CHAR '        '))
A            @SL            3S 0P
A            @SP            3S 0P
```

Notice the variables passed to the program, @SL and @SP, are used as the starting line and position for the window. The display file must also contain a definition for these variables as program-to-system fields in the same record format as shown in the example.

As you can see, about half of the lines of RPG code for SFR211 are needed to allow the user to position the window on the display. That's not bad, considering the program without this capability would have only required 29 lines of code.

Limiting the Loading Process

There is another function built into this program: the ability to limit loading a subfile up to its subfile size. The subfile automatically extends itself unless you tell it not to. This program is an example of how to limit the loading process up to the subfile size. The subfile size in SFD211 is seven. Notice the following lines of code in the RPG program.

```
C           @LOAD     BEGSR
C*
C*  Perform this loop until you reach the end of
C*  the Customer file or the maximum subfile records
C*  allowed. Both cases would turn on indicator 99.
C*
C           *IN99     DOWNE*ON
C                     READ CUSTOMER                 99
C*
C*  If indicator 99 is on, then leave this do loop.
C*
C           *IN99     IFEQ *ON
C                     LEAVE
C                     END
C*
C*  Add 1 to the subfile relative record number. If you
C*  reach the maximum allowable subfile records, indicator
C*  99 will turn on.
C*
C                     ADD  1         @RRN
C                     WRITESFD211A                  99
C                     END
```

This subfile load subroutine is where the program performs the load-all method of loading records into the subfile. The program performs a loop until indicator 99 is set on. There are two ways 99 gets set on. The first is if the program reaches end of file on the CUSTOMER file. If this happens, indicator 99 is set on and the program leaves the loop.

But in this example the program does not reach end of file. As a matter of fact, the subfile does not contain all of the records in the CUSTOMER file. Why not? Because the program loads only a maximum of seven records. When the program writes to the subfile, indicator 99 turns on if the program tries to write more records than the value of the subfile size. The program checks the value of indicator 99 at the top of the loop and exits the loop when indicator 99 is on.

You can notice this effect by executing the subfile program, rolling the screens, and counting the number of records. There are seven records displayed before you get a message stating that you attempted to roll past the last record on the subfile.

Why would you want to do this? Well, this program is a load-all subfile program. It handles the roll keys for you and is very simple. However, without the resulting indicator on the subfile write, the subfile would load all records in the CUSTOMER file. Ordinarily, this is what you would want to do. But there may be times where you would want to keep the number of records in the subfile relatively low—say 200 or less. To keep the number of records low, you could set the subfile size to 200, place a resulting indicator on the write to the subfile, and never load more than 200 records. If you do limit the number of subfile records loaded, then you would probably want to provide a command key to allow the user to load the next 200 records. This way, you can limit the subfile from loading an excessive number of records, which can severely impact the response time for the program, and can adversely affect the system as a whole. If you do not place a resulting indicator on the write statement, it is possible to load 9,999 records into the subfile. If your program attempts to load more than 9,999 records, an error is issued stating that you tried to load more than the maximum possible subfile records.

Coding Considerations For a Load-All Subfile

To code a load-all subfile, you should set the subfile size (SFLSIZ) to the normal amount of records that could be loaded. For instance, if your freight master file only has 30 records, and this file is rarely ever added to, then a value of 30 for the subfile size would be sufficient. However, if the file expands past 30 records, the subfile will automatically extend itself to contain more records. You do not want the value of the subfile size to be too small. If you normally load 50 records into a subfile program, then you should set the SFLSIZ to 50, If you set it to 10, the subfile will automatically extend itself. But this extension method costs some CPU time to handle, and the subfile records that are located beyond the initial 10 are not contiguous with the initial 10. This all adds up to slightly slower performance. So you want to size the subfile at a good logical level.

You do not want to set the subfile size value too high either, because space is allocated in main memory to handle the number of records in the subfile. If you set the subfile size to 1000, but never load more than 30 records in the subfile, there will be areas in memory that contain 970 blank subfile records. Since this memory is associated with a job's PAG (process access group), it can adversely affect the job's overall performance if the size value is set too high.

The value of subfile page (SFLPAG) should be set to the number of records that you want to appear on the screen at one time.

For a load-all subfile, the subfile size must be greater than the subfile page.

Common Uses For Load-All Subfiles

Load-all subfiles are the subfile of choice if the number of records to be loaded are minimal. You may ask, "What is minimal?" That really depends on the size of your AS/400. The user must wait while records are being loaded before receiving the display on the screen, so you want to keep this wait time to a minimum. The amount of time a user has to wait for the display to come up is directly related to the number of records that are loaded into the subfile.

You must consider if the number of records loaded will change significantly over time. Your subfile may only be loading about 20 records now, but by this time next year, the application may be loading 300. As applications and businesses mature, the amount of data that is processed goes up. This program might initially perform well as a load-all subfile, but the wait time for the user may continue to grow day after day until it gets unbearable.

A freight code inquiry would be a good candidate for a load-all subfile. Most businesses only have, at most, a few ways to ship a product. Even if there are 80 different shipping companies you work with, it is still a good candidate for a load-all subfile. Also, over time, you will probably add a very few, if any, new shipping companies to your file.

Another good candidate would be a state file inquiry. Since there are 50 states, you would load 50 records into your subfile. As the number of states is not likely to change in the near future, the state file inquiry would be an excellent candidate for the load-all method now, and for some time into the future.

Another possible candidate for a load-all subfile would be an invoice file inquiry program. Suppose you enter a customer number and would like a subfile to load all invoices for that customer. If the most invoices you have for any customer is less than 100, this program would be a candidate for a load-all subfile. After all, you would not be loading all invoices, you would only load all invoices for that specific customer. If you want to work on another customer's invoices, you could clear the subfile and load all invoices for the next customer. Clearing a subfile will be discussed later in this book.

However, you need to think about what could happen over time. You need to consider what may happen in a year. Is it possible that a lot of customers may end

up having several hundred invoices. If so, the load time for the invoices will cause the user to wait a long time before being able to view the data. Always think about the future when you design your applications.

Performance Considerations

If a load-all subfile is properly designed, the performance of your application will be good. Once the subfile is loaded, rolling through the subfile data is virtually instantaneous. No further disk accesses are needed because all records are in memory.

Improperly using the load-all subfile technique can have a severe performance impact for not just the user, but the system as a whole. You must be prepared for the following:

1. The user must wait for the subfile to be loaded. The maximum wait time should not exceed a couple of seconds. Beyond that, it gets frustrating for the user, especially if it is a program that is used often.

2. Loading too many records at a time will tax the system. Loading records requires a large number of disk accesses to be performed if the data is not currently in main memory. Loading this many records also requires more CPU time than most basic transaction programs. Unlike a single record transaction, you are dealing with dozens or possibly hundreds of records. Since this program will probably be running at priority 20 with the rest of the interactive jobs, it could have a severe impact on the rest of the system while it is loading.

3. The response time data for your system may be a bit skewed. Even if loading the subfile has no affect on system performance, it will probably increase the average response time of your system. This increase is likely if you are loading a good number of records, but the information the user needs is almost always on the first screen or two.

When the load-all subfile technique is improperly used, there can be direct impact on the end user.

For example, it may take three seconds to load the subfile and only half the time the information the user needs is on the first screen. The other times, when the required information needed is on the second subfile page, the user must perform one roll forward, which takes .1 seconds.

If this program is run 500 times in the day, the average response time for this job would be (500 * 3(load) + 250 * .1(roll))/750 or 2.03 seconds.

If you average 10,000 transactions a day, which most small systems do, and you have an average response time of 1 second, adding the following program to your system would cause a difference in the average response time.

The difference in system response times could be calculated as:

(10000 * 1 + 750 * 2.03)/10750 = 1.1 seconds. This one program causes the average response time for the system to increase by 10%. Although the response time for the system is higher, the actual response time that everyone individually experiences remains the same.

On the flip side, a well designed load-all subfile may cause the average response time for the system to drop. Now, use the above scenario with the exception that the user must roll an average of 5 times to see the data they need. The average response time for the program would be:

(500 * 3 + 2500 * .1)/3000 = .58 seconds

The system response time would then be:

(10000 * 1 + 3000 * .58) / 13000 =.9 seconds

Again, everyone on the system experiences the same response time, but by adding the program into the system, the overall response time drops. If this program is even more highly used, the response time for the system would continue to drop.

Changing the Subfile Page and Size

Now, change the look and characteristics of the program a bit by manipulating the subfile page and size values. For example, change the subfile page down to 5 and the subfile size to 100.

The impact of these changes are shown in the following source members for SFD220, SFR220, SFD221 and SFR221:

```
A*============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD220) SRCFILE(XXX/QDDSSRC)
A*
A*============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03 'Exit')
A          R SFD220A
A                                       CF04(04 'Call Search Program')
A                                  1  3DATE
A                                       EDTCDE(Y)
```

```
A                                    1 23'Customer Master Maintenance'
A                                        DSPATR(HI)
A                                    2  3TIME
A                                    1 71USER
A                                    2 71'SFD220A'
A                                   23  2'F3-Exit   F4-Customer Search'
A                                        DSPATR(HI)
A                                    5  1'Customer Number....:'
A                                        COLOR(BLU)
A                                    7  1'Customer Name......:'
A                                        COLOR(BLU)
A                                    9  1'Customer Address...:'
A                                        COLOR(BLU)
A                                   11  1'Customer City......:'
A                                        COLOR(BLU)
A                                   13  1'Customer State.....:'
A                                        COLOR(BLU)
A                                   15  1'Customer Zip.......:'
A                                        COLOR(BLU)
A                                   17  1'Phone Number.......:'
A                                        COLOR(BLU)
A            CUSNO     R     Y  B  5 23REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                        COLOR(WHT)
A                                        CHECK(ER)
A                                        CHECK(FE)
A                                        CHECK(RB)
A                                   24  1'Customer Number Not Found In Custo-
A                                        mer Master File-Key In A Valid Numb-
A                                        er'
A                                        COLOR(RED)
A                                        DSPATR(BL)
A N99                                    DSPATR(ND)
A          R SFD220B
A                                        CLRL(*NO)
A                                    2 71'SFD220B'
A            CUSNO     R        O  5 23REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                        COLOR(PNK)
A            CUSNM     R        B  7 22REFFLD(RCUST/CUSNM *LIBL/CUSTOMER)
A                                        COLOR(WHT)
A            CUSAD     R        B  9 22REFFLD(RCUST/CUSAD *LIBL/CUSTOMER)
A                                        COLOR(WHT)
A            CUSCT     R        B 11 22REFFLD(RCUST/CUSCT *LIBL/CUSTOMER)
A                                        COLOR(WHT)
A            CUSST     R        B 13 22REFFLD(RCUST/CUSST *LIBL/CUSTOMER)
A                                        COLOR(WHT)
A            CUSCP     R        B 15 22REFFLD(RCUST/CUSCP *LIBL/CUSTOMER)
A                                        COLOR(WHT)
A            CUSPH     R        B 17 22REFFLD(RCUST/CUSPH *LIBL/CUSTOMER)
A                                        COLOR(WHT)
A                                        EDTWRD('   -   -    ')
A                                   23  1'F3-Exit                          -
A                                           '
A                                        COLOR(WHT)
A                                   24  1'                                  -
A                                                                           -
A                                                   '
```

```
F*=============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR220) SRCFILE(XXX/QRPGSRC)
F*
F*=============================================================
FSFD220  CF  E                    WORKSTN
F                                            KINFDS INFDS
FCUSTOMERIF  E           K        DISK
IINFDS       DS
I                                     370 371 LINPOS
I            DS
I                                  B   1   20@LIN
I                                      2   2 @L
I            DS
I                                  B   1   20@POS
I                                      2   2 @P
C*
C                     EXFMTSFD220A
C                     EXSR @CMD
C*
C           CUSNO     CHAINCUSTOMER             99
C*
C           *IN99     IFEQ *OFF
C                     EXFMTSFD220B
C                     EXSR @CMD
C                     END
C*
C*********************************************************
C*    @ C M D   S U B R O U T I N E                      *
C*                                                       *
C*    Executed after every EXFMT or display read op code.  *
C*    Used to process command keys.                      *
C*********************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, seton LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C* If F4 was pressed, call program SFR221.
C*
C           *IN04     IFEQ *ON
C*
C* Convert the hex values of the cursor line/position into
C* binary, then to decimal.
C*
C                     MOVE *OFF      *IN04
C                     MOVE *ZEROS    @LIN
C                     MOVE *ZEROS    @POS
C                     MOVELLINPOS    @L
```

```
C                     MOVE LINPOS    @P
C                     Z-ADD@LIN      @SL     30
C                     Z-ADD@POS      @SP     30
C*
C                     CALL 'SFR221'
C                     PARM           @SL
C                     PARM           @SP
C*
C                     END
C*
C                     ENDSR


A*============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD221) SRCFILE(XXX/QDDSSRC)
A*
A*============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03)
A          R ASSUME
A                                       ASSUME
A                                  24  2' '
A          R SFD221A                    SFL
A            CUSNO     R        0  2  1REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                       COLOR(PNK)
A            CUSNM     R        0  2  9REFFLD(RCUST/CUSNM *LIBL/CUSTOMER)
A                                       COLOR(PNK)
A          R SFD221B                    SFLCTL(SFD221A)
A                                       SFLSIZ(0100)
A                                       SFLPAG(0005)
A                                       WINDOW(WINDOW)
A                                       OVERLAY
A  30                                   SFLDSP
A                                       SFLDSPCTL
A                                   1  1'Number'
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A                                   1  9'Customer Name                  '
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A          R WINDOW
A                                       WINDOW(&@SL &@SP 10 40)
A                                       WDWBORDER((*COLOR BLU) (*DSPATR RI)-
A                                        (*CHAR '        '))
A            @SL            3S 0P
A            @SP            3S 0P
A                                   9  1'F3-Exit'
A                                       DSPATR(HI)


F*============================================================
F* To compile:
F*
```

```
F*      CRTRPGPGM  PGM(XXX/SFR221) SRCFILE(XXX/QRPGSRC)
F*
F*==============================================================
FCUSTOMERIF  E           K        DISK
FSFD221  CF  E                    WORKSTN
F                                        @RRN  KSFILE SFD221A
C*
C                     WRITEWINDOW
C                     EXFMTSFD221B
C                     EXSR @CMD
C*
C*********************************************************
C*    @ C M D   S U B R O U T I N E                      *
C*                                                       *
C*    Executed after every EXFMT or display read op code.  *
C*    Used to process command keys.                      *
C*********************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, seton LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C                     ENDSR
C*
C*********************************************************
C*    * I N Z S R   S U B R O U T I N E                  *
C*                                                       *
C*    Retrieve the starting line and position for the    *
C*    window. Check to make sure that these values are   *
C*    valid. If not, put the closest valid number in.    *
C*********************************************************
C*
C           *INZSR    BEGSR
C*
C           *ENTRY    PLIST
C                     PARM           @SL
C                     PARM           @SP
C*
C                     Z-ADD0         @RRN    50
C*
C*  If the start line is greater than 12, the window will not
C*  fit on the display. Reset the line to 12.
C*
C           @SL       IFGT 12
C                     Z-ADD12        @SL
C                     END
C*
C*  If the start position is greater than 36, the window will not
C*  fit on the display. Reset the position to 36.
C*
C           @SP       IFGT 36
C                     Z-ADD36        @SP
```

```
C                     END
C*
C*   Nothing can be displayed from position 1,1 on the display.
C*   If the position is 1,1, reset it to 1,2. Also, check and
C*   make sure that the start line or start position is not zero.
C*
C           @SL       IFLE 1
C                     Z-ADD1         @SL
C           @SP       IFLE 1
C                     Z-ADD2         @SP
C                     END
C                     END
C*
C           @SP       IFLT 1
C                     Z-ADD1         @SP
C                     END
C*
C*  Load the subfile
C*
C                     EXSR @LOAD
C*
C                     ENDSR
C*
C*********************************************************
C*    @ L O A D     S U B R O U T I N E                  *
C*                                                       *
C*    This subroutine will read the customer file and    *
C*    load the subfile records. Since this is a load all *
C*    subfile program, all records in the customer file  *
C*    will be read and loaded.                           *
C*********************************************************
C*
C           @LOAD     BEGSR
C*
C*  Perform this loop until you reach the end of
C*  the Customer file or the maximum subfile records
C*  allowed. Both cases would turn on indicator 99.
C*
C           *IN99     DOWNE*ON
C                     READ CUSTOMER                 99
C*
C*  If indicator 99 is on, then leave this do loop.
C*
C           *IN99     IFEQ *ON
C                     LEAVE
C                     END
C*
C*  Add 1 to the subfile relative record number. If you
C*  reach the maximum allowable subfile records, indicator
C*  99 will turn on.
C*
C                     ADD  1         @RRN
C                     WRITESFD221A                  99
C                     END
C*
```

```
C* If you have loaded at least one record into the subfile,
C* turn on indicator 30. This indicator conditions the
C* SFLDSP keyword. If you have loaded at least one record
C* into the subfile, it will be all right to display the subfile
C* format. If you did not load any records into the subfile, you
C* would not turn on indicator 30. Attempting to display
C* an empty subfile would result in an error being issued.
C*
C           @RRN      IFGT 0
C                     MOVE *ON       *IN30
C                     END
C*
C                     ENDSR
```

Now execute program SFR220. Press F4 to call program SFR221.

You should get a screen like this:

```
  xx/xx/xx            Customer Master Maintenance                    QPGMR
  xx:xx:xx                                                           SFD220A


Customer Number....:  ..........................................
                      : Number  Customer Name                    :
Customer Name......:  : 00001   David's Computers and Bait Shop  :
                      : 00002   AS/400s Are Us                   :
Customer Address...:  : 00003   Tennessee Minicomputer Sales     :
                      : 00004   PCs For Less                     :
Customer City......:  : 00005   Mid-South Computer Sales         :
                      :                                          :
Customer State.....:  :                                          :
                      : F3-Exit                                  :
Customer Zip.......:  :                                          :
                      :..........................................:
Phone Number.......:




 F3-Exit   F4-Customer Search
```

Notice that this time only five records are displayed on the screen at a time. Roll forward and notice how many records are in the subfile. The subfile load process did not end after seven records this time, but ended after 100 were loaded, which happens to be the subfile size.

Subfile End Message

Now you can start adding some tangible functions. First, there is nothing currently being displayed on the screen specifying if there are more records in the subfile. If on a roll forward there were no more records, you could add a message saying that the roll forward key was invalid. You can also show whether or not there are more records on any screen. You can indicate that there are more records in two ways. The first is to allow a plus symbol "+" to be displayed if there are more records, and no symbol if there are no more records. You can do this by making a single line change in the subfile DDS and in the RPG program. Look at the source code for SFD230, SFR230, SFD231, and SFR231:

```
A*==============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD230) SRCFILE(XXX/QDDSSRC)
A*
A*==============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03 'Exit')
A          R SFD230A
A                                       CF04(04 'Call Search Program')
A                                   1  3DATE
A                                       EDTCDE(Y)
A                                   1 23'Customer Master Maintenance'
A                                       DSPATR(HI)
A                                   2  3TIME
A                                   1 71USER
A                                   2 71'SFD230A'
A                                  23  2'F3-Exit   F4-Customer Search'
A                                       DSPATR(HI)
A                                   5  1'Customer Number....:'
A                                       COLOR(BLU)
A                                   7  1'Customer Name......:'
A                                       COLOR(BLU)
A                                   9  1'Customer Address...:'
A                                       COLOR(BLU)
A                                  11  1'Customer City......:'
A                                       COLOR(BLU)
A                                  13  1'Customer State.....:'
A                                       COLOR(BLU)
A                                  15  1'Customer Zip.......:'
A                                       COLOR(BLU)
A                                  17  1'Phone Number.......:'
A                                       COLOR(BLU)
A            CUSNO     R     Y  B  5 23REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A                                       CHECK(ER)
A                                       CHECK(FE)
A                                       CHECK(RB)
A                                  24  1'Customer Number Not Found In Custo-
A                                       mer Master File-Key In A Valid Numb-
A                                       er'
```

```
A                                       COLOR(RED)
A                                       DSPATR(BL)
A N99                                   DSPATR(ND)
A          R SFD230B
A                                       CLRL(*NO)
A                                  2 71'SFD230B'
A            CUSNO     R        O  5 23REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                       COLOR(PNK)
A            CUSNM     R        B  7 22REFFLD(RCUST/CUSNM *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A            CUSAD     R        B  9 22REFFLD(RCUST/CUSAD *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A            CUSCT     R        B 11 22REFFLD(RCUST/CUSCT *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A            CUSST     R        B 13 22REFFLD(RCUST/CUSST *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A            CUSCP     R        B 15 22REFFLD(RCUST/CUSCP *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A                                 23  1'F3-Exit                          -
A                                            '
A                                       COLOR(WHT)
A            CUSPH     R        B 17 22REFFLD(RCUST/CUSPH *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A                                       EDTWRD('   -   -    ')
A                                 24  1'                                  -
A                                                                         -
A                                                 '


F*==============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR230) SRCFILE(XXX/QRPGSRC)
F*
F*==============================================================
FSFD230  CF  E                    WORKSTN
F                                            KINFDS INFDS
FCUSTOMERIF  E           K        DISK
IINFDS       DS
I                                       370 371 LINPOS
I            DS
I                                  B   1   20@LIN
I                                      2   2 @L
I            DS
I                                  B   1   20@POS
I                                      2   2 @P
C*
C                     EXFMTSFD230A
C                     EXSR @CMD
C*
C           CUSNO     CHAINCUSTOMER             99
C*
C           *IN99     IFEQ *OFF
C                     EXFMTSFD230B
C                     EXSR @CMD
C                     END
```

```
C*
C*************************************************************
C*    @ C M D   S U B R O U T I N E                          *
C*                                                           *
C*    Executed after every EXFMT or display read op code.    *
C*    Used to process command keys.                          *
C*************************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, seton LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C* If F4 was pressed, call program SFR231.
C*
C           *IN04     IFEQ *ON
C*
C* Convert the hex values of the cursor line/position into
C* binary, then to decimal.
C*
C                     MOVE *OFF      *IN04
C                     MOVE *ZEROS    @LIN
C                     MOVE *ZEROS    @POS
C                     MOVELLINPOS    @L
C                     MOVE LINPOS    @P
C                     Z-ADD@LIN      @SL     30
C                     Z-ADD@POS      @SP     30
C*
C                     CALL 'SFR231'
C                     PARM           @SL
C                     PARM           @SP
C*
C                     END
C*
C                     ENDSR


A*==============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD231) SRCFILE(XXX/QDDSSRC)
A*
A*==============================================================
A                                      DSPSIZ(24 80 *DS3)
A                                      CF03(03)
A          R ASSUME
A                                      ASSUME
A                                 24  2' '
A          R SFD231A                   SFL
A            CUSNO     R        0  2  1REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                      COLOR(PNK)
A            CUSNM     R        0  2  9REFFLD(RCUST/CUSNM *LIBL/CUSTOMER)
```

```
A                                       COLOR(PNK)
A          R SFD231B                    SFLCTL(SFD231A)
A                                       OVERLAY
A  30                                   SFLDSP
A                                       SFLDSPCTL
A  35                                   SFLEND
A                                       SFLSIZ(0100)
A                                       SFLPAG(0005)
A                                       WINDOW(WINDOW)
A                                   1  1'Number'
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A                                   1  9'Customer Name                  '
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A          R WINDOW
A                                       WINDOW(&@SL &@SP 10 40)
A                                       WDWBORDER((*COLOR BLU) (*DSPATR RI)-
A                                        (*CHAR '        '))
A            @SL            3S 0P
A            @SP            3S 0P
A                                   9  1'F3-Exit'
A                                       DSPATR(HI)


F*=============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR231) SRCFILE(XXX/QRPGSRC)
F*
F*=============================================================
FCUSTOMERIF  E           K        DISK
FSFD231  CF  E                    WORKSTN
F                                       @RRN  KSFILE SFD231A
C*
C                     WRITEWINDOW
C                     EXFMTSFD231B
C                     EXSR @CMD
C*
C*********************************************************
C*    @ C M D   S U B R O U T I N E                      *
C*                                                       *
C*    Executed after every EXFMT or display read op code.  *
C*    Used to process command keys.                      *
C*********************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, seton LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C                     ENDSR
```

```
C*
C*************************************************************
C*    * I N Z S R   S U B R O U T I N E                      *
C*                                                           *
C*    Retrieve the starting line and position for the        *
C*    window. Check to make sure that these values are       *
C*    valid. If not, put the closest valid number in.        *
C*************************************************************
C*
C           *INZSR    BEGSR
C*
C           *ENTRY    PLIST
C                     PARM           @SL
C                     PARM           @SP
C*
C                     Z-ADD0         @RRN    50
C*
C*  If the start line is greater than 12, the window will not
C*  fit on the display. Reset the line to 12.
C*
C           @SL       IFGT 12
C                     Z-ADD12        @SL
C                     END
C*
C*  If the start position is greater than 36, The window will not
C*  fit on the display. Reset the position to 36.
C*
C           @SP       IFGT 36
C                     Z-ADD36        @SP
C                     END
C*
C*   Nothing can be displayed from position 1,1 on the display.
C*   If the position is 1,1, reset it to 1,2. Also, check and
C*   make sure that the start line or start position is not zero.
C*
C           @SL       IFLE 1
C                     Z-ADD1         @SL
C           @SP       IFLE 1
C                     Z-ADD2         @SP
C                     END
C                     END
C*
C           @SP       IFLT 1
C                     Z-ADD1         @SP
C                     END
C*
C*  Load the subfile
C*
C                     EXSR @LOAD
C*
C                     ENDSR
C*
C*************************************************************
C*    @ L O A D     S U B R O U T I N E                      *
C*                                                           *
C*    This subroutine will read the customer file and        *
C*    load the subfile records. Since this is a load all     *
```

```
C*    subfile program, all records in the customer file     *
C*    will be read and loaded.                              *
C*********************************************************
C*
C           @LOAD     BEGSR
C*
C*  Perform this loop until you reach the end of
C*  the Customer file or the maximum
C*  subfile records allowed. Both cases would turn on
C*  indicator 99.
C*
C           *IN99     DOWNE*ON
C                     READ CUSTOMER                 99
C*
C*  If indicator 99 is on, then leave this do loop.
C*
C           *IN99     IFEQ *ON
C                     LEAVE
C                     END
C*
C*  Add 1 to our subfile relative record number. If you
C*  reach the maximum allowable subfile records, indicator
C*  99 will turn on.
C*
C                     ADD  1         @RRN
C                     WRITESFD231A                  99
C                     END
C*
C* If you have loaded at least one record into the subfile,
C* turn on inducator 30. This indicator conditions the
C* SFLDS0 keyword. If you have loaded at least one record
C* into the subfile, it will be all right to display the subfile
C* format. If you did not load any records into the subfile, you
C* would not turn on inidicator 30. Attempting to display
C* an empty subfile would result in an error being issued.
C*
C           @RRN      IFGT 0
C                     MOVE *ON       *IN30
C                     END
C*
C* Since you have loaded all records into the subfile, allow the
C* SFLEND keyword to display the end message.
C*
C                     MOVE *ON       *IN35
C*
C                     ENDSR
```

As you can see in display file SFD231, the following line has been added to the subfile control format:

```
A  35                                   SFLEND
```

This statement could also be written :

```
A  35                                   SFLEND(*PLUS)
```

By specifying the Subfile End (SFLEND) keyword, you activate the "+" symbol to show up on the last line of the subfile. If indicator 35 is on indicating you have reached the end of the subfile, the "+" symbol disappears. If indicator 35 is not on, the "+" symbol is displayed even if you are at the end of the subfile. So the sole purpose of indicator 35 is to allow the subfile to show if you have reached the end of the subfile. It does not activate the keyword. As long as the keyword is specified, it is activated. The indicator activates to show that you have reached end of subfile. For a load-all subfile, you will set this indicator on and leave it on, because if you reach end of subfile, you have no more records to display. This will not be the case with other subfiles. In expanding and single-page subfile load method subfiles, the timing of when to turn on this indicator is important.

A change is also necessary in program SFR231. The following line of code was added to the RPG program.

```
C                     MOVE *ON       *IN35
```

It doesn't matter where you add this statement, as long as indicator 35 is on before you perform the EXFMT statement for the subfile control format.

After making these changes, you can see the effect they have by running program SFR230. You should get a screen like this:

```
  xx/xx/xx            Customer Master Maintenance                    QPGMR
  xx:xx:xx                                                           SFD230A


Customer Number....:  ..........................................
                      : Number  Customer Name                    :
Customer Name......:  : 00001   David's Computers and Bait Shop  :
                      : 00002   AS/400s Are Us                   :
Customer Address...:  : 00003   Tennessee Minicomputer Sales     :
                      : 00004   PCs For Less                     :
Customer City......:  : 00005   Mid-South Computer Sales       + :
                      :                                          :
Customer State.....:  :                                          :
                      : F3-Exit                                  :
Customer Zip.......:  :                                          :
                      :..........................................:
Phone Number.......:




 F3-Exit   F4-Customer Search
```

Now a "+" symbol is displayed on the last line of the subfile if more records can be displayed. The "+" is not displayed if you are on the last page of the subfile. If you roll to the end of the subfile, your display would look like this:

```
  xx/xx/xx            Customer Master Maintenance                    QPGMR
  xx:xx:xx                                                           SFD230A


Customer Number....:  ..........................................
                      : Number  Customer Name                    :
Customer Name......:  : 00054   Open Systems Inc.                 :
                      : 00087   SemiCondutor City                 :
Customer Address...:  : 00099   Hard Drive Warehouse              :
                      : 00386   Memory Systems Inc.               :
Customer City......:  :                                           :
                      :                                           :
Customer State.....:  :                                           :
                      : F3-Exit                                   :
Customer Zip.......:  :                                           :
                      :...........................................:
Phone Number.......:




 F3-Exit   F4-Customer Search
```

Subfile End Option

Until V2R1, the plus sign was the only way you had to show that there were more records. But, in some of IBM's programs that appear to be subfiles, IBM used a "MORE..." message to show that there were more records that could be displayed. IBM's programs are not really subfiles, but they looked like subfiles, so programmers thought they should be able to display "MORE" too.

So, IBM made a change that allows you to display the "MORE..." message if there are more records in the subfile. There is one catch. The "MORE..." message resides on a line below the last subfile record, whereas the "+" symbol was actually displayed on the last subfile record. So if you have a window subfile like the one in the example, you will need to make sure that you have an extra line available. This modification was made in program SFR221, when the subfile page was changed to 5 and gave you one extra line inside the window that you were not using. You will now use it for displaying the "MORE..." symbol.

Look at the source code for SFD240, SFR240, SFD241, and SFR241:

```
A*==============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD240) SRCFILE(XXX/QDDSSRC)
A*
A*==============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03 'Exit')
A          R SFD240A
A                                       CF04(04 'Call Search Program')
A                                   1  3DATE
A                                       EDTCDE(Y)
A                                   1 23'Customer Master Maintenance'
A                                       DSPATR(HI)
A                                   2  3TIME
A                                   1 71USER
A                                   2 71'SFD240A'
A                                  23  2'F3-Exit   F4-Customer Search'
A                                       DSPATR(HI)
A                                   5  1'Customer Number....:'
A                                       COLOR(BLU)
A                                   7  1'Customer Name......:'
A                                       COLOR(BLU)
A                                   9  1'Customer Address...:'
A                                       COLOR(BLU)
A                                  11  1'Customer City......:'
A                                       COLOR(BLU)
A                                  13  1'Customer State.....:'
A                                       COLOR(BLU)
A                                  15  1'Customer Zip.......:'
A                                       COLOR(BLU)
A                                  17  1'Phone Number.......:'
A                                       COLOR(BLU)
A            CUSNO     R     Y  B  5 23REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A                                       CHECK(ER)
A                                       CHECK(FE)
A                                       CHECK(RB)
A                                  24  1'Customer Number Not Found In Custo-
A                                       mer Master File-Key In A Valid Numb-
A                                       er'
A                                       COLOR(RED)
A                                       DSPATR(BL)
A N99                                   DSPATR(ND)
A          R SFD240B
A                                       CLRL(*NO)
A                                   2 71'SFD240B'
A            CUSNO     R        O  5 23REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                       COLOR(PNK)
A            CUSNM     R        B  7 22REFFLD(RCUST/CUSNM *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A            CUSAD     R        B  9 22REFFLD(RCUST/CUSAD *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A            CUSCT     R        B 11 22REFFLD(RCUST/CUSCT *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A            CUSST     R        B 13 22REFFLD(RCUST/CUSST *LIBL/CUSTOMER)
A                                       COLOR(WHT)
```

```
A            CUSCP     R        B 15 22REFFLD(RCUST/CUSCP *LIBL/CUSTOMER)
A                                     COLOR(WHT)
A                                23  1'F3-Exit                          -
A                                        '
A                                     COLOR(WHT)
A            CUSPH     R        B 17 22REFFLD(RCUST/CUSPH *LIBL/CUSTOMER)
A                                     COLOR(WHT)
A                                     EDTWRD('   -   -    ')
A                                24  1'                                  -
A                                                                        -
A                                            '


F*============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR240) SRCFILE(XXX/QRPGSRC)
F*
F*============================================================
FSFD240  CF  E                    WORKSTN
F                                            KINFDS INFDS
FCUSTOMERIF  E           K        DISK
IINFDS       DS
I                                     370 371 LINPOS
I            DS
I                                  B   1   20@LIN
I                                      2   2 @L
I            DS
I                                  B   1   20@POS
I                                      2   2 @P
C*
C                     EXFMTSFD240A
C                     EXSR @CMD
C*
C           CUSNO     CHAINCUSTOMER            99
C*
C           *IN99     IFEQ *OFF
C                     EXFMTSFD240B
C                     EXSR @CMD
C                     END
C*
C*********************************************************
C*    @ C M D   S U B R O U T I N E                      *
C*                                                       *
C*    Executed after every EXFMT or display read op code.  *
C*    Used to process command keys.                      *
C*********************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, seton LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
```

```
C*
C* If F4 was pressed, call program SFR241.
C*
C           *IN04     IFEQ *ON
C*
C* Convert the hex values of the cursor line/position into
C* binary, then to decimal.
C*
C                     MOVE *OFF      *IN04
C                     MOVE *ZEROS    @LIN
C                     MOVE *ZEROS    @POS
C                     MOVELLINPOS    @L
C                     MOVE LINPOS    @P
C                     Z-ADD@LIN      @SL     30
C                     Z-ADD@POS      @SP     30
C*
C                     CALL 'SFR241'
C                     PARM           @SL
C                     PARM           @SP
C*
C                     END
C*
C                     ENDSR


A*============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD241) SRCFILE(XXX/QDDSSRC)
A*
A*============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03)
A          R ASSUME
A                                       ASSUME
A                                  24  2' '
A          R SFD241A                    SFL
A            CUSNO     R        0  2  1REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                       COLOR(PNK)
A            CUSNM     R        0  2  9REFFLD(RCUST/CUSNM *LIBL/CUSTOMER)
A                                       COLOR(PNK)
A          R SFD241B                    SFLCTL(SFD241A)
A                                       OVERLAY
A  30                                   SFLDSP
A                                       SFLDSPCTL
A  35                                   SFLEND(*MORE)
A                                       SFLSIZ(0100)
A                                       SFLPAG(0005)
A                                       WINDOW(WINDOW)
A                                   1  1'Number'
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A                                   1  9'Customer Name                  '
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A          R WINDOW
```

```
A                                       WINDOW(&@SL &@SP 10 40)
A                                       WDWBORDER((*COLOR BLU) (*DSPATR RI)-
A                                        (*CHAR '        '))
A            @SL            3S 0P
A            @SP            3S 0P
A                                 9  1'F3-Exit'
A                                       DSPATR(HI)


F*============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR241) SRCFILE(XXX/QRPGSRC)
F*
F*============================================================
FCUSTOMERIF  E           K        DISK
FSFD241  CF  E                    WORKSTN
F                                        @RRN  KSFILE SFD241A
C*
C                     WRITEWINDOW
C                     EXFMTSFD241B
C                     EXSR @CMD
C*
C*********************************************************
C*    @ C M D   S U B R O U T I N E                      *
C*                                                       *
C*    Executed after every EXFMT or display read op code.  *
C*    Used to process command keys.                      *
C*********************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, seton LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C                     ENDSR
C*
C*********************************************************
C*    * I N Z S R   S U B R O U T I N E                  *
C*                                                       *
C*    Retrieve the starting line and position for the    *
C*    window. Check to make sure that these values are   *
C*    valid. If not, put the closest valid number in.    *
C*********************************************************
C*
C           *INZSR    BEGSR
C*
C           *ENTRY    PLIST
C                     PARM           @SL
C                     PARM           @SP
C*
C                     Z-ADD0         @RRN    50
```

```
C*
C*  If the start line is greater than 12, the window will not
C*  fit on the display. Reset the line to 12.
C*
C           @SL       IFGT 12
C                     Z-ADD12        @SL
C                     END
C*
C*  If the start position is greater than 36, the window will not
C*  fit on the display. Reset the position to 36.
C*
C           @SP       IFGT 36
C                     Z-ADD36        @SP
C                     END
C*
C*   Nothing can be displayed from position 1,1 on the display.
C*   If the position is 1,1, reset it to 1,2. Also, check and
C*   make sure that the start line or start position is not zero.
C*
C           @SL       IFLE 1
C           @SP       ANDLE1
C                     Z-ADD1         @SL
C                     Z-ADD2         @SP
C                     END
C*
C*  Load the subfile
C*
C                     EXSR @LOAD
C*
C                     ENDSR
C*
C*************************************************************
C*    @ L O A D     S U B R O U T I N E                     *
C*                                                            *
C*    This subroutine will read the customer file and         *
C*    load the subfile records. Since this is a load all      *
C*    subfile program, all records in the customer file       *
C*    will be read and loaded.                                *
C*************************************************************
C*
C           @LOAD     BEGSR
C*
C*  Perform this loop until you reach the end of
C*  the Customer file or the maximum subfile records
C*  allowed. Both cases would turn on indicator 99.
C*
C           *IN99     DOWNE*ON
C                     READ CUSTOMER                 99
C*
C*  If indicator 99 is on, then leave this do loop.
C*
C           *IN99     IFEQ *ON
C                     LEAVE
C                     END
C*
C*  Add 1 to the subfile relative record number. If you
C*  reach the maximum allowable subfile records, indicator
```

```
C*  99 will turn on.
C*
C                     ADD  1         @RRN
C                     WRITESFD241A                  99
C                     END
C*
C* If you have loaded at least one record into the subfile,
C* turn on indicator 30. This indicator conditions the
C* SFLDSP keyword. If you have loaded at least one record
C* into the subfile, it will be all right to display the subfile
C* format. If you did not load any records into the subfile, you
C* would not turn on indicator 30. Attempting to display
C* an empty subfile would result in an error being issued.
C*
C           @RRN      IFGT 0
C                     MOVE *ON       *IN30
C                     END
C*
C* Since you have loaded all records into the subfile, allow the
C* SFLEND keyword to display the end message.
C*
C                     MOVE *ON       *IN35
C*
C                     ENDSR
```

To allow the "MORE..." message to be displayed, you only need to change one line in the subfile DDS:

```
A  35                                   SFLEND
```

To:

```
A  35                                   SFLEND(*MORE)
```

You can see the effects by runnning SFR240. You will get a screen that looks like this:

```
  xx/xx/xx            Customer Master Maintenance                    QPGMR
  xx:xx:xx                                                           SFD240A


Customer Number....:  ..........................................
                      : Number  Customer Name                    :
Customer Name......:  : 00001   David's Computers and Bait Shop  :
                      : 00002   AS/400s Are Us                   :
Customer Address...:  : 00003   Tennessee Minicomputer Sales     :
                      : 00004   PCs For Less                     :
Customer City......:  : 00005   Mid-South Computer Sales         :
                      :                                  More... :
Customer State.....:  :                                          :
                      : F3-Exit                                  :
Customer Zip.......:  :                                          :
                      :..........................................:
Phone Number.......:




 F3-Exit   F4-Customer Search
```

Notice the "MORE..." message if there are more records that can be displayed. If you roll to the bottom of the subfile, your display would look like this:

```
  xx/xx/xx            Customer Master Maintenance                    QPGMR
  xx:xx:xx                                                           SFD240A


Customer Number....:  ..........................................
                      : Number  Customer Name                    :
Customer Name......:  : 00054   Open Systems Inc.                 :
                      : 00087   SemiCondutor City                 :
Customer Address...:  : 00099   Hard Drive Warehouse              :
                      : 00386   Memory Systems Inc.               :
Customer City......:  :                                           :
                      :                                    Bottom :
Customer State.....:  :                                           :
                      : F3-Exit                                   :
Customer Zip.......:  :                                           :
                      :...........................................:
Phone Number.......:




 F3-Exit   F4-Customer Search
```

Notice the "Bottom" message if the end of the subfile has been reached. These messages are much more user friendly than showing either a "+" or a " ".

SFLEND Summary

To summarize again the SFLEND keyword, as long as it is specified in DDS, you will get a "+" or the "MORE..." message at the end of your subfile, regardless of whether the controlling indicator is on or off. If the end of subfile has been reached, and the controlling indicator is off, then you will still receive the "+" or "MORE..." message. But if the controlling indicator is on, and you reach the end of subfile, the " " or "Bottom" message is displayed. Also, if this indicator is on and you have not reached the bottom of the subfile, the "+" or "MORE..." message is displayed. So in the load-all subfile, set on 35 immediately and leave it on. Whenever the subfile hits the last record, it will be allowed to display the " " or "Bottom" message.

Removing the Load Limitation

Now you are ready to make one final structural change to the subfile before getting really serious. In the previous examples, the number of records that were to be loaded to 100 was limited to 100. In this example, you will remove this restriction and allow the maximum number of records to be loaded. You will also change the subfile page to allow six records to be displayed and set the subfile size to seven. In doing this, you will need to modify the WINDOW characteristics to allow more lines since you are currently using all the available lines inside the window. You will also have to modify the RPG code window validation routine to check to see if the starting positions and line for the window is valid as the window size is being modified.

To make the changes in the RPG program to allow the maximum number of records to be loaded (9,999), the indicator is removed from the subfile write statement. Then you must add a routine to check if the relative record number for the subfile is 9,999. If it is, turn on indicator 99. This routine causes an exit from the load loop if you load 9,999 records into the subfile and prevents you from attempting to load more than the maximum number of subfile records allowed.

Now, if you arrive at the end of the file, or if the relative record number for the subfile is 9,999, you will leave the loop.

You can see these changes in SFR250, SFD250, SFR251, and SFD251:

```
F*==============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR250) SRCFILE(XXX/QRPGSRC)
F*
F*==============================================================
FSFD250  CF  E                    WORKSTN
F                                            KINFDS INFDS
FCUSTOMERIF  E           K        DISK
IINFDS       DS
I                                     370 371 LINPOS
I            DS
I                                   B   1   20@LIN
I                                       2   2 @L
I            DS
I                                   B   1   20@POS
I                                       2   2 @P
C*
C                     EXFMTSFD250A
C                     EXSR @CMD
C*
C           CUSNO     CHAINCUSTOMER             99
C*
C           *IN99     IFEQ *OFF
C                     EXFMTSFD250B
C                     EXSR @CMD
C                     END
```

```
C*
C*************************************************************
C*    @ C M D   S U B R O U T I N E                          *
C*                                                           *
C*    Executed after every EXFMT or display read op code.    *
C*    Used to process command keys.                          *
C*************************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, seton LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C* If F4 was pressed, call program SFR251.
C*
C           *IN04     IFEQ *ON
C                     MOVE *OFF      *IN04
C                     MOVE *ZEROS    @LIN
C                     MOVE *ZEROS    @POS
C                     MOVELLINPOS    @L
C                     MOVE LINPOS    @P
C                     Z-ADD@LIN      @SL     30
C                     Z-ADD@POS      @SP     30
C*
C                     CALL 'SFR251'
C                     PARM           @SL
C                     PARM           @SP
C*
C                     END
C*
C                     ENDSR


A*==============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD250) SRCFILE(XXX/QDDSSRC)
A*
A*==============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03 'Exit')
A          R SFD250A
A                                       CF04(04 'Call Search Program')
A                                  1  3DATE
A                                       EDTCDE(Y)
A                                  1 23'Customer Master Maintenance'
A                                       DSPATR(HI)
A                                  2  3TIME
A                                  1 71USER
A                                  2 71'SFD250A'
A                                 23  2'F3-Exit   F4-Customer Search'
A                                       DSPATR(HI)
```

```
A                                    5  1'Customer Number....:'
A                                       COLOR(BLU)
A                                    7  1'Customer Name......:'
A                                       COLOR(BLU)
A                                    9  1'Customer Address...:'
A                                       COLOR(BLU)
A                                   11  1'Customer City......:'
A                                       COLOR(BLU)
A                                   13  1'Customer State.....:'
A                                       COLOR(BLU)
A                                   15  1'Customer Zip.......:'
A                                       COLOR(BLU)
A                                   17  1'Phone Number.......:'
A                                       COLOR(BLU)
A            CUSNO     R     Y  B  5 23REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A                                       CHECK(ER)
A                                       CHECK(FE)
A                                       CHECK(RB)
A                                   24  1'Customer Number Not Found In Custo-
A                                       mer Master File-Key In A Valid Numb-
A                                       er'
A                                       COLOR(RED)
A                                       DSPATR(BL)
A N99                                   DSPATR(ND)
A          R SFD250B
A                                       CLRL(*NO)
A                                    2 71'SFD250B'
A            CUSNO     R        O  5 23REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                       COLOR(PNK)
A            CUSNM     R        B  7 22REFFLD(RCUST/CUSNM *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A            CUSAD     R        B  9 22REFFLD(RCUST/CUSAD *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A            CUSCT     R        B 11 22REFFLD(RCUST/CUSCT *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A            CUSST     R        B 13 22REFFLD(RCUST/CUSST *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A            CUSCP     R        B 15 22REFFLD(RCUST/CUSCP *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A                                   23  1'F3-Exit                          -
A                                          '
A                                       COLOR(WHT)
A            CUSPH     R        B 17 22REFFLD(RCUST/CUSPH *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A                                       EDTWRD('   -   -    ')
A                                   24  1'                                  -
A                                                                           -
A                                                  '


F*==============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR251) SRCFILE(XXX/QRPGSRC)
F*
```

```
F*=============================================================
FCUSTOMERIF  E           K        DISK
FSFD251  CF  E                    WORKSTN
F                                        @RRN  KSFILE SFD251A
C*
C                     WRITEWINDOW
C                     EXFMTSFD251B
C                     EXSR @CMD
C*
C*********************************************************
C*    @ C M D   S U B R O U T I N E                      *
C*                                                       *
C*    Executed after every EXFMT or display read op code.  *
C*    Used to process command keys.                      *
C*********************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, seton LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C                     ENDSR
C*
C*********************************************************
C*    * I N Z S R   S U B R O U T I N E                  *
C*                                                       *
C*    Retrieve the starting line and position for the    *
C*    window. Check to make sure that these values are   *
C*    valid. If not, put the closest valid number in.    *
C*********************************************************
C*
C           *INZSR    BEGSR
C*
C           *ENTRY    PLIST
C                     PARM           @SL
C                     PARM           @SP
C*
C                     Z-ADD0         @RRN    50
C*
C*  If the start line is greater than 12, the window will not
C*  fit on the display. Reset the line to 12.
C*
C           @SL       IFGT 12
C                     Z-ADD12        @SL
C                     END
C*
C*  If the start position is greater than 36, the window will not
C*  fit on the display. Reset the position to 36.
C*
C           @SP       IFGT 36
C                     Z-ADD36        @SP
C                     END
C*
```

```
C*   Nothing can be displayed from position 1,1 on the display.
C*   If the position is 1,1, reset it to 1,2. Also, check and
C*   make sure that the start line or start position is not zero.
C*
C           @SL       IFLE 1
C           @SP       ANDLE1
C                     Z-ADD1         @SL
C                     Z-ADD2         @SP
C                     END
C*
C*  Load the subfile
C*
C                     EXSR @LOAD
C*
C                     ENDSR
C*
C*************************************************************
C*    @ L O A D     S U B R O U T I N E                      *
C*                                                           *
C*    This subroutine will read the customer file and        *
C*    load the subfile records. Since this is a load all     *
C*    subfile program, all records in the customer file      *
C*    will be read and loaded.                               *
C*************************************************************
C*
C           @LOAD     BEGSR
C*
C*  Perform this loop until you reach the end of
C*  the Customer file.
C*
C           *IN99     DOWNE*ON
C                     READ CUSTOMER                 99
C*
C*  If indicator 99 is on, or if you have already loaded
C*  9999 subfile records, (the maximum allowed), then exit
C*  this loop.
C*
C           *IN99     IFEQ *ON
C           @RRN      OREQ 9999
C                     LEAVE
C                     END
C*
C*  Add 1 to the subfile relative record number.
C*
C                     ADD  1         @RRN
C                     WRITESFD251A
C                     END
C*
C* If you have loaded at least one record into the subfile,
C* turn on indicator 30. This indicator conditions the
C* SFLDSP keyword. If you have loaded at least one record
C* into the subfile, it will be all right to display the subfile
C* format. If you did not load any records into the subfile, you
C* would not turn on indicator 30. Attempting to display
C* an empty subfile would result in an error being issued.
C*
C           @RRN      IFGT 0
```

```
C                     MOVE *ON       *IN30
C                     END
C*
C* Since you have loaded all records into the subfile, allow the
C* SFLEND keyword to display the end message.
C*
C                     MOVE *ON       *IN35
C*
C                     ENDSR


A*============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD251) SRCFILE(XXX/QDDSSRC)
A*
A*============================================================
A                                      DSPSIZ(24 80 *DS3)
A                                      CF03(03)
A          R ASSUME
A                                      ASSUME
A                                 24  2' '
A          R SFD251A                   SFL
A            CUSNO     R        0  2  1REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                      COLOR(PNK)
A            CUSNM     R        0  2  9REFFLD(RCUST/CUSNM *LIBL/CUSTOMER)
A                                      COLOR(PNK)
A          R SFD251B                   SFLCTL(SFD251A)
A                                      OVERLAY
A  30                                  SFLDSP
A                                      SFLDSPCTL
A  35                                  SFLEND(*MORE)
A                                      SFLSIZ(0007)
A                                      SFLPAG(0006)
A                                      WINDOW(WINDOW)
A                                  1  1'Number'
A                                      COLOR(YLW)
A                                      DSPATR(UL)
A                                  1  9'Customer Name                  '
A                                      COLOR(YLW)
A                                      DSPATR(UL)
A          R WINDOW
A                                      WINDOW(&@SL &@SP 10 40)
A                                      WDWBORDER((*COLOR BLU) (*DSPATR RI)-
A                                       (*CHAR '        '))
A            @SL            3S OP
A            @SP            3S OP
A                                  9  1'F3-Exit'
A                                      DSPATR(HI)
```

You can view these changes by executing SFR250. Your screen will look like this:

```
  xx/xx/xx            Customer Master Maintenance                    QPGMR
  xx:xx:xx                                                           SFD250A


Customer Number....:  ..............................................
                      : Number  Customer Name                      :
Customer Name......:  : 00001   David's Computers and Bait Shop    :
                      : 00002   AS/400s Are Us                     :
Customer Address...:  : 00003   Tennessee Minicomputer Sales       :
                      : 00004   PCs For Less                       :
Customer City......:  : 00005   Mid-South Computer Sales           :
                      : 00010   Mini Computing Clearance           :
Customer State.....:  :                                   More...  :
                      : F3-Exit                                    :
Customer Zip.......:  :                                            :
                      :............................................:
Phone Number.......:




 F3-Exit   F4-Customer Search
```

Now you have six subfile records to a page. Use the roll forward key. Every record that was in the customer file is now in the subfile because the number of subfile records to load is unrestricted.

Functionality Added So Far

Notice in these previous examples that you did not have to add many, if any, lines of code to the RPG programs. Also note that the additions to the display file were extremely minimal. But these previous functionality changes are "cream puffs" compared to what you are about to do.

Multiple Line Subfile

Now it is time to get serious. Suppose a user put in a request to modify the customer inquiry subfile program. They would like for it to look the way it does now, but they would like to also have a command key available to show the state and phone number for each customer. When they press this command key, they would like the state and phone number for the customer to be displayed on a separate line on the display. If

this program were not a subfile program, you would be in trouble. You would have to add twelve more fields to the screen and a lot of additional code to the RPG program.

But, this *is* a subfile program, so you can make the requested changes by adding six lines of code to the display file. No changes will be necessary to the RPG program.

Make the following changes to the display file. Add the CUSST and CUSPH fields to the subfile format. Add them to the line below the customer number and name fields. The necessary changes have been made in SFD260, SFR260, SFD261, and SFR261:

```
A*============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD260) SRCFILE(XXX/QDDSSRC)
A*
A*============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03 'Exit')
A          R SFD260A
A                                       CF04(04 'Call Search Program')
A                                  1  3DATE
A                                       EDTCDE(Y)
A                                  1 23'Customer Master Maintenance'
A                                       DSPATR(HI)
A                                  2  3TIME
A                                  1 71USER
A                                  2 71'SFD260A'
A                                 23  2'F3-Exit   F4-Customer Search'
A                                       DSPATR(HI)
A                                  5  1'Customer Number....:'
A                                       COLOR(BLU)
A                                  7  1'Customer Name......:'
A                                       COLOR(BLU)
A                                  9  1'Customer Address...:'
A                                       COLOR(BLU)
A                                 11  1'Customer City......:'
A                                       COLOR(BLU)
A                                 13  1'Customer State.....:'
A                                       COLOR(BLU)
A                                 15  1'Customer Zip.......:'
A                                       COLOR(BLU)
A                                 17  1'Phone Number.......:'
A                                       COLOR(BLU)
A            CUSNO     R     Y  B  5 23REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A                                       CHECK(ER)
A                                       CHECK(FE)
A                                       CHECK(RB)
A                                 24  1'Customer Number Not Found In Custo-
A                                       mer Master File-Key In A Valid Numb-
A                                       er'
A                                       COLOR(RED)
A                                       DSPATR(BL)
A N99                                   DSPATR(ND)
```

```
A          R SFD260B
A                                     CLRL(*NO)
A                                  2 71'SFD260B'
A            CUSNO     R        O  5 23REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                     COLOR(PNK)
A            CUSNM     R        B  7 22REFFLD(RCUST/CUSNM *LIBL/CUSTOMER)
A                                     COLOR(WHT)
A            CUSAD     R        B  9 22REFFLD(RCUST/CUSAD *LIBL/CUSTOMER)
A                                     COLOR(WHT)
A            CUSCT     R        B 11 22REFFLD(RCUST/CUSCT *LIBL/CUSTOMER)
A                                     COLOR(WHT)
A            CUSST     R        B 13 22REFFLD(RCUST/CUSST *LIBL/CUSTOMER)
A                                     COLOR(WHT)
A            CUSCP     R        B 15 22REFFLD(RCUST/CUSCP *LIBL/CUSTOMER)
A                                     COLOR(WHT)
A                                 23  1'F3-Exit                          -
A                                        '
A                                     COLOR(WHT)
A            CUSPH     R        B 17 22REFFLD(RCUST/CUSPH *LIBL/CUSTOMER)
A                                     COLOR(WHT)
A                                     EDTWRD('   -   -    ')
A                                 24  1'                                  -
A                                                                         -
A                                              '


F*==============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR260) SRCFILE(XXX/QRPGSRC)
F*
F*==============================================================
FSFD260  CF  E                    WORKSTN
F                                            KINFDS INFDS
FCUSTOMERIF  E           K        DISK
IINFDS       DS
I                                     370 371 LINPOS
I            DS
I                                   B   1   20@LIN
I                                       2   2 @L
I            DS
I                                   B   1   20@POS
I                                       2   2 @P
C*
C                     EXFMTSFD260A
C                     EXSR @CMD
C*
C           CUSNO     CHAINCUSTOMER             99
C*
C           *IN99     IFEQ *OFF
C                     EXFMTSFD260B
C                     EXSR @CMD
C                     END
C*
C*********************************************************
C*    @ C M D   S U B R O U T I N E                      *
```

```
C*                                                          *
C*    Executed after every EXFMT or display read op code.   *
C*    Used to process command keys.                         *
C*************************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, seton LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C* If F4 was pressed, call program SFR261.
C*
C           *IN04     IFEQ *ON
C*
C* Convert the hex values of the cursor line/position into
C* binary, then to decimal.
C*
C                     MOVE *OFF      *IN04
C                     MOVE *ZEROS    @LIN
C                     MOVE *ZEROS    @POS
C                     MOVELLINPOS    @L
C                     MOVE LINPOS    @P
C                     Z-ADD@LIN      @SL     30
C                     Z-ADD@POS      @SP     30
C*
C                     CALL 'SFR261'
C                     PARM           @SL
C                     PARM           @SP
C*
C                     END
C*
C                     ENDSR


A*==============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD261) SRCFILE(XXX/QDDSSRC)
A*
A*==============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03)
A          R ASSUME
A                                       ASSUME
A                                  24  2' '
A          R SFD261A                    SFL
A            CUSNO     R        O  2  1REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                       COLOR(PNK)
A            CUSNM     R        O  2  9REFFLD(RCUST/CUSNM *LIBL/CUSTOMER)
A                                       COLOR(PNK)
A                                  3 10'State:'
A                                  3 21'Phone:'
```

```
A            CUSPH     R        O  3 28REFFLD(RCUST/CUSPH *LIBL/CUSTOMER)
A                                     EDTWRD('   -   -    ')
A            CUSST     R        O  3 17REFFLD(RCUST/CUSST *LIBL/CUSTOMER)
A          R SFD261B                  SFLCTL(SFD261A)
A                                     OVERLAY
A  30                                 SFLDSP
A                                     SFLDSPCTL
A  35                                 SFLEND(*MORE)
A                                     SFLDROP(CF10)
A                                     SFLSIZ(0007)
A                                     SFLPAG(0003)
A                                     WINDOW(WINDOW)
A                                  1  1'Number'
A                                     COLOR(YLW)
A                                     DSPATR(UL)
A                                  1  9'Customer Name                  '
A                                     COLOR(YLW)
A                                     DSPATR(UL)
A          R WINDOW
A                                     WINDOW(&@SL &@SP 10 40)
A                                     WDWBORDER((*COLOR BLU) (*DSPATR RI)-
A                                      (*CHAR '        '))
A            @SL            3S OP
A            @SP            3S OP
A                                  9  1'F3-Exit  F10-Display All Info'
A                                     DSPATR(HI)


F*==============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR261) SRCFILE(XXX/QRPGSRC)
F*
F*==============================================================
FCUSTOMERIF  E           K        DISK
FSFD261  CF  E                    WORKSTN
F                                       @RRN  KSFILE SFD261A
C*
C                     WRITEWINDOW
C                     EXFMTSFD261B
C                     EXSR @CMD
C*
C*********************************************************
C*    @ C M D   S U B R O U T I N E                      *
C*                                                       *
C*    Executed after every EXFMT or display read op code.  *
C*    Used to process command keys.                      *
C*********************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, seton LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
```

```
C                     END
C*
C                     ENDSR
C*
C*********************************************************
C*    * I N Z S R   S U B R O U T I N E                  *
C*                                                       *
C*    Retrieve the starting line and position for the    *
C*    window. Check to make sure that these values are   *
C*    valid. If not, put the closest valid number in.    *
C*********************************************************
C*
C           *INZSR    BEGSR
C*
C           *ENTRY    PLIST
C                     PARM           @SL
C                     PARM           @SP
C*
C                     Z-ADD0         @RRN    50
C*
C*  If the start line is greater than 12, the window will not
C*  fit on the display. Reset the line to 12.
C*
C           @SL       IFGT 12
C                     Z-ADD12        @SL
C                     END
C*
C*  If the start position is greater than 36, the window will not
C*  fit on the display. Reset the position to 36.
C*
C           @SP       IFGT 36
C                     Z-ADD36        @SP
C                     END
C*
C*   Nothing can be displayed from position 1,1 on the display.
C*   If the position is 1,1, reset it to 1,2. Also, check and
C*   make sure that the start line or start position is not zero.
C*
C           @SL       IFLE 1
C           @SP       ANDLE1
C                     Z-ADD1         @SL
C                     Z-ADD2         @SP
C                     END
C*
C*  Load the subfile
C*
C                     EXSR @LOAD
C*
C                     ENDSR
C*
C*********************************************************
C*    @ L O A D     S U B R O U T I N E                  *
C*                                                       *
C*    This subroutine will read the customer file and    *
C*    load the subfile records. Since this is a load all *
C*    subfile program, all records in the customer file  *
C*    will be read and loaded.                           *
```

```
C*************************************************************
C*
C           @LOAD     BEGSR
C*
C*  Perform this loop until you reach the end of
C*  the Customer file.
C*
C           *IN99     DOWNE*ON
C                     READ CUSTOMER                 99
C*
C*  If indicator 99 is on, or if you have already loaded
C*  9999 subfile records, (the maximum allowed), then exit
C*  this loop.
C*
C           *IN99     IFEQ *ON
C           @RRN      OREQ 9999
C                     LEAVE
C                     END
C*
C*  Add 1 to our subfile relative record number.
C*
C                     ADD  1         @RRN
C                     WRITESFD261A
C                     END
C*
C* If you have loaded at least one record into the subfile,
C* turn on indicator 30. This indicator conditions the
C* SFLDSP keyword. If you have loaded at least one record
C* into the subfile, it will be all right to display the subfile
C* format. If you did not load any records into the subfile, you
C* would not turn on indicator 30. Attempting to display
C* an empty subfile would result in an error being issued.
C*
C           @RRN      IFGT 0
C                     MOVE *ON       *IN30
C                     END
C*
C* Since you have loaded all records into the subfile, allow the
C* SFLEND keyword to display the end message.
C*
C                     MOVE *ON       *IN35
C*
C                     ENDSR
```

Change the command text line on the window format for the F10 key. Add the following statement to the subfile control format:

```
A                                       SFLDROP(CF10)
```

Now change the subfile page to three. To see the effects of these changes, execute program SFR260.

```
  xx/xx/xx            Customer Master Maintenance                    QPGMR
  xx:xx:xx                                                           SFD260A


Customer Number....:  ........................................
                      : Number  Customer Name                  :
Customer Name......:  : 00001   David's Computers and Bait Shop  :
                      : 00002   AS/400s Are Us                  :
Customer Address...:  : 00003   Tennessee Minicomputer Sales    :
                      : 00004   PCs For Less                    :
Customer City......:  : 00005   Mid-South Computer Sales        :
                      : 00010   Mini Computing Clearance        :
Customer State.....:  :                                 More... :
                      : F3-Exit  F10-Display All Info           :
Customer Zip.......:  :                                         :
                      :.........................................:
Phone Number.......:




 F3-Exit   F4-Customer Search
```

Press F10 to show all info.

```
  xx/xx/xx            Customer Master Maintenance                    QPGMR
  xx:xx:xx                                                           SFD260A


Customer Number....:  ........................................
                      : Number  Customer Name                  :
Customer Name......:  : 00001   David's Computers and Bait Shop  :
                      :          State: AR  Phone: 555-123-6666  :
Customer Address...:  : 00002   AS/400s Are Us                  :
                      :          State: TN  Phone: 666-455-4444  :
Customer City......:  : 00003   Tennessee Minicomputer Sales    :
                      :          State: KY  Phone: 880-555-1111  :
Customer State.....:  :                                 More... :
                      : F3-Exit  F10-Display All Info           :
Customer Zip.......:  :                                         :
                      :.........................................:
Phone Number.......:




 F3-Exit   F4-Customer Search
```

Subfile Drop

Now, instead of six lines of customers, three customers are displayed with two lines for each customer. The second line shows the state and phone number. Pressing F10 toggles this effect.

You have created a multiline subfile. Each subfile record consists of two lines of data. You can condition the subfile to display all lines of a subfile record or only the first. You can do this with the Subfile Drop (SFLDROP) and Subfile Fold (SFLFOLD) keywords. In this example, SFLDROP is specified, which tells the subfile program that it should initially be displayed with only the first subfile line in view. The rest of the subfile record, which exists on separate lines, should be dropped from view.

You must change the subfile page size to three because each subfile record now takes up two lines. If you were to display all subfile data, with each record taking two lines, displaying three subfile records would yield six lines. This number is the maximum you can display inside your window. Note, however, that when you display only one line of subfile data (when the additional subfile data is dropped), that six subfile records are displayed. The subfile handles this for you automatically.

Subfile Fold

You can specify that all subfile data should be displayed initially by changing the SFLDROP keyword to SFLFOLD. This change can be seen in SFD270, SFR270, SFD271, and SFR271:

```
A*============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD270) SRCFILE(XXX/QDDSSRC)
A*
A*============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03 'Exit')
A          R SFD270A
A                                       CF04(04 'Call Search Program')
A                                   1  3DATE
A                                       EDTCDE(Y)
A                                   1 23'Customer Master Maintenance'
A                                       DSPATR(HI)
A                                   2  3TIME
A                                   1 71USER
A                                   2 71'SFD270A'
A                                  23  2'F3-Exit   F4-Customer Search'
A                                       DSPATR(HI)
A                                   5  1'Customer Number....:'
A                                       COLOR(BLU)
```

```
A                                   7  1'Customer Name......:'
A                                       COLOR(BLU)
A                                   9  1'Customer Address...:'
A                                       COLOR(BLU)
A                                  11  1'Customer City......:'
A                                       COLOR(BLU)
A                                  13  1'Customer State.....:'
A                                       COLOR(BLU)
A                                  15  1'Customer Zip.......:'
A                                       COLOR(BLU)
A                                  17  1'Phone Number.......:'
A                                       COLOR(BLU)
A            CUSNO     R     Y  B  5 23REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A                                       CHECK(ER)
A                                       CHECK(FE)
A                                       CHECK(RB)
A                                  24  1'Customer Number Not Found In Custo-
A                                       mer Master File-Key In A Valid Numb-
A                                       er'
A                                       COLOR(RED)
A                                       DSPATR(BL)
A N99                                   DSPATR(ND)
A          R SFD270B
A                                       CLRL(*NO)
A                                   2 71'SFD270B'
A            CUSNO     R        O  5 23REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                       COLOR(PNK)
A            CUSNM     R        B  7 22REFFLD(RCUST/CUSNM *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A            CUSAD     R        B  9 22REFFLD(RCUST/CUSAD *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A            CUSCT     R        B 11 22REFFLD(RCUST/CUSCT *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A            CUSST     R        B 13 22REFFLD(RCUST/CUSST *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A            CUSCP     R        B 15 22REFFLD(RCUST/CUSCP *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A                                  23  1'F3-Exit                          -
A                                          '
A                                       COLOR(WHT)
A            CUSPH     R        B 17 22REFFLD(RCUST/CUSPH *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A                                       EDTWRD('   -   -    ')
A                                  24  1'                                  -
A                                                                          -
A                                                  '


F*============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR270) SRCFILE(XXX/QRPGSRC)
F*
F*============================================================
FSFD270  CF  E                     WORKSTN
```

```
F                                                KINFDS INFDS
FCUSTOMERIF  E           K        DISK
IINFDS       DS
I                                     370 371 LINPOS
I            DS
I                                   B   1   20@LIN
I                                       2   2 @L
I            DS
I                                   B   1   20@POS
I                                       2   2 @P
C*
C                     EXFMTSFD270A
C                     EXSR @CMD
C*
C            CUSNO    CHAINCUSTOMER             99
C*
C            *IN99    IFEQ *OFF
C                     EXFMTSFD270B
C                     EXSR @CMD
C                     END
C*
C*************************************************************
C*    @ C M D   S U B R O U T I N E                          *
C*                                                           *
C*    Executed after every EXFMT or display read op code.    *
C*    Used to process command keys.                          *
C*************************************************************
C*
C            @CMD     BEGSR
C*
C* If F3 was pressed, seton LR and exit.
C*
C            *IN03    IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C* If F4 was pressed, call program SFR271.
C*
C            *IN04    IFEQ *ON
C*
C* Convert the hex values of the cursor line/position into
C* binary, then to decimal.
C*
C                     MOVE *OFF      *IN04
C                     MOVE *ZEROS    @LIN
C                     MOVE *ZEROS    @POS
C                     MOVELLINPOS    @L
C                     MOVE LINPOS    @P
C                     Z-ADD@LIN      @SL     30
C                     Z-ADD@POS      @SP     30
C*
C                     CALL 'SFR271'
C                     PARM           @SL
C                     PARM           @SP
C*
C                     END
```

```
C*
C                     ENDSR


A*============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD271) SRCFILE(XXX/QDDSSRC)
A*
A*============================================================
A                                     DSPSIZ(24 80 *DS3)
A                                     CF03(03)
A          R ASSUME
A                                     ASSUME
A                                24  2' '
A          R SFD271A                  SFL
A            CUSNO     R        O  2  1REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                     COLOR(PNK)
A            CUSNM     R        O  2  9REFFLD(RCUST/CUSNM *LIBL/CUSTOMER)
A                                     COLOR(PNK)
A                                 3 10'State:'
A                                 3 21'Phone:'
A            CUSPH     R        O  3 28REFFLD(RCUST/CUSPH *LIBL/CUSTOMER)
A                                     EDTWRD('   -   -    ')
A            CUSST     R        O  3 17REFFLD(RCUST/CUSST *LIBL/CUSTOMER)
A          R SFD271B                  SFLCTL(SFD271A)
A                                     OVERLAY
A  30                                 SFLDSP
A                                     SFLDSPCTL
A  35                                 SFLEND(*MORE)
A                                     SFLFOLD(CF10)
A                                     SFLSIZ(0007)
A                                     SFLPAG(0003)
A                                     WINDOW(WINDOW)
A                                 1  1'Number'
A                                     COLOR(YLW)
A                                     DSPATR(UL)
A                                 1  9'Customer Name                  '
A                                     COLOR(YLW)
A                                     DSPATR(UL)
A          R WINDOW
A                                     WINDOW(&@SL &@SP 10 40)
A                                     WDWBORDER((*COLOR BLU) (*DSPATR RI)-
A                                      (*CHAR '        '))
A            @SL            3S OP
A            @SP            3S OP
A                                 9  1'F3-Exit  F10-Remove Additonal Info'
A                                     DSPATR(HI)


F*============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR271) SRCFILE(XXX/QRPGSRC)
F*
```

```
F*==============================================================
FCUSTOMERIF  E           K        DISK
FSFD271  CF  E                    WORKSTN
F                                        @RRN  KSFILE SFD271A
C*
C                     WRITEWINDOW
C                     EXFMTSFD271B
C                     EXSR @CMD
C*
C*********************************************************
C*    @ C M D   S U B R O U T I N E                      *
C*                                                       *
C*    Executed after every EXFMT or display read op code.  *
C*    Used to process command keys.                      *
C*********************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, seton LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C                     ENDSR
C*
C*********************************************************
C*    * I N Z S R   S U B R O U T I N E                  *
C*                                                       *
C*    Retrieve the starting line and position for the    *
C*    window. Check to make sure that these values are   *
C*    valid. If not, put the closest valid number in.    *
C*********************************************************
C*
C           *INZSR    BEGSR
C*
C           *ENTRY    PLIST
C                     PARM           @SL
C                     PARM           @SP
C*
C                     Z-ADD0         @RRN    50
C*
C*  If the start line is greater than 12, the window will not
C*  fit on the display. Reset the line to 12.
C*
C           @SL       IFGT 12
C                     Z-ADD12        @SL
C                     END
C*
C*  If the start position is greater than 36, the window will not
C*  fit on the display. Reset the position to 36.
C*
C           @SP       IFGT 36
C                     Z-ADD36        @SP
C                     END
C*
```

```
C*   Nothing can be displayed from position 1,1 on the display.
C*   If the position is 1,1, reset it to 1,2. Also, check and
C*   make sure that the start line or start position is not zero.
C*
C           @SL       IFLE 1
C           @SP       ANDLE1
C                     Z-ADD1         @SL
C                     Z-ADD2         @SP
C                     END
C*
C*  Load the subfile
C*
C                     EXSR @LOAD
C*
C                     ENDSR
C*
C*********************************************************
C*    @ L O A D     S U B R O U T I N E                  *
C*                                                       *
C*    This subroutine will read the customer file and    *
C*    load the subfile records. Since this is a load all *
C*    subfile program, all records in the customer file  *
C*    will be read and loaded.                           *
C*********************************************************
C*
C           @LOAD     BEGSR
C*
C*  Perform this loop until you reach the end of
C*  the Customer file.
C*
C           *IN99     DOWNE*ON
C                     READ CUSTOMER                99
C*
C*  If indicator 99 is on, or if you have already loaded
C*  9999 subfile records, (the maximum allowed), then exit
C*  this loop.
C*
C           *IN99     IFEQ *ON
C           @RRN      OREQ 9999
C                     LEAVE
C                     END
C*
C*  Add 1 to the subfile relative record number.
C*
C                     ADD  1         @RRN
C                     WRITESFD271A
C                     END
C*
C* If you have loaded at least one record into the subfile,
C* turn on indicator 30. This indicator conditions the
C* SFLDSP keyword. If you have loaded at least one record
C* into the subfile, it will be all right to display the subfile
C* format. If you did not load any records into the subfile, you
C* would not turn on indicator 30. Attempting to display
C* an empty subfile would result in an error being issued.
C*
C           @RRN      IFGT 0
```

```
C                       MOVE *ON       *IN30
C                       END
C*
C* Since you have loaded all records into the subfile, allow the
C* SFLEND keyword to display the end message.
C*
C                       MOVE *ON       *IN35
C*
C                       ENDSR
```

When you execute SFR270, you can see that the subfile now initially shows all data.

```
  xx/xx/xx            Customer Master Maintenance                    QPGMR
  xx:xx:xx                                                           SFD270A


Customer Number....:  ..........................................
                      : Number  Customer Name                    :
Customer Name......:  : 00001   David's Computers and Bait Shop  :
                      :          State: AR  Phone: 555-123-6666  :
Customer Address...:  : 00002   AS/400s Are Us                   :
                      :          State: TN  Phone: 666-455-4444  :
Customer City......:  : 00003   Tennessee Minicomputer Sales     :
                      :          State: KY  Phone: 880-555-1111  :
Customer State.....:  :                                 More... :
                      : F3-Exit  F10-Display All Info            :
Customer Zip.......:  :                                          :
                      :..........................................:
Phone Number.......:




 F3-Exit   F4-Customer Search
```

If you press F10, you can still toggle between one line per record subfile and all subfile lines being displayed.

```
  xx/xx/xx            Customer Master Maintenance                    QPGMR
  xx:xx:xx                                                           SFD260A


Customer Number....:  ..........................................
                      : Number  Customer Name                    :
Customer Name......:  : 00001   David's Computers and Bait Shop  :
                      : 00002   AS/400s Are Us                   :
Customer Address...:  : 00003   Tennessee Minicomputer Sales     :
                      : 00004   PCs For Less                     :
Customer City......:  : 00005   Mid-South Computer Sales         :
                      : 00010   Mini Computing Clearance         :
Customer State.....:  :                                 More... :
                      : F3-Exit  F10-Display All Info            :
Customer Zip.......:  :                                          :
                      :..........................................:
Phone Number.......:




 F3-Exit   F4-Customer Search
```

Subfile Cursor Relative Record Number

The Subfile Cursor Relative Record Number (SFLCSRRRN) is a new and very powerful keyword. Using this keyword, the subfile will return the relative record number of the subfile record on which the cursor was positioned. You can let users pull up a subfile inquiry screen, roll until they find the information they need, place the cursor on the record they want to select, press Enter, and the subfile returns the subfile relative record number. You can easily modify the last subfile program you looked at to work this way. Look at the code for SFD280 and SFR280:

```
A*==============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD280) SRCFILE(XXX/QDDSSRC)
A*
A*==============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03 'Exit')
A          R SFD280A
A                                       CF04(04 'Call Search Program')
A                                   1  3DATE
A                                       EDTCDE(Y)
A                                   1 23'Customer Master Maintenance'
A                                       DSPATR(HI)
A                                   2  3TIME
A                                   1 71USER
A                                   2 71'SFD280A'
A                                  23  2'F3-Exit   F4-Customer Search'
A                                       DSPATR(HI)
A                                   5  1'Customer Number....:'
A                                       COLOR(BLU)
A                                   7  1'Customer Name......:'
A                                       COLOR(BLU)
A                                   9  1'Customer Address...:'
A                                       COLOR(BLU)
A                                  11  1'Customer City......:'
A                                       COLOR(BLU)
A                                  13  1'Customer State.....:'
A                                       COLOR(BLU)
A                                  15  1'Customer Zip.......:'
A                                       COLOR(BLU)
A                                  17  1'Phone Number.......:'
A                                       COLOR(BLU)
A            CUSNO     R     Y  B  5 23REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A                                       CHECK(ER)
A                                       CHECK(FE)
A                                       CHECK(RB)
A                                  24  1'Customer Number Not Found In Custo-
A                                       mer Master File-Key In A Valid Numb-
A                                       er'
A                                       COLOR(RED)
A                                       DSPATR(BL)
A N99                                   DSPATR(ND)
A          R SFD280B
A                                       CLRL(*NO)
A                                   2 71'SFD280B'
A            CUSNO     R        O  5 23REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                       COLOR(PNK)
A            CUSNM     R        B  7 22REFFLD(RCUST/CUSNM *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A            CUSAD     R        B  9 22REFFLD(RCUST/CUSAD *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A            CUSCT     R        B 11 22REFFLD(RCUST/CUSCT *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A            CUSST     R        B 13 22REFFLD(RCUST/CUSST *LIBL/CUSTOMER)
A                                       COLOR(WHT)
```

```
A            CUSCP     R        B 15 22REFFLD(RCUST/CUSCP *LIBL/CUSTOMER)
A                                     COLOR(WHT)
A                                 23  1'F3-Exit                          -
A                                         '
A                                     COLOR(WHT)
A            CUSPH     R        B 17 22REFFLD(RCUST/CUSPH *LIBL/CUSTOMER)
A                                     COLOR(WHT)
A                                     EDTWRD('   -   -    ')
A                                 24  1'                                 -
A                                                                        -
A                                             '


F*============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR280) SRCFILE(XXX/QRPGSRC)
F*
F*============================================================
FSFD280  CF  E                    WORKSTN
F                                            KINFDS INFDS
FCUSTOMERIF  E           K        DISK
IINFDS       DS
I                                     370 371 LINPOS
I            DS
I                                  B   1   20@LIN
I                                      2   2 @L
I            DS
I                                  B   1   20@POS
I                                      2   2 @P
C*
C                     EXFMTSFD280A
C                     EXSR @CMD
C*
C           CUSNO     CHAINCUSTOMER            99
C*
C           *IN99     IFEQ *OFF
C                     EXFMTSFD280B
C                     EXSR @CMD
C                     END
C*
C*********************************************************
C*    @ C M D   S U B R O U T I N E                      *
C*                                                       *
C*    Executed after every EXFMT or display read op code.  *
C*    Used to process command keys.                      *
C*********************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, seton LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
```

```
C*
C* If F4 was pressed, call program SFR281. Pass the customer
C* number field. If the customer number field greater than 0
C* upon return, then the user has selected a customer.
C*
C           *IN04     IFEQ *ON
C*
C* Convert the hex values of the cursor line/position into
C* binary, then to decimal.
C*
C                     Z-ADD0         CUSNO
C                     MOVE *OFF      *IN04
C                     MOVE *ZEROS    @LIN
C                     MOVE *ZEROS    @POS
C                     MOVELLINPOS    @L
C                     MOVE LINPOS    @P
C                     Z-ADD@LIN      @SL     30
C                     Z-ADD@POS      @SP     30
C*
C                     CALL 'SFR281'
C                     PARM           @SL
C                     PARM           @SP
C                     PARM           CUSNO
C*
C                     WRITESFD280A
C*
C                     END
C*
C                     ENDSR
```

In this case, you must change the CALL statement parameters to pass the customer number field to the subfile program. This change to the program allows this program to receive back in the customer number that the user selected from the subfile inquiry program. The subfile inquiry program (SFR281) does not use the field for anything, but moves the selected customer number that the user wanted into this field.

```
C                     CALL 'SFR281'
C                     PARM           @SL
C                     PARM           @SP
C                     PARM           CUSNO
```

You must also add another line to write the SFD280A format again, so you can be assured this format is on the screen after returning back from the call to SFR281.

So, if F4 is pressed, execute the @CMD subroutine. After the F4 routine is processed, call SFR281 and pass it the starting line and position for the window, as well as the customer number. After SFR281 is executed, return back to SFR280 and write the SFD280A format again. Use the customer number field to chain out to the customer file.

If the user does not select a customer from the inquiry program, but pressed F3 to exit, the customer number is 0. When you chain to the customer master file and do

not retrieve a record, set on indicator 99. In this case, you fall all the way through the program and perform the EXFMT statement. Since indicator 99 is on, a message is displayed at the bottom of the screen indicating that the customer number is invalid.

If the user does select a customer in SFR281, you will retrieve records from the customer master file. In this case, perform the EXFMT on SFD280B display format, which shows the customer detail.

Now, look at the changes needed for the subfile program to implement the SFLCSRRRN keyword:

```
A*============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD281) SRCFILE(XXX/QDDSSRC)
A*
A*============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03)
A          R ASSUME
A                                       ASSUME
A                                  24  2' '
A          R SFD281A                    SFL
A            CUSNO     R        O  2  1REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                       COLOR(PNK)
A            CUSNM     R        O  2  9REFFLD(RCUST/CUSNM *LIBL/CUSTOMER)
A                                       COLOR(PNK)
A                                   3 10'State:'
A                                   3 21'Phone:'
A            CUSPH     R        O  3 28REFFLD(RCUST/CUSPH *LIBL/CUSTOMER)
A                                       EDTWRD('   -   -    ')
A            CUSST     R        O  3 17REFFLD(RCUST/CUSST *LIBL/CUSTOMER)
A          R SFD281B                    SFLCTL(SFD281A)
A                                       OVERLAY
A                                       SFLCSRRRN(&@REC)
A  30                                   SFLDSP
A                                       SFLDSPCTL
A  35                                   SFLEND(*MORE)
A                                       SFLDROP(CF10)
A                                       SFLSIZ(0007)
A                                       SFLPAG(0003)
A                                       WINDOW(WINDOW)
A                                   1  1'Number'
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A                                   1  9'Customer Name                  '
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A            @REC           5S 0H
A          R WINDOW
A                                       WINDOW(&@SL &@SP 10 40)
A                                       WDWBORDER((*COLOR BLU) (*DSPATR RI)-
A                                        (*CHAR '        '))
A            @SL            3S 0P
A            @SP            3S 0P
```

```
A                                   9  1'F3-Exit  F10-Show Additional Info '
A                                       DSPATR(HI)


F*=============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR281) SRCFILE(XXX/QRPGSRC)
F*
F*=============================================================
FCUSTOMERIF  E           K        DISK
FSFD281  CF  E                    WORKSTN
F                                        @RRN  KSFILE SFD281A
C*
C                     WRITEWINDOW
C                     EXFMTSFD281B
C                     EXSR @CMD
C*
C*********************************************************
C*    @ C M D   S U B R O U T I N E                      *
C*                                                       *
C*    Executed after every EXFMT or display read op code.  *
C*    Used to process command keys.                      *
C*********************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, seton LR and exit.  Since the user did
C* not select a customer, 0 out the CUSN0 field
C*
C           *IN03     IFEQ *ON
C                     Z-ADD0         CUSN0
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C* If the field @REC is greater than 0, it means that the user
C* has selected a record. Chain to the subfile to retrieve the
C* customer number, then return back to the calling program.
C*
C           @REC      IFGT 0
C           @REC      CHAINSFD281A              98
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C                     ENDSR
C*
C*********************************************************
C*    * I N Z S R   S U B R O U T I N E                  *
C*                                                       *
C*    Retrieve the starting line and position for the    *
C*    window. Check to make sure that these values are   *
C*    valid. If not, put the closest valid number in.    *
C*********************************************************
C*
```

```
C           *INZSR    BEGSR
C*
C           *ENTRY    PLIST
C                     PARM           @SL
C                     PARM           @SP
C                     PARM           CUSNO
C*
C                     Z-ADD0         @RRN    50
C*
C*  If the start line is greater than 12, the window will not
C*  fit on the display. Reset the line to 12.
C*
C           @SL       IFGT 12
C                     Z-ADD12        @SL
C                     END
C*
C*  If the start position is greater than 36, the window will not
C*  fit on the display. Reset the position to 36.
C*
C           @SP       IFGT 36
C                     Z-ADD36        @SP
C                     END
C*
C*   Nothing can be displayed from position 1,1 on the display.
C*   If the position is 1,1, reset it to 1,2. Also, check and
C*   make sure that the start line or start position is not zero.
C*
C           @SL       IFLE 1
C           @SP       ANDLE1
C                     Z-ADD1         @SL
C                     Z-ADD2         @SP
C                     END
C*
C*  Load the subfile
C*
C                     EXSR @LOAD
C*
C                     ENDSR
C*
C*************************************************************
C*    @ L O A D     S U B R O U T I N E                     *
C*                                                          *
C*    This subroutine will read the customer file and       *
C*    load the subfile records. Since this is a load all    *
C*    subfile program, all records in the customer file     *
C*    will be read and loaded.                              *
C*************************************************************
C*
C           @LOAD     BEGSR
C*
C*  Perform this loop until you reach the end of
C*  the Customer file.
C*
C           *IN99     DOWNE*ON
C                     READ CUSTOMER                 99
C*
C*  If indicator 99 is on, or if you have already loaded
```

```
C*  9999 subfile records, (the maximum allowed), then exit
C*  this loop.
C*
C           *IN99     IFEQ *ON
C           @RRN      OREQ 9999
C                     LEAVE
C                     END
C*
C*  Add 1 to our subfile relative record number.
C*
C                     ADD  1         @RRN
C                     WRITESFD281A
C                     END
C*
C* If you have loaded at least one record into the subfile,
C* turn on indicator 30. This indicator conditions the
C* SFLDSP keyword. If you have loaded at least one record
C* into the subfile, it will be all right to display the subfile
C* format. If you did not load any records into the subfile, you
C* would not turn on indicator 30. Attempting to display
C* an empty subfile would result in an error being issued.
C*
C           @RRN      IFGT 0
C                     MOVE *ON       *IN30
C                     END
C*
C* Since you have loaded all records into the subfile, allow the
C* SFLEND keyword to display the end message.
C*
C                     MOVE *ON       *IN35
C*
C                     ENDSR
```

In the RPG program, the following lines have been added to the @CMD subroutine:

```
C           @REC      IFGT 0
C           @REC      CHAINSFD281A              98
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
```

Field @REC is the SFLCSRRRN field that contains the subfile relative record number if the cursor is positioned on a subfile record. If the cursor is not positioned on a valid subfile record, then the value of @REC is 0.

The program checks for this condition first. If @REC is 0, then it redisplays the subfile. If @REC is not 0, it means the user placed the cursor on a valid subfile record, and pressed a valid function key or the Enter key. The program now has to retrieve the customer number that is in the subfile at the relative record number that is specified by field @REC. The program should always retrieve the record at the chain statement, unless it has a bug. If it doesn't retrieve a record, control is returned back to SFR280

without the selected customer number. If it does retrieve a record (and it should), the field CUSNO now contains the customer number the user wants. It retrieves a record because the name of the customer field in the subfile is the same as the name of the customer number field in the customer file, CUSNO. Control is passed back to SFR280 and field CUSNO contains the value of the customer number that the user selected.

```
Also note one added line in the F3 routine.
```

```
C                   Z-ADD0        CUSNO
```

If F3 is pressed (which means the user wants to exit the program without selecting a customer) the program forces the customer number to be 0. This value is passed back to SFR280 which clears the screen since the user did not select a customer.

The display file has only two additional lines. The first is the SFLCSRRRN keyword. Notice that you have to specify a parameter for this keyword. The parameter is the field name in which you want the value of the relative record number to be placed. In this example, the value is placed in field @REC. The field in this parameter must also be defined on this record format as a 5-byte, signed numeric, 0 decimal, hidden field.

Now call SFR280. Your screen should look like this:

```
  xx/xx/xx            Customer Master Maintenance                    QPGMR
  xx:xx:xx                                                           SFD280A


Customer Number....:  00000

Customer Name......:
                                          _____________  Previous Phone#
Customer Address...:

Customer City......:

Customer State.....:

Customer Zip.......:

Phone Number.......:




 F3-Exit   F4-Customer Search
```

Now press F4. Your display should look like this:

```
  xx/xx/xx            Customer Master Maintenance                    QPGMR
  xx:xx:xx                                                           SFD280A


Customer Number....:  ..........................................
                      : Number  Customer Name                    :
Customer Name......:  : 00001   David's Computers and Bait Shop  :
                      : 00002   AS/400s Are Us                   :
Customer Address...:  : 00003   Tennessee Minicomputer Sales     :
                      : 00004   PCs For Less                     :
Customer City......:  : 00005   Mid-South Computer Sales         :
                      : 00010   Mini Computing Clearance         :
Customer State.....:  :                                 More... :
                      : F3-Exit  F10-Display All Info            :
Customer Zip.......:  :                                          :
                      :..........................................:
Phone Number.......:




 F3-Exit   F4-Customer Search
```

Place the cursor on "PCs For Less" and press Enter:

```
  xx/xx/xx            Customer Master Maintenance                    QPGMR
  xx:xx:xx                                                           SFD280B


Customer Number....:  00004

Customer Name......: PCs For Less

Customer Address...: 3222 S. Main St.

Customer City......: Sardis

Customer State.....: MS

Customer Zip.......: 66555

Phone Number.......: 555-789-9889




F3-Exit
```

See how this works? You have added the ability to search and select for a customer and you only added 9 lines of code. What is especially nice about this technique is

that you did not have to make the subfile input capable. You did not have to put input fields next to each subfile record and let the user key a character into the field of the customer that they wanted to select. If you did, the subfile would have looked like this:

```
.............................................
: Opt  Number  Customer Name                 :
:  _   00001   David's Computers and Bait Shop  :
:  _   00002   AS/400s Are Us                  :
:  _   00003   Tennessee Minicomputer Sales    :
:  _   00004   PCs For Less                    :
:  _   00005   Mid-South Computer Sales        :
:  _   00010   Mini Computing Clearance        :
:                                     More... :
: F3-Exit  F10-Display All Info                :
:                                              :
:............................................:
```

Then the user would have to place a character, such as a "1", to select the record they wanted:

```
.............................................
: Opt  Number  Customer Name                 :
:  _   00001   David's Computers and Bait Shop  :
:  _   00002   AS/400s Are Us                  :
:  _   00003   Tennessee Minicomputer Sales    :
:  1   00004   PCs For Less                    :
:  _   00005   Mid-South Computer Sales        :
:  _   00010   Mini Computing Clearance        :
:                                     More... :
: F3-Exit  F10-Display All Info                :
:                                              :
:............................................:
```

Then, inside the subfile program, you would have to check these input fields for a selection. There will be a case for doing this type of checking, and you will examine these types of subfiles later in the book. But, you can see that using the SFLCSRRRN keyword can greatly reduce the programming effort. It also allows the users with mouse control to "point and click" to select the information they want back into the main program.

Subfile Line

The next program shows one use of the Subfile Line (SFLLIN) keyword. The name subfile line is a misnomer because this keyword actually works with spaces.

By using the SFLLIN keyword, you can create a horizontal subfile. In other words, you can cause a subfile to display more than one subfile record per line.

So far, the examples have addressed vertical subfiles where only one subfile record fits on one line on the display. You have seen single line vertical subfiles:

```
Subfile Record 1
Subfile Record 2
Subfile Record 3
Subfile Record 4
```

You have also seen multiple line vertical subfiles:

```
Subfile Record 1
  Subfile Record 1
Subfile Record 2
  Subfile Record 2
Subfile Record 3
  Subfile Record 3
```

Now you will learn how to create a horizontal subfile in which more than one subfile record is on a single display line.

```
Subfile Record 1  Subfile Record 2   Subfile Record 3
Subfile Record 4  Subfile Record 5   Subfile Record 6
```

The format of the SFLLIN keyword is

```
SFLLIN(value)
```

where value is the number of positions, or spaces, between subfile records. If a subfile record is 10-bytes long, and you specify a subfile line of four, then five subfile records would fit on an 80-column display. If you are displaying the subfile inside a 40-byte wide window, then only two subfile records would be displayed per line.

Now take program SFR280 and SFR281 and copy them into SFR290 and SFR291. You will need to make one change to SFR290 to include the ability to press F4 on the SFD290B screen. This change allows your program to call the state search inquiry program, SFR292. Now, you can look at the code:

```
A*=============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD290) SRCFILE(XXX/QDDSSRC)
A*
A*=============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03 'Exit')
A                                       CF04(04 'Call Search Program')
A          R SFD290A
A                                       RTNCSRLOC(&@RCD &@FLD)
A                                   1  3DATE
A                                       EDTCDE(Y)
```

```
A                                   1 23'Customer Master Maintenance'
A                                       DSPATR(HI)
A                                   2  3TIME
A                                   1 71USER
A                                   2 71'SFD290A'
A                                  23  2'F3-Exit   F4-Customer Search'
A                                       DSPATR(HI)
A                                   5  1'Customer Number....:'
A                                       COLOR(BLU)
A                                   7  1'Customer Name......:'
A                                       COLOR(BLU)
A                                   9  1'Customer Address...:'
A                                       COLOR(BLU)
A                                  11  1'Customer City......:'
A                                       COLOR(BLU)
A                                  13  1'Customer State.....:'
A                                       COLOR(BLU)
A                                  15  1'Customer Zip.......:'
A                                       COLOR(BLU)
A                                  17  1'Phone Number.......:'
A                                       COLOR(BLU)
A            CUSNO     R     Y  B  5 23REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A                                       CHECK(ER)
A                                       CHECK(FE)
A                                       CHECK(RB)
A                                  24  1'Customer Number Not Found In Custo-
A                                       mer Master File-Key In A Valid Numb-
A                                       er'
A                                       COLOR(RED)
A                                       DSPATR(BL)
A N99                                   DSPATR(ND)
A            @RCD          10   H
A            @FLD          10   H
A          R SFD290B
A                                       RTNCSRLOC(&@RCD &@FLD)
A                                       CLRL(*NO)
A                                   2 71'SFD290B'
A            CUSNO     R        O  5 23REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                       COLOR(PNK)
A            CUSNM     R        B  7 22REFFLD(RCUST/CUSNM *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A            CUSAD     R        B  9 22REFFLD(RCUST/CUSAD *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A            CUSCT     R        B 11 22REFFLD(RCUST/CUSCT *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A            CUSST     R        B 13 22REFFLD(RCUST/CUSST *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A            CUSCP     R        B 15 22REFFLD(RCUST/CUSCP *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A                                  23  1'F3-Exit    F4-State Search        -
A                                                  '
A                                       COLOR(WHT)
A            CUSPH     R        B 17 22REFFLD(RCUST/CUSPH *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A                                       EDTWRD('   -   -    ')
A                                  24  1'                                  -
```

```
A                                                                       -
A                                                  '
A            @RCD          10A  H
A            @FLD          10A  H


F*==============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR290) SRCFILE(XXX/QRPGSRC)
F*
F*==============================================================
FSFD290  CF  E                    WORKSTN
F                                              KINFDS INFDS
FCUSTOMERIF  E           K        DISK
IINFDS       DS
I                                     370 371 LINPOS
I            DS
I                                   B   1   20@LIN
I                                       2   2 @L
I            DS
I                                   B   1   20@POS
I                                       2   2 @P
C*
C                     EXFMTSFD290A
C                     EXSR @CMD
C*
C           CUSNO     CHAINCUSTOMER             99
C*
C           *IN99     IFEQ *OFF
C                     EXFMTSFD290B
C                     EXSR @CMD
C                     END
C*
C*********************************************************
C*    @ C M D   S U B R O U T I N E                      *
C*                                                       *
C*    Executed after every EXFMT or display read op code.  *
C*    Used to process command keys.                      *
C*********************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, seton LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C* If F4 was pressed, call program SFR291. Pass the customer
C* number field. If the customer number field greater than 0
C* upon return, then the user has selected a customer.
C*
C           *IN04     IFEQ *ON
C*
```

```
C* Convert the hex values of the cursor line/position into
C* binary, then to decimal.
C*
C                     Z-ADD0         CUSNO
C                     MOVE *OFF      *IN04
C                     MOVE *ZEROS    @LIN
C                     MOVE *ZEROS    @POS
C                     MOVELLINPOS    @L
C                     MOVE LINPOS    @P
C                     Z-ADD@LIN      @SL     30
C                     Z-ADD@POS      @SP     30
C*
C* If the F4 key was pressed on the  A  format, call the customer
C* search program, else call the state search program.
C*
C           @RCD      IFEQ 'SFD290A'
C*
C                     CALL 'SFR291'
C                     PARM           @SL
C                     PARM           @SP
C                     PARM           CUSNO
C*
C                     ELSE
C*
C                     CALL 'SFR292'
C                     PARM           @SL
C                     PARM           @SP
C                     END
C*
C*
C*
C                     WRITESFD290A
C*
C                     END
C*
C                     ENDSR


A*============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD291) SRCFILE(XXX/QDDSSRC)
A*
A*============================================================
A                                      DSPSIZ(24 80 *DS3)
A                                      CF03(03)
A          R ASSUME
A                                      ASSUME
A                                 24  2' '
A          R SFD291A                   SFL
A            CUSNO     R        O  2  1REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                      COLOR(PNK)
A            CUSNM     R        O  2  9REFFLD(RCUST/CUSNM *LIBL/CUSTOMER)
A                                      COLOR(PNK)
A                                  3 10'State:'
A                                  3 21'Phone:'
```

```
A            CUSPH     R        O  3 28REFFLD(RCUST/CUSPH *LIBL/CUSTOMER)
A                                     EDTWRD('   -   -    ')
A            CUSST     R        O  3 17REFFLD(RCUST/CUSST *LIBL/CUSTOMER)
A          R SFD291B                  SFLCTL(SFD291A)
A                                     OVERLAY
A                                     SFLCSRRRN(&@REC)
A  30                                 SFLDSP
A                                     SFLDSPCTL
A  35                                 SFLEND(*MORE)
A                                     SFLDROP(CF10)
A                                     SFLSIZ(0007)
A                                     SFLPAG(0003)
A                                     WINDOW(WINDOW)
A                                  1  1'Number'
A                                     COLOR(YLW)
A                                     DSPATR(UL)
A                                  1  9'Customer Name                  '
A                                     COLOR(YLW)
A                                     DSPATR(UL)
A            @REC           5S 0H
A          R WINDOW
A                                     WINDOW(&@SL &@SP 10 40)
A                                     WDWBORDER((*COLOR BLU) (*DSPATR RI)-
A                                      (*CHAR '        '))
A            @SL            3S 0P
A            @SP            3S 0P
A                                  9  1'F3-Exit  F10-Show Additional Info '
A                                     DSPATR(HI)


F*==============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR291) SRCFILE(XXX/QRPGSRC)
F*
F*==============================================================
FCUSTOMERIF  E           K        DISK
FSFD291  CF  E                    WORKSTN
F                                       @RRN  KSFILE SFD291A
C*
C                     WRITEWINDOW
C                     EXFMTSFD291B
C                     EXSR @CMD
C*
C*************************************************************
C*    @ C M D   S U B R O U T I N E                          *
C*                                                           *
C*    Executed after every EXFMT or display read op code.    *
C*    Used to process command keys.                          *
C*************************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, seton LR and exit.  Since the user did
C* not select a customer, 0 out the CUSN0 field
C*
```

```
C           *IN03     IFEQ *ON
C                     Z-ADD0         CUSNO
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C* If the field @REC is greater than 0, it means that the user
C* has selected a record. Chain to the subfile to retrieve the
C* customer number, then return back to the calling program.
C*
C           @REC      IFGT 0
C           @REC      CHAINSFD291A              98
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C                     ENDSR
C*
C*********************************************************
C*    * I N Z S R   S U B R O U T I N E                  *
C*                                                       *
C*    Retrieve the starting line and position for the    *
C*    window. Check to make sure that these values are   *
C*    valid. If not, put the closest valid number in.    *
C*********************************************************
C*
C           *INZSR    BEGSR
C*
C           *ENTRY    PLIST
C                     PARM           @SL
C                     PARM           @SP
C                     PARM           CUSNO
C*
C                     Z-ADD0         @RRN    50
C*
C*  If the start line is greater than 12, the window will not
C*  fit on the display. Reset the line to 12.
C*
C           @SL       IFGT 12
C                     Z-ADD12        @SL
C                     END
C*
C*  If the start position is greater than 36, the window will not
C*  fit on the display. Reset the position to 36.
C*
C           @SP       IFGT 36
C                     Z-ADD36        @SP
C                     END
C*
C*   Nothing can be displayed from position 1,1 on the display.
C*   If the position is 1,1, reset it to 1,2. Also, check and
C*   make sure that the start line or start position is not zero.
C*
C           @SL       IFLE 1
C           @SP       ANDLE1
C                     Z-ADD1         @SL
C                     Z-ADD2         @SP
```

```
C                     END
C*
C*  Load the subfile
C*
C                     EXSR @LOAD
C*
C                     ENDSR
C*
C*************************************************************
C*    @ L O A D     S U B R O U T I N E                     *
C*                                                          *
C*    This subroutine will read the customer file and       *
C*    load the subfile records. Since this is a load all    *
C*    subfile program, all records in the customer file     *
C*    will be read and loaded.                              *
C*************************************************************
C*
C           @LOAD     BEGSR
C*
C*  Perform this loop until you reach the end of
C*  the Customer file.
C*
C           *IN99     DOWNE*ON
C                     READ CUSTOMER                 99
C*
C*  If indicator 99 is on, or if you have already loaded
C*  9999 subfile records, (the maximum allowed), then exit
C*  this loop.
C*
C           *IN99     IFEQ *ON
C           @RRN      OREQ 9999
C                     LEAVE
C                     END
C*
C*  Add 1 to our subfile relative record number.
C*
C                     ADD  1         @RRN
C                     WRITESFD291A
C                     END
C*
C* If you have loaded at least one record into the subfile,
C* turn on indicator 30. This indicator conditions the
C* SFLDSP keyword. If you have loaded at least one record
C* into the subfile, it will be all right to display the subfile
C* format. If you did not load any records into the subfile, you
C* would not turn on indicator 30. Attempting to display
C* an empty subfile would result in an error being issued.
C*
C           @RRN      IFGT 0
C                     MOVE *ON       *IN30
C                     END
C*
C* Since you have loaded all records into the subfile, allow the
C* SFLEND keyword to display the end message.
C*
C                     MOVE *ON       *IN35
```

```
C*
C                     ENDSR


A*============================================================
A* To compile:
A*
A*      CRTPF      FILE(XXX/STATES) SRCFILE(XXX/QDDSSRC)
A*
A*============================================================
A                                       UNIQUE
A          R RSTATES
A            STATCD         2
A            STATDS        15
A          K STATCD


A*============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD292) SRCFILE(XXX/QDDSSRC)
A*
A*============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03)
A          R ASSUME
A                                       ASSUME
A                                  24  2' '
A          R SFD292A                    SFL
A            STATCD    R        0  3  1REFFLD(RSTATES/STATCD *LIBL/STATES)
A                                       COLOR(PNK)
A            STATDS    R        0  3  4REFFLD(RSTATES/STATDS *LIBL/STATES)
A                                       COLOR(PNK)
A          R SFD292B                    SFLCTL(SFD292A)
A                                       OVERLAY
A  30                                   SFLDSP
A                                       SFLDSPCTL
A  35                                   SFLEND(*MORE)
A                                       SFLSIZ(0050)
A                                       SFLPAG(0015)
A                                       SFLLIN(0003)
A                                       WINDOW(WINDOW)
A                                  2  1'Valid State Codes '
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A                                  2 22'Valid State Codes '
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A                                  2 43'Valid State Codes '
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A          R WINDOW
A                                       WINDOW(&@SL &@SP 10 61)
A                                       WDWBORDER((*COLOR BLU) (*DSPATR RI)-
A                                        (*CHAR '        '))
```

```
A            @SL            3S OP
A            @SP            3S OP
A                                  9  1'F3-Exit'
A                                      DSPATR(HI)


F*==============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR292) SRCFILE(XXX/QRPGSRC)
F*
F*==============================================================
FSTATES  IF  E           K        DISK
FSFD292  CF  E                    WORKSTN
F                                       @RRN  KSFILE SFD292A
C*
C                     WRITEWINDOW
C                     EXFMTSFD292B
C                     EXSR @CMD
C*
C*********************************************************
C*    @ C M D   S U B R O U T I N E                      *
C*                                                       *
C*    Executed after every EXFMT or display read op code.  *
C*    Used to process command keys.                      *
C*********************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, seton LR and exit.  Since the user did
C* not select a customer, 0 out the CUSN0 field
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C                     ENDSR
C*
C*********************************************************
C*    * I N Z S R   S U B R O U T I N E                  *
C*                                                       *
C*    Retrieve the starting line and position for the    *
C*    window. Check to make sure that these values are   *
C*    valid. If not, put the closest valid number in.    *
C*********************************************************
C*
C           *INZSR    BEGSR
C*
C           *ENTRY    PLIST
C                     PARM           @SL
C                     PARM           @SP
C*
C                     Z-ADD0         @RRN    50
C*
C*  If the start line is greater than 12, our window will not
```

```
C*  fit on the display. Reset the line to 12.
C*
C           @SL       IFGT 12
C                     Z-ADD12        @SL
C                     END
C*
C*  If the start position is greater than 36, our window will not
C*  fit on the display. Reset the position to 36.
C*
C           @SP       IFGT 15
C                     Z-ADD15        @SP
C                     END
C*
C*   Nothing can be displayed from position 1,1 on the display.
C*   If the position is 1,1, reset it to 1,2. Also, check and
C*   make sure that the start line or start position is not zero.
C*
C           @SL       IFLE 1
C           @SP       ANDLE1
C                     Z-ADD1         @SL
C                     Z-ADD2         @SP
C                     END
C*
C*  Load the subfile
C*
C                     EXSR @LOAD
C*
C                     ENDSR
C*
C*************************************************************
C*    @ L O A D     S U B R O U T I N E                      *
C*                                                           *
C*    This subroutine will read the STATES file and          *
C*    load the subfile records. Since this is a load all     *
C*    subfile program, all records in the states file        *
C*    will be read and loaded.                               *
C*************************************************************
C*
C           @LOAD     BEGSR
C*
C*  Perform this loop until you reach the end of
C*  the Customer file.
C*
C           *IN99     DOWNE*ON
C                     READ STATES                  99
C*
C*  If indicator 99 is on, or if you have already loaded
C*  9999 subfile records, (the maximum allowed), then exit
C*  this loop.
C*
C           *IN99     IFEQ *ON
C           @RRN      OREQ 9999
C                     LEAVE
C                     END
C*
C*  Add 1 to the subfile relative record number.
C*
```

```
C                       ADD  1         @RRN
C                       WRITESFD292A
C                       END
C*
C* If you have loaded at least one record into the subfile,
C* turn on indicator 30. This indicator conditions the
C* SFLDSP keyword. If you have loaded at least one record
C* into the subfile, it will be all right to display the subfile
C* format. If you did not load any records into the subfile, you
C* would not turn on indicator 30. Attempting to display
C* an empty subfile would result in an error being issued.
C*
C            @RRN       IFGT 0
C                       MOVE *ON       *IN30
C                       END
C*
C* Since you have loaded all records into the subfile, allow the
C* SFLEND keyword to display the end message.
C*
C                       MOVE *ON       *IN35
C*
C                       ENDSR
```

Notice in SFD292 that the subfile size is set to 50. If the subfile size were set to anything less than this amount and you tried to load 50 states every time, the subfile program would expend a very small amount of extra CPU power to extend the subfile. But over time, these small amounts of CPU add up.

Notice that the value of the subfile page is set to 15. You will have 15 lines of subfile records on each display. If you were creating a single line vertical subfile, setting the subfile page to 15 would mean that you were handling 15 subfile records on each display. A multiline vertical subfile that had three lines per subfile record would mean you would be seeing (15 / 3 = 5) five subfile records per display.

But this subfile is a horizontal subfile, and you have set the SFLLIN to three. You want three spaces in between subfile records. With a window width of 61, the system can fit three subfile records onto each line on the display. So you can view (15 lines * 3 records per line) or 45 subfile records per display.

To see this example, call SFR290. Key in customer 1. Press Enter. Your display should look like this:

```
  xx/xx/xx            Customer Master Maintenance                    QPGMR
  xx:xx:xx                                                           SFD290B


Customer Number....:  00001

Customer Name......: David's Computers and Bait Shop

Customer Address...: 2455 Beuragard

Customer City......: Norfork

Customer State.....: AR

Customer Zip.......: 33455

Phone Number.......: 555-123-6666



F3-Exit    F4-State Search
```

Now press F4 for the state search. Your display should look like this:

```
  xx/xx/xx            Customer Master Maintenance                    QPGMR
  xx:xx:xx                                                           SFD290B


Customer Number....:  00001

Customer Name :.............................................................
              :                                                            :
Customer Addr : Valid State Codes    Valid State Codes    Valid State Codes  :
              : AK ALASKA            FL FLORIDA           ND NORTH DAKOTA    :
Customer City : AL ALABAMA           GA GEORGIA           NM NEW MEXICO      :
              : AR ARKANSAS          LA LOUISIANA         NV NEVADA          :
Customer Stat : CA CALIFORNIA        MS MISSISSIPPI       SC SOUTH CAROLINA  :
              : CN CANADA            NC NORTH CAROLINA    SD SOUTH DAKOTA    :
Customer Zip. :                                                   More... :
              : F3-Exit                                                    :
Phone Number. :                                                            :
              :............................................................:



F3-Exit    F4-State Search
```

The SFLLIN allows you to display the maximum amount of subfile information on a given screen size. If you not specify the SFLLIN keyword, the system automatically places enough spaces in between subfile records to allow one subfile record per line.

Now you can convert the load-all subfile into an expanding subfile. The following reviews the expanding subfile load technique.

Expanding Subfiles

Expanding subfiles are very similar to load-all subfiles, except for the way the loading process takes place. For a load-all subfile, you would load all the records into the subfile before displaying the subfile. The subfile is then able to handle the roll-up and roll-down keys automatically.

With an expanding subfile, you do not load all the records in before displaying the subfile. You load in a fixed number of records, and display the subfile. If the user presses a roll key, the subfile can still perform the rolling of the data back and forth automatically until the user attempts to roll past the last loaded subfile record. At this point, the subfile returns control back to your program, and you can code a routine to load some more records into the subfile.

The roll back key is still handled automatically by the subfile, so you don't have to program for this key.

What you are doing on an expanding subfile is quickly loading a logical number of records into the subfile and displaying these records to the user. If the data that the user needs is not in the subfile, he will eventually roll until he reaches the end of the loaded records. At that time, control returns to your program, and you have to load more records. These records are loaded behind those that you initially loaded. If the user continues to roll past the end of loaded records, your subfile continues to extend itself.

The reason you would want to use an expanding subfile is the possibility of a large number of records that could possibly be loaded. The term large number of records to be loaded is different per machine. One hundred records may be a large number of records for a B10 to load at one time, whereas a model F70 could load several hundred at one time.

In essence, you don't want the user to have to wait too long for the initial subfile screen to come up. With a load-all subfile, the time it takes for the subfile to be displayed is directly related to the number of records to load. By utilizing an extend subfile, you cut the loading process down. Instead of loading all records before displaying the subfile, you load only a fixed number at a time.

Suppose there were a possibility of loading 2,000 subfile records. A load-all subfile would take quite a bit of time, regardless of the size of system you were on, to display the subfile. The load-all subfile would load all 2,000 records first, then display the subfile.

An expanding subfile, on the other hand, loads a fixed number of records at a time, then displays the subfile. If you tell the subfile to load only 20 records at a time, the subfile loads the first 20 of the 2,000 records into the subfile, then displays the subfile. The user may roll forward or backward. The subfile will handle the roll keys automatically, unless the user attempts to roll past subfile record number 20. At that point, the subfile returns control to your program. You then load the next 20 records behind the first 20. Again, the roll keys are handled automatically by the display file unless the user attempts to roll past the last loaded record, in which case you load another 20 records.

Coding an Expanding Subfile Program

Next to load-all subfiles, expanding subfiles are the easiest to program. First, you do not have to worry about coding for the roll-back key, because the display file will

handle this process automatically. You will have to handle the roll-forward key inside your program, but this addition usually takes only a few more lines of code.

For an expanding subfile, the subfile size must be at least 1 more than the subfile page , just like the load-all subfile. Some thought should be placed into this value, and you should set this value closer to the highest number of subfile records that would normally be loaded. This value is not the number of records you will load at a time, but how many records are normally loaded altogether. You might only load the subfile with 20 records at a time, but if the user almost always winds up with 100 records in the subfile, you should set this number to 100. The subfile is initially loaded with 20 records, and then subsequently loaded four more times with 20 records each.

Expanding subfiles are usually the best choice in subfiles for most practical applications, especially when you are loading an unpredictable number of records. Expanding subfiles take just a few more lines of code than a load-all subfile.

To see what kind of coding changes are needed for an expanding subfile versus a load-all subfile, look at program SFR311. It is called by program SFR310.

```
A*==============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD310) SRCFILE(XXX/QDDSSRC)
A*
A*==============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03 'Exit')
A          R SFD310A
A                                       CF04(04 'Call Search Program')
A                                   1  3DATE
A                                       EDTCDE(Y)
A                                   1 23'Customer Master Maintenance'
A                                       DSPATR(HI)
A                                   2  3TIME
A                                   1 71USER
A                                   2 71'SFD310A'
A                                  23  2'F3-Exit   F4-Customer Search'
A                                       DSPATR(HI)
A                                   5  1'Customer Number....:'
A                                       COLOR(BLU)
A                                   7  1'Customer Name......:'
A                                       COLOR(BLU)
A                                   9  1'Customer Address...:'
A                                       COLOR(BLU)
A                                  11  1'Customer City......:'
A                                       COLOR(BLU)
A                                  13  1'Customer State.....:'
A                                       COLOR(BLU)
A                                  15  1'Customer Zip.......:'
A                                       COLOR(BLU)
A                                  17  1'Phone Number.......:'
A                                       COLOR(BLU)
```

```
A            CUSNO     R     Y  B  5 23REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                     COLOR(WHT)
A                                     CHECK(ER)
A                                     CHECK(FE)
A                                     CHECK(RB)
A                                24  1'Customer Number Not Found In Custo-
A                                     mer Master File-Key In A Valid Numb-
A                                     er'
A                                     COLOR(RED)
A                                     DSPATR(BL)
A N99                                 DSPATR(ND)
A          R SFD310B
A                                     CLRL(*NO)
A                                 2 71'SFD310B'
A            CUSNO     R        O  5 23REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                     COLOR(PNK)
A            CUSNM     R        B  7 22REFFLD(RCUST/CUSNM *LIBL/CUSTOMER)
A                                     COLOR(WHT)
A            CUSAD     R        B  9 22REFFLD(RCUST/CUSAD *LIBL/CUSTOMER)
A                                     COLOR(WHT)
A            CUSCT     R        B 11 22REFFLD(RCUST/CUSCT *LIBL/CUSTOMER)
A                                     COLOR(WHT)
A            CUSST     R        B 13 22REFFLD(RCUST/CUSST *LIBL/CUSTOMER)
A                                     COLOR(WHT)
A            CUSCP     R        B 15 22REFFLD(RCUST/CUSCP *LIBL/CUSTOMER)
A                                     COLOR(WHT)
A                                23  1'F3-Exit                           -
A                                        '
A                                     COLOR(WHT)
A            CUSPH     R        B 17 22REFFLD(RCUST/CUSPH *LIBL/CUSTOMER)
A                                     COLOR(WHT)
A                                     EDTWRD('   -   -    ')
A                                24  1'                                   -
A                                                                        -
A                                                 '

F*==============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR310) SRCFILE(XXX/QRPGSRC)
F*
F*==============================================================
FSFD310  CF  E                    WORKSTN
F                                            KINFDS INFDS
FCUSTOMERIF  E           K        DISK
IINFDS       DS
I                                     370 371 LINPOS
I            DS
I                                  B   1   20@LIN
I                                      2   2 @L
I            DS
I                                  B   1   20@POS
I                                      2   2 @P
C*
C                     EXFMTSFD310A
C                     EXSR @CMD
C*
```

```
C           CUSNO     CHAINCUSTOMER            99
C*
C           *IN99     IFEQ *OFF
C                     EXFMTSFD310B
C                     EXSR @CMD
C                     END
C*
C*************************************************************
C*    @ C M D   S U B R O U T I N E                          *
C*                                                           *
C*    Executed after every EXFMT or display read op code.    *
C*    Used to process command keys.                          *
C*************************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, seton LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C* If F4 was pressed, call program SFR381. Pass the customer
C* number field. If the customer number field greater than 0
C* upon return, then the user has selected a customer.
C*
C           *IN04     IFEQ *ON
C*
C* Convert the hex values of the cursor line/position into
C* binary, then to decimal.
C*
C                     Z-ADD0         CUSNO
C                     MOVE *OFF      *IN04
C                     MOVE *ZEROS    @LIN
C                     MOVE *ZEROS    @POS
C                     MOVELLINPOS    @L
C                     MOVE LINPOS    @P
C                     Z-ADD@LIN      @SL     30
C                     Z-ADD@POS      @SP     30
C*
C                     CALL 'SFR311'
C                     PARM           @SL
C                     PARM           @SP
C                     PARM           CUSNO
C*
C                     WRITESFD310A
C*
C                     END
C*
C                     ENDSR


A*============================================================
A* To compile:
A*
```

```
A*      CRTDSPF    FILE(XXX/SFD311) SRCFILE(XXX/QDDSSRC)
A*
A*============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03)
A          R ASSUME
A                                       ASSUME
A                                  24  2' '
A          R SFD311A                    SFL
A            CUSNO     R        O  2  1REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                       COLOR(PNK)
A            CUSNM     R        O  2  9REFFLD(RCUST/CUSNM *LIBL/CUSTOMER)
A                                       COLOR(PNK)
A                                   3 10'State:'
A                                   3 21'Phone:'
A            CUSPH     R        O  3 28REFFLD(RCUST/CUSPH *LIBL/CUSTOMER)
A                                       EDTWRD('   -   -    ')
A            CUSST     R        O  3 17REFFLD(RCUST/CUSST *LIBL/CUSTOMER)
A          R SFD311B                    SFLCTL(SFD311A)
A                                       ROLLUP(25)
A                                       OVERLAY
A                                       SFLCSRRRN(&@REC)
A  30                                   SFLDSP
A                                       SFLDSPCTL
A  35                                   SFLEND(*MORE)
A                                       SFLDROP(CF10)
A                                       SFLSIZ(0012)
A                                       SFLPAG(0003)
A                                       WINDOW(WINDOW)
A                                   1  1'Number'
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A                                   1  9'Customer Name                  '
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A            @REC           5S 0H
A          R WINDOW
A                                       WINDOW(&@SL &@SP 10 40)
A                                       WDWBORDER((*COLOR BLU) (*DSPATR RI)-
A                                        (*CHAR '        '))
A            @SL            3S 0P
A            @SP            3S 0P
A                                   9  1'F3-Exit  F10-Show Additional Info'
A                                       DSPATR(HI)


F*============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR311) SRCFILE(XXX/QRPGSRC)
F*
F*============================================================
FCUSTOMERIF  E           K        DISK
FSFD311  CF  E                    WORKSTN
F                                       @RRN  KSFILE SFD311A
C*
```

```
C                     WRITEWINDOW
C                     EXFMTSFD311B
C                     EXSR @CMD
C*
C*********************************************************
C*    @ C M D   S U B R O U T I N E                      *
C*                                                       *
C*    Executed after every EXFMT or display read op code.  *
C*    Used to process command keys.                      *
C*********************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, seton LR and exit.  Since the user did
C* not select a customer, 0 out the CUSNO field
C*
C           *IN03     IFEQ *ON
C                     Z-ADD0         CUSNO
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C* If *IN25 is on, it means that the user pressed the roll key
C* and that more subfile records were needed to be loaded.
C* (The subfile passed control back to the program.) If the
C* subfile already had the records for the next subfile page
C* in the subfile, control would not pass back to the program. The
C* subfile would handle the rolling, and the program would remain
C* on the EXFMT SFD311B statement.)
C*
C           *IN25     IFEQ *ON
C                     MOVE *OFF      *IN25
C                     EXSR @LOAD
C                     Z-ADD0         @REC
C                     END
C*
C* If the field @REC is greater than 0, it means that the user
C* has selected a record. Chain to the subfile to retrieve the
C* customer number, then return back to the calling program.
C*
C           @REC      IFGT 0
C           @REC      CHAINSFD311A              98
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C                     ENDSR
C*
C*********************************************************
C*    * I N Z S R   S U B R O U T I N E                  *
C*                                                       *
C*    Retrieve the starting line and position for the    *
C*    window. Check to make sure that these values are   *
C*    valid. If not, put the closest valid number in.    *
C*********************************************************
C*
C           *INZSR    BEGSR
```

```
C*
C           *ENTRY    PLIST
C                     PARM           @SL
C                     PARM           @SP
C                     PARM           CUSNO
C*
C                     Z-ADD0         @RRN    50
C*
C*  If the start line is greater than 12, the window will not
C*  fit on the display. Reset the line to 12.
C*
C           @SL       IFGT 12
C                     Z-ADD12        @SL
C                     END
C*
C*  If the start position is greater than 36, the window will not
C*  fit on the display. Reset the position to 36.
C*
C           @SP       IFGT 36
C                     Z-ADD36        @SP
C                     END
C*
C*   Nothing can be displayed from position 1,1 on the display.
C*   If the position is 1,1, reset it to 1,2. Also, check and
C*   make sure that the start line or start position is not zero.
C*
C           @SL       IFLE 1
C           @SP       ANDLE1
C                     Z-ADD1         @SL
C                     Z-ADD2         @SP
C                     END
C*
C*  Load the subfile
C*
C                     EXSR @LOAD
C*
C                     ENDSR
C*
C*************************************************************
C*    @ L O A D     S U B R O U T I N E                      *
C*                                                           *
C*    This subroutine will read the customer file and        *
C*    load the subfile records. Since this is an extended    *
C*    subfile, and the SFLPAG has been set to 10, 10 records*
C*    maximum will be read and loaded at a time.             *
C*                                                           *
C*************************************************************
C*
C           @LOAD     BEGSR
C*
C*  Perform this loop until you either reach the end of
C*  the Customer file or until we load 12 records, which
C*  is 2 pages worth of subfile records. (SFLPAG=6)
C*
C                     DO   12
C                     READ CUSTOMER                 99
C*
```

```
C*  If indicator 99 is on, or if you have already loaded
C*  9999 subfile records, (the maximum allowed), then exit
C*  this loop.
C*
C           *IN99     IFEQ *ON
C           @RRN      OREQ 9999
C                     LEAVE
C                     END
C*
C*  Add 1 to the subfile relative record number.
C*
C                     ADD  1         @RRN
C                     WRITESFD311A
C                     END
C*
C* If you have loaded at least one record into the subfile,
C* turn on indicator 30. This indicator conditions the
C* SFLDSP keyword. If you have loaded at least one record
C* into the subfile, it will be all right to display the subfile
C* format. If you did not load any records into the subfile, you
C* would not turn on indicator 30. Attempting to display
C* an empty subfile would result in an error being issued.
C*
C           @RRN      IFGT 0
C                     MOVE *ON       *IN30
C                     END
C*
C* Allow the SFLEND keyword to display the END messsage.
C*
C                     MOVE *ON       *IN35
C*
C                     ENDSR
```

SFR311 is a copy of SFR281, which was a load-all subfile. The only change made is that you must now define the roll-up key. If you did not define it, the subfile handles the roll up for you, but when you reach the end of the loaded records, it would just tell that you are at the end of the subfile. Instead, what you want to happen is the subfile to return control back to the RPG program, so you can load more records. This is the only structural difference inside the DDS between a load-all and an expanding subfile. When the roll-up key is pressed, you should turn on indicator 25 if control is returned to the program. If control was not returned to the program, as it would not if it was not at the end of the loaded subfile records, then indicator 25 is not turned on.

Also, the value of subfile size was changed, which was not required. As long as the subfile size is one more than the subfile page, you would be OK. However, if most users of this program will roll through at least 12 subfile records, you should set the SFLSIZ to 12.

Now, call SFR310 and see how it works.

```
  xx/xx/xx            Customer Master Maintenance                    QPGMR
  xx:xx:xx                                                           SFD310A


Customer Number....:  00000

Customer Name......:

Customer Address...:

Customer City......:

Customer State.....:

Customer Zip.......:

Phone Number.......:




 F3-Exit   F4-Customer Search
```

Press F4. Your display should look like this:

```
  xx/xx/xx            Customer Master Maintenance                    QPGMR
  xx:xx:xx                                                           SFD310A


Customer Number....:  ..........................................
                      : Number  Customer Name                  :
Customer Name......:  : 00001   David's Computers and Bait Shop  :
                      : 00002   AS/400s Are Us                   :
Customer Address...:  : 00003   Tennessee Minicomputer Sales     :
                      : 00004   PCs For Less                     :
Customer City......:  : 00005   Mid-South Computer Sales         :
                      : 00010   Mini Computing Clearance         :
Customer State.....:  :                                More... :
                      : F3-Exit  F10-Display All Info            :
Customer Zip.......:  :                                          :
                      :..........................................:
Phone Number.......:




 F3-Exit   F4-Customer Search
```

Now roll forward until you see the "Bottom" message, which means that there are no more subfile records to be loaded.

```
  xx/xx/xx            Customer Master Maintenance                    QPGMR
  xx:xx:xx                                                           SFD310A


Customer Number....:  ..........................................
                      : Number  Customer Name                    :
Customer Name......:  : 00099   Hard Drive Warehouse             :
                      : 00386   Memory Systems Inc.              :
Customer Address...:  :                                          :
                      :                                          :
Customer City......:  :                                          :
                      :                                          :
Customer State.....:  :                                   Bottom :
                      : F3-Exit  F10-Display All Info            :
Customer Zip.......:  :                                          :
                      :..........................................:
Phone Number.......:




 F3-Exit   F4-Customer Search
```

Now press roll forward again.

```
  xx/xx/xx            Customer Master Maintenance                    QPGMR
  xx:xx:xx                                                           SFD310A


Customer Number....:  ..........................................
                      : Number  Customer Name                    :
Customer Name......:  : 00001   David's Computers and Bait Shop  :
                      : 00002   AS/400s Are Us                   :
Customer Address...:  : 00003   Tennessee Minicomputer Sales     :
                      : 00004   PCs For Less                     :
Customer City......:  : 00005   Mid-South Computer Sales         :
                      : 00010   Mini Computing Clearance         :
Customer State.....:  :                                  More... :
                      : F3-Exit  F10-Display All Info            :
Customer Zip.......:  :                                          :
                      :..........................................:
Phone Number.......:




 F3-Exit   F4-Customer Search
```

Notice that the display now shows the first subfile screen again instead of telling you that the roll forward key was not valid, like it did on the load-all subfile programs.

The display did this because you defined the roll key in the display file. And if you do not want the roll key to be pressed if the end of the subfile has been reached, then you must code for it.

Also notice that the "Bottom" message does not necessarily mean that there are not more subfile records that can be displayed. The "Bottom" message is displayed every time you display the last page of loaded records. This misrepresents the message, but the subfile doesn't know any difference because it is currently displaying the last subfile page.

A couple of ways to handle this message are described in the next section. Where indicator 35 was turned on to condition the SFLEND keyword, you will now use indicator 99. You will see why in just a minute.

Look at the code for SFD320, SFR320, SFD321, and SFR321.

```
A*============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD320) SRCFILE(XXX/QDDSSRC)
A*
A*============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03 'Exit')
A          R SFD320A
A                                       CF04(04 'Call Search Program')
A                                   1  3DATE
A                                       EDTCDE(Y)
A                                   1 23'Customer Master Maintenance'
A                                       DSPATR(HI)
A                                   2  3TIME
A                                   1 71USER
A                                   2 71'SFD320A'
A                                  23  2'F3-Exit   F4-Customer Search'
A                                       DSPATR(HI)
A                                   5  1'Customer Number....:'
A                                       COLOR(BLU)
A                                   7  1'Customer Name......:'
A                                       COLOR(BLU)
A                                   9  1'Customer Address...:'
A                                       COLOR(BLU)
A                                  11  1'Customer City......:'
A                                       COLOR(BLU)
A                                  13  1'Customer State.....:'
A                                       COLOR(BLU)
A                                  15  1'Customer Zip.......:'
A                                       COLOR(BLU)
A                                  17  1'Phone Number.......:'
A                                       COLOR(BLU)
A            CUSNO     R     Y  B  5 23REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A                                       CHECK(ER)
A                                       CHECK(FE)
```

```
A                                        CHECK(RB)
A                                   24  1'Customer Number Not Found In Custo-
A                                        mer Master File-Key In A Valid Numb-
A                                        er'
A                                        COLOR(RED)
A                                        DSPATR(BL)
A N99                                    DSPATR(ND)
A          R SFD320B
A                                        CLRL(*NO)
A                                    2 71'SFD320B'
A            CUSNO     R        O  5 23REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                        COLOR(PNK)
A            CUSNM     R        B  7 22REFFLD(RCUST/CUSNM *LIBL/CUSTOMER)
A                                        COLOR(WHT)
A            CUSAD     R        B  9 22REFFLD(RCUST/CUSAD *LIBL/CUSTOMER)
A                                        COLOR(WHT)
A            CUSCT     R        B 11 22REFFLD(RCUST/CUSCT *LIBL/CUSTOMER)
A                                        COLOR(WHT)
A            CUSST     R        B 13 22REFFLD(RCUST/CUSST *LIBL/CUSTOMER)
A                                        COLOR(WHT)
A            CUSCP     R        B 15 22REFFLD(RCUST/CUSCP *LIBL/CUSTOMER)
A                                        COLOR(WHT)
A                                   23  1'F3-Exit                          -
A                                           '
A                                        COLOR(WHT)
A            CUSPH     R        B 17 22REFFLD(RCUST/CUSPH *LIBL/CUSTOMER)
A                                        COLOR(WHT)
A                                        EDTWRD('   -   -    ')
A                                   24  1'                                  -
A                                                                           -
A                                                  '


F*==============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR320) SRCFILE(XXX/QRPGSRC)
F*
F*==============================================================
FSFD320  CF  E                   WORKSTN
F                                                 KINFDS INFDS
FCUSTOMERIF  E           K       DISK
IINFDS       DS
I                                        370 371 LINPOS
I            DS
I                                     B   1   20@LIN
I                                         2   2 @L
I            DS
I                                     B   1   20@POS
I                                         2   2 @P
C*
C                      EXFMTSFD320A
C                      EXSR @CMD
C*
C            CUSNO     CHAINCUSTOMER            99
C*
```

```
C           *IN99     IFEQ *OFF
C                     EXFMTSFD320B
C                     EXSR @CMD
C                     END
C*
C*************************************************************
C*    @ C M D   S U B R O U T I N E                          *
C*                                                           *
C*    Executed after every EXFMT or display read op code.    *
C*    Used to process command keys.                          *
C*************************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, seton LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C* If F4 was pressed, call program SFR381. Pass the customer
C* number field. If the customer number field greater than 0
C* upon return, then the user has selected a customer.
C*
C           *IN04     IFEQ *ON
C*
C* Convert the hex values of the cursor line/position into
C* binary, then to decimal.
C*
C                     Z-ADD0         CUSNO
C                     MOVE *OFF      *IN04
C                     MOVE *ZEROS    @LIN
C                     MOVE *ZEROS    @POS
C                     MOVELLINPOS    @L
C                     MOVE LINPOS    @P
C                     Z-ADD@LIN      @SL     30
C                     Z-ADD@POS      @SP     30
C*
C                     CALL 'SFR321'
C                     PARM           @SL
C                     PARM           @SP
C                     PARM           CUSNO
C*
C                     WRITESFD320A
C*
C                     END
C*
C                     ENDSR


A*==============================================================
A* To compile:
A*
```

```
A*      CRTDSPF    FILE(XXX/SFD321) SRCFILE(XXX/QDDSSRC)
A*
A*============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03)
A          R ASSUME
A                                       ASSUME
A                                  24  2' '
A          R SFD321A                    SFL
A            CUSNO     R        O  2  1REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                       COLOR(PNK)
A            CUSNM     R        O  2  9REFFLD(RCUST/CUSNM *LIBL/CUSTOMER)
A                                       COLOR(PNK)
A                                   3 10'State:'
A                                   3 21'Phone:'
A            CUSPH     R        O  3 28REFFLD(RCUST/CUSPH *LIBL/CUSTOMER)
A                                       EDTWRD('   -   -    ')
A            CUSST     R        O  3 17REFFLD(RCUST/CUSST *LIBL/CUSTOMER)
A          R SFD321B                    SFLCTL(SFD321A)
A N99                                   ROLLUP(25)
A                                       OVERLAY
A                                       SFLCSRRRN(&@REC)
A  30                                   SFLDSP
A                                       SFLDSPCTL
A  99                                   SFLEND(*MORE)
A                                       SFLDROP(CF10)
A                                       SFLSIZ(0012)
A                                       SFLPAG(0003)
A                                       WINDOW(WINDOW)
A                                   1  1'Number'
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A                                   1  9'Customer Name                  '
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A            @REC           5S OH
A          R WINDOW
A                                       WINDOW(&@SL &@SP 10 40)
A                                       WDWBORDER((*COLOR BLU) (*DSPATR RI)-
A                                        (*CHAR '        '))
A            @SL            3S OP
A            @SP            3S OP
A                                   9  1'F3-Exit  F10-Show Additonal Info'
A                                       DSPATR(HI)


F*============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR321) SRCFILE(XXX/QRPGSRC)
F*
F*============================================================
FCUSTOMERIF  E           K        DISK
FSFD321  CF  E                    WORKSTN
F                                       @RRN  KSFILE SFD321A
C*
```

```
C                     WRITEWINDOW
C                     EXFMTSFD321B
C                     EXSR @CMD
C*
C*********************************************************
C*    @ C M D   S U B R O U T I N E                      *
C*                                                       *
C*    Executed after every EXFMT or display read op code.  *
C*    Used to process command keys.                      *
C*********************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, seton LR and exit.  Since the user did
C* not select a customer, 0 out the CUSNO field
C*
C           *IN03     IFEQ *ON
C                     Z-ADD0         CUSNO
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C* If *IN25 is on, it means that the user pressed the roll key
C* and that more subfile records were needed to be loaded.
C* (The subfile passed control back to the program.) If the
C* subfile already had the records for the next subfile page
C* in the subfile, control would not pass back to the program. The
C* subfile would handle the rolling, and the program would remain
C* on the EXFMT SFD321B statement.)
C*
C           *IN25     IFEQ *ON
C                     MOVE *OFF      *IN25
C                     EXSR @LOAD
C                     Z-ADD0         @REC
C                     END
C*
C* If the field @REC is greater than 0, it means that the user
C* has selected a record. Chain to the subfile to retrieve the
C* customer number, then return back to the calling program.
C*
C           @REC      IFGT 0
C           @REC      CHAINSFD321A              98
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C                     ENDSR
C*
C*********************************************************
C*    * I N Z S R   S U B R O U T I N E                  *
C*                                                       *
C*    Retrieve the starting line and position for the    *
C*    window. Check to make sure that these values are   *
C*    valid. If not, put the closest valid number in.    *
C*********************************************************
C*
C           *INZSR    BEGSR
```

```
C*
C           *ENTRY    PLIST
C                     PARM           @SL
C                     PARM           @SP
C                     PARM           CUSNO
C*
C                     Z-ADD0         @RRN    50
C*
C*  If the start line is greater than 12, the window will not
C*  fit on the display. Reset the line to 12.
C*
C           @SL       IFGT 12
C                     Z-ADD12        @SL
C                     END
C*
C*  If the start position is greater than 36, the window will not
C*  fit on the display. Reset the position to 36.
C*
C           @SP       IFGT 36
C                     Z-ADD36        @SP
C                     END
C*
C*   Nothing can be displayed from position 1,1 on the display.
C*   If the position is 1,1, reset it to 1,2. Also, check and
C*   make sure that the start line or start position is not zero.
C*
C           @SL       IFLE 1
C           @SP       ANDLE1
C                     Z-ADD1         @SL
C                     Z-ADD2         @SP
C                     END
C*
C*  Load the subfile
C*
C                     EXSR @LOAD
C*
C                     ENDSR
C*
C*************************************************************
C*    @ L O A D     S U B R O U T I N E                      *
C*                                                           *
C*    This subroutine will read the customer file and        *
C*    load the subfile records. Since this is an extended    *
C*    subfile, and the SFLPAG has been set to 10, 10 records*
C*    maximum will be read and loaded at a time.             *
C*                                                           *
C*************************************************************
C*
C           @LOAD     BEGSR
C*
C*  Perform this loop until you reach the end of
C*  the Customer file or until you load 12 records, which
C*  is 2 pages worth of subfile records. (SFLPAG=6)
C*
C                     DO   12
C                     READ CUSTOMER                 99
C*
```

```
C*  If indicator 99 is on, or if you have already loaded
C*  9999 subfile records, (the maximum allowed), then exit
C*  this loop.
C*
C           *IN99     IFEQ *ON
C           @RRN      OREQ 9999
C                     LEAVE
C                     END
C*
C*  Add 1 to our subfile relative record number.
C*
C                     ADD  1         @RRN
C                     WRITESFD321A
C                     END
C*
C* If you have loaded at least one record into the subfile,
C* turn on indicator 30. This indicator conditions the
C* SFLDSP keyword. If you have loaded at least one record
C* into the subfile, it will be all right to display the subfile
C* format. If you did not load any records into the subfile, you
C* would not turn on indicator 30. Attempting to display
C* an empty subfile would result in an error being issued.
C*
C           @RRN      IFGT 0
C                     MOVE *ON       *IN30
C                     END
C*
C                     ENDSR
```

The changes needed were:

```
A N99                                   ROLLUP(25)
```

and

```
A  99                                   SFLEND(*MORE)
```

If you are conditioning the subfile end keyword with indicator 99, the subfile does not display the "Bottom" message unless the indicator 99 is on and the display detects the end of the subfile has been reached.

The problem with the way the subfile end keyword was coded is that it automatically turned on indicator 35. Whenever the subfile detected an end of subfile had been reached (which it would have in this program after every twelve records were displayed), it would have displayed the "Bottom" message even though there were more records to load.

By leaving the SFLEND indicator off, you are telling the subfile not to display the end of subfile message even though it detects an end of subfile condition. There might be more records you can load.

Also notice what was done with the roll key. The roll-up key is a valid key unless indicator 99 is on. If indicator 99 is on, it means you have reached an end of file in

the customer master file. Run the program to see how it works. Then you can look at the program code for an expanding subfile.

Call SFR320. Press the F4 key. Now attempt to roll past the last record. Your display should look like this:

```
  xx/xx/xx            Customer Master Maintenance                    QPGMR
  xx:xx:xx                                                           SFD320A


Customer Number....:  ..........................................
                      : Number  Customer Name                    :
Customer Name......:  : 00099   Hard Drive Warehouse              :
                      : 00386   Memory Systems Inc.               :
Customer Address...:  :                                           :
                      :                                           :
Customer City......:  :                                           :
                      :                                           :
Customer State.....:  :                                   Bottom  :
                      : F3-Exit  F10-Show Additonal Info          :
Customer Zip.......:  : Roll up or down past the first or las...  :
                      :.........................................:
Phone Number.......:




 F3-Exit   F4-Customer Search
```

If you attempt to roll past the last record, the subfile gives you the message that the end of the file has been reached.

Also, notice how the subfile end message now says “MORE....” until the very last subfile page is displayed where it says “Bottom.” With this change, the “Bottom” message is no longer displayed every other page.

Now, look at the program code to see how an expanding subfile works. First, look at the load routine. It is identical to the load-all program except for the DO statement.

```
C                     DO   12
```

This DO statement causes only 12 records to be loaded. When the program is first started, the relative record number field, @RRN, is zero. Therefore, the first 12 records are added to the subfile at relative record number 1 through 12.

Now the program displays the subfile. The program “hangs” at this point in the program unless the user attempts to roll past the twelfth record. If the user does roll past the twelfth record, control is passed back to the program. Since the user pressed

the roll-up key, indicator 25 will be on. The program then runs through the @CMD subroutine.

In the subroutine, the program checks to see if the F3 key was pressed. It was not. Then it checks to see if the roll-up key was pressed by seeing if indicator 25 is on. It is on. The program sets indicator 25 off (you also could have set this indicator within the display file by using the SETOF keyword). Then you perform the load routine again.

This time, in the load routine, the program loads another 12 records. But the relative record numbers is 13 through 24. Then the program displays the subfile again. Now the user has 24 records he can roll back and forth.

Now, if during the last load, the program hits end of file on the customer file, indicator 99 is set on. Then the program displays the subfile again. The user is allowed to use the roll-up key as long they do not attempt to roll past the last subfile record. Once the user rolls past the end of the subfile, control is returned to the program. The display file notices that the roll-up key was pressed, but it was not active because indicator 99 was on. So the display file sends a message telling the user that the key is not valid.

You might have noticed one aggravating thing about this subfile program. It initially loads 12 records. You see six records initially. If you roll forward, you see the next six. If you roll forward again, you would attempt to roll past the 12 loaded subfile records. So the program loads 12 more records and redisplays the subfile. When the subfile is redisplayed, the first subfile page is displayed.

Whenever you display a subfile, it assumes that you want to display the page with subfile relative record number one on it. By default, every time you display a subfile, the first page is displayed.

In a load-all subfile, that is usually what you want. In an expanding subfile, however, you want to initially display the first page, but if you load more records and redisplay the subfile, you don't want to redisplay the first screen. You want the subfile to display the next screen of records, as if you were rolling forward. For example, call SFR320 again. Your display should look like this:

```
  xx/xx/xx            Customer Master Maintenance                    QPGMR
  xx:xx:xx                                                           SFD320A


Customer Number....:  ..........................................
                      : Number  Customer Name                    :
Customer Name......:  : 00001   David's Computers and Bait Shop  :
                      : 00002   AS/400s Are Us                   :
Customer Address...:  : 00003   Tennessee Minicomputer Sales     :
                      : 00004   PCs For Less                     :
Customer City......:  : 00005   Mid-South Computer Sales         :
                      : 00010   Mini Computing Clearance         :
Customer State.....:  :                                 More...  :
                      : F3-Exit  F10-Display All Info            :
Customer Zip.......:  :                                          :
                      :..........................................:
Phone Number.......:




 F3-Exit   F4-Customer Search
```

The display initially shows the first subfile page. By default, whenever you output a subfile, it displays page 1.

The program loads 12 records into the subfile. You see the first six. So, if you roll forward, the subfile handles the rolling to the next page of records. Control is not passed back to the program. Your display looks like this:

```
  xx/xx/xx            Customer Master Maintenance                    QPGMR
  xx:xx:xx                                                           SFD320A


Customer Number....:  ........................................
                      : Number  Customer Name                  :
Customer Name......:  : 00011   Computer Mart                  :
                      : 00025   Westmoreland Computers          :
Customer Address...:  : 00042   Midrange Supercenter            :
                      : 00045   Full Range Computing            :
Customer City......:  : 00054   Open Systems Inc.               :
                      : 00087   SemiCondutor City               :
Customer State.....:  :                                 More... :
                      : F3-Exit  F10-Display All Info           :
Customer Zip.......:  :                                         :
                      :.........................................:
Phone Number.......:




 F3-Exit   F4-Customer Search
```

The subfile automatically handles rolling to page 2. But now you are at the end of the loaded subfile records. If you roll forward again, control passes back to the program. The program loads another twelve records and redisplays the subfile. When subfile is redisplayed, by default it displays page 1. The display looks like this:

```
  xx/xx/xx            Customer Master Maintenance                    QPGMR
  xx:xx:xx                                                           SFD320A


Customer Number....:  ..........................................
                      : Number  Customer Name                    :
Customer Name......:  : 00001   David's Computers and Bait Shop  :
                      : 00002   AS/400s Are Us                   :
Customer Address...:  : 00003   Tennessee Minicomputer Sales     :
                      : 00004   PCs For Less                     :
Customer City......:  : 00005   Mid-South Computer Sales         :
                      : 00010   Mini Computing Clearance         :
Customer State.....:  :                                  More... :
                      : F3-Exit  F10-Display All Info            :
Customer Zip.......:  :                                          :
                      :..........................................:
Phone Number.......:




 F3-Exit   F4-Customer Search
```

But this isn't what you want to happen. If the user rolls forward, you want the next subfile page, which is page 3, to be displayed.

Subfile Record Number

You can control which subfile page the subfile displays when you output the subfile by using the Subfile Record Number (SFLRCDNBR) keyword.

By placing a value in the subfile record number keyword field, the subfile will locate the page on which that subfile record resides. When it finds the page, it displays that page first. Then you can roll up and down from that point.

In our current expanding subfile example, you load two pages worth of subfile records at a time. When the subfile first comes up, you see page one. When you roll forward, you see page two. Since you are now at the end of the subfile, the next time you roll, control is passed back to the program to load 12 more records. When the display comes back up, you would like to see page three, which comes after page two. But what you see after page two is page one again. Then you can roll forward and see pages two, three, and four. If you roll forward again, the program would load twelve more records and the subfile would redisplay at page one, and so on.

To rectify this problem, you must save the first subfile relative record number that was loaded during this process into the SFLRCDNBR field. When you first call the program, the first relative record number is one. If you roll past the 12th record, the program will load another 12 records into the subfile. Now the program will save relative record number 13 in the SFLRCDNBR field. When the program displays the subfile, it will initially display the subfile page on which this record exists. Since there are six records per display, relative record number 13 resides on page three, which is the page that you want displayed after page two.

The necessary coding changes have been made in SFD324, SFR324, SFD325, SFR325:

```
A*============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD324) SRCFILE(XXX/QDDSSRC)
A*
A*============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03 'Exit')
A          R SFD324A
A                                       CF04(04 'Call Search Program')
A                                  1  3DATE
A                                       EDTCDE(Y)
A                                  1 23'Customer Master Maintenance'
A                                       DSPATR(HI)
A                                  2  3TIME
A                                  1 71USER
A                                  2 71'SFD324A'
A                                 23  2'F3-Exit   F4-Customer Search'
A                                       DSPATR(HI)
A                                  5  1'Customer Number....:'
A                                       COLOR(BLU)
A                                  7  1'Customer Name......:'
A                                       COLOR(BLU)
A                                  9  1'Customer Address...:'
A                                       COLOR(BLU)
A                                 11  1'Customer City......:'
A                                       COLOR(BLU)
A                                 13  1'Customer State.....:'
A                                       COLOR(BLU)
A                                 15  1'Customer Zip.......:'
A                                       COLOR(BLU)
A                                 17  1'Phone Number.......:'
A                                       COLOR(BLU)
A            CUSNO     R     Y  B  5 23REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A                                       CHECK(ER)
A                                       CHECK(FE)
A                                       CHECK(RB)
A                                 24  1'Customer Number Not Found In Custo-
A                                       mer Master File-Key In A Valid Numb-
A                                       er'
A                                       COLOR(RED)
A                                       DSPATR(BL)
```

```
A N99                                   DSPATR(ND)
A          R SFD324B
A                                       CLRL(*NO)
A                                  2 71'SFD324B'
A            CUSNO     R        O  5 23REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                       COLOR(PNK)
A            CUSNM     R        B  7 22REFFLD(RCUST/CUSNM *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A            CUSAD     R        B  9 22REFFLD(RCUST/CUSAD *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A            CUSCT     R        B 11 22REFFLD(RCUST/CUSCT *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A            CUSST     R        B 13 22REFFLD(RCUST/CUSST *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A            CUSCP     R        B 15 22REFFLD(RCUST/CUSCP *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A                                 23  1'F3-Exit                          -
A                                         '
A                                       COLOR(WHT)
A            CUSPH     R        B 17 22REFFLD(RCUST/CUSPH *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A                                       EDTWRD('   -   -    ')
A                                 24  1'                                  -
A                                                                         -
A                                                 '


F*============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR324) SRCFILE(XXX/QRPGSRC)
F*
F*============================================================
FSFD324  CF  E                    WORKSTN
F                                            KINFDS INFDS
FCUSTOMERIF  E           K        DISK
IINFDS       DS
I                                       370 371 LINPOS
I            DS
I                                    B   1   20@LIN
I                                        2   2 @L
I            DS
I                                    B   1   20@POS
I                                        2   2 @P
C*
C                      EXFMTSFD324A
C                      EXSR @CMD
C*
C           CUSNO      CHAINCUSTOMER            99
C*
C           *IN99      IFEQ *OFF
C                      EXFMTSFD324B
C                      EXSR @CMD
C                      END
C*
C*************************************************************
```

```
C*    @ C M D   S U B R O U T I N E                          *
C*                                                           *
C*    Executed after every EXFMT or display read op code.    *
C*    Used to process command keys.                          *
C*************************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, seton LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C* If F4 was pressed, call program SFR381. Pass the customer
C* number field. If the customer number field greater than 0
C* upon return, then the user has selected a customer.
C*
C           *IN04     IFEQ *ON
C*
C* Convert the hex values of the cursor line/position into
C* binary, then to decimal.
C*
C                     Z-ADD0         CUSNO
C                     MOVE *OFF      *IN04
C                     MOVE *ZEROS    @LIN
C                     MOVE *ZEROS    @POS
C                     MOVELLINPOS    @L
C                     MOVE LINPOS    @P
C                     Z-ADD@LIN      @SL     30
C                     Z-ADD@POS      @SP     30
C*
C                     CALL 'SFR325'
C                     PARM           @SL
C                     PARM           @SP
C                     PARM           CUSNO
C*
C                     WRITESFD324A
C*
C                     END
C*
C                     ENDSR


A*==============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD325) SRCFILE(XXX/QDDSSRC)
A*
A*==============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03)
A          R ASSUME
A                                       ASSUME
A                                  24  2' '
```

```
A          R SFD325A                    SFL
A            CUSNO     R        O  2  1REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                      COLOR(PNK)
A            CUSNM     R        O  2  9REFFLD(RCUST/CUSNM *LIBL/CUSTOMER)
A                                      COLOR(PNK)
A                                 3 10'State:'
A                                 3 21'Phone:'
A            CUSPH     R        O  3 28REFFLD(RCUST/CUSPH *LIBL/CUSTOMER)
A                                      EDTWRD('   -   -    ')
A            CUSST     R        O  3 17REFFLD(RCUST/CUSST *LIBL/CUSTOMER)
A          R SFD325B                    SFLCTL(SFD325A)
A N99                                   ROLLUP(25)
A                                       OVERLAY
A                                       SFLCSRRRN(&@REC)
A  30                                   SFLDSP
A                                       SFLDSPCTL
A  99                                   SFLEND(*MORE)
A                                       SFLDROP(CF10)
A                                       SFLSIZ(0012)
A                                       SFLPAG(0003)
A                                       WINDOW(WINDOW)
A            @PAG            4  0H      SFLRCDNBR
A                                 1  1'Number'
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A                                 1  9'Customer Name                  '
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A            @REC            5S 0H
A          R WINDOW
A                                       WINDOW(&@SL &@SP 10 40)
A                                       WDWBORDER((*COLOR BLU) (*DSPATR RI)-
A                                        (*CHAR '        '))
A            @SL             3S 0P
A            @SP             3S 0P
A                                 9  1'F3-Exit  F10-Show Additonal Info'
A                                       DSPATR(HI)


F*=============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR325) SRCFILE(XXX/QRPGSRC)
F*
F*=============================================================
FCUSTOMERIF  E           K        DISK
FSFD325  CF  E                    WORKSTN
F                                       @RRN  KSFILE SFD325A
C*
C                     WRITEWINDOW
C                     EXFMTSFD325B
C                     EXSR @CMD
C*
C*************************************************************
C*    @ C M D   S U B R O U T I N E                          *
C*                                                           *
```

```
C*    Executed after every EXFMT or display read op code.   *
C*    Used to process command keys.                         *
C*********************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, seton LR and exit.  Since the user did
C* not select a customer, 0 out the CUSNO field
C*
C           *IN03     IFEQ *ON
C                     Z-ADD0         CUSNO
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C* If *IN25 is on, it means that the user pressed the roll key
C* and that more subfile records were needed to be loaded.
C* (The subfile passed control back to the program.) If the
C* subfile already had the records for the next subfile page
C* in the subfile, control would not pass back to the program. The
C* subfile would handle the rolling, and the program would remain
C* on the EXFMT SFD325B statement.)
C*
C           *IN25     IFEQ *ON
C                     MOVE *OFF      *IN25
C                     EXSR @LOAD
C                     Z-ADD0         @REC
C                     END
C*
C* If the field @REC is greater than 0, it means that the user
C* has selected a record. Chain to the subfile to retrieve the
C* customer number, then return back to the calling program.
C*
C           @REC      IFGT 0
C           @REC      CHAINSFD325A              98
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C                     ENDSR
C*
C*********************************************************
C*    * I N Z S R   S U B R O U T I N E                  *
C*                                                       *
C*    Retrieve the starting line and position for the    *
C*    window. Check to make sure that these values are   *
C*    valid. If not, put the closest valid number in.    *
C*********************************************************
C*
C           *INZSR    BEGSR
C*
C           *ENTRY    PLIST
C                     PARM           @SL
C                     PARM           @SP
C                     PARM           CUSNO
C*
C                     Z-ADD0         @RRN    50
```

```
C*
C*  If the start line is greater than 12, the window will not
C*  fit on the display. Reset the line to 12.
C*
C           @SL       IFGT 12
C                     Z-ADD12        @SL
C                     END
C*
C*  If the start position is greater than 36, the window will not
C*  fit on the display. Reset the position to 36.
C*
C           @SP       IFGT 36
C                     Z-ADD36        @SP
C                     END
C*
C*   Nothing can be displayed from position 1,1 on the display.
C*   If the position is 1,1, reset it to 1,2. Also, check and
C*   make sure that the start line or start position is not zero.
C*
C           @SL       IFLE 1
C           @SP       ANDLE1
C                     Z-ADD1         @SL
C                     Z-ADD2         @SP
C                     END
C*
C*  Load the subfile
C*
C                     EXSR @LOAD
C*
C                     ENDSR
C*
C*************************************************************
C*    @ L O A D     S U B R O U T I N E                     *
C*                                                            *
C*    This subroutine will read the customer file and         *
C*    load the subfile records. Since this is an extended     *
C*    subfile, and the SFLPAG has been set to 10, 10 records*
C*    maximum will be read and loaded at a time.              *
C*                                                            *
C*************************************************************
C*
C           @LOAD     BEGSR
C*
C* Set the first subfile page to display
C*
C                     Z-ADD@RRN      @PAG
C*
C*  Perform this loop until you either reach the end of
C*  the Customer file or until you load 12 records, which
C*  is 2 pages worth of subfile records. (SFLPAG=6)
C*
C                     DO   12
C                     READ CUSTOMER                 99
C*
C*  If indicator 99 is on, or if you have already loaded
C*  9999 subfile records, (the maximum allowed), then exit
C*  this loop.
```

```
C*
C           *IN99     IFEQ *ON
C           @RRN      OREQ 9999
C                     LEAVE
C                     END
C*
C*  Add 1 to the subfile relative record number.
C*
C                     ADD  1         @RRN
C                     WRITESFD325A
C                     END
C*
C* If you have loaded at least one record into the subfile,
C* turn on indicator 30. This indicator conditions the
C* SFLDSP keyword. If you have loaded at least one record
C* into the subfile, it will be all right to display the subfile
C* format. If you did not load any records into the subfile, you
C* would not turn on indicator 30. Attempting to display
C* an empty subfile would result in an error being issued.
C*
C           @RRN      IFGT 0
C                     MOVE *ON       *IN30
C                     ELSE
C                     MOVE *OFF      *IN30
C                     END
C*
C*  If you added at least one record during the load routine,
C*  then cause the subfile to display the next page. If you did
C*  not add any records, display the last subfile page that was
C*  on the display.
C*
C           @PAG      IFNE @RRN
C                     ADD  1         @PAG
C                     END
C*
C                     ENDSR
```

Call SFR324. Press F4 to call SFR325. Your display should look like this:

```
  xx/xx/xx            Customer Master Maintenance                    QPGMR
  xx:xx:xx                                                           SFD325A


Customer Number....:  ..........................................
                      : Number  Customer Name                    :
Customer Name......:  : 00001   David's Computers and Bait Shop  :
                      : 00002   AS/400s Are Us                   :
Customer Address...:  : 00003   Tennessee Minicomputer Sales     :
                      : 00004   PCs For Less                     :
Customer City......:  : 00005   Mid-South Computer Sales         :
                      : 00010   Mini Computing Clearance         :
Customer State.....:  :                                 More... :
                      : F3-Exit  F10-Display All Info            :
Customer Zip.......:  :                                          :
                      :..........................................:
Phone Number.......:




 F3-Exit   F4-Customer Search
```

Now roll forward. The subfile will automatically place us on page 2:

```
  xx/xx/xx            Customer Master Maintenance                    QPGMR
  xx:xx:xx                                                           SFD325A


Customer Number....:  ..........................................
                      : Number  Customer Name                    :
Customer Name......:  : 00011   Computer Mart                    :
                      : 00025   Westmoreland Computers           :
Customer Address...:  : 00042   Midrange Supercenter             :
                      : 00045   Full Range Computing             :
Customer City......:  : 00054   Open Systems Inc.                :
                      : 00087   SemiCondutor City                :
Customer State.....:  :                                 More... :
                      : F3-Exit  F10-Display All Info            :
Customer Zip.......:  :                                          :
                      :..........................................:
Phone Number.......:




 F3-Exit   F4-Customer Search
```

Now roll forward again. Since the program has to load 12 more records, it saves the first loaded relative record number into the SFLRCDNBR field, which is 13. Now when the program outputs the subfile again, you get the following:

```
  xx/xx/xx            Customer Master Maintenance                    QPGMR
  xx:xx:xx                                                           SFD325A


Customer Number....:  ..........................................
                      : Number  Customer Name                    :
Customer Name......:  : 00099   Hard Drive Warehouse             :
                      : 00386   Memory Systems Inc.              :
Customer Address...:  :                                          :
                      :                                          :
Customer City......:  :                                          :
                      :                                          :
Customer State.....:  :                                   Bottom :
                      : F3-Exit  F10-Display All Info            :
Customer Zip.......:  :                                          :
                      :..........................................:
Phone Number.......:




 F3-Exit   F4-Customer Search
```

The subfile displayed page 3. Notice how the display continues to roll, instead of being reset to page 1 every time it needs to load more records.

The code required to do this was very minimal. First the SFLRCDNBR keyword was added to the DDS.

```
A             @PAG            4  0H       SFLRCDNBR
```

By placing a value in the field @PAG, the program tells the subfile to position itself to the page on which the relative record number of the value @PAG resides.

It is important that this number contain an actual relative record number that resides in the subfile, or else the program will receive an error. The value of SFLRCDNBR must be the value of an existing subfile record. It cannot be zero when the program displays the subfile. In other words, when the program outputs the control format, if the conditioning indicator on the SFLDSP is true, then it attempts to display the subfile format, and SFLRCDNBR must contain a valid value. SFLRCDNBR can contain an invalid value as long as the program does not display the subfile, such as would be the case if the program did not load any records into the subfile.

The code required for the SFLRCDNBR keyword in our RPG program is as follows:

```
C                       Z-ADD@RRN      @PAG
```

This routine resides at the very top of the load procedure. The program saves the current relative record number value into the SFLRCDNBR field @PAG. The first time the program performs a load, the value is zero. The second time the program performs a load, the value is 12, and so on.

Right after the load routine has processed, the program checks to see if it added any records to the subfile, which is accomplished with the following code:

```
C            @PAG      IFNE @RRN
C                      ADD  1         @PAG
C                      END
```

If the program adds records to the subfile, it increments the relative record pointer for the SFLRCDNBR field. So the first time it loads records, @PAG initially gets set to 0. If the program adds records during the load routine, it increments the number to 1. When it displays the subfile, it locates the page on which subfile relative record 12 resides, and displays it.

The next time the program loads records, it saves the current value of @RRN into @PAG, so @PAG has the value of 12. If the program did not load any records into the subfile during the second load process, @PAG retains the value 12. The program cannot automatically increment the value to 13, because if it did not load any records and @PAG contained the value 13, it would be issued an error. So if the program does not load any records, it is telling the system to redisplay the last subfile page. If the program does load records, then @PAG gets incremented, and it contains the value 13, which causes the third subfile page to be displayed.

Using a Position To Field

Suppose you want to allow a users to key in a customer name, or part of a customer name, in which to start the display. You would have to give them an input field in which to do that. This program is shown in the following example:

```
A*==============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD330) SRCFILE(XXX/QDDSSRC)
A*
A*==============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03 'Exit')
A          R SFD330A
A                                       CF04(04 'Call Search Program')
A                                   1  3DATE
A                                       EDTCDE(Y)
```

```
A                                   1 23'Customer Master Maintenance'
A                                       DSPATR(HI)
A                                   2  3TIME
A                                   1 71USER
A                                   2 71'SFD330A'
A                                  23  2'F3-Exit   F4-Customer Search'
A                                       DSPATR(HI)
A                                   5  1'Customer Number....:'
A                                       COLOR(BLU)
A                                   7  1'Customer Name......:'
A                                       COLOR(BLU)
A                                   9  1'Customer Address...:'
A                                       COLOR(BLU)
A                                  11  1'Customer City......:'
A                                       COLOR(BLU)
A                                  13  1'Customer State.....:'
A                                       COLOR(BLU)
A                                  15  1'Customer Zip.......:'
A                                       COLOR(BLU)
A                                  17  1'Phone Number.......:'
A                                       COLOR(BLU)
A            CUSNO     R     Y  B  5 23REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A                                       CHECK(ER)
A                                       CHECK(FE)
A                                       CHECK(RB)
A                                  24  1'Customer Number Not Found In Custo-
A                                       mer Master File-Key In A Valid Numb-
A                                       er'
A                                       COLOR(RED)
A                                       DSPATR(BL)
A N99                                   DSPATR(ND)
A          R SFD330B
A                                       CLRL(*NO)
A                                   2 71'SFD330B'
A            CUSNO     R        O  5 23REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                       COLOR(PNK)
A            CUSNM     R        B  7 22REFFLD(RCUST/CUSNM *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A            CUSAD     R        B  9 22REFFLD(RCUST/CUSAD *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A            CUSCT     R        B 11 22REFFLD(RCUST/CUSCT *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A            CUSST     R        B 13 22REFFLD(RCUST/CUSST *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A            CUSCP     R        B 15 22REFFLD(RCUST/CUSCP *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A                                  23  1'F3-Exit                          -
A                                          '
A                                       COLOR(WHT)
A            CUSPH     R        B 17 22REFFLD(RCUST/CUSPH *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A                                       EDTWRD('   -   -    ')
A                                  24  1'                                  -
A                                                                          -
A                                                 '
```

```
F*==============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR330) SRCFILE(XXX/QRPGSRC)
F*
F*==============================================================
FSFD330  CF  E                    WORKSTN
F                                            KINFDS INFDS
FCUSTOMERIF  E           K        DISK
IINFDS       DS
I                                     370 371 LINPOS
I            DS
I                                   B   1   20@LIN
I                                       2   2 @L
I            DS
I                                   B   1   20@POS
I                                       2   2 @P
C*
C                     EXFMTSFD330A
C                     EXSR @CMD
C*
C           CUSNO     CHAINCUSTOMER             99
C*
C           *IN99     IFEQ *OFF
C                     EXFMTSFD330B
C                     EXSR @CMD
C                     END
C*
C*************************************************************
C*    @ C M D   S U B R O U T I N E                          *
C*                                                           *
C*    Executed after every EXFMT or display read op code.    *
C*    Used to process command keys.                          *
C*************************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, seton LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C* If F4 was pressed, call program SFR381. Pass the customer
C* number field. If the customer number field greater than 0
C* upon return, then the user has selected a customer.
C*
C           *IN04     IFEQ *ON
C*
C* Convert the hex values of the cursor line/position into
C* binary, then to decimal.
C*
C                     Z-ADD0         CUSNO
C                     MOVE *OFF      *IN04
C                     MOVE *ZEROS    @LIN
C                     MOVE *ZEROS    @POS
```

```
C                     MOVELLINPOS    @L
C                     MOVE LINPOS    @P
C                     Z-ADD@LIN      @SL     30
C                     Z-ADD@POS      @SP     30
C*
C                     CALL 'SFR331'
C                     PARM           @SL
C                     PARM           @SP
C                     PARM           CUSNO
C*
C                     WRITESFD330A
C*
C                     END
C*
C                     ENDSR


A*============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD331) SRCFILE(XXX/QDDSSRC)
A*
A*============================================================
A                                      DSPSIZ(24 80 *DS3)
A                                      CF03(03)
A          R ASSUME
A                                      ASSUME
A                                 24  2' '
A          R SFD331A                   SFL
A            CUSNO     R        O  2  1REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                      COLOR(PNK)
A            CUSNM     R        O  2  9REFFLD(RCUST/CUSNM *LIBL/CUSTOMER)
A                                      COLOR(PNK)
A                                  3 10'State:'
A                                  3 21'Phone:'
A            CUSPH     R        O  3 28REFFLD(RCUST/CUSPH *LIBL/CUSTOMER)
A                                      EDTWRD('   -   -    ')
A            CUSST     R        O  3 17REFFLD(RCUST/CUSST *LIBL/CUSTOMER)
A          R SFD331B                   SFLCTL(SFD331A)
A                                      SFLSIZ(0012)
A                                      SFLPAG(0003)
A                                      WINDOW(WINDOW)
A N99                                  ROLLUP(25)
A                                      OVERLAY
A                                      SFLCSRRRN(&@REC)
A  30                                  SFLDSP
A                                      SFLDSPCTL
A  99                                  SFLEND(*MORE)
A                                      SFLDROP(CF10)
A            @PAG           4  0H      SFLRCDNBR
A            @REC           5S 0H
A                                  1  3'Name:'
A            @SRCH     R        B  1  9REFFLD(RCUST/CUSNM *LIBL/CUSTOMER)
A                                      DSPATR(HI)
A          R WINDOW
A                                      WINDOW(&@SL &@SP 10 40)
```

```
A                                       WDWBORDER((*COLOR BLU) (*DSPATR RI)-
A                                        (*CHAR '        '))
A            @SL            3S OP
A            @SP            3S OP
A                                 9  1'F3-Exit  F10-Show Additonal Info'
A                                     DSPATR(HI)


F*==============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR331) SRCFILE(XXX/QRPGSRC)
F*
F*==============================================================
FCUSTL1  IF  E           K        DISK
FSFD331  CF  E                    WORKSTN
F                                       @RRN  KSFILE SFD331A
C*
C                     WRITEWINDOW
C                     EXFMTSFD331B
C                     EXSR @CMD
C*
C*************************************************************
C*    @ C M D   S U B R O U T I N E                          *
C*                                                           *
C*    Executed after every EXFMT or display read op code.    *
C*    Used to process command keys.                          *
C*************************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, seton LR and exit.  Since the user did
C* not select a CUSTL1  , 0 out the CUSN0 field
C*
C           *IN03     IFEQ *ON
C                     Z-ADD0         CUSN0
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C* If *IN25 is on, it means that the user pressed the roll key
C* and that more subfile records were needed to be loaded.
C* (The subfile passed control back to the program.) If the
C* subfile already had the records for the next subfile page
C* in the subfile, control would not pass back to the program. The
C* subfile would handle the rolling, and the program would remain
C* on the EXFMT SFD331B statement.)
C*
C           *IN25     IFEQ *ON
C                     MOVE *OFF      *IN25
C                     EXSR @LOAD
C                     Z-ADD0         @REC
C                     END
C*
C* If the field @REC is greater than 0, it means that the user
C* has selected a record. Chain to the subfile to retrieve the
```

```
C* CUSTL1   number, then return back to the calling program.
C*
C           @REC      IFGT 0
C           @REC      CHAINSFD331A              98
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C* If the fields @SRCH and @SAVE are not equal, then the user
C* requested to load the subfile starting from a specific name.
C*
C           @SRCH     IFNE @SAVE
C                     MOVE @SRCH     @SAVE
C           @SRCH     SETLLCUSTL1
C                     EXSR @LOAD
C                     END
C*
C                     ENDSR
C*
C*********************************************************
C*    * I N Z S R   S U B R O U T I N E                  *
C*                                                       *
C*    Retrieve the starting line and position for the    *
C*    window. Check to make sure that these values are   *
C*    valid. If not, put the closest valid number in.    *
C*********************************************************
C*
C           *INZSR    BEGSR
C*
C           *ENTRY    PLIST
C                     PARM           @SL
C                     PARM           @SP
C                     PARM           CUSNO
C*
C                     Z-ADD0         @RRN    50
C           *LIKE     DEFN @SRCH     @SAVE
C*
C*  If the start line is greater than 12, the window will not
C*  fit on the display. Reset the line to 12.
C*
C           @SL       IFGT 12
C                     Z-ADD12        @SL
C                     END
C*
C*  If the start position is greater than 36, the window will not
C*  fit on the display. Reset the position to 36.
C*
C           @SP       IFGT 36
C                     Z-ADD36        @SP
C                     END
C*
C*   Nothing can be displayed from position 1,1 on the display.
C*   If the position is 1,1, reset it to 1,2. Also, check and
C*   make sure that the start line or start position is not zero.
C*
C           @SL       IFLE 1
C           @SP       ANDLE1
```

```
C                     Z-ADD1         @SL
C                     Z-ADD2         @SP
C                     END
C*
C*  Load the subfile
C*
C                     EXSR @LOAD
C*
C                     ENDSR
C*
C*************************************************************
C*    @ L O A D     S U B R O U T I N E                     *
C*                                                            *
C*    This subroutine will read the CUSTL1   file and         *
C*    load the subfile records. Since this is an extended     *
C*    subfile, and the SFLPAG has been set to 10, 10 records*
C*    maximum will be read and loaded at a time.              *
C*                                                            *
C*************************************************************
C*
C           @LOAD     BEGSR
C*
C                     MOVE *OFF      *IN99
C*
C* Set the first subfile page to display
C*
C                     Z-ADD@RRN      @PAG
C*
C*  Perform this loop until you reach the end of
C*  the CUSTL1   file or until we load 12 records, which
C*  is 2 pages worth of subfile records. (SFLPAG=6)
C*
C                     DO   12
C                     READ CUSTL1                   99
C*
C*  If indicator 99 is on, or if you have already loaded
C*  9999 subfile records, (the maximum allowed), then exit
C*  this loop.
C*
C           *IN99     IFEQ *ON
C           @RRN      OREQ 9999
C                     LEAVE
C                     END
C*
C*  Add 1 to our subfile relative record number.
C*
C                     ADD  1         @RRN
C                     WRITESFD331A
C                     END
C*
C* If you have loaded at least one record into the subfile,
C* turn on indicator 30. This indicator conditions the
C* SFLDSP keyword. If you have loaded at least one record
C* into the subfile, it will be all right to display the subfile
C* format. If you did not load any records into the subfile, you
C* would not turn on indicator 30. Attempting to display
C* an empty subfile would result in an error being issued.
```

```
C*
C           @RRN      IFGT 0
C                     MOVE *ON       *IN30
C                     END
C*
C*
C*  If you added at least one record during the load routine,
C*  then cause the subfile to display the next page. If you did
C*  not add any records, display the last subfile page that was
C*  on the display.
C*
C           @PAG      IFNE @RRN
C                     ADD  1         @PAG
C                     END
C*
C*
C* Ensure that the field @PAG is not 0, if so force it to 1
C*
C           @PAG      IFEQ 0
C                     Z-ADD1         @PAG
C                     END
C                     ENDSR
```

The user can key in a partial name in which to start the subfile display. Since the name field is now the key, instead of reading the customer file to load the subfile records, the program continues to read the CUSTL1 file. This file is the customer file by customer name key sequence.

The following routine was added to the subfile program.

```
C           @SRCH     IFNE @SAVE
C                     MOVE @SRCH     @SAVE
C           @SRCH     SETLLCUSTL1
C                     EXSR @LOAD
C                     END
```

The program initially displays the subfile starting from the beginning of the file. If the user decides to position the subfile starting at a specific name, they key into the @SRCH field. The program detects that they have keyed something by comparing the value of @SRCH to @SAVE. If they are not equal, the user has keyed something into this field.

If the user keys something into the field, the program saves this new value into @SAVE. Then it sets lower limits on the customer file and begins the subfile loading process from there. But this program has a problem with it. To see the problem, run SFR330. Press F4. Your display should look like this:

```
  xx/xx/xx            Customer Master Maintenance                    QPGMR
  xx:xx:xx                                                           SFD330A


Customer Number....:  ........................................
                      :   Name: ________________________________   :
Customer Name......:  : 00002   AS/400s Are Us                    :
                      : 00011   Computer Mart                     :
Customer Address...:  : 00001   David's Computers and Bait Shop   :
                      : 00045   Full Range Computing              :
Customer City......:  : 00099   Hard Drive Warehouse              :
                      : 00386   Memory Systems Inc.               :
Customer State.....:  :                                  More...  :
                      : F3-Exit  F10-Show Additonal Info          :
Customer Zip.......:  :                                           :
                      :...........................................:
Phone Number.......:




 F3-Exit   F4-Customer Search
```

Key a "W" into the search field and press Enter. Your display should look like this:

```
  xx/xx/xx            Customer Master Maintenance                    QPGMR
  xx:xx:xx                                                           SFD330A


Customer Number....:  ........................................
                      :   Name: W_______________________________   :
Customer Name......:  : 00025   Westmoreland Computers            :
                      :                                           :
Customer Address...:  :                                           :
                      :                                           :
Customer City......:  :                                           :
                      :                                           :
Customer State.....:  :                                   Bottom  :
                      : F3-Exit  F10-Show Additonal Info          :
Customer Zip.......:  :                                           :
                      :...........................................:
Phone Number.......:




 F3-Exit   F4-Customer Search
```

Your display positions itself to the Ws, so it looks like it worked. But key in the letter "A," and press Enter. Your display will look like this:

```
  xx/xx/xx            Customer Master Maintenance                    QPGMR
  xx:xx:xx                                                           SFD330A


Customer Number....:  ........................................
                      :   Name: A_______________________________   :
Customer Name......:  : 00025   Westmoreland Computers            :
                      : 00002   AS/400s Are Us                    :
Customer Address...:  : 00011   Computer Mart                     :
                      : 00001   David's Computers and Bait Shop   :
Customer City......:  : 00045   Full Range Computing              :
                      : 00099   Hard Drive Warehouse              :
Customer State.....:  :                                 More...   :
                      : F3-Exit  F10-Show Additonal Info          :
Customer Zip.......:  :                                           :
                      :...........................................:
Phone Number.......:




 F3-Exit   F4-Customer Search
```

What happened? When you roll all the way through the subfile, you have every record in the customer file loaded into the subfile. When you keyed in a "W," the program positioned itself to the Ws in the customer file. When you keyed in the "A," the program positioned itself back to the top of the customer file. Then it loaded 12 more records into the subfile behind the ones that were already loaded. In the next section, you'll see how to solve this problem.

Subfile Clear

If the user changes the position-to field, the program needs to clear out all the existing records in the subfile. Then it can set lower limits on the file and start filling up the subfile up. You can do this with the subfile clear (SFLCLR) keyword. Subfile clear removes all records out of a subfile.

The next example contains the code for the subfile clear keyword:

```
A*==============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD340) SRCFILE(XXX/QDDSSRC)
A*
```

```
A*==============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03 'Exit')
A          R SFD340A
A                                       CF04(04 'Call Search Program')
A                                   1  3DATE
A                                       EDTCDE(Y)
A                                   1 23'Customer Master Maintenance'
A                                       DSPATR(HI)
A                                   2  3TIME
A                                   1 71USER
A                                   2 71'SFD340A'
A                                  23  2'F3-Exit   F4-Customer Search'
A                                       DSPATR(HI)
A                                   5  1'Customer Number....:'
A                                       COLOR(BLU)
A                                   7  1'Customer Name......:'
A                                       COLOR(BLU)
A                                   9  1'Customer Address...:'
A                                       COLOR(BLU)
A                                  11  1'Customer City......:'
A                                       COLOR(BLU)
A                                  13  1'Customer State.....:'
A                                       COLOR(BLU)
A                                  15  1'Customer Zip.......:'
A                                       COLOR(BLU)
A                                  17  1'Phone Number.......:'
A                                       COLOR(BLU)
A            CUSNO     R     Y  B  5 23REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A                                       CHECK(ER)
A                                       CHECK(FE)
A                                       CHECK(RB)
A                                  24  1'Customer Number Not Found In Custo-
A                                       mer Master File-Key In A Valid Numb-
A                                       er'
A                                       COLOR(RED)
A                                       DSPATR(BL)
A N99                                   DSPATR(ND)
A          R SFD340B
A                                       CLRL(*NO)
A                                   2 71'SFD340B'
A            CUSNO     R        O  5 23REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                       COLOR(PNK)
A            CUSNM     R        B  7 22REFFLD(RCUST/CUSNM *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A            CUSAD     R        B  9 22REFFLD(RCUST/CUSAD *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A            CUSCT     R        B 11 22REFFLD(RCUST/CUSCT *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A            CUSST     R        B 13 22REFFLD(RCUST/CUSST *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A            CUSCP     R        B 15 22REFFLD(RCUST/CUSCP *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A                                  23  1'F3-Exit                          -
A                                          '
A                                       COLOR(WHT)
```

```
A            CUSPH     R        B 17 22REFFLD(RCUST/CUSPH *LIBL/CUSTOMER)
A                                     COLOR(WHT)
A                                     EDTWRD('   -   -    ')
A                                24  1'                                  -
A                                                                        -
A                                            '


F*============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR340) SRCFILE(XXX/QRPGSRC)
F*
F*============================================================
FSFD340  CF  E                    WORKSTN
F                                            KINFDS INFDS
FCUSTOMERIF  E           K        DISK
IINFDS       DS
I                                     370 371 LINPOS
I            DS
I                                  B   1   20@LIN
I                                      2   2 @L
I            DS
I                                  B   1   20@POS
I                                      2   2 @P
C*
C                     EXFMTSFD340A
C                     EXSR @CMD
C*
C           CUSNO     CHAINCUSTOMER             99
C*
C           *IN99     IFEQ *OFF
C                     EXFMTSFD340B
C                     EXSR @CMD
C                     END
C*
C*********************************************************
C*    @ C M D   S U B R O U T I N E                      *
C*                                                       *
C*    Executed after every EXFMT or display read op code.  *
C*    Used to process command keys.                      *
C*********************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, seton LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C* If F4 was pressed, call program SFR381. Pass the customer
C* number field. If the customer number field greater than 0
C* upon return, then the user has selected a customer.
C*
```

```
C           *IN04     IFEQ *ON
C*
C* Convert the hex values of the cursor line/position into
C* binary, then to decimal.
C*
C                     Z-ADD0         CUSNO
C                     MOVE *OFF      *IN04
C                     MOVE *ZEROS    @LIN
C                     MOVE *ZEROS    @POS
C                     MOVELLINPOS    @L
C                     MOVE LINPOS    @P
C                     Z-ADD@LIN      @SL     30
C                     Z-ADD@POS      @SP     30
C*
C                     CALL 'SFR341'
C                     PARM           @SL
C                     PARM           @SP
C                     PARM           CUSNO
C*
C                     WRITESFD340A
C*
C                     END
C*
C                     ENDSR


A*============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD341) SRCFILE(XXX/QDDSSRC)
A*
A*============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03)
A          R ASSUME
A                                       ASSUME
A                                  24  2' '
A          R SFD341A                    SFL
A            CUSNO     R        O  2  1REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                       COLOR(PNK)
A            CUSNM     R        O  2  9REFFLD(RCUST/CUSNM *LIBL/CUSTOMER)
A                                       COLOR(PNK)
A                                   3 10'State:'
A                                   3 21'Phone:'
A            CUSPH     R        O  3 28REFFLD(RCUST/CUSPH *LIBL/CUSTOMER)
A                                       EDTWRD('   -   -    ')
A            CUSST     R        O  3 17REFFLD(RCUST/CUSST *LIBL/CUSTOMER)
A          R SFD341B                    SFLCTL(SFD341A)
A N99                                   ROLLUP(25)
A                                       OVERLAY
A                                       SFLCSRRRN(&@REC)
A  30                                   SFLDSP
A                                       SFLDSPCTL
A  31                                   SFLCLR
A  99                                   SFLEND(*MORE)
A                                       SFLDROP(CF10)
```

```
A                                       SFLSIZ(0012)
A                                       SFLPAG(0003)
A                                       WINDOW(WINDOW)
A            @PAG           4  0H       SFLRCDNBR
A            @REC           5S 0H
A                                  1  3'Name:'
A            @SRCH     R        B  1  9REFFLD(RCUST/CUSNM *LIBL/CUSTOMER)
A                                       DSPATR(HI)
A          R WINDOW
A*%%TS  SD  19931203  223011  QSECOFR     REL-V2R2M0  5738-PW1
A                                       WINDOW(&@SL &@SP 10 40)
A                                       WDWBORDER((*COLOR BLU) (*DSPATR RI)-
A                                        (*CHAR '        '))
A            @SL            3S 0P
A            @SP            3S 0P
A                                  9  1'F3-Exit  F10-Show Additonal Info'
A                                       DSPATR(HI)


F*=============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR341) SRCFILE(XXX/QRPGSRC)
F*
F*=============================================================
FCUSTL1  IF  E           K        DISK
FSFD341  CF  E                    WORKSTN
F                                       @RRN  KSFILE SFD341A
C*
C                     WRITEWINDOW
C                     EXFMTSFD341B
C                     EXSR @CMD
C*
C*********************************************************
C*    @ C M D   S U B R O U T I N E                      *
C*                                                       *
C*    Executed after every EXFMT or display read op code.  *
C*    Used to process command keys.                      *
C*********************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, seton LR and exit.  Since the user did
C* not select a CUSTL1  , 0 out the CUSN0 field
C*
C           *IN03     IFEQ *ON
C                     Z-ADD0         CUSN0
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C* If *IN25 is on, it means that the user pressed the roll key
C* and that more subfile records were needed to be loaded.
C* (The subfile passed control back to the program.) If the
C* subfile already had the records for the next subfile page
C* in the subfile, control would not pass back to the program. The
```

```
C* subfile would handle the rolling, and the program would remain
C* on the EXFMT SFD341B statement.)
C*
C           *IN25     IFEQ *ON
C                     MOVE *OFF      *IN25
C                     EXSR @LOAD
C                     Z-ADD0         @REC
C                     END
C*
C* If the field @REC is greater than 0, it means that the user
C* has selected a record. Chain to the subfile to retrieve the
C* CUSTL1   number, then return back to the calling program.
C*
C           @REC      IFGT 0
C           @REC      CHAINSFD341A              98
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C* If the fields @SRCH and @SAVE are not equal, then the user
C* requested to load the subfile starting from a specific name.
C*
C           @SRCH     IFNE @SAVE
C                     MOVE @SRCH     @SAVE
C*
C* The following three lines of code will clear the subfile
C*
C                     MOVE *ON       *IN31
C                     WRITESFD341B
C                     MOVE *OFF      *IN31
C*
C* Since you cleared the subfile, reset the subfile relative
C* record number to 0.
C*
C                     Z-ADD0         @RRN
C           @SRCH     SETLLCUSTL1
C                     EXSR @LOAD
C                     END
C*
C                     ENDSR
C*
C*************************************************************
C*    * I N Z S R   S U B R O U T I N E                      *
C*                                                           *
C*    Retrieve the starting line and position for the        *
C*    window. Check to make sure that these values are       *
C*    valid. If not, put the closest valid number in.        *
C*************************************************************
C*
C           *INZSR    BEGSR
C*
C           *ENTRY    PLIST
C                     PARM           @SL
C                     PARM           @SP
C                     PARM           CUSNO
C*
C                     Z-ADD0         @RRN    50
```

```
C           *LIKE     DEFN @SRCH     @SAVE
C*
C*  If the start line is greater than 12, the window will not
C*  fit on the display. Reset the line to 12.
C*
C           @SL       IFGT 12
C                     Z-ADD12        @SL
C                     END
C*
C*  If the start position is greater than 36, the window will not
C*  fit on the display. Reset the position to 36.
C*
C           @SP       IFGT 36
C                     Z-ADD36        @SP
C                     END
C*
C*   Nothing can be displayed from position 1,1 on the display.
C*   If the position is 1,1, reset it to 1,2. Also, check and
C*   make sure that the start line or start position is not zero.
C*
C           @SL       IFLE 1
C           @SP       ANDLE1
C                     Z-ADD1         @SL
C                     Z-ADD2         @SP
C                     END
C*
C*  Load the subfile
C*
C                     EXSR @LOAD
C*
C                     ENDSR
C*
C*********************************************************
C*    @ L O A D     S U B R O U T I N E                  *
C*                                                       *
C*    This subroutine will read the CUSTL1   file and    *
C*    load the subfile records. Since this is an extended *
C*    subfile, and the SFLPAG has been set to 10, 10 records*
C*    maximum will be read and loaded at a time.         *
C*                                                       *
C*********************************************************
C*
C           @LOAD     BEGSR
C*
C                     MOVE *OFF      *IN99
C*
C* Set the first subfile page to display
C*
C                     Z-ADD@RRN      @PAG
C*
C*  Perform this loop until you reach the end of
C*  the CUSTL1   file or until you load 12 records, which
C*  is 2 pageS worth of subfile records. (SFLPAG=6)
C*
C                     DO   12
C                     READ CUSTL1                  99
C*
```

```
C*  If indicator 99 is on, or if you have already loaded
C*  9999 subfile records, (the maximum allowed), then exit
C*  this loop.
C*
C           *IN99     IFEQ *ON
C           @RRN      OREQ 9999
C                     LEAVE
C                     END
C*
C*  Add 1 to the subfile relative record number.
C*
C                     ADD  1         @RRN
C                     WRITESFD341A
C                     END
C*
C* If you have loaded at least one record into the subfile,
C* turn on indicator 30. This indicator conditions the
C* SFLDSP keyword. If you have loaded at least one record
C* into the subfile, it will be all right to display the subfile
C* format. If you did not load any records into the subfile, you
C* would not turn on indicator 30. Attempting to display
C* an empty subfile would result in an error being issued.
C*
C           @RRN      IFGT 0
C                     MOVE *ON       *IN30
C                     ELSE
C                     MOVE *OFF      *IN30
C                     END
C*
C*  If you added at least one record during the load routine,
C*  then cause the subfile to display the next page. If you did
C*  not add any records, display the last subfile page that was
C*  on the display.
C*
C           @PAG      IFNE @RRN
C                     ADD  1         @PAG
C                     END
C*
C                     ENDSR
```

Notice in the DDS specifications for SFD341 that the SFLCLR keyword was added to the subfile control format.

```
A  31                                   SFLCLR
```

If the program outputs the subfile control format and indicator 31 is on, then the subfile clears itself of all subfile records. If the program outputs to the subfile control format and 31 is not on, then the subfile is not cleared. If the program attempts to clear a subfile that is already empty, the subfile ignores the request. The RPG program controls when to clear the subfile with the following code:

```
C           @SRCH     IFNE @SAVE
C                     MOVE @SRCH     @SAVE
C*
C                     MOVE *ON       *IN31
```

```
C                       WRITESFD341B
C                       MOVE *OFF       *IN31
C*
C                       Z-ADDO          @RRN
C             @SRCH     SETLLCUSTL1
C                       EXSR @LOAD
C                       END
```

If the user changes the position-to field, the program sets indicator 31 on and writes the subfile control format. This write clears the subfile of all records. Then the program sets off indicator 31, so that when it is ready to display the newly loaded subfile records, the subfile won't clear itself again. The program also sets the value of @RRN to 0, since it will be starting from the beginning of the subfile again.

To see what these changes have done, call SFR420 and press F4. Your display should look like this:

```
  xx/xx/xx            Customer Master Maintenance                    QPGMR
  xx:xx:xx                                                           SFD340A


Customer Number....:  ..........................................
                      :   Name: ______________________________   :
Customer Name......:  : 00002   AS/400s Are Us                   :
                      : 00011   Computer Mart                    :
Customer Address...:  : 00001   David's Computers and Bait Shop  :
                      : 00045   Full Range Computing             :
Customer City......:  : 00099   Hard Drive Warehouse             :
                      : 00386   Memory Systems Inc.              :
Customer State.....:  :                                 More... :
                      : F3-Exit  F10-Show Additonal Info         :
Customer Zip.......:  :                                          :
                      :..........................................:
Phone Number.......:




 F3-Exit   F4-Customer Search
```

When the display comes up, key in a “W”. Now press Enter. Your display should look like this:

```
  xx/xx/xx            Customer Master Maintenance                    QPGMR
  xx:xx:xx                                                           SFD340A


Customer Number....:  ........................................
                      :   Name: W_______________________________   :
Customer Name......:  : 00025   Westmoreland Computers           :
                      :                                           :
Customer Address...:  :                                           :
                      :                                           :
Customer City......:  :                                           :
                      :                                           :
Customer State.....:  :                                   Bottom  :
                      : F3-Exit  F10-Show Additonal Info          :
Customer Zip.......:  :                                           :
                      :...........................................:
Phone Number.......:




 F3-Exit   F4-Customer Search
```

When you key in a “W,” the program clears the subfile and sets the file pointer for CUSTL1 to the Ws. Then the program attempts to load 12 records from the CUSTL1 file into the subfile. In this case, it finds only one record before it reaches the end of the file.

Attempt to roll backwards. You don’t see any records and you are told that you are at the beginning of the subfile. When you key in a value to start searching, the program doesn’t load all the records from the customer file. It starts loading from the value keyed into the field @SRCH.

Now key in the letter "Z" and press Enter. Your screen should look like this:

```
  xx/xx/xx            Customer Master Maintenance                    QPGMR
  xx:xx:xx                                                           SFD340A


Customer Number....:  ..........................................
                      :   Name: Z_______________________________   :
Customer Name......:  :                                            :
                      :                                            :
Customer Address...:  :                                            :
                      :                                            :
Customer City......:  :                                            :
                      :                                            :
Customer State.....:  :                                     Bottom :
                      : F3-Exit  F10-Show Additonal Info           :
Customer Zip.......:  :                                            :
                      :............................................:
Phone Number.......:




 F3-Exit   F4-Customer Search
```

Notice that nothing comes up in the subfile because there are no customers that start with a "Z." Since there are no records in the subfile, the program has to ensure that it does not display the subfile. This change was made when the indicator 30 routine was modified :

```
C            @RRN       IFGT 0
C                       MOVE *ON       *IN30
C                       ELSE
C                       MOVE *OFF      *IN30
C                       END
```

When you keyed a "Z," the subfile contained no records because none were loaded. Therefore, @RRN has a value of 0. In this case, indicator 30 is set off, which conditions the SFLDSP keyword and keeps the subfile format from being displayed. If the program attempts to display the subfile format, and it does not contain any records, it is issued an error. This routine keeps the program from displaying an empty subfile format. The program does, however, display the subfile control format because you did not condition the SFLDSPCTL keyword.

Subfile Enter

Now take a look at some more subfile functions you can use. The first one is the Subfile Enter (SFLENTER) keyword. This keyword makes the Enter key act like the roll-up key. You can then define a function key to act as the Enter key.

One reason you might want to do this is for mouse-controlled terminals. A user can point and click for a window subfile to pop up, and then use the Enter mouse button to roll the subfile forward. The user has more accessibility with the mouse. Instead of having to flip back and forth between the mouse and keyboard, the user can just use the mouse. There are other ways to use the mouse to control rolling forward or backward, which you will see later.

The changes needed for the SFLENTER keyword have been made in the following source members:

```
A*==============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD350) SRCFILE(XXX/QDDSSRC)
A*
A*==============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03 'Exit')
A          R SFD350A
A                                       CF04(04 'Call Search Program')
A                                   1  3DATE
A                                       EDTCDE(Y)
A                                   1 23'Customer Master Maintenance'
A                                       DSPATR(HI)
A                                   2  3TIME
A                                   1 71USER
A                                   2 71'SFD350A'
A                                  23  2'F3-Exit   F4-Customer Search'
A                                       DSPATR(HI)
A                                   5  1'Customer Number....:'
A                                       COLOR(BLU)
A                                   7  1'Customer Name......:'
A                                       COLOR(BLU)
A                                   9  1'Customer Address...:'
A                                       COLOR(BLU)
A                                  11  1'Customer City......:'
A                                       COLOR(BLU)
A                                  13  1'Customer State.....:'
A                                       COLOR(BLU)
A                                  15  1'Customer Zip.......:'
A                                       COLOR(BLU)
A                                  17  1'Phone Number.......:'
A                                       COLOR(BLU)
A            CUSNO     R     Y  B  5 23REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A                                       CHECK(ER)
A                                       CHECK(FE)
```

```
A                                        CHECK(RB)
A                                   24  1'Customer Number Not Found In Custo-
A                                        mer Master File-Key In A Valid Numb-
A                                        er'
A                                        COLOR(RED)
A                                        DSPATR(BL)
A N99                                    DSPATR(ND)
A          R SFD350B
A                                        CLRL(*NO)
A                                    2 71'SFD350B'
A            CUSNO     R        O  5 23REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                        COLOR(PNK)
A            CUSNM     R        B  7 22REFFLD(RCUST/CUSNM *LIBL/CUSTOMER)
A                                        COLOR(WHT)
A            CUSAD     R        B  9 22REFFLD(RCUST/CUSAD *LIBL/CUSTOMER)
A                                        COLOR(WHT)
A            CUSCT     R        B 11 22REFFLD(RCUST/CUSCT *LIBL/CUSTOMER)
A                                        COLOR(WHT)
A            CUSST     R        B 13 22REFFLD(RCUST/CUSST *LIBL/CUSTOMER)
A                                        COLOR(WHT)
A            CUSCP     R        B 15 22REFFLD(RCUST/CUSCP *LIBL/CUSTOMER)
A                                        COLOR(WHT)
A                                   23  1'F3-Exit                          -
A                                           '
A                                        COLOR(WHT)
A            CUSPH     R        B 17 22REFFLD(RCUST/CUSPH *LIBL/CUSTOMER)
A                                        COLOR(WHT)
A                                        EDTWRD('   -   -    ')
A                                   24  1'                                  -
A                                                                           -
A                                                   '


F*============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR350) SRCFILE(XXX/QRPGSRC)
F*
F*============================================================
FSFD350  CF  E                   WORKSTN
F                                             KINFDS INFDS
FCUSTOMERIF  E           K       DISK
IINFDS       DS
I                                        370 371 LINPOS
I            DS
I                                     B   1   20@LIN
I                                         2   2 @L
I            DS
I                                     B   1   20@POS
I                                         2   2 @P
C*
C                      EXFMTSFD350A
C                      EXSR @CMD
C*
C           CUSNO      CHAINCUSTOMER            99
C*
```

```
C           *IN99     IFEQ *OFF
C                     EXFMTSFD350B
C                     EXSR @CMD
C                     END
C*
C*************************************************************
C*    @ C M D   S U B R O U T I N E                          *
C*                                                           *
C*    Executed after every EXFMT or display read op code.    *
C*    Used to process command keys.                          *
C*************************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, seton LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C* If F4 was pressed, call program SFR381. Pass the customer
C* number field. If the customer number field greater than 0
C* upon return, then the user has selected a customer.
C*
C           *IN04     IFEQ *ON
C*
C* Convert the hex values of the cursor line/position into
C* binary, then to decimal.
C*
C                     Z-ADD0         CUSNO
C                     MOVE *OFF      *IN04
C                     MOVE *ZEROS    @LIN
C                     MOVE *ZEROS    @POS
C                     MOVELLINPOS    @L
C                     MOVE LINPOS    @P
C                     Z-ADD@LIN      @SL     30
C                     Z-ADD@POS      @SP     30
C*
C                     CALL 'SFR351'
C                     PARM           @SL
C                     PARM           @SP
C                     PARM           CUSNO
C*
C                     WRITESFD350A
C*
C                     END
C*
C                     ENDSR


A*==============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD351) SRCFILE(XXX/QDDSSRC)
A*
```

```
A*============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03)
A          R ASSUME
A                                       ASSUME
A                                  24  2' '
A          R SFD351A                    SFL
A            CUSNO     R        O  2  1REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                       COLOR(PNK)
A            CUSNM     R        O  2  9REFFLD(RCUST/CUSNM *LIBL/CUSTOMER)
A                                       COLOR(PNK)
A                                   3 10'State:'
A                                   3 21'Phone:'
A            CUSPH     R        O  3 28REFFLD(RCUST/CUSPH *LIBL/CUSTOMER)
A                                       EDTWRD('   -   -    ')
A            CUSST     R        O  3 17REFFLD(RCUST/CUSST *LIBL/CUSTOMER)
A          R SFD351B                    SFLCTL(SFD351A)
A N99                                   ROLLUP(25)
A                                       OVERLAY
A                                       SFLCSRRRN(&@REC)
A  30                                   SFLDSP
A                                       SFLDSPCTL
A  31                                   SFLCLR
A  99                                   SFLEND(*MORE)
A                                       SFLDROP(CF10)
A                                       SFLENTER(CF20)
A                                       SFLSIZ(0012)
A                                       SFLPAG(0003)
A                                       WINDOW(WINDOW)
A            @PAG           4S OH       SFLRCDNBR
A            @REC           5S OH
A                                   1  3'Name:'
A            @SRCH     R        B  1  9REFFLD(RCUST/CUSNM *LIBL/CUSTOMER)
A                                       DSPATR(HI)
A          R WINDOW
A                                       WINDOW(&@SL &@SP 10 40)
A                                       WDWBORDER((*COLOR BLU) (*DSPATR RI)-
A                                        (*CHAR '        '))
A            @SL            3S OP
A            @SP            3S OP
A                                   9  1'F3-Exit  F10-Show Additonal Info'
A                                       DSPATR(HI)


F*============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR351) SRCFILE(XXX/QRPGSRC)
F*
F*============================================================
FCUSTL1  IF  E           K        DISK
FSFD351  CF  E                    WORKSTN
F                                       @RRN  KSFILE SFD351A
C*
C                     WRITEWINDOW
C                     EXFMTSFD351B
```

```
C                     EXSR @CMD
C*
C*************************************************************
C*    @ C M D   S U B R O U T I N E                          *
C*                                                           *
C*    Executed after every EXFMT or display read op code.    *
C*    Used to process command keys.                          *
C*************************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, seton LR and exit.  Since the user did
C* not select a CUSTL1  , 0 out the CUSN0 field
C*
C           *IN03     IFEQ *ON
C                     Z-ADD0         CUSN0
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C* If *IN25 is on, it means that the user pressed the roll key
C* and that more subfile records were needed to be loaded.
C* (The subfile passed control back to the program.) If the
C* subfile already had the records for the next subfile page
C* in the subfile, control would not pass back to the program. The
C* subfile would handle the rolling, and the program would remain
C* on the EXFMT SFD351B statement.)
C*
C           *IN25     IFEQ *ON
C                     MOVE *OFF      *IN25
C                     EXSR @LOAD
C                     Z-ADD0         @REC
C                     END
C*
C* If the field @REC is greater than 0, it means that the user
C* has selected a record. Chain to the subfile to retrieve the
C* CUSTL1   number, then return back to the calling program.
C*
C           @REC      IFGT 0
C           @REC      CHAINSFD351A              98
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C* If the fields @SRCH and @SAVE are not equal, then the user
C* requested to load the subfile starting from a specific name.
C*
C           @SRCH     IFNE @SAVE
C                     MOVE @SRCH     @SAVE
C*
C* The following three lines of code will clear the subfile
C*
C                     MOVE *ON       *IN31
C                     WRITESFD351B
C                     MOVE *OFF      *IN31
C*
C* Since you cleared the subfile, reset the subfile relative
```

```
C* record number to 0.
C*
C                     Z-ADD0         @RRN
C           @SRCH     SETLLCUSTL1
C                     EXSR @LOAD
C                     END
C*
C                     ENDSR
C*
C*********************************************************
C*    * I N Z S R   S U B R O U T I N E                  *
C*                                                       *
C*    Retrieve the starting line and position for the    *
C*    window. Check to make sure that these values are   *
C*    valid. If not, put the closest valid number in.    *
C*********************************************************
C*
C           *INZSR    BEGSR
C*
C           *ENTRY    PLIST
C                     PARM           @SL
C                     PARM           @SP
C                     PARM           CUSNO
C*
C                     Z-ADD0         @RRN    50
C           *LIKE     DEFN @SRCH     @SAVE
C*
C*  If the start line is greater than 12, the window will not
C*  fit on the display. Reset the line to 12.
C*
C           @SL       IFGT 12
C                     Z-ADD12        @SL
C                     END
C*
C*  If the start position is greater than 36, the window will not
C*  fit on the display. Reset the position to 36.
C*
C           @SP       IFGT 36
C                     Z-ADD36        @SP
C                     END
C*
C*   Nothing can be displayed from position 1,1 on the display.
C*   If the position is 1,1, reset it to 1,2. Also, check and
C*   make sure that the start line or start position is not zero.
C*
C           @SL       IFLE 1
C           @SP       ANDLE1
C                     Z-ADD1         @SL
C                     Z-ADD2         @SP
C                     END
C*
C*  Load the subfile
C*
C                     EXSR @LOAD
C*
C                     ENDSR
C*
```

```
C*********************************************************
C*    @ L O A D     S U B R O U T I N E                  *
C*                                                       *
C*    This subroutine will read the CUSTL1   file and    *
C*    load the subfile records. Since this is an extended *
C*    subfile, and the SFLPAG has been set to 10, 10 records*
C*    maximum will be read and loaded at a time.         *
C*                                                       *
C*********************************************************
C*
C           @LOAD     BEGSR
C*
C                     MOVE *OFF      *IN99
C*
C* Set the first subfile page to display
C*
C                     Z-ADD@RRN      @PAG
C*
C*  Perform this loop until you reach the end of
C*  the CUSTL1   file or until you load 12 records, which
C*  is 2 pages worth of subfile records. (SFLPAG=6)
C*
C                     DO   12
C                     READ CUSTL1                  99
C*
C*  If indicator 99 is on, or if you have already loaded
C*  9999 subfile records, (the maximum allowed), then exit
C*  this loop.
C*
C           *IN99     IFEQ *ON
C           @RRN      OREQ 9999
C                     LEAVE
C                     END
C*
C*  Add 1 to our subfile relative record number.
C*
C                     ADD  1         @RRN
C                     WRITESFD351A
C                     END
C*
C* If you have loaded at least one record into the subfile,
C* turn on indicator 30. This indicator conditions the
C* SFLDSP keyword. If you have loaded at least one record
C* into the subfile, it will be all right to display the subfile
C* format. If you did not load any records into the subfile, you
C* would turn off indicator 30. Attempting to display
C* an empty subfile would result in an error being issued.
C*
C           @RRN      IFGT 0
C                     MOVE *ON       *IN30
C                     ELSE
C                     MOVE *OFF      *IN30
C                     END
C*
C*  If you added at least one record during the load routine,
C*  then cause the subfile to display the next page. If you did
C*  not add any records, display the last subfile page that was
```

```
C*  on the display.
C*
C           @PAG      IFNE @RRN
C                     ADD  1         @PAG
C                     END
C*
C                     ENDSR
```

You can see that the DDS keyword SFLENTER was added to SFD351.

```
     A                                        SFLENTER(CF20)
```

This line tells the subfile to allow both the roll-up key and the Enter key to function as the roll-up key. It also specifies that the F20 key act like the Enter key. You can test this function by pressing the roll up and Enter key, and you will find that the subfile rolls forward. Now place the cursor on a subfile record and press F20. The record is selected into SFR350, just like the Enter key functioned previously.

What is nice about this keyword is that the roll up and roll down keys still function as usual. So if you have a mixture of mouse controlled terminals and keyboard-controlled terminals, the keyboard-controlled terminals can use the roll-up key, whereas the mouse-controlled terminals can use the mouse button. Again, there are better ways of creating mouse-controlled rolling, which will be explored later. The most common use of the Subfile Enter keyword is to allow this application to simulate other applications that you have in which the roll key is used to roll a display forward. This simulation frequently occurs in S/36 code. The Subfile Enter keyword allows you to keep all of your application's function keys identical. However, it should be stated that when you develop new applications, the old, out-of-date standards should be abandoned in favor of the newer programming techniques. For instance, S/36 code used the Enter key because it was very difficult to program for the roll-up key. As a matter of fact, you had to trap the roll-up key as an error condition. For this reason, a lot of programmers used the Enter key to roll a display. Since there were a lot of applications that used the Enter key as the roll-up key, the Subfile Enter keyword was created to allow you to keep your existing and new application function keys identical.

Subfile Roll Value

You can also add the ability to roll less than one page worth of records to duplicate what some PC programs do. You may have noticed that in Microsoft Windows a display that resembles a subfile display can be set to roll up only one line at a time (when you attempt to open a file, for example).

You can duplicate this effect by using the Subfile Roll Value (SFLROLVAL) keyword. If this keyword is not specified, then the subfile is set to roll one full page of records each time you use the roll-up or roll-down keys. If you specify the SFLROLVAL keyword, and set the value greater than the number of subfile records on a page, the subfile still only rolls one page at a time. But if you set the value of SFLROLVAL to anything less than the page size, the subfile rolls backwards and forwards the number of records specified on the SFLROLVAL keyword.

The following source members have been modified to make use of the SFLROLVAL keyword:

```
A*==============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD360) SRCFILE(XXX/QDDSSRC)
A*
A*==============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03 'Exit')
A          R SFD360A
A                                       CF04(04 'Call Search Program')
A                                   1  3DATE
A                                       EDTCDE(Y)
A                                   1 23'Customer Master Maintenance'
A                                       DSPATR(HI)
A                                   2  3TIME
A                                   1 71USER
A                                   2 71'SFD360A'
A                                  23  2'F3-Exit   F4-Customer Search'
A                                       DSPATR(HI)
A                                   5  1'Customer Number....:'
A                                       COLOR(BLU)
A                                   7  1'Customer Name......:'
A                                       COLOR(BLU)
A                                   9  1'Customer Address...:'
A                                       COLOR(BLU)
A                                  11  1'Customer City......:'
A                                       COLOR(BLU)
A                                  13  1'Customer State.....:'
A                                       COLOR(BLU)
A                                  15  1'Customer Zip.......:'
A                                       COLOR(BLU)
A                                  17  1'Phone Number.......:'
A                                       COLOR(BLU)
A            CUSNO     R     Y  B  5 23REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A                                       CHECK(ER)
A                                       CHECK(FE)
A                                       CHECK(RB)
A                                  24  1'Customer Number Not Found In Custo-
A                                       mer Master File-Key In A Valid Numb-
A                                       er'
A                                       COLOR(RED)
A                                       DSPATR(BL)
```

```
A N99                                   DSPATR(ND)
A          R SFD360B
A                                       CLRL(*NO)
A                                  2 71'SFD360B'
A            CUSNO     R        O  5 23REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                       COLOR(PNK)
A            CUSNM     R        B  7 22REFFLD(RCUST/CUSNM *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A            CUSAD     R        B  9 22REFFLD(RCUST/CUSAD *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A            CUSCT     R        B 11 22REFFLD(RCUST/CUSCT *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A            CUSST     R        B 13 22REFFLD(RCUST/CUSST *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A            CUSCP     R        B 15 22REFFLD(RCUST/CUSCP *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A                                 23  1'F3-Exit                          -
A                                         '
A                                       COLOR(WHT)
A            CUSPH     R        B 17 22REFFLD(RCUST/CUSPH *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A                                       EDTWRD('   -   -    ')
A                                 24  1'                                 -
A                                                                        -
A                                                 '


F*============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR360) SRCFILE(XXX/QRPGSRC)
F*
F*============================================================
FSFD360  CF  E                    WORKSTN
F                                            KINFDS INFDS
FCUSTOMERIF  E           K        DISK
IINFDS       DS
I                                       370 371 LINPOS
I            DS
I                                    B   1   20@LIN
I                                        2   2 @L
I            DS
I                                    B   1   20@POS
I                                        2   2 @P
C*
C                     EXFMTSFD360A
C                     EXSR @CMD
C*
C           CUSNO     CHAINCUSTOMER             99
C*
C           *IN99     IFEQ *OFF
C                     EXFMTSFD360B
C                     EXSR @CMD
C                     END
C*
C*************************************************************
```

```
C*    @ C M D   S U B R O U T I N E                          *
C*                                                           *
C*    Executed after every EXFMT or display read op code.    *
C*    Used to process command keys.                          *
C*************************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, seton LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C* If F4 was pressed, call program SFR381. Pass the customer
C* number field. If the customer number field greater than 0
C* upon return, then the user has selected a customer.
C*
C           *IN04     IFEQ *ON
C*
C* Convert the hex values of the cursor line/position into
C* binary, then to decimal.
C*
C                     Z-ADD0         CUSNO
C                     MOVE *OFF      *IN04
C                     MOVE *ZEROS    @LIN
C                     MOVE *ZEROS    @POS
C                     MOVELLINPOS    @L
C                     MOVE LINPOS    @P
C                     Z-ADD@LIN      @SL     30
C                     Z-ADD@POS      @SP     30
C*
C                     CALL 'SFR361'
C                     PARM           @SL
C                     PARM           @SP
C                     PARM           CUSNO
C*
C                     WRITESFD360A
C*
C                     END
C*
C                     ENDSR


A*==============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD361) SRCFILE(XXX/QDDSSRC)
A*
A*==============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03)
A          R ASSUME
A                                       ASSUME
A                                  24  2' '
```

```
A          R SFD361A                    SFL
A            CUSNO     R        O  2  1REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                      COLOR(PNK)
A            CUSNM     R        O  2  9REFFLD(RCUST/CUSNM *LIBL/CUSTOMER)
A                                      COLOR(PNK)
A                                 3 10'State:'
A                                 3 21'Phone:'
A            CUSPH     R        O  3 28REFFLD(RCUST/CUSPH *LIBL/CUSTOMER)
A                                      EDTWRD('   -   -    ')
A            CUSST     R        O  3 17REFFLD(RCUST/CUSST *LIBL/CUSTOMER)
A          R SFD361B                    SFLCTL(SFD361A)
A                                       SFLSIZ(0012)
A                                       SFLPAG(0003)
A                                       WINDOW(WINDOW)
A N99                                   ROLLUP(25)
A                                       OVERLAY
A                                       SFLCSRRRN(&@REC)
A  30                                   SFLDSP
A                                       SFLDSPCTL
A  31                                   SFLCLR
A  99                                   SFLEND(*MORE)
A                                       SFLDROP(CF10)
A            @PAG           4  0H       SFLRCDNBR
A            @REC           5S 0H
A            @ROL           2S 0B  1  2SFLROLVAL
A                                       COLOR(YLW)
A            @SRCH     R        B  1  9REFFLD(RCUST/CUSNM *LIBL/CUSTOMER)
A                                       DSPATR(HI)
A                                       DSPATR(PC)
A          R WINDOW
A                                       WINDOW(&@SL &@SP 10 40)
A                                       WDWBORDER((*COLOR BLU) (*DSPATR RI)-
A                                        (*CHAR '        '))
A            @SL            3S 0P
A            @SP            3S 0P
A                                 9  1'F3-Exit  F10-Show Additonal Info'
A                                       DSPATR(HI)


F*============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR361) SRCFILE(XXX/QRPGSRC)
F*
F*============================================================
FCUSTL1  IF  E           K        DISK
FSFD361  CF  E                    WORKSTN
F                                       @RRN  KSFILE SFD361A
C*
C                    WRITEWINDOW
C                    EXFMTSFD361B
C                    EXSR @CMD
C*
C*********************************************************
C*    @ C M D   S U B R O U T I N E                      *
C*                                                       *
```

```
C*    Executed after every EXFMT or display read op code.   *
C*    Used to process command keys.                         *
C*********************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, seton LR and exit.  Since the user did
C* not select a CUSTL1  , 0 out the CUSN0 field
C*
C           *IN03     IFEQ *ON
C                     Z-ADD0         CUSN0
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C* If *IN25 is on, it means that the user pressed the roll key
C* and that more subfile records were needed to be loaded.
C* (The subfile passed control back to the program.) If the
C* subfile already had the records for the next subfile page
C* in the subfile, control would not pass back to the program. The
C* subfile would handle the rolling, and the program would remain
C* on the EXFMT SFD361B statement.)
C*
C           *IN25     IFEQ *ON
C                     MOVE *OFF      *IN25
C                     EXSR @LOAD
C                     Z-ADD0         @REC
C                     END
C*
C* If the field @REC is greater than 0, it means that the user
C* has selected a record. Chain to the subfile to retrieve the
C* CUSTL1   number, then return back to the calling program.
C*
C           @REC      IFGT 0
C           @REC      CHAINSFD361A              98
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C* If the fields @SRCH and @SAVE are not equal, then the user
C* requested to load the subfile starting from a specific name.
C*
C           @SRCH     IFNE @SAVE
C                     MOVE @SRCH     @SAVE
C*
C* The following three lines of code will clear the subfile
C*
C                     MOVE *ON       *IN31
C                     WRITESFD361B
C                     MOVE *OFF      *IN31
C*
C* Since we cleared the subfile, reset the subfile relative
C* record number to 0.
C*
C                     Z-ADD0         @RRN
C           @SRCH     SETLLCUSTL1
C                     EXSR @LOAD
```

```
C                     END
C*
C                     ENDSR
C*
C*********************************************************
C*    * I N Z S R   S U B R O U T I N E                  *
C*                                                       *
C*    Retrieve the starting line and position for the    *
C*    window. Check to make sure that these values are   *
C*    valid. If not, put the closest valid number in.    *
C*********************************************************
C*
C           *INZSR    BEGSR
C*
C           *ENTRY    PLIST
C                     PARM           @SL
C                     PARM           @SP
C                     PARM           CUSNO
C*
C                     Z-ADD0         @RRN    50
C                     Z-ADD99        @ROL
C           *LIKE     DEFN @SRCH     @SAVE
C*
C*  If the start line is greater than 12, our window will not
C*  fit on the display. Reset the line to 12.
C*
C           @SL       IFGT 12
C                     Z-ADD12        @SL
C                     END
C*
C*  If the start position is greater than 36, our window will not
C*  fit on the display. Reset the position to 36.
C*
C           @SP       IFGT 36
C                     Z-ADD36        @SP
C                     END
C*
C*   Nothing can be displayed from position 1,1 on the display.
C*   If the position is 1,1, reset it to 1,2. Also, check and
C*   make sure that the start line or start position is not zero.
C*
C           @SL       IFLE 1
C           @SP       ANDLE1
C                     Z-ADD1         @SL
C                     Z-ADD2         @SP
C                     END
C*
C*  Load the subfile
C*
C                     EXSR @LOAD
C*
C                     ENDSR
C*
C*********************************************************
C*    @ L O A D     S U B R O U T I N E                  *
C*                                                       *
C*    This subroutine will read the CUSTL1   file and    *
```

```
C*    load the subfile records. Since this is an extended   *
C*    subfile, and the SFLPAG has been set to 10, 10 records*
C*    maximum will be read and loaded at a time.            *
C*                                                          *
C*************************************************************
C*
C           @LOAD     BEGSR
C*
C                     MOVE *OFF      *IN99
C*
C* Set the first subfile page to display
C*
C                     Z-ADD@RRN      @PAG
C*
C*  Perform this loop until you reach the end of
C*  the CUSTL1   file or until you load 12 records, which
C*  is 2 pages worth of subfile records. (SFLPAG=6)
C*
C                     DO   12
C                     READ CUSTL1                  99
C*
C*  If indicator 99 is on, or if you have already loaded
C*  9999 subfile records, (the maximum allowed), then exit
C*  this loop.
C*
C           *IN99     IFEQ *ON
C           @RRN      OREQ 9999
C                     LEAVE
C                     END
C*
C*  Add 1 to our subfile relative record number.
C*
C                     ADD  1         @RRN
C                     WRITESFD361A
C                     END
C*
C* If you have loaded at least one record into the subfile,
C* turn on indicator 30. This indicator conditions the
C* SFLDSP keyword. If you have loaded at least one record
C* into the subfile, it will be all right to display the subfile
C* format. If you did not load any records into the subfile, you
C* would turn off indicator 30, because attempting to display
C* an empty subfile would result in an error being issued.
C*
C           @RRN      IFGT 0
C                     MOVE *ON       *IN30
C                     ELSE
C                     MOVE *OFF      *IN30
C                     END
C*
C*
C*  If you added at least one record during the load routine,
C*  then cause the subfile to display the next page. If you did
C*  not add any records, display the last subfile page that was
C*  on the display.
C*
C           @PAG      IFNE @RRN
```

```
C                       ADD  1          @PAG
C                       END
C*
C                       ENDSR
```

In this example, the SFLROLVAL keyword was added to the subfile control format.

```
A             @ROL            2S 0B  1  2SFLROLVAL
```

A field called @ROL was defined to be the Subfile roll value field. It is a 2-byte, signed numeric field. It will be displayed on the screen for the user to change.

To see the effect of the SFLROLVAL keyword, call SFR360 and press F4. Your display should look like this:

```
  xx/xx/xx            Customer Master Maintenance                    QPGMR
  xx:xx:xx                                                           SFD360A


Customer Number....:  ..........................................
                      :  99    ______________________________    :
Customer Name......:  : 00002   AS/400s Are Us                    :
                      : 00011   Computer Mart                     :
Customer Address...:  : 00001   David's Computers and Bait Shop   :
                      : 00045   Full Range Computing              :
Customer City......:  : 00099   Hard Drive Warehouse              :
                      : 00386   Memory Systems Inc.               :
Customer State.....:  :                                 More...   :
                      : F3-Exit  F10-Show Additonal Info          :
Customer Zip.......:  :                                           :
                      :...........................................:
Phone Number.......:




 F3-Exit   F4-Customer Search
```

The initial value of the SFLROLVAL keyword is 99. Since this value is greater than the subfile page value of 6, the subfile still rolls forwards and backwards 6 records at a time.

Now change the value of the Subfile Roll Value field to 2 and roll forward. Your display should look like this:

```
  xx/xx/xx            Customer Master Maintenance                    QPGMR
  xx:xx:xx                                                           SFD360A


Customer Number....:  ........................................
                      :  2     ______________________________   :
Customer Name......:  : 00001   David's Computers and Bait Shop   :
                      : 00045   Full Range Computing             :
Customer Address...:  : 00099   Hard Drive Warehouse             :
                      : 00386   Memory Systems Inc.              :
Customer City......:  : 00005   Mid-South Computer Sales         :
                      : 00042   Midrange Supercenter             :
Customer State.....:  :                                 More... :
                      : F3-Exit  F10-Show Additonal Info         :
Customer Zip.......:  :                                          :
                      :..........................................:
Phone Number.......:




 F3-Exit   F4-Customer Search
```

The subfile is now rolling forwards and backwards two records at a time. Remember the changes needed for proper processing of expanding load subfiles.

1. The roll up key must be defined.
2. The SFLEND keyword must be conditioned correctly.
3. The SFLRCDNBR keyword should be used.

Now you're ready to convert the expanding subfile into a single-page subfile. First, you may want to review the single-page loading method.

Single-Page Subfiles

Single-page subfiles only contain subfile records that are displayed in the screen. For a single-page subfile, the subfile size and subfile page are equal values. Single-page subfiles cannot be extended beyond the value of the subfile size parameter. An error is issued if an attempt is made to go beyond this value.

Single-page subfiles are the most difficult subfile loading method to program. They are not very difficult, but the code required is usually more than a load-all or an expanding subfile. It is still much easier to code a single-page subfile than to try to achieve the same effect with a non-subfile program.

The major difference between a single-page subfile and the load-all and expanding methods is that only one subfile page is loaded and displayed. The roll keys must be programmed for and handled inside the program.

The basic structure of a single-page subfile is:

1. Clear the subfile.
2. Load the subfile.
3. Display the subfile.
4. If a roll key is pressed, go to step 1.

Uses for Single-Page Subfiles

The main reason to use a single-page subfile is to (possibly) load many records. If you attempt to load too many records into a load-all subfile, the user wait time could be substantial. If you do attempt to load too many records with an expanding subfile, the PAG size may be so large that it could adversely affect performance. For example, if you display the history of an item, there is the possibility of eventually loading more than 9,999 subfile records. There could be thousands of transaction records for an item, and a load-all subfile could take minutes to load the records before the user sees the information. In addition, the load-all subfiles PAG would be very large to accommodate all of the records. Expanding subfiles cut the loading time down dramatically over the load-all subfile, but its PAG would still be large. In this case, a single-page subfile could be used. The program would load only the number of records that it would display at a time. If the program displays only 16 records at a time, then it would only load in 16 records, instead of thousands. On the flip side, you have to handle the roll keys inside your program by clearing the subfile and loading 16 more records into the subfile.

It needs to be said that the above scenario could still be effectively handled by a load all, or especially an expanding subfile. Remember, you can control the total number of records that can be loaded.

Coding Considerations for a Single-Page Subfile

You make a subfile a single-page subfile by setting the subfile page and size parameters to equal values. If you want a single-page subfile to display 10 records at a time, then you set SFLPAG and SFLSIZ to 10. Therefore, for a Single-page subfile, the subfile page equals the subfile size.

To roll through a single-page subfile, you must program for the roll keys inside your program. When a roll key is pressed, you must essentially destroy the current information stored in the subfile by clearing the subfile, then load in the new records. Whenever a roll key is pressed on a single-page subfile, you clear and reload the subfile every time, usually requiring disk requests. These rolls are slower than the rolls of a load-all subfile. The load-all subfile contains all records in memory, whereas the single-page subfile must read each page of records into memory from disk each time. You must also keep track of file pointers to ensure you roll backwards and forwards correctly through your data file. Therefore, programming becomes a bit more complex.

Should You Use a Single-Page Subfile?

When deciding whether or not to use a page-at-a-time subfile, the size of the file and the probability of rolling comes into play. If the file you are building a subfile over is small, and will always be small, it may not be a bad idea to create a load-all subfile because a load-all subfile is the simplest in nature. In addition, if the user needs to roll, performance will be better. If the file has the capability of maturing into a large file, then the decision will rest between the expanding and page at a time. The size of the file actually means the numbers of records that are to be selected. A file may be millions of records in size, but if you are loading only the current orders for a single customer, a load all may still be the best alternative. If you are reading the order history for a customer, then the design may be different. Even though the file may have few records per customer now, after some time has passed, the number of selected records per customer may run into the thousands or tens of thousands. Over the maturing life of this file, the load-all subfile program will continue to become slower and slower to load.

Let's look at an example of a page-at-a-time subfile. Examine the code for SFD410, SFR410, SFD411 and SFR411. This example shows a page-at-a-time subfile.

```
A*==============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD410) SRCFILE(XXX/QDDSSRC)
A*
A*==============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03 'Exit')
A          R SFD410A
A                                       CF04(04 'Call Search Program')
A                                   1  3DATE
A                                       EDTCDE(Y)
A                                   1 23'Customer Master Maintenance'
A                                       DSPATR(HI)
A                                   2  3TIME
```

```
A                                   1 71USER
A                                   2 71'SFD410A'
A                                  23  2'F3-Exit   F4-Customer Search'
A                                       DSPATR(HI)
A                                   5  1'Customer Number....:'
A                                       COLOR(BLU)
A                                   7  1'Customer Name......:'
A                                       COLOR(BLU)
A                                   9  1'Customer Address...:'
A                                       COLOR(BLU)
A                                  11  1'Customer City......:'
A                                       COLOR(BLU)
A                                  13  1'Customer State.....:'
A                                       COLOR(BLU)
A                                  15  1'Customer Zip.......:'
A                                       COLOR(BLU)
A                                  17  1'Phone Number.......:'
A                                       COLOR(BLU)
A            CUSNO     R     Y  B  5 23REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A                                       CHECK(ER)
A                                       CHECK(FE)
A                                       CHECK(RB)
A                                  24  1'Customer Number Not Found In Custo-
A                                       mer Master File-Key In A Valid Numb-
A                                       er'
A                                       COLOR(RED)
A                                       DSPATR(BL)
A N99                                   DSPATR(ND)
A          R SFD410B
A                                       CLRL(*NO)
A                                   2 71'SFD410B'
A            CUSNO     R        O  5 23REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                       COLOR(PNK)
A            CUSNM     R        B  7 22REFFLD(RCUST/CUSNM *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A            CUSAD     R        B  9 22REFFLD(RCUST/CUSAD *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A            CUSCT     R        B 11 22REFFLD(RCUST/CUSCT *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A            CUSST     R        B 13 22REFFLD(RCUST/CUSST *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A            CUSCP     R        B 15 22REFFLD(RCUST/CUSCP *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A                                  23  1'F3-Exit                          -
A                                          '
A                                       COLOR(WHT)
A            CUSPH     R        B 17 22REFFLD(RCUST/CUSPH *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A                                       EDTWRD('   -   -    ')
A                                  24  1'                                  -
A                                                                          -
A                                                 '
```

```
F*==============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR410) SRCFILE(XXX/QRPGSRC)
F*
F*==============================================================
FSFD410  CF  E                    WORKSTN
F                                            KINFDS INFDS
FCUSTOMERIF  E           K        DISK
IINFDS       DS
I                                     370 371 LINPOS
I            DS
I                                   B   1   20@LIN
I                                       2   2 @L
I            DS
I                                   B   1   20@POS
I                                       2   2 @P
C*
C                     EXFMTSFD410A
C                     EXSR @CMD
C*
C           CUSNO     CHAINCUSTOMER             99
C*
C           *IN99     IFEQ *OFF
C                     EXFMTSFD410B
C                     EXSR @CMD
C                     END
C*
C*************************************************************
C*    @ C M D   S U B R O U T I N E                          *
C*                                                           *
C*    Executed after every EXFMT or display read op code.    *
C*    Used to process command keys.                          *
C*************************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, seton LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C* If F4 was pressed, call program SFR381. Pass the customer
C* number field. If the customer number field greater than 0
C* upon return, then the user has selected a customer.
C*
C           *IN04     IFEQ *ON
C*
C* Convert the hex values of the cursor line/position into
C* binary, then to decimal.
C*
C                     Z-ADD0         CUSNO
C                     MOVE *OFF      *IN04
C                     MOVE *ZEROS    @LIN
C                     MOVE *ZEROS    @POS
```

```
C                     MOVELLINPOS    @L
C                     MOVE LINPOS    @P
C                     Z-ADD@LIN      @SL     30
C                     Z-ADD@POS      @SP     30
C*
C                     CALL 'SFR411'
C                     PARM           @SL
C                     PARM           @SP
C                     PARM           CUSNO
C*
C                     WRITESFD410A
C*
C                     END
C*
C                     ENDSR


A*============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD411) SRCFILE(XXX/QDDSSRC)
A*
A*============================================================
A                                      DSPSIZ(24 80 *DS3)
A                                      CF03(03)
A          R ASSUME
A                                      ASSUME
A                                 24  2' '
A          R SFD411A                   SFL
A            CUSNO     R        O  2  1REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                      COLOR(PNK)
A            CUSNM     R        O  2  9REFFLD(RCUST/CUSNM *LIBL/CUSTOMER)
A                                      COLOR(PNK)
A                                  3 10'State:'
A                                  3 21'Phone:'
A            CUSPH     R        O  3 28REFFLD(RCUST/CUSPH *LIBL/CUSTOMER)
A                                      EDTWRD('   -   -    ')
A            CUSST     R        O  3 17REFFLD(RCUST/CUSST *LIBL/CUSTOMER)
A          R SFD411B                   SFLCTL(SFD411A)
A N99                                  ROLLUP(25)
A                                      OVERLAY
A                                      SFLCSRRRN(&@REC)
A  30                                  SFLDSP
A                                      SFLDSPCTL
A  31                                  SFLCLR
A  35                                  SFLEND(*MORE)
A                                      SFLSIZ(0003)
A                                      SFLPAG(0003)
A                                      WINDOW(WINDOW)
A            @REC           5S 0H
A                                  1  3'Name:'
A            @SRCH     R        B  1  9REFFLD(RCUST/CUSNM *LIBL/CUSTOMER)
A                                      DSPATR(HI)
A          R WINDOW
A                                      WINDOW(&@SL &@SP 10 40)
A                                      WDWBORDER((*COLOR BLU) (*DSPATR RI)-
```

```
A                                        (*CHAR '        '))
A            @SL            3S OP
A            @SP            3S OP
A                                  9  1'F3-Exit                          '
A                                      DSPATR(HI)


F*==============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR411) SRCFILE(XXX/QRPGSRC)
F*
F*==============================================================
FCUSTL1  IF  E           K        DISK
FSFD411  CF  E                    WORKSTN
F                                        @RRN  KSFILE SFD411A
C*
C                     WRITEWINDOW
C                     EXFMTSFD411B
C                     EXSR @CMD
C*
C*********************************************************
C*    @ C M D   S U B R O U T I N E                      *
C*                                                       *
C*    Executed after every EXFMT or display read op code.  *
C*    Used to process command keys.                      *
C*********************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, seton LR and exit.  Since the user did
C* not select a CUSTL1  , 0 out the CUSN0 field
C*
C           *IN03     IFEQ *ON
C                     Z-ADD0         CUSN0
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C* If *IN25 is on, it means that the user pressed the roll key
C* and that more subfile records were needed to be loaded.
C* (The subfile passed control back to the program.) If the
C* subfile already had the records for the next subfile page
C* in the subfile, control would not pass back to the program. The
C* subfile would handle the rolling, and the program would remain
C* on the EXFMT SFD411B statement.)
C*
C           *IN25     IFEQ *ON
C                     MOVE *ON       *IN31
C                     WRITESFD411B
C                     MOVE *OFF      *IN31
C                     Z-ADD0         @RRN
C                     MOVE *OFF      *IN25
C                     EXSR @LOAD
C                     END
C*
```

```
C* If the field @REC is greater than 0, it means that the user
C* has selected a record. Chain to the subfile to retrieve the
C* CUSTL1   number, then return back to the calling program.
C*
C           @REC      IFGT 0
C           @REC      CHAINSFD411A              98
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C* If the fields @SRCH and @SAVE are not equal, then the user
C* requested to load the subfile starting from a specific name.
C*
C           @SRCH     IFNE @SAVE
C                     MOVE @SRCH     @SAVE
C*
C* The following three lines of code will clear the subfile
C*
C                     MOVE *ON       *IN31
C                     WRITESFD411B
C                     MOVE *OFF      *IN31
C*
C* Since you cleared the subfile, reset the subfile relative
C* record number to 0.
C*
C                     Z-ADD0         @RRN
C           @SRCH     SETLLCUSTL1
C                     EXSR @LOAD
C                     END
C*
C                     ENDSR
C*
C*************************************************************
C*    * I N Z S R   S U B R O U T I N E                      *
C*                                                           *
C*    Retrieve the starting line and position for the        *
C*    window. Check to make sure that these values are       *
C*    valid. If not, put the closest valid number in.        *
C*************************************************************
C*
C           *INZSR    BEGSR
C*
C           *ENTRY    PLIST
C                     PARM           @SL
C                     PARM           @SP
C                     PARM           CUSNO
C*
C                     Z-ADD0         @RRN    50
C           *LIKE     DEFN @SRCH     @SAVE
C*
C*  If the start line is greater than 12, our window will not
C*  fit on the display. Reset the line to 12.
C*
C           @SL       IFGT 12
C                     Z-ADD12        @SL
C                     END
C*
```

```
C*  If the start position is greater than 36, our window will not
C*  fit on the display. Reset the position to 36.
C*
C           @SP       IFGT 36
C                     Z-ADD36        @SP
C                     END
C*
C*   Nothing can be displayed from position 1,1 on the display.
C*   If the position is 1,1, reset it to 1,2. Also, check and
C*   make sure that the start line or start position is not zero.
C*
C           @SL       IFLE 1
C           @SP       ANDLE1
C                     Z-ADD1         @SL
C                     Z-ADD2         @SP
C                     END
C*
C*  Load the subfile
C*
C                     EXSR @LOAD
C*
C                     ENDSR
C*
C*************************************************************
C*    @ L O A D     S U B R O U T I N E                      *
C*                                                           *
C*    This subroutine will read the CUSTL1   file and        *
C*    load the subfile records. Since this is an extended    *
C*    subfile, and the SFLPAG has been set to 10, 10 records*
C*    maximum will be read and loaded at a time.             *
C*                                                           *
C*************************************************************
C*
C           @LOAD     BEGSR
C*
C                     MOVE *OFF      *IN99
C*
C*  Perform this loop until you reach the end of
C*  the CUSTL1 file or until you load  3 records, which
C*  is 1 page  worth of subfile records. (SFLPAG=3)
C*
C                     DO   3
C                     READ CUSTL1                   99
C*
C*  If indicator 99 is on, or if you have already loaded
C*  9999 subfile records, (the maximum allowed), then exit
C*  this loop.
C*
C           *IN99     IFEQ *ON
C           @RRN      OREQ 9999
C                     LEAVE
C                     END
C*
C*  Add 1 to the subfile relative record number.
C*
C                     ADD  1         @RRN
C                     WRITESFD411A
```

```
C                       END
C*
C* If you have loaded at least one record into the subfile,
C* turn on indicator 30. This indicator conditions the
C* SFLDSP keyword. If you have loaded at least one record
C* into the subfile, it will be alright to display the subfile
C* format. If you did not load any records into the subfile, you
C* would turn off indicator 30, because attempting to display
C* an empty subfile would result in an error being issued.
C*
C            @RRN       IFGT 0
C                       MOVE *ON       *IN30
C                       ELSE
C                       MOVE *OFF      *IN30
C                       END
C                       MOVE *ON       *IN35
C*
C                       ENDSR
```

Call SFR410 and press the F4 key. Your display should look like this:

```
  xx/xx/xx            Customer Master Maintenance                    QPGMR
  xx:xx:xx                                                           SFD410A


Customer Number....:  ........................................
                      :   Name: ______________________________   :
Customer Name......:  : 00045   Full Range Computing             :
                      :          State: FL  Phone: 245-465-2125  :
Customer Address...:  : 00099   Hard Drive Warehouse             :
                      :          State: LA  Phone: 553-654-3435  :
Customer City......:  : 00386   Memory Systems Inc.              :
                      :          State: MN  Phone: 654-989-6565  :
Customer State.....:  :                                  Bottom :
                      : F3-Exit                                  :
Customer Zip.......:  :                                          :
                      :..........................................:
Phone Number.......:




 F3-Exit   F4-Customer Search
```

The subfile displays three two line subfile records. Press the roll forward key. Your display should look like this:

```
  xx/xx/xx            Customer Master Maintenance                    QPGMR
  xx:xx:xx                                                           SFD410A


Customer Number....:  ........................................
                      :   Name: ______________________________   :
Customer Name......:  : 00005   Mid-South Computer Sales         :
                      :          State: CN  Phone: 332-778-4555  :
Customer Address...:  : 00042   Midrange Supercenter             :
                      :          State: CA  Phone: 543-418-7643  :
Customer City......:  : 00010   Mini Computing Clearance         :
                      :          State: FL  Phone: 788-266-9221  :
Customer State.....:  :                                   Bottom :
                      : F3-Exit                                  :
Customer Zip.......:  :                                          :
                      :..........................................:
Phone Number.......:




 F3-Exit   F4-Customer Search
```

The screen rolled forward like the other subfiles did. The only difference in the page-at-a-time subfile is that the program would be physically loading records each time a roll key is pressed. On the load-all subfile, the program never loads records when the roll keys are pressed. On the expanding subfile, the program only loads records when the roll keys are pressed if it has displayed all of the loaded records already.

Press roll forward until you reach the end of the subfile. Notice on the bottom of each display the subfile says it is at the bottom of the subfile. It says this because indicator 35, which controls the SFLEND keyword, is turned on. Therefore, the subfile detects a subfile end condition, and since the conditioning indicator 35 is on, it displays the “Bottom” message. What the program needs to do is to only turn on the SFLEND indicator if it has reached the end of the customer file, which is controlled by indicator 99. That way, even though the subfile reaches an end of subfile condition after displaying each page, it will not display the “Bottom” message unless the program has also reached the end of file from which it is loading the subfile records.

Continue to roll until all the records have been displayed. Your display should look like this:

```
  xx/xx/xx            Customer Master Maintenance                    QPGMR
  xx:xx:xx                                                           SFD410A


Customer Number....:  ........................................
                      :   Name: ______________________________   :
Customer Name......:  : 00003   Tennessee Minicomputer Sales     :
                      :          State: KY  Phone: 880-555-1111  :
Customer Address...:  : 00025   Westmoreland Computers           :
                      :          State: KY  Phone: 684-321-3542  :
Customer City......:  :                                          :
                      :                                          :
Customer State.....:  :                                   Bottom :
                      : F3-Exit                                  :
Customer Zip.......:  :  Function key not allowed.                :
                      :..........................................:
Phone Number.......:




 F3-Exit   F4-Customer Search
```

Now that the program doesn't have any more records to load into the subfile, it tells us the roll up function key is not valid.

Now attempt to roll backwards. The display tells you that you can't roll backwards, because the key is not valid. The page-at-a-time subfile does not handle the roll keys automatically. You must define both roll keys in your display and code logic for them in your RPG program for a page-at-a-time subfile. If you code for the roll keys in the display file, then they are allowed. But the subfile will not automatically roll the screen, because there is nothing to roll to. The subfile only contains the records that are being displayed. Therefore, any time you press a roll key on a single-page subfile, control is passed back to the program. You must handle the rolling process from within your program.

The program already has logic set up for the roll forward key, so we need to add logic for the roll back key. Also, you want to change the SFLEND message so that it will show the "More..." and "Bottom" messages correctly.

Look at the source code for SFD420, SFR420, SFD421 and SFR421. The changes for the roll keys and the subfile messages have been added.

```
A*==============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD420) SRCFILE(XXX/QDDSSRC)
A*
A*==============================================================
```

```
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03 'Exit')
A          R SFD420A
A                                       CF04(04 'Call Search Program')
A                                   1  3DATE
A                                       EDTCDE(Y)
A                                   1 23'Customer Master Maintenance'
A                                       DSPATR(HI)
A                                   2  3TIME
A                                   1 71USER
A                                   2 71'SFD420A'
A                                  23  2'F3-Exit   F4-Customer Search'
A                                       DSPATR(HI)
A                                   5  1'Customer Number....:'
A                                       COLOR(BLU)
A                                   7  1'Customer Name......:'
A                                       COLOR(BLU)
A                                   9  1'Customer Address...:'
A                                       COLOR(BLU)
A                                  11  1'Customer City......:'
A                                       COLOR(BLU)
A                                  13  1'Customer State.....:'
A                                       COLOR(BLU)
A                                  15  1'Customer Zip.......:'
A                                       COLOR(BLU)
A                                  17  1'Phone Number.......:'
A                                       COLOR(BLU)
A            CUSNO     R     Y  B  5 23REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A                                       CHECK(ER)
A                                       CHECK(FE)
A                                       CHECK(RB)
A                                  24  1'Customer Number Not Found In Custo-
A                                       mer Master File-Key In A Valid Numb-
A                                       er'
A                                       COLOR(RED)
A                                       DSPATR(BL)
A N99                                   DSPATR(ND)
A          R SFD420B
A                                       CLRL(*NO)
A                                   2 71'SFD420B'
A            CUSNO     R        O  5 23REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                       COLOR(PNK)
A            CUSNM     R        B  7 22REFFLD(RCUST/CUSNM *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A            CUSAD     R        B  9 22REFFLD(RCUST/CUSAD *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A            CUSCT     R        B 11 22REFFLD(RCUST/CUSCT *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A            CUSST     R        B 13 22REFFLD(RCUST/CUSST *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A            CUSCP     R        B 15 22REFFLD(RCUST/CUSCP *LIBL/CUSTOMER)
A                                       COLOR(WHT)
A                                  23  1'F3-Exit                          -
A                                          '
A                                       COLOR(WHT)
A            CUSPH     R        B 17 22REFFLD(RCUST/CUSPH *LIBL/CUSTOMER)
```

```
A                                        COLOR(WHT)
A                                        EDTWRD('   -   -    ')
A                                  24  1'                                  -
A                                                                          -
A                                                 '


F*============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR420) SRCFILE(XXX/QRPGSRC)
F*
F*============================================================
FSFD420  CF  E                     WORKSTN
F                                              KINFDS INFDS
FCUSTOMERIF  E           K         DISK
IINFDS       DS
I                                        370 371 LINPOS
I            DS
I                                     B   1   20@LIN
I                                         2   2 @L
I            DS
I                                     B   1   20@POS
I                                         2   2 @P
C*
C                      EXFMTSFD420A
C                      EXSR @CMD
C*
C           CUSNO      CHAINCUSTOMER             99
C*
C           *IN99      IFEQ *OFF
C                      EXFMTSFD420B
C                      EXSR @CMD
C                      END
C*
C*********************************************************
C*    @ C M D   S U B R O U T I N E                      *
C*                                                       *
C*    Executed after every EXFMT or display read op code.  *
C*    Used to process command keys.                      *
C*********************************************************
C*
C           @CMD       BEGSR
C*
C* If F3 was pressed, seton LR and exit.
C*
C           *IN03      IFEQ *ON
C                      MOVE *ON       *INLR
C                      RETRN
C                      END
C*
C* If F4 was pressed, call program SFR381. Pass the customer
C* number field. If the customer number field greater than 0
C* upon return, then the user has selected a customer.
C*
C           *IN04      IFEQ *ON
```

```
C*
C* Convert the hex values of the cursor line/position into
C* binary, then to decimal.
C*
C                     Z-ADD0         CUSNO
C                     MOVE *OFF      *IN04
C                     MOVE *ZEROS    @LIN
C                     MOVE *ZEROS    @POS
C                     MOVELLINPOS    @L
C                     MOVE LINPOS    @P
C                     Z-ADD@LIN      @SL     30
C                     Z-ADD@POS      @SP     30
C*
C                     CALL 'SFR421'
C                     PARM           @SL
C                     PARM           @SP
C                     PARM           CUSNO
C*
C                     WRITESFD420A
C*
C                     END
C*
C                     ENDSR


A*==============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD421) SRCFILE(XXX/QDDSSRC)
A*
A*==============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03)
A          R SFD421A                    SFL
A                                       CHGINPDFT
A            CUSNO     R        O  2  1REFFLD(RCUST/CUSNO *LIBL/CUSTOMER)
A                                       COLOR(PNK)
A            CUSNM     R        O  2  9REFFLD(RCUST/CUSNM *LIBL/CUSTOMER)
A                                       COLOR(PNK)
A                                    3 10'State:'
A                                    3 21'Phone:'
A            CUSPH     R        O  3 28REFFLD(RCUST/CUSPH *LIBL/CUSTOMER)
A                                       EDTWRD('   -   -    ')
A            CUSST     R        O  3 17REFFLD(RCUST/CUSST *LIBL/CUSTOMER)
A          R SFD421B                    SFLCTL(SFD421A)
A N99                                   ROLLUP(25)
A N98                                   ROLLDOWN(26)
A                                       OVERLAY
A                                       SFLCSRRRN(&@REC)
A  30                                   SFLDSP
A                                       SFLDSPCTL
A  31                                   SFLCLR
A  99                                   SFLEND(*MORE)
A                                       SFLSIZ(0003)
A                                       SFLPAG(0003)
A  98                                   SFLMSG('You are at the top of the f-
```

```
A                                       ile')
A N99                                   SFLMSG('More records can be display-
A                                       ed')
A  99                                   SFLMSG('You have reached the end of-
A                                        file')
A                                       WINDOW(WINDOW)
A            @REC           5S OH
A                                  1  3'Name:'
A            @SRCH     R        B  1  9REFFLD(RCUST/CUSNM *LIBL/CUSTOMER)
A                                       DSPATR(HI)
A          R WINDOW
A                                       WINDOW(&@SL &@SP 10 40)
A                                       WDWBORDER((*COLOR BLU) (*DSPATR RI)-
A                                        (*CHAR '        '))
A            @SL            3S OP
A            @SP            3S OP
A                                  9  1'F3-Exit                        '
A                                       DSPATR(HI)
A          R ASSUME
A                                       ASSUME
A                                 24  2' '


F*==============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR421) SRCFILE(XXX/QRPGSRC)
F*
F*==============================================================
FCUSTL1  IF  E           K        DISK
FSFD421  CF  E                    WORKSTN
F                                        @RRN  KSFILE SFD421A
C*
C                     WRITEWINDOW
C                     EXFMTSFD421B
C                     EXSR @CMD
C*
C*********************************************************
C*    @ C M D   S U B R O U T I N E                      *
C*                                                       *
C*    Executed after every EXFMT or display read op code.  *
C*    Used to process command keys.                      *
C*********************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, seton LR and exit.  Since the user did
C* not select a CUSTL1  , 0 out the CUSN0 field
C*
C           *IN03     IFEQ *ON
C                     Z-ADD0         CUSN0
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C* If *IN25 is on, it means that the user pressed the roll key
```

```
C* and that more subfile records were needed to be loaded.
C* (The subfile passed control back to the program.) If the
C* subfile already had the records for the next subfile page
C* in the subfile, control would not pass back to the program. The
C* subfile would handle the rolling, and the program would remain
C* on the EXFMT SFD421B statement.)
C*
C           *IN25     IFEQ *ON
C                     MOVE *ON       *IN31
C                     WRITESFD421B
C                     MOVE *OFF      *IN31
C                     Z-ADD0         @RRN
C                     MOVE *OFF      *IN25
C                     MOVE *OFF      *IN98
C                     EXSR @LOAD
C                     END
C*
C* If *IN26 is on, it means that the user pressed the roll key
C* and that more subfile records were needed to be loaded.
C* (The subfile passed control back to the program.) If the
C* subfile already had the records for the next subfile page
C* in the subfile, control would not pass back to the program. The
C* subfile would handle the rolling, and the program would remain
C* on the EXFMT SFD421B statement.)
C*
C           *IN26     IFEQ *ON
C                     MOVE *ON       *IN31
C                     WRITESFD421B
C                     MOVE *OFF      *IN31
C                     MOVE *OFF      *IN26
C                     Z-ADD4         @D0     30
C                     ADD  @RRN      @D0
C           *IN99     IFEQ '1'
C           *HIVAL    SETGTCUSTL1
C                     END
C                     DO   @D0
C                     READPCUSTL1                    91
C           *IN91     IFEQ '1'
C                     MOVE *ON       *IN98
C           *LOVAL    SETLLCUSTL1
C                     LEAVE
C                     END
C                     END
C                     Z-ADD0         @RRN
C                     EXSR @LOAD
C                     END
C*
C* If the field @REC is greater than 0, it means that the user
C* has selected a record. Chain to the subfile to retrieve the
C* CUSTL1   number, then return back to the calling program.
C*
C           @REC      IFGT 0
C           @REC      CHAINSFD421A              98
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
```

```
C* If the fields @SRCH and @SAVE are not equal, then the user
C* requested to load the subfile starting from a specific name.
C*
C           @SRCH     IFNE @SAVE
C                     MOVE *OFF      *IN98
C                     MOVE @SRCH     @SAVE
C*
C* The following three lines of code will clear the subfile
C*
C                     MOVE *ON       *IN31
C                     WRITESFD421B
C                     MOVE *OFF      *IN31
C*
C* Since you cleared the subfile, reset the subfile relative
C* record number to 0.
C*
C                     Z-ADD0         @RRN
C           @SRCH     SETLLCUSTL1
C                     EXSR @LOAD
C                     END
C*
C                     ENDSR
C*
C*********************************************************
C*    * I N Z S R   S U B R O U T I N E                  *
C*                                                       *
C*    Retrieve the starting line and position for the    *
C*    window. Check to make sure that these values are   *
C*    valid. If not, put the closest valid number in.    *
C*********************************************************
C*
C           *INZSR    BEGSR
C*
C           *ENTRY    PLIST
C                     PARM           @SL
C                     PARM           @SP
C                     PARM           CUSNO
C*
C                     Z-ADD0         @RRN    50
C           *LIKE     DEFN @SRCH     @SAVE
C*
C*  If the start line is greater than 12, our window will not
C*  fit on the display. Reset the line to 12.
C*
C           @SL       IFGT 12
C                     Z-ADD12        @SL
C                     END
C*
C*  If the start position is greater than 36, our window will not
C*  fit on the display. Reset the position to 36.
C*
C           @SP       IFGT 36
C                     Z-ADD36        @SP
C                     END
C*
C*   Nothing can be displayed from position 1,1 on the display.
C*   If the position is 1,1, reset it to 1,2. Also, check and
```

```
C*   make sure that the start line or start position is not zero.
C*
C           @SL       IFLE 1
C           @SP       ANDLE1
C                     Z-ADD1         @SL
C                     Z-ADD2         @SP
C                     END
C*
C*  Load the subfile
C*
C                     MOVE *ON       *IN98
C                     EXSR @LOAD
C*
C                     ENDSR
C*
C*************************************************************
C*    @ L O A D     S U B R O U T I N E                      *
C*                                                           *
C*    This subroutine will read the CUSTL1   file and        *
C*    load the subfile records. Since this is an extended    *
C*    subfile, and the SFLPAG has been set to 10, 10 records*
C*    maximum will be read and loaded at a time.             *
C*                                                           *
C*************************************************************
C*
C           @LOAD     BEGSR
C*
C                     MOVE *OFF      *IN99
C*
C*  Perform this loop until you reach the end of
C*  the CUSTL1   file or until you load  3 records, which
C*  is 1 page  worth of subfile records. (SFLPAG=3)
C*
C                     DO   3
C                     READ CUSTL1                   99
C*
C*  If indicator 99 is on, or if you have already loaded
C*  9999 subfile records, (the maximum allowed), then exit
C*  this loop.
C*
C           *IN99     IFEQ *ON
C           @RRN      OREQ 9999
C                     LEAVE
C                     END
C*
C*  Add 1 to the subfile relative record number.
C*
C                     ADD  1         @RRN
C                     WRITESFD421A
C                     END
C*
C* If you have loaded at least one record into the subfile,
C* turn on indicator 30. This indicator conditions the
C* SFLDSP keyword. If you have loaded at least one record
C* into the subfile, it will be alright to display the subfile
C* format. If you did not load any records into the subfile, you
C* would not turn on indicator 30, because attempting to display
```

```
C* an empty subfile would result in an error being issued.
C*
C           @RRN      IFGT 0
C                     MOVE *ON       *IN30
C                     ELSE
C                     MOVE *OFF      *IN30
C                     END
C*
C                     ENDSR
```

This example conditions the subfile end message with the end of customer file indicator 99.

```
A  99                                   SFLEND(*MORE)
```

Subfile Message

The Subfile End keyword is nice, because it is handled automatically by the subfile. You just have to control it when you want to display the "END" message. But suppose you want to notify users when they are at the top of the subfile, whether or not there are more records, and when they are at the bottom of the subfile. Subfile End can handle the "More" and "Bottom" messages, but it cannot handle the top message.

You can indicate the top of the subfile by using a subfile message. You display to the user on the message line if they are at the top of the subfile, if they are in the middle of the subfile, or if they are at the end of the subfile.

You control these messages with the Subfile Message (SFLMSG) keyword. Do not confuse the subfile message keyword with message subfiles. They are not the same. Message subfiles will be covered later. The subfile message keyword allows you to specify a hard coded message that is displayed on the message line of a subfile. If you want to use a message file to extract the message using a message key instead of hard coding the message, you can use the Subfile Message Identifier (SFLMSGID) keyword instead.

The code for the SFLMSG keyword is:

```
A  98                          SFLMSG('You are at the top of the f-
A                             ile')
A N99                           SFLMSG('More records can be display-
A                             ed')
A  99                          SFLMSG('You have reached the end of-
A                              file')
```

If indicator 98 is on, the program displays a message stating that the user is currently at the top of the file. This message tells the user that he cannot roll backwards any further, because he is already at the top of the data. If indicator 99 is on, it signifies that there are more records that can be loaded into the subfile and display. If indicator 99 is off, then the user has reached the end of all displayed records.

Under normal circumstances, such as when the program displays the first page of subfile records, indicator 98 is on and indicator 99 is off. But the user does not get two messages on the bottom of the display. Whichever message is encountered first is the one that is displayed. You will see the effect of these messages shortly.

Notice the roll key definitions on the DDS.

```
A N99                                    ROLLUP(25)
A N98                                    ROLLDOWN(26)
```

The roll-up key is only allowed when indicator 25 is off. Indicator 99 gets set on if the program reaches the end of the customer file. Therefore, you can restrict the use of the roll-up key after the end of file has been reached. If the roll-up key is pressed, indicator 26 is turned on inside the program stating that the roll-up key has been pressed.

The roll-down key is allowed if indicator 98 is off. When the program is initially called, indicator 98 is turned on because the program is at the top of the file, so it can't roll backwards anyway. Indicator 98 is subsequently turned on if the user rolls back to the top of the file. Whenever the user rolls down, the program turns off indicator 98, which activates the roll-down key. If the roll-down key is pressed, indicator 26 is turned on inside the program.

Look at the portion of the RPG program where it is processing the roll-down key, which is detected when indicator 26 is on. The program isn't reading the file backwards. Instead, it is resetting the customer file pointer back two pages worth of records, and then reading the file forward again. But there are some checks it has to perform. If the program is at the end of file, and only has two subfile records on the display, it could set the pointer too far back, or not far back enough. This is how the program sets the pointer back.

First, the goal is to go backwards through the file seven records at a time. This movement positions the file pointer back, so the program will read the file forward starting with the previous page of records.

But notice that you don't hard code the value in the program to go backwards seven times. You hard code to go backwards four times, plus the relative record number of the subfile. If the program is at the end of the subfile, it might only have one or two records on the display. In this case, going back seven records causes the program

to jump backwards over and miss one record. So instead it goes backwards four records plus the subfile relative record number. If the program has three records on the display, that would cause it to go backwards seven times. If it only has one record displayed, it only goes backwards five times.

Call SFR420 and press F4. Your display should look like this:

```
  xx/xx/xx            Customer Master Maintenance                    QPGMR
  xx:xx:xx                                                           SFD420A


Customer Number....:  ..........................................
                      :   Name: ________________________________   :
Customer Name......:  : 00002   AS/400s Are Us                    :
                      :          State: TN  Phone: 666-455-4444   :
Customer Address...:  : 00011   Computer Mart                     :
                      :          State: MD  Phone: 684-324-6543   :
Customer City......:  : 00001   David's Computers and Bait Shop   :
                      :          State: AR  Phone: 555-123-6666   :
Customer State.....:  :                                 More...  :
                      : F3-Exit                                  :
Customer Zip.......:  : You are at the top of the file            :
                      :..........................................:
Phone Number.......:




 F3-Exit   F4-Customer Search
```

Notice that the message says that you are at the top of the subfile. Attempt to roll backwards. See how the message comes up stating that the function key is invalid.

Now press the roll forward key. Your screen should look like this:

```
  xx/xx/xx            Customer Master Maintenance                    QPGMR
  xx:xx:xx                                                           SFD420A


Customer Number....:  ..........................................
                      :   Name: ________________________________   :
Customer Name......:  : 00045   Full Range Computing              :
                      :          State: FL  Phone: 245-465-2125   :
Customer Address...:  : 00099   Hard Drive Warehouse              :
                      :          State: LA  Phone: 553-654-3435   :
Customer City......:  : 00386   Memory Systems Inc.               :
                      :          State: MN  Phone: 654-989-6565   :
Customer State.....:  :                                  More...  :
                      : F3-Exit                                   :
Customer Zip.......:  : More records can be displayed             :
                      :.........................................:
Phone Number.......:




 F3-Exit   F4-Customer Search
```

Notice how the message has now changed to say that you have more records to display.

Now press the roll key until you reach the end of the file. Your display should look like this:

```
  xx/xx/xx            Customer Master Maintenance                    QPGMR
  xx:xx:xx                                                           SFD420A


Customer Number....:  ........................................
                      :   Name: ______________________________   :
Customer Name......:  : 00003   Tennessee Minicomputer Sales     :
                      :          State: KY  Phone: 880-555-1111  :
Customer Address...:  : 00025   Westmoreland Computers           :
                      :          State: KY  Phone: 684-321-3542  :
Customer City......:  :                                          :
                      :                                          :
Customer State.....:  :                                   Bottom :
                      : F3-Exit                                  :
Customer Zip.......:  : You have reached the end of file         :
                      :..........................................:
Phone Number.......:




 F3-Exit   F4-Customer Search
```

You can see how easy it is to program for messages to appear on the message line of a subfile, and what a difference they can make to the users. You can also use the SFLMSGID keyword to bring messages in from a message file. Subfile messages can be used for much more than an alternate use of the subfile end keyword. But if you like the subfile message keyword, you will definitely want to investigate the use of message subfiles, which are covered in a later chapter.

Performance Considerations

Single-page subfiles suffer from some performance issues. First, notice that to roll backwards, the routine actually reads the file ten times for just a three line subfile. (Seven times to go backwards, plus three to go forwards.) If the file had a unique key, the program could have performed a set lower limits with a key value and performed three REDPE (read previous equal). But this technique only works with files that have a unique key. You might also be able to save the relative record number of the file you are reading inside the subfile as a hidden field. With some fancy coding, you could chain to the first subfile record, and get the file's relative record number. Then you could read the file backwards from this point.

Also, notice that every time the roll key is pressed, you have to clear the subfile and load records. The page-at-a-time subfile is definitely not easiest type of subfile to code, but there are times you should use them. You should use them in update

subfiles, which will be covered shortly. For output subfiles, it is better to stick with either a load-all or an expanding subfile.

5

Displaying Multiple Subfiles

Multiple subfiles are a simple way to display more than one subfile on the screen at a time. A user can benefit from multiple subfiles in some applications.

Suppose you are designing an inventory inquiry screen for a user who is responsible for purchasing inventory and keeping the stock at an appropriate level. The user could benefit from at least three active subfiles. One subfile could show the current purchase orders that are open for an item. Another could show the open customer orders for the item. A third could show the past sales history of the item. In addition, the user may want another panel to show what the past order quantities and prices were.

The user may also want some additional subfile power in this program. For instance, he may want to place the cursor on any item in any of the subfiles, and have a detail inquiry window pop up with detail information about the item. Also, the user may want the ability to roll any one of the subfiles.

In this scenario, you are giving the user a tremendous amount of information on a very limited display size. The subfiles themselves contain just a few fields, but the user can expand the information by zooming in on a particular subfile record.

The Over/Under Subfile Method

Following is an example of a multiple subfile layout. Look at display file SFD500 and program SFR500:

```
A*==============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD500) SRCFILE(XXX/QDDSSRC)
A*
A*==============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03)
A          R SFD500A                    SFL
A            ODLBNM    R        O  5  2REFFLD(QLIDOBJD/ODLBNM *LIBL/FILEOB-
A                                       J)
A                                       COLOR(PNK)
A            ODOBNM    R        O  5 15REFFLD(QLIDOBJD/ODOBNM *LIBL/FILEOB-
A                                       J)
A                                       COLOR(PNK)
A            ODOBAT    R        O  5 30REFFLD(QLIDOBJD/ODOBAT *LIBL/FILEOB-
A                                       J)
A                                       COLOR(PNK)
A            ODCRTU    R        O  5 67REFFLD(QLIDOBJD/ODCRTU *LIBL/FILEOB-
A                                       J)
A                                       COLOR(PNK)
A            ODOBSZ    R        O  5 47REFFLD(QLIDOBJD/ODOBSZ *LIBL/FILEOB-
A                                       J)
A                                       EDTCDE(J)
A                                       COLOR(PNK)
A          R SFD500B                    SFLCTL(SFD500A)
A                                       SFLSIZ(0020)
A                                       SFLPAG(0005)
A                                       OVERLAY
A  30                                   SFLDSP
A                                       SFLDSPCTL
A  35                                   SFLEND(*MORE)
A                                   1 27'ZZSUBFILE Programs Inquiry'
A                                       COLOR(WHT)
A                                       DSPATR(RI)
A                                   1  3DATE
A                                       EDTCDE(Y)
A                                       COLOR(BLU)
A                                   2  3TIME
A                                       COLOR(BLU)
A                                   1 71USER
A                                       COLOR(BLU)
A                                   2 71'SFD500A'
A                                       COLOR(BLU)
A                                   4  2'Library   '
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A                                   4 15'Program '
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A                                   4 30'Attribute'
```

```
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A                                   4 47'Object Size  '
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A                                   4 67'Created By'
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A          R SFD500C                    SFL
A            ODLBNM    R        O 17  2REFFLD(QLIDOBJD/ODLBNM *LIBL/FILEOB-
A                                       J)
A                                       COLOR(PNK)
A            ODOBNM    R        O 17 15REFFLD(QLIDOBJD/ODOBNM *LIBL/FILEOB-
A                                       J)
A                                       COLOR(PNK)
A            ODOBAT    R        O 17 30REFFLD(QLIDOBJD/ODOBAT *LIBL/FILEOB-
A                                       J)
A                                       COLOR(PNK)
A            ODOBSZ    R        O 17 47REFFLD(QLIDOBJD/ODOBSZ *LIBL/FILEOB-
A                                       J)
A                                       COLOR(PNK)
A                                       EDTCDE(J)
A            ODCRTU    R        O 17 67REFFLD(QLIDOBJD/ODCRTU *LIBL/FILEOB-
A                                       J)
A                                       COLOR(PNK)
A          R SFD500D                    SFLCTL(SFD500C)
A                                       SFLSIZ(0020)
A                                       SFLPAG(0005)
A                                       OVERLAY
A  31                                   SFLDSP
A                                       SFLDSPCTL
A  35                                   SFLEND(*MORE)
A                                  14 27'ZZSUBFILE Files Inquiry   '
A                                       COLOR(WHT)
A                                       DSPATR(RI)
A                                  16  2'Library  '
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A                                  16 15'File    '
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A                                  16 30'Attribute'
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A                                  16 47'Object Size  '
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A                                  16 67'Created By'
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A          R TRAILER
A                                  23  1'F3-Exit'
A                                       DSPATR(HI)
```

```
F*============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR500) SRCFILE(XXX/QRPGSRC)
F*
F*============================================================
FFILEOBJ IF  E           K        DISK
FSFD500  CF  E                    WORKSTN
F                                       @RRN1 KSFILE SFD500A
F                                       @RRN2 KSFILE SFD500C
C*
C                     WRITETRAILER
C                     WRITESFD500B
C                     WRITESFD500D
C                     READ SFD500B                  80
C                     READ SFD500D                  80
C                     EXSR @CMD
C*
C*********************************************************
C*    @ C M D   S U B R O U T I N E                      *
C*                                                       *
C*    Executed after every EXFMT or display read op code.  *
C*    Used to process command keys.                      *
C*********************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, seton LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C                     ENDSR
C*
C*********************************************************
C*    * I N Z S R   S U B R O U T I N E                  *
C*                                                       *
C*    Retrieve the starting line and position for the    *
C*    window. Check to make sure that these values are   *
C*    valid. If not, put the closest valid number in.    *
C*********************************************************
C*
C           *INZSR    BEGSR
C*
C                     Z-ADD0         @RRN1   50
C                     Z-ADD0         @RRN2   50
C*
C*  Load the subfiles
C*
C                     EXSR @LOAD
C*
C                     ENDSR
C*
C*********************************************************
C*    @ L O A D     S U B R O U T I N E                  *
```

```
C*                                                          *
C*    This subroutine will read the file and program        *
C*    objects frmo file FILEOBJ. If the object is a         *
C*    program, it will be loaded into SFD500A. If it is     *
C*    a file, load it into SFD500C.                         *
C*************************************************************
C*
C           @LOAD     BEGSR
C*
C*  Perform this loop until you reach the end of the end
C*  of FILEOBJ file.
C*
C           *IN99     DOWNE*ON
C                     READ FILEOBJ                  99
C*
C*  If indicator 99 is on, or if you have already loaded
C*  9999 subfile records, (the maximum allowed), then exit
C*  this loop.
C*
C           *IN99     IFEQ *ON
C           @RRN1     OREQ 9999
C           @RRN2     OREQ 9999
C                     LEAVE
C                     END
C*
C*  If its a program, load it into SFD500A.
C*
C           ODOBTP    IFEQ '*PGM'
C                     ADD  1         @RRN1
C                     WRITESFD500A
C                     END
C*
C*  If its a file, load it into SFD500C.
C*
C           ODOBTP    IFEQ '*FILE'
C                     ADD  1         @RRN2
C                     WRITESFD500C
C                     END
C*
C                     END
C*
C* If you have loaded at least one record into the subfile,
C* turn on indicator 30. This indicator conditions the
C* SFLDSP keyword. If you have loaded at least one record
C* into the subfile, you can display the subfile
C* format. If you did not load any records into the subfile, you
C* would not turn on indicator 30. Attempting to display
C* an empty subfile would result in an error being issued.
C*
C                     MOVE *ON       *IN35
C*
C*   If you loaded records into SFD500A, it will be ok
C*   to display it.
C*
C           @RRN1     IFGT 0
C                     MOVE *ON       *IN30
C                     END
```

```
C*
C*   If you loaded recordsd into SFD500C, it will be ok
C*   to display it.
C*
C           @RRN2     IFGT 0
C                     MOVE *ON       *IN31
C                     END
C*
C                     ENDSR
```

The display file SFD500 and program SFR500 use a file called FILEOBJ. There is no DDS for this file. Instead, you can use the OUTFILE parameter of the Display Object Description (DSPOBJD) command to create the file. In the example, the command is run against all objects in a library called ZZSUBFILE, but you can choose any library you want. Run a command similar to the following to create the FILEOBJ file on your system before attempting to compile SFD500 and SFR500:

```
DSPOBJD OBJ(ZZSUBFILE/*ALL) +
        OBJTYPE(*ALL) +
        OUTPUT(*OUTFILE) +
        OUTFILE(FILEOBJ)
```

SFR500 is what is known as an "over/under" subfile. The display actually contains two active subfile records—one is displayed on top and one below. Call SFR500 to see this effect:

```
 xx/xx/xx                ZZSUBFILE Programs Inquiry                 QPGMR
 xx:xx:xx                                                           SFD500A

Library      Program        Attribute        Object Size         Created By
ZZSUBFILE    SFC900A        CLP                      8,192       MICHAEL
ZZSUBFILE    SFC900B        CLP                      8,192       MICHAEL
ZZSUBFILE    SFC900C        CLP                      8,192       MICHAEL
ZZSUBFILE    SFC900D        CLP                      8,192       MICHAEL
ZZSUBFILE    SFC901A        CLP                      8,192       MICHAEL
                                                                       More...



                        ZZSUBFILE Files Inquiry

Library      File           Attribute        Object Size         Created By
ZZSUBFILE    BIGF           PF                   1,187,840       QSECOFR
ZZSUBFILE    CONTACT        PF                      15,872       QSECOFR
ZZSUBFILE    CUSTL1         LF                      10,752       QSECOFR
ZZSUBFILE    CUSTOMER       PF                      30,208       QSECOFR
ZZSUBFILE    FILEOBJ        PF                      99,328       MICHAEL
                                                                       More...
```

You can roll either subfile by placing the cursor on the part of the screen controlled by that subfile. If your cursor is in the top part of the screen, you can roll forward

and backward through the top subfile. If the cursor is on the lower subfile, you can roll forward and backward through the lower subfile. Control is never passed back to the program during the roll key processing. It is all handled by the subfile formats automatically, even though there are two of them. This example shows how you can have more than one format active at a time and read from all formats. The function occurs in SFR500 in this routine:

```
C                     WRITETRAILER
C                     WRITESFD500B
C                     WRITESFD500D
C                     READ SFD500B                  80
C                     READ SFD500D                  80
```

Since both of the subfiles use the load-all method, the program never leaves the read statements when the roll keys are processed.

Notice that the user has to use the arrow keys to maneuver the cursor into the subfile record part of the screen. If the user has a mouse, that is no problem. But for the unlucky users who do not have a mouse, what can you do?

You can use a trick to allow the user to maneuver around the different subfiles easily. Specifically, you can describe an input field on each subfile control format that does nothing. This trick, however, allows users to use the tab key, and in one keystroke, change which subfile they are on.

For an example of this trick, see display file SFD510 and program SFR510.

```
A*============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD510) SRCFILE(XXX/QDDSSRC)
A*
A*============================================================
A                                     DSPSIZ(24 80 *DS3)
A                                     CF03(03)
A          R SFD510A                  SFL
A            ODLBNM    R        O  5  2REFFLD(QLIDOBJD/ODLBNM *LIBL/FILEOB-
A                                     J)
A                                     COLOR(PNK)
A            ODOBNM    R        O  5 15REFFLD(QLIDOBJD/ODOBNM *LIBL/FILEOB-
A                                     J)
A                                     COLOR(PNK)
A            ODOBAT    R        O  5 30REFFLD(QLIDOBJD/ODOBAT *LIBL/FILEOB-
A                                     J)
A                                     COLOR(PNK)
A            ODCRTU    R        O  5 67REFFLD(QLIDOBJD/ODCRTU *LIBL/FILEOB-
A                                     J)
A                                     COLOR(PNK)
A            ODOBSZ    R        O  5 47REFFLD(QLIDOBJD/ODOBSZ *LIBL/FILEOB-
A                                     J)
A                                     EDTCDE(J)
A                                     COLOR(PNK)
```

```
A          R SFD510B                    SFLCTL(SFD510A)
A                                       SFLSIZ(0020)
A                                       SFLPAG(0005)
A                                       OVERLAY
A  30                                   SFLDSP
A                                       SFLDSPCTL
A  35                                   SFLEND(*MORE)
A                                   1 27'ZZSUBFILE Programs Inquiry'
A                                       COLOR(WHT)
A                                       DSPATR(RI)
A                                   1  3DATE
A                                       EDTCDE(Y)
A                                       COLOR(BLU)
A                                   2  3TIME
A                                       COLOR(BLU)
A                                   1 71USER
A                                       COLOR(BLU)
A                                   2 71'SFD510A'
A                                       COLOR(BLU)
A                                   4  2'Library   '
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A                                   4 15'Program '
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A                                   4 30'Attribute'
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A                                   4 47'Object Size  '
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A                                   4 67'Created By'
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A            @CSR           1   I  3  3DSPATR(ND)
A          R SFD510C                    SFL
A            ODLBNM    R        O 17  2REFFLD(QLIDOBJD/ODLBNM *LIBL/FILEOB-
A                                       J)
A                                       COLOR(PNK)
A            ODOBNM    R        O 17 15REFFLD(QLIDOBJD/ODOBNM *LIBL/FILEOB-
A                                       J)
A                                       COLOR(PNK)
A            ODOBAT    R        O 17 30REFFLD(QLIDOBJD/ODOBAT *LIBL/FILEOB-
A                                       J)
A                                       COLOR(PNK)
A            ODOBSZ    R        O 17 47REFFLD(QLIDOBJD/ODOBSZ *LIBL/FILEOB-
A                                       J)
A                                       COLOR(PNK)
A                                       EDTCDE(J)
A            ODCRTU    R        O 17 67REFFLD(QLIDOBJD/ODCRTU *LIBL/FILEOB-
A                                       J)
A                                       COLOR(PNK)
A          R SFD510D                    SFLCTL(SFD510C)
A                                       SFLSIZ(0020)
A                                       SFLPAG(0005)
A                                       OVERLAY
A  31                                   SFLDSP
```

```
A                                        SFLDSPCTL
A  35                                    SFLEND(*MORE)
A                                   14 27'ZZSUBFILE Files Inquiry   '
A                                        COLOR(WHT)
A                                        DSPATR(RI)
A                                   16  2'Library  '
A                                        COLOR(YLW)
A                                        DSPATR(UL)
A                                   16 15'File    '
A                                        COLOR(YLW)
A                                        DSPATR(UL)
A                                   16 30'Attribute'
A                                        COLOR(YLW)
A                                        DSPATR(UL)
A                                   16 47'Object Size  '
A                                        COLOR(YLW)
A                                        DSPATR(UL)
A                                   16 67'Created By'
A                                        COLOR(YLW)
A                                        DSPATR(UL)
A            @CSR           1   I 15  3DSPATR(ND)
A          R TRAILER
A                                   23  1'F3-Exit'
A                                        DSPATR(HI)


F*==============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR510) SRCFILE(XXX/QRPGSRC)
F*
F*==============================================================
FFILEOBJ IF  E           K        DISK
FSFD510  CF  E                    WORKSTN
F                                       @RRN1 KSFILE SFD510A
F                                       @RRN2 KSFILE SFD510C
C*
C                     WRITETRAILER
C                     WRITESFD510B
C                     WRITESFD510D
C                     READ SFD510B                 80
C                     READ SFD510D                 80
C                     EXSR @CMD
C*
C*************************************************************
C*    @ C M D   S U B R O U T I N E                          *
C*                                                           *
C*    Executed after every EXFMT or display read op code.    *
C*    Used to process command keys.                          *
C*************************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, seton LR and exit.
C*
C           *IN03     IFEQ *ON
```

```
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C                     ENDSR
C*
C*************************************************************
C*    * I N Z S R   S U B R O U T I N E                      *
C*                                                           *
C*    Retrieve the starting line and position for the        *
C*    window. Check to make sure that these values are       *
C*    valid. If not, put the closest valid number in.        *
C*************************************************************
C*
C           *INZSR    BEGSR
C*
C                     Z-ADD0         @RRN1   50
C                     Z-ADD0         @RRN2   50
C*
C*  Load the subfiles
C*
C                     EXSR @LOAD
C*
C                     ENDSR
C*
C*************************************************************
C*    @ L O A D     S U B R O U T I N E                      *
C*                                                           *
C*    This subroutine will read the file and program         *
C*    objects frmo file FILE0BJ. If the object is a          *
C*    program, it will be loaded into SFD510A. If it is      *
C*    a file, load it into SFD510C.                          *
C*************************************************************
C*
C           @LOAD     BEGSR
C*
C*  Perform this loop until you reach the end of the end
C*  of FILE0BJ file.
C*
C           *IN99     DOWNE*ON
C                     READ FILE0BJ                 99
C*
C*  If indicator 99 is on, or if you have already loaded
C*  9999 subfile records, (the maximum allowed), then exit
C*  this loop.
C*
C           *IN99     IFEQ *ON
C           @RRN1     OREQ 9999
C           @RRN2     OREQ 9999
C                     LEAVE
C                     END
C*
C*  If its a program, load it into SFD510A.
C*
C           0D0BTP    IFEQ '*PGM'
C                     ADD  1         @RRN1
C                     WRITESFD510A
```

```
C                     END
C*
C*  If its a file, load it into SFD510C.
C*
C           ODOBTP    IFEQ '*FILE'
C                     ADD  1         @RRN2
C                     WRITESFD510C
C                     END
C*
C                     END
C*
C* If you have loaded at least one record into the subfile,
C* turn on indicator 30. This indicator conditions the
C* SFLDSP keyword. If you have loaded at least one record
C* into the subfile, it will be alright to display the subfile
C* format. If you did not load any records into the subfile, you
C* would not turn on indicator 30, because attempting to display
C* an empty subfile would result in an error being issued.
C*
C                     MOVE *ON       *IN35
C*
C*   If you loaded recordsd into SFD510A, it will be ok
C*   to display it.
C*
C           @RRN1     IFGT 0
C                     MOVE *ON       *IN30
C                     END
C*
C*   If you loaded recordsd into SFD510C, it will be ok
C*   to display it.
C*
C           @RRN2     IFGT 0
C                     MOVE *ON       *IN31
C                     END
C*
C                     ENDSR
```

The user can now move the cursor between the two subfile screens by using the Tab or Field Exit key. This movement is possible because each record format now contains an input field. Since both subfiles are active, both input fields are active, and the user can toggle between them. The input fields are defined as nondisplayed. You can see this feature by calling SFR510. Your display should look like this.

```
 xx/xx/xx                ZZSUBFILE Programs Inquiry                 QPGMR
 xx:xx:xx                                                           SFD510A

Library      Program        Attribute        Object Size         Created By
ZZSUBFILE    SFC900A        CLP                      8,192       MICHAEL
ZZSUBFILE    SFC900B        CLP                      8,192       MICHAEL
ZZSUBFILE    SFC900C        CLP                      8,192       MICHAEL
ZZSUBFILE    SFC900D        CLP                      8,192       MICHAEL
ZZSUBFILE    SFC901A        CLP                      8,192       MICHAEL
                                                                       More...



                         ZZSUBFILE Files Inquiry

Library      File           Attribute        Object Size         Created By
ZZSUBFILE    BIGF           PF                   1,187,840       QSECOFR
ZZSUBFILE    CONTACT        PF                      15,872       QSECOFR
ZZSUBFILE    CUSTL1         LF                      10,752       QSECOFR
ZZSUBFILE    CUSTOMER       PF                      30,208       QSECOFR
ZZSUBFILE    FILEOBJ        PF                      99,328       MICHAEL
                                                                       More...
```

Now use the Tab or Field Exit keys to move back and forth between the subfile formats. Notice how quickly you can roll each format.

Complex Multiple Subfiles Displays

You don't have to limit yourself to just the over and under subfile concept. You can do side-by-side subfiles, or side-by-side on top, and side-by-side below. Actually, practically any combination is possible.

Unfortunately, you can't do side-by-side subfiles like you do over/under subfiles. The problem is that a subfile record that is displayed in full-screen mode occupies an entire display line. Therefore, you can't have two subfile formats side by side, because one actually overlays the other.

Simulating Side-by-Side Subfiles

There are a couple of ways to do side-by-side subfiles. The first method is to create the illusion that two or more are being displayed side by side. What the user sees on their display could be something like this:

```
**** Items Booked ****      ***** Items Shipped  *****

Item #       Quantity       Item #        Quantity

ACC-24        200.00        ACC-24         4565.00
ACC-27        213.00        ACC-28          345.00
BDD-123       432.00        CCS-2        12,334.00
```

This screen appears to be two subfiles: one for the booked items and one for the shipped items. The user can place the cursor on the "Items Booked" side of the screen and just roll the booked items and quantity, or move the cursor to the "Items Shipped" side of the screen and just roll the shipped items and quantity.

You would think you were seeing two subfiles. Actually, this method is usually accomplished using one subfile. Each subfile record looks like this:

```
Booked Item | Booked Quantity | Shipped Item | Shipped Quantity
```

To get each side of the subfile to roll independently, this method uses two multiple occurrence data structures . One data structure contains the booked information and the other the shipped information.

The type of subfile used for this process is the page-at-a-time subfile. The program actually performs the "load-all" logic to the data structures first.

Following is how the loading and displaying process breaks down.

1. Load-all subfile records into the multiple occurrence data structures.
2. Clear the subfile.
3. Set the multiple occurrence data structure pointers. (Initially both are set to 1.)

The next steps are looped until one page of subfile records have been written.

4. Write a subfile record.
5. Add 1 to the data structure pointers.

End of loop

6. Display Subfile.

Now the subfile is initially displayed. One page of subfile records are displayed, but all subfile record data has been loaded into the multiple occurrence data structures.

If the user presses the roll keys, the cursor position is extracted from the display screen. If the cursor is on the booked side of the screen, the pointer for the shipped side is reset to its saved value, which in this case is still 1. The pointer for the booked

side is not reset. Then the program goes back to step 2 above and repeats the process to load another page of subfile records.

The problem with this method is that you have to predefine in your program how many occurrences of each data structure can occur. Then you have to make sure that you only load to a maximum of the occurrences of the data structures. You have placed some of the subfile control inside the RPG program, and this is not really a good idea. The beauty behind subfiles is that their functions reside in DDS, and not the RPG program.

Windowed Side-by-Side Subfiles

A better approach to side-by-side subfiles is to use windows. As stated before, subfile formats that are displayed in the full-screen mode cannot reside side by side. But subfiles displayed inside windows can.

You can define multiple windows in different parts of the screen, and you can display many different subfiles. However, there is one catch to this method. Only one window can be active at a time, which causes some limitations, but as usual, there is a way around them.

First, look at display file SFD520 and program SFR520:

```
A*============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD520) SRCFILE(XXX/QDDSSRC)
A*
A*============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03 'Exit')
A                                       CF10(10 'Toggle Active Window')
A          R HEADER
A                                  1  3DATE
A                                       EDTCDE(Y)
A                                       COLOR(BLU)
A                                  2  3TIME
A                                       COLOR(BLU)
A                                  1 31'Object Inquiry'
A                                       COLOR(WHT)
A                                  1 70USER
A                                       COLOR(BLU)
A                                  2 70'SFR520'
A                                       COLOR(BLU)
A                                 23  1'F3-Exit   F10-Change Active Window'
A                                       COLOR(WHT)
A                                  4  2'** Programs **'
A                                       COLOR(GRN)
A                                  4 56'** Files **'
```

```
A                                        COLOR(GRN)
A                                   14  2'** Source Files **'
A                                        COLOR(GRN)
A          R SFD520A                     SFL
A            OD0BSZ    R        0  2 12REFFLD(QLID0BJD/OD0BSZ *LIBL/FILE0B-
A                                        J)
A                                        COLOR(PNK)
A                                        EDTCDE(Z)
A            OD0BNM    R        0  2  1REFFLD(QLID0BJD/OD0BNM *LIBL/FILE0B-
A                                        J)
A                                        COLOR(PNK)
A            ODCRTU    R        0  2 23REFFLD(QLID0BJD/ODCRTU *LIBL/FILE0B-
A                                        J)
A                                        COLOR(PNK)
A          R SFD520B                     SFLCTL(SFD520A)
A                                        OVERLAY
A  30                                    SFLDSP
A                                        SFLDSPCTL
A  35                                    SFLEND(*MORE)
A                                        SFLSIZ(0020)
A                                        SFLPAG(0004)
A                                        WINDOW(5 2 7 33)
A                                        WDWBORDER((*CHAR '        '))
A                                   1  1'Program   '
A                                        COLOR(YLW)
A                                        DSPATR(UL)
A                                   1 12'Size     '
A                                        COLOR(YLW)
A                                        DSPATR(UL)
A                                   1 23'User      '
A                                        COLOR(YLW)
A                                        DSPATR(UL)
A          R SFD520C                     SFL
A            OD0BSZ    R        0  2 12REFFLD(QLID0BJD/OD0BSZ *LIBL/FILE0B-
A                                        J)
A                                        COLOR(PNK)
A                                        EDTCDE(Z)
A            OD0BNM    R        0  2  1REFFLD(QLID0BJD/OD0BNM *LIBL/FILE0B-
A                                        J)
A                                        COLOR(PNK)
A            ODCRTU    R        0  2 23REFFLD(QLID0BJD/ODCRTU *LIBL/FILE0B-
A                                        J)
A                                        COLOR(PNK)
A          R SFD520D                     SFLCTL(SFD520C)
A                                        OVERLAY
A  31                                    SFLDSP
A                                        SFLDSPCTL
A  35                                    SFLEND(*MORE)
A                                        SFLSIZ(0020)
A                                        SFLPAG(0004)
A                                        WINDOW(5 43 7 33)
A                                        WDWBORDER((*CHAR '        '))
A                                   1  1'Data Files'
A                                        COLOR(YLW)
A                                        DSPATR(UL)
A                                   1 12'Size     '
A                                        COLOR(YLW)
```

```
A                                       DSPATR(UL)
A                                   1 23'User      '
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A          R SFD52OE                    SFL
A            ODOBSZ    R        O  2 14REFFLD(QLIDOBJD/ODOBSZ *LIBL/FILEOB-
A                                       J)
A                                       COLOR(PNK)
A                                       EDTCDE(Z)
A            ODOBNM    R        O  2  1REFFLD(QLIDOBJD/ODOBNM *LIBL/FILEOB-
A                                       J)
A                                       COLOR(PNK)
A            ODCRTU    R        O  2 25REFFLD(QLIDOBJD/ODCRTU *LIBL/FILEOB-
A                                       J)
A                                       COLOR(PNK)
A          R SFD52OF                    SFLCTL(SFD52OE)
A                                       OVERLAY
A  32                                   SFLDSP
A                                       SFLDSPCTL
A  35                                   SFLEND(*MORE)
A                                       SFLSIZ(0020)
A                                       SFLPAG(0002)
A                                       WINDOW(15 2 5 35)
A                                       WDWBORDER((*CHAR '        '))
A                                   1  1'Disply Files'
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A                                   1 14'Size     '
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A                                   1 25'User      '
A                                       COLOR(YLW)
A                                       DSPATR(UL)


F*==============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR52O) SRCFILE(XXX/QRPGSRC)
F*
F*==============================================================
FFILEOBJ IF  E           K        DISK
FSFD52O  CF  E                    WORKSTN
F                                       @RRN1 KSFILE SFD52OA
F                                       @RRN2 KSFILE SFD52OC
F                                       @RRN3 KSFILE SFD52OE
C*
C* Write the windows, activating the correct window for rolling
C*
C           *IN41     IFEQ '1'
C                     WRITEHEADER
C                     WRITESFD52OD
C                     WRITESFD52OF
C                     EXFMTSFD52OB
C                     END
C*
```

```
C           *IN42     IFEQ '1'
C                     WRITEHEADER
C                     WRITESFD520B
C                     WRITESFD520F
C                     EXFMTSFD520D
C                     END
C*
C           *IN43     IFEQ '1'
C                     WRITEHEADER
C                     WRITESFD520B
C                     WRITESFD520D
C                     EXFMTSFD520F
C                     END
C*
C                     EXSR @CMD
C*
C*************************************************************
C*    @ C M D   S U B R O U T I N E                          *
C*                                                           *
C*    Executed after every EXFMT or display read op code.    *
C*    Used to process command keys.                          *
C*************************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, seton LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C* If F10 was pressed, then toggle the active window
C*
C           *IN10     DOWEQ*ON
C                     MOVE *OFF      *IN10
C*
C           *IN41     IFEQ *ON
C                     MOVE *OFF      *IN41
C                     MOVE *ON       *IN42
C                     LEAVE
C                     END
C*
C           *IN42     IFEQ *ON
C                     MOVE *OFF      *IN42
C                     MOVE *ON       *IN43
C                     LEAVE
C                     END
C*
C           *IN43     IFEQ *ON
C                     MOVE *OFF      *IN43
C                     MOVE *ON       *IN41
C                     LEAVE
C                     END
C*
C                     END
C*
```

```
C                     ENDSR
C*
C*********************************************************
C*    * I N Z S R   S U B R O U T I N E                  *
C*                                                       *
C*    Retrieve the starting line and position for the    *
C*    window. Check to make sure that these values are   *
C*    valid. If not, put the closest valid number in.    *
C*********************************************************
C*
C           *INZSR    BEGSR
C*
C                     Z-ADD0         @RRN1   50
C                     Z-ADD0         @RRN2   50
C                     Z-ADD0         @RRN3   50
C                     MOVE *ON       *IN41
C*
C*  Load the subfiles
C*
C                     EXSR @LOAD
C*
C                     ENDSR
C*
C*********************************************************
C*    @ L O A D     S U B R O U T I N E                  *
C*                                                       *
C*    This subroutine will read the file and program     *
C*    objects frmo file FILEOBJ. If the object is a      *
C*    program, it will be loaded into SFD520A. If it is  *
C*    a file, load it into SFD520C.                      *
C*********************************************************
C*
C           @LOAD     BEGSR
C*
C*  Perform this loop until you reach the end of the end
C*  of FILEOBJ file.
C*
C           *IN99     DOWNE*ON
C                     READ FILEOBJ                 99
C*
C*  If indicator 99 is on, or if you have already loaded
C*  9999 subfile records, (the maximum allowed), then exit
C*  this loop.
C*
C           *IN99     IFEQ *ON
C           @RRN1     OREQ 9999
C           @RRN2     OREQ 9999
C           @RRN3     OREQ 9999
C                     LEAVE
C                     END
C*
C*  If its a program, load it into SFD520A.
C*
C           ODOBTP    IFEQ '*PGM'
C                     ADD  1         @RRN1
C                     WRITESFD520A
C                     END
```

```
C*
C*  If its a file, load it into SFD520C.
C*
C           0D0BTP    IFEQ '*FILE'
C           0D0BAT    ANDNE'DSPF'
C                     ADD  1         @RRN2
C                     WRITESFD520C
C                     END
C*
C*
C*  If its a source file, load it into SFD520E.
C*
C           0D0BTP    IFEQ '*FILE'
C           0D0BAT    ANDEQ'DSPF'
C                     ADD  1         @RRN3
C                     WRITESFD520E
C                     END
C                     END
C*
C* Since all three subfiles are load all, it will be ok to turn
C* on indicator 35.
C*
C                     MOVE *ON       *IN35
C*
C*   If you loaded records into SFD520A, it will be ok
C*   to display it.
C*
C           @RRN1     IFGT 0
C                     MOVE *ON       *IN30
C                     END
C*
C*   If you loaded records into SFD520C, it will be ok
C*   to display it.
C*
C           @RRN2     IFGT 0
C                     MOVE *ON       *IN31
C                     END
C*
C*   If you loaded records into SFD520E, it will be ok
C*   to display it.
C*
C           @RRN3     IFGT 0
C                     MOVE *ON       *IN32
C                     END
C*
C                     ENDSR
```

SFR520 displays three subfiles on the display. They are not an illusion. There are actually three separate subfiles on the display. Call SFR520. Your display should look like this:

```
 xx/xx/xx                    Object Inquiry                        QPGMR
 xx:xx:xx                                                          SFR520

** Programs **                                        ** Files **

  Program    Size        User          Data Files Size        User
  SFC900A          8192 MICHAEL         BIGF          1187840 QSECOFR
  SFC900B          8192 MICHAEL         CONTACT         15872 QSECOFR
  SFC900C          8192 MICHAEL         CUSTL1          10752 QSECOFR
  SFC900D          8192 MICHAEL         CUSTOMER        30208 QSECOFR
                            More...                                 More...


** Source Files **

  Disply Files Size        User
  SFD001             3584 MICHAEL
  SFD002             3072 MICHAEL
                              More...



F3-Exit   F10-Change Active Window
```

As you can see, this program displays three subfiles. Two of them are side by side, and one is below the other two. Notice how this effect was accomplished. In the example, three windows were specified, but the window border attributes were not displayed. By specifying the window border attribute of " ", no window border attributes were displayed. Notice how each subfile has its own subfile end message—a nice touch for the user.

Press the roll key. Notice how the first subfile rolls forward, while the others remain stationary. Now move the cursor to the next subfile and press the roll key. The cursor is returned to the first subfile, and the display station sounds the alarm because only one window can be active at one time.

There is a way to get around this limitation and provide a quick way for the user to jump from subfile to subfile. Remember in the over/under subfile example how there were two nondisplay input fields in which you could toggle between the upper and lower subfiles. That won't work in this case because the other windows aren't active. Instead, you can use a function key as a toggle between subfiles. If the user presses F10, the subfiles are redisplayed so that the second subfile is now active, and the cursor is now positioned on the second subfile. You can now roll the second subfile. By pressing F10 again, you would toggle to the third subfile.

Look at the program logic of how this feature works. Before you write the subfile formats to the display, you must execute the following routines:

```
C           *IN41     IFEQ '1'
C                     WRITEHEADER
C                     WRITESFD520D
C                     WRITESFD520F
C                     EXFMTSFD520B
C                     END
C*
C           *IN42     IFEQ '1'
C                     WRITEHEADER
C                     WRITESFD520B
C                     WRITESFD520F
C                     EXFMTSFD520D
C                     END
C*
C           *IN43     IFEQ '1'
C                     WRITEHEADER
C                     WRITESFD520B
C                     WRITESFD520D
C                     EXFMTSFD520F
C                     END
```

When the program initially starts, indicator 41 is set on. Therefore, the first routine is processed. In this case, the program writes the subfile windows SFD520D and SFD520F, then performs a write/read on SFD520B. This write/read causes SFD520B to be the active window because it was the last window to which something was written.

Down in the program, you have the logic for situations when the F10 key is pressed.

```
C           *IN10     DOWEQ*ON
C                     MOVE *OFF      *IN10
C*
C           *IN41     IFEQ *ON
C                     MOVE *OFF      *IN41
C                     MOVE *ON       *IN42
C                     LEAVE
C                     END
C*
C           *IN42     IFEQ *ON
C                     MOVE *OFF      *IN42
C                     MOVE *ON       *IN43
C                     LEAVE
C                     END
C*
C           *IN43     IFEQ *ON
C                     MOVE *OFF      *IN43
C                     MOVE *ON       *IN41
C                     LEAVE
C                     END
```

These routines cause the indicators 41, 42, and 43 to toggle on in succession each time the F10 key is pressed. Initially, indicator 41 is on. The next time the F10 key is pressed, indicator 41 is set off, and indicator 42 is set on. Then the next time the program displays the subfiles, this routine is performed:

```
C           *IN42     IFEQ '1'
C                     WRITEHEADER
C                     WRITESFD520B
C                     WRITESFD520F
C                     EXFMTSFD520D
C                     END
```

So with the use of one function key, the user can activate a different subfile window. This process automatically positions the cursor into the active window, which allows the user to use the roll keys for that subfile.

Now call SFR520 again. Press the roll forward key. The first subfile now rolls to the second page of subfile records. Now press F10. The second subfile window is activated, but notice how the first subfile is now displaying the first subfile page again. This display is not usually a desirable effect. The user usually wants his subfiles to "stick" in the position he leaves them. You can do this by adding a couple of lines of code to the display and RPG program.

You can see these changes in SFD530 and SFR530.

```
A*============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD530) SRCFILE(XXX/QDDSSRC)
A*
A*============================================================
A                                        DSPSIZ(24 80 *DS3)
A                                        CF03(03 'Exit')
A                                        CF10(10 'Toggle Active Window')
A          R HEADER
A                                   1  3DATE
A                                        EDTCDE(Y)
A                                        COLOR(BLU)
A                                   2  3TIME
A                                        COLOR(BLU)
A                                   1 31'Object Inquiry'
A                                        COLOR(WHT)
A                                   1 70USER
A                                        COLOR(BLU)
A                                   2 70'SFR530'
A                                        COLOR(BLU)
A                                  23  1'F3-Exit   F10-Change Active Window'
A                                        COLOR(WHT)
A                                   4  2'** Programs **'
A                                        COLOR(GRN)
A                                   4 56'** Files **'
A                                        COLOR(GRN)
A                                  14  2'** Source Files **'
A                                        COLOR(GRN)
A          R SFD530A                     SFL
A            ODOBSZ    R        O  2 12REFFLD(QLIDOBJD/ODOBSZ *LIBL/FILEOB-
A                                        J)
A                                        COLOR(PNK)
A                                        EDTCDE(Z)
```

```
A            ODOBNM    R        O  2  1REFFLD(QLIDOBJD/ODOBNM *LIBL/FILEOB-
A                                      J)
A                                      COLOR(PNK)
A            ODCRTU    R        O  2 23REFFLD(QLIDOBJD/ODCRTU *LIBL/FILEOB-
A                                      J)
A                                      COLOR(PNK)
A          R SFD530B                   SFLCTL(SFD530A)
A                                      OVERLAY
A  30                                  SFLDSP
A                                      SFLDSPCTL
A  35                                  SFLEND(*MORE)
A                                      SFLSIZ(0020)
A                                      SFLPAG(0004)
A                                      WINDOW(5 2 7 33)
A                                      WDWBORDER((*CHAR '        '))
A            @REC1          4S OH      SFLRCDNBR
A                                 1  1'Program   '
A                                      COLOR(YLW)
A                                      DSPATR(UL)
A                                 1 12'Size     '
A                                      COLOR(YLW)
A                                      DSPATR(UL)
A                                 1 23'User      '
A                                      COLOR(YLW)
A                                      DSPATR(UL)
A          R SFD530C                   SFL
A            ODOBSZ    R        O  2 12REFFLD(QLIDOBJD/ODOBSZ *LIBL/FILEOB-
A                                      J)
A                                      COLOR(PNK)
A                                      EDTCDE(Z)
A            ODOBNM    R        O  2  1REFFLD(QLIDOBJD/ODOBNM *LIBL/FILEOB-
A                                      J)
A                                      COLOR(PNK)
A            ODCRTU    R        O  2 23REFFLD(QLIDOBJD/ODCRTU *LIBL/FILEOB-
A                                      J)
A                                      COLOR(PNK)
A          R SFD530D                   SFLCTL(SFD530C)
A                                      OVERLAY
A  31                                  SFLDSP
A                                      SFLDSPCTL
A  35                                  SFLEND(*MORE)
A                                      SFLSIZ(0020)
A                                      SFLPAG(0004)
A                                      WINDOW(5 43 7 33)
A                                      WDWBORDER((*CHAR '        '))
A            @REC2          4S OH      SFLRCDNBR
A                                 1  1'Data Files'
A                                      COLOR(YLW)
A                                      DSPATR(UL)
A                                 1 12'Size     '
A                                      COLOR(YLW)
A                                      DSPATR(UL)
A                                 1 23'User      '
A                                      COLOR(YLW)
A                                      DSPATR(UL)
A          R SFD530E                   SFL
A            ODOBSZ    R        O  2 14REFFLD(QLIDOBJD/ODOBSZ *LIBL/FILEOB-
```

```
A                                       J)
A                                       COLOR(PNK)
A                                       EDTCDE(Z)
A            ODOBNM    R        O  2  1REFFLD(QLIDOBJD/ODOBNM *LIBL/FILEOB-
A                                       J)
A                                       COLOR(PNK)
A            ODCRTU    R        O  2 25REFFLD(QLIDOBJD/ODCRTU *LIBL/FILEOB-
A                                       J)
A                                       COLOR(PNK)
A          R SFD530F                    SFLCTL(SFD530E)
A                                       OVERLAY
A  32                                   SFLDSP
A                                       SFLDSPCTL
A  35                                   SFLEND(*MORE)
A                                       SFLSIZ(0020)
A                                       SFLPAG(0002)
A                                       WINDOW(15 2 5 35)
A                                       WDWBORDER((*CHAR '        '))
A            @REC3          4S OH       SFLRCDNBR
A                                  1  1'Disply Files'
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A                                  1 14'Size     '
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A                                  1 25'User      '
A                                       COLOR(YLW)
A                                       DSPATR(UL)


F*==============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR530) SRCFILE(XXX/QRPGSRC)
F*
F*==============================================================
FFILEOBJ IF  E           K        DISK
FSFD530  CF  E                    WORKSTN      KINFDS INFDS
F                                        @RRN1 KSFILE SFD530A
F                                        @RRN2 KSFILE SFD530C
F                                        @RRN3 KSFILE SFD530E
IINFDS       DS
I                                    B 378 3790@SFL
C*
C* Write the windows, activating the correct window for rolling
C*
C           *IN41     IFEQ '1'
C                     WRITEHEADER
C                     WRITESFD530D
C                     WRITESFD530F
C                     EXFMTSFD530B
C                     END
C*
C           *IN42     IFEQ '1'
C                     WRITEHEADER
C                     WRITESFD530B
```

```
C                     WRITESFD530F
C                     EXFMTSFD530D
C                     END
C*
C           *IN43     IFEQ '1'
C                     WRITEHEADER
C                     WRITESFD530B
C                     WRITESFD530D
C                     EXFMTSFD530F
C                     END
C*
C                     EXSR @CMD
C*
C*************************************************************
C*    @ C M D   S U B R O U T I N E                          *
C*                                                           *
C*    Executed after every EXFMT or display read op code.    *
C*    Used to process command keys.                          *
C*************************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, seton LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C* If F10 was pressed, then toggle the active window
C*
C           *IN10     IFEQ *ON
C           *IN10     DOWEQ*ON
C                     MOVE *OFF      *IN10
C*
C           *IN41     IFEQ *ON
C                     Z-ADD@SFL      @REC1
C                     MOVE *OFF      *IN41
C                     MOVE *ON       *IN42
C                     LEAVE
C                     END
C*
C           *IN42     IFEQ *ON
C                     Z-ADD@SFL      @REC2
C                     MOVE *OFF      *IN42
C                     MOVE *ON       *IN43
C                     LEAVE
C                     END
C*
C           *IN43     IFEQ *ON
C                     Z-ADD@SFL      @REC3
C                     MOVE *OFF      *IN43
C                     MOVE *ON       *IN41
C                     LEAVE
C                     END
C*
C                     END
```

```
C*
C                     ELSE
C*
C           *IN41     IFEQ *ON
C                     Z-ADD@SFL      @REC1
C                     END
C*
C           *IN42     IFEQ *ON
C                     Z-ADD@SFL      @REC2
C                     END
C*
C           *IN43     IFEQ *ON
C                     Z-ADD@SFL      @REC3
C                     END
C*
C                     END
C*
C                     ENDSR
C*
C*********************************************************
C*    * I N Z S R   S U B R O U T I N E                  *
C*                                                       *
C*    Retrieve the starting line and position for the    *
C*    window. Check to make sure that these values are   *
C*    valid. If not, put the closest valid number in.    *
C*********************************************************
C*
C           *INZSR    BEGSR
C*
C                     Z-ADD0         @RRN1   50
C                     Z-ADD0         @RRN2   50
C                     Z-ADD0         @RRN3   50
C                     MOVE *ON       *IN41
C*
C*  Load the subfiles
C*
C                     EXSR @LOAD
C*
C*  Set each subfile to initially display the 1st page
C*
C                     Z-ADD1         @REC1
C                     Z-ADD1         @REC2
C                     Z-ADD1         @REC3
C                     Z-ADD1         @SFL
C*
C                     ENDSR
C*
C*********************************************************
C*    @ L O A D     S U B R O U T I N E                  *
C*                                                       *
C*    This subroutine will read the file and program     *
C*    objects from file FILEOBJ. If the object is a      *
C*    program, it will be loaded into SFD530A. If it is  *
C*    a file, load it into SFD530C.                      *
C*********************************************************
C*
C           @LOAD     BEGSR
```

```
C*
C*  Perform this loop until you reach the end of the end
C*  of FILEOBJ file.
C*
C           *IN99     DOWNE*ON
C                     READ FILEOBJ                  99
C*
C*  If indicator 99 is on, or if you have already loaded
C*  9999 subfile records, (the maximum allowed), then exit
C*  this loop.
C*
C           *IN99     IFEQ *ON
C           @RRN1     OREQ 9999
C           @RRN2     OREQ 9999
C           @RRN3     OREQ 9999
C                     LEAVE
C                     END
C*
C*  If its a program, load it into SFD53OA.
C*
C           ODOBTP    IFEQ '*PGM'
C                     ADD  1         @RRN1
C                     WRITESFD53OA
C                     END
C*
C*  If its a file, load it into SFD53OC.
C*
C           ODOBTP    IFEQ '*FILE'
C           ODOBAT    ANDNE'DSPF'
C                     ADD  1         @RRN2
C                     WRITESFD53OC
C                     END
C*
C*
C*  If its a source file, load it into SFD53OE.
C*
C           ODOBTP    IFEQ '*FILE'
C           ODOBAT    ANDEQ'DSPF'
C                     ADD  1         @RRN3
C                     WRITESFD53OE
C                     END
C                     END
C*
C*   Since all three subfiles are load all, it will be ok
C*   to turn on the subfile end message.
C*
C                     MOVE *ON       *IN35
C*
C*   If you loaded records into SFD53OA, it will be ok
C*   to display it.
C*
C           @RRN1     IFGT O
C                     MOVE *ON       *IN3O
C                     END
C*
C*   If you loaded records into SFD53OC, it will be ok
C*   to display it.
```

```
C*
C           @RRN2     IFGT 0
C                     MOVE *ON       *IN31
C                     END
C*
C*   If you loaded records into SFD530E, it will be ok
C*   to display it.
C*
C           @RRN3     IFGT 0
C                     MOVE *ON       *IN32
C                     END
C*
C                     ENDSR
```

Making Windowed Subfiles "Stick"

To prevent the first subfile from repositioning when you switch to the second subfile, you can use a keyword with which you are already familiar—SFLRCDNBR. Remember, the SRFLRCDNBR keyword allows you to specify which page the subfile should initially display.

You used this keyword when you were designing an expanding subfile. Whenever you loaded the subfile with more records, you saved the first loaded subfile record number in the SFLRCDNBR field, so that subfile page would be displayed.

But remember, these are load-all subfiles. When you roll, the subfile will handle the scrolling of records back and forth, and control is not returned to the program unless a function key is pressed. How can you save the current page of the subfile that is being displayed?

The answer lies in the file information data structure for the subfile. When the user presses F10 to toggle the subfiles, control is passed back to the program. The program can access the file information data structure and find the lowest subfile relative record number on the active display, and save it in field associated with the SFLRCDNBR keyword.

First, you must define each subfile to have its own SFLRCDNBR keyword. The first subfile's field that contains the subfile record number keyword is @REC1, the second subfile's field is @REC2, and the third is @REC3. Therefore, each subfile has its own subfile record number assigned to it.

Now you must program the RPG code to extract the lowest subfile relative record number on the active display. You can do this in the RPG program in the following way:

```
FSFD530  CF  E                    WORKSTN      KINFDS INFDS

IINFDS       DS
I                                    B 378 3790@SFL
```

This code defines the file information data structure for the display file. In the file information data structure at positions 378 through 379, binary, is the lowest subfile relative record number that exists on an active display format. If a user rolls through a subfile a few times, then presses F10 to switch to another subfile, your program can first retrieve the lowest subfile relative record number for the current subfile. It can then move it into the SFLRCDNBR field for that subfile. The program does this in the screen toggle routines.

```
C           *IN10     IFEQ *ON
C           *IN10     DOWEQ*ON
C                     MOVE *OFF      *IN10
C*
C           *IN41     IFEQ *ON
C                     Z-ADD@SFL      @REC1
C                     MOVE *OFF      *IN41
C                     MOVE *ON       *IN42
C                     LEAVE
C                     END
C*
C           *IN42     IFEQ *ON
C                     Z-ADD@SFL      @REC2
C                     MOVE *OFF      *IN42
C                     MOVE *ON       *IN43
C                     LEAVE
C                     END
C*
C           *IN43     IFEQ *ON
C                     Z-ADD@SFL      @REC3
C                     MOVE *OFF      *IN43
C                     MOVE *ON       *IN41
C                     LEAVE
C                     END
C*
C                     END
C*
C                     ELSE
C*
C           *IN41     IFEQ *ON
C                     Z-ADD@SFL      @REC1
C                     END
C*
C           *IN42     IFEQ *ON
C                     Z-ADD@SFL      @REC2
C                     END
C*
C           *IN43     IFEQ *ON
C                     Z-ADD@SFL      @REC3
C                     END
C*
C                     END
```

Notice that when indicator 41 is on and F10 is pressed, the program moves the current lowest subfile relative record number field @SFL into @REC1, which is the SFLRCDNBR for the first subfile. It moves @SFL into @REC1 because if the program is executing this routine, then the subfile that was last active would have been the first subfile. This procedure also works if the user presses the Enter key. In this case, indicator 10 is not on, but 41 is still on. That is why the ELSE statement routines exist—to handle situations when the Enter key is pressed after the subfile has been rolled. If the program had not been coded this way, and the user pressed the Enter key after rolling a subfile, the subfile would redisplay the first page.

To see this effect call SFR530. Your display should look like this:

```
 xx/xx/xx                    Object Inquiry                        QPGMR
 xx:xx:xx                                                          SFR530

** Programs **                                        ** Files **

  Program    Size       User        Data Files Size       User
  SFC900A          8192 MICHAEL     BIGF          1187840 QSECOFR
  SFC900B          8192 MICHAEL     CONTACT         15872 QSECOFR
  SFC900C          8192 MICHAEL     CUSTL1          10752 QSECOFR
  SFC900D          8192 MICHAEL     CUSTOMER        30208 QSECOFR
                            More...                                More...


** Source Files **

  Disply Files Size       User
  SFD001             3584 MICHAEL
  SFD002             3072 MICHAEL
                              More...



F3-Exit   F10-Change Active Window
```

Now roll forward. The display should look like this:

```
 xx/xx/xx                    Object Inquiry                        QPGMR
 xx:xx:xx                                                          SFR530

** Programs **                                          ** Files **

  Program    Size       User        Data Files Size       User
  SFC901A          8192 MICHAEL     BIGF          1187840 QSECOFR
  SFC901B          8192 MICHAEL     CONTACT         15872 QSECOFR
  SFC901C          8192 MICHAEL     CUSTL1          10752 QSECOFR
  SFC901D          8192 MICHAEL     CUSTOMER        30208 QSECOFR
                            More...                                More...


** Source Files **

  Disply Files Size       User
  SFD001             3584 MICHAEL
  SFD002             3072 MICHAEL
                              More...



F3-Exit   F10-Change Active Window
```

Now press the F10 key. Your display should look the same except that the cursor is now positioned on the second subfile.

Notice how the first subfile “stuck” in its position when you changed active subfiles. You can also see this effect by rolling a few times on the second subfile and pressing the Enter key. The subfiles “stick” wherever you last left them, which is probably how your users will want the subfiles to perform.

As you can see, multiple subfiles don’t have to be difficult and can make a user far more productive by giving him a wealth of information he can easily access on a small portion of the display screen.

6

Input and Update Subfiles

Input Subfiles

Subfiles aren't just for outputting data. You can also use them to input or update data. Input subfiles are great for fast data entry operators.

If you go back to the S/36, S/38, and very early AS/400s that did not have keyboard buffering, you'll remember that these systems were a nightmare for fast keyboard operators. If they were keying in on an order entry detail screen, they would have to enter just one item and quantity, then wait until the system displayed the next line. Attempting to go faster than the system caused the user to have to press the error reset key.

The S/38 allowed input subfiles, which was a big improvement and allowed the operators to keypunch somewhat faster. However, they still needed to constantly look at the screen to make sure that they didn't outperform the system and end up having to rekey a lot of information.

The advent of the keyboard buffering in OS/400 V1R2 allowed users to outkey the system. Keyboard buffering improved productivity, but when you couple input-capable subfiles with keyboard buffering, your users have the greatest potential for reaching maximum keying speeds.

For instance, suppose a user is keying in the line items for an order. You could design a screen that allowed the user to key in 20 line items and quantities. If the user needs more than 20 lines, your program can display another 20 lines in which to key. So far, the system has not checked to validate that the items and quantities are valid. The user can press a function or Enter key when he has finished keying the order, and have the system validate the entire order at one time. Your program can display control totals on the bottom of the screen, such as the number of line items keyed in. It can display on the bottom of the display whether or not there were any errors encountered. The user can check the order one time at the end of keying it, which is usually more natural to do than after keying in each item.

Now, take look at an example of an input subfile. This subfile allows you to key in a state abbreviation code and the state description:

```
A*============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD600) SRCFILE(XXX/QDDSSRC)
A*
A*============================================================
A                                      DSPSIZ(24 80 *DS3)
A                                      CF03(03)
A          R SFD600A                   SFL
A            STATCD    R        B  8  3REFFLD(RSTATES/STATCD *LIBL/STATES)
A                                      COLOR(WHT)
A            STATDS    R        B  8 13REFFLD(RSTATES/STATDS *LIBL/STATES)
A                                      COLOR(WHT)
A          R SFD600B                   SFLCTL(SFD600A)
A                                      OVERLAY
A  30                                  SFLDSP
A                                      SFLDSPCTL
A                                      SFLSIZ(0100)
A                                      SFLPAG(0010)
A                                  1  4DATE
A                                      EDTCDE(Y)
A                                      COLOR(BLU)
A                                  2  4TIME
A                                      COLOR(BLU)
A                                  1 26'Input State Codes'
A                                      COLOR(WHT)
A                                  1 69USER
A                                      COLOR(BLU)
A                                  2 69'SFR600'
A                                      COLOR(BLU)
A                                  6  1'State Code'
A                                      COLOR(PNK)
A                                      DSPATR(UL)
A                                  6 13'Description    '
A                                      COLOR(PNK)
A                                      DSPATR(UL)
A          R TRAILER
A                                 23  2'F3-Exit'
A                                      COLOR(WHT)
```

```
A          R READS
A                                    CLRL(*NO)
A                                 13 44'Number Of Reads'
A                                 15 44'Number Of Records Entered'
A            @READS         5Y 00 13 73COLOR(PNK)
A                                    EDTCDE(Z)
A            @RCDS          5Y 00 15 73COLOR(PNK)
A                                    EDTCDE(Z)


F*=============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR600) SRCFILE(XXX/QRPGSRC)
F*
F*=============================================================
FSFD600  CF  E                    WORKSTN
F                                       @RRN  KSFILE SFD600A
C*
C                     WRITETRAILER
C                     EXFMTSFD600B
C                     EXSR @CMD
C*
C                     EXSR @READ
C*
C*********************************************************
C*    @ C M D   S U B R O U T I N E                      *
C*                                                       *
C*    Executed after every EXFMT or display read op code.  *
C*    Used to process command keys.                      *
C*********************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, set on LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C                     ENDSR
C*
C*********************************************************
C*    @ R E A D   S U B R O U T I N E                    *
C*                                                       *
C*    Read the records inputted from the subfile         *
C*********************************************************
C*
C           @READ     BEGSR
C*
C* Do until you reach the end of the subfile (100 records)
C*
C           *IN99     DOWEQ*OFF
C                     ADD  1         @RRN
C           @RRN      CHAINSFD600A             99
```

```
C           *IN99     IFEQ *ON
C                     LEAVE
C                     END
C*
C                     ADD  1         @READS
C*
C           STATCD    IFNE *BLANKS
C           STATDS    ANDNE*BLANKS
C                     ADD  1         @RCDS
C                     END
C*
C                     END
C*
C                     EXFMTREADS
C                     EXSR @CMD
C                     ENDSR
C*
C*************************************************************
C*    * I N Z S R   S U B R O U T I N E                      *
C*                                                           *
C*    Retrieve the starting line and position for the        *
C*    window. Check to make sure that these values are       *
C*    valid. If not, put the closest valid number in.        *
C*************************************************************
C*
C           *INZSR    BEGSR
C*
C                     Z-ADD0         @RRN    50
C                     Z-ADD0         @READS
C                     Z-ADD0         @RCDS
C*
C                     ENDSR
```

Call program SFR600. Your display should look like this:

```
   xx/xx/xx              Input State Codes                        QPGMR
   xx:xx:xx                                                       SFR600


State Code  Description    ___



 F3-Exit
```

All you got was an output screen. There are no lines on which to input data. The program forgot to turn on the indicator 30 to display the subfile. Make this change and recompile the program. The change has already been made in SFR601.

```
F*==============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR601) SRCFILE(XXX/QRPGSRC)
F*
F*==============================================================
FSFD600  CF  E                    WORKSTN
F                                        @RRN  KSFILE SFD600A
C*
C                     WRITETRAILER
C                     MOVE *ON       *IN30
C                     EXFMTSFD600B
C                     EXSR @CMD
C*
C                     EXSR @READ
C*
C*********************************************************
C*    @ C M D   S U B R O U T I N E                      *
C*                                                       *
C*    Executed after every EXFMT or display read op code.  *
C*    Used to process command keys.                      *
C*********************************************************
C*
C           @CMD      BEGSR
```

```
C*
C* If F3 was pressed, set on LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C                     ENDSR
C*
C*********************************************************
C*    @ R E A D   S U B R O U T I N E                    *
C*                                                       *
C*    Read the records inputted from the subfile         *
C*********************************************************
C*
C           @READ     BEGSR
C*
C* Do until you reach the end of the subfile (100 records)
C*
C           *IN99     DOWEQ*OFF
C                     ADD  1         @RRN
C           @RRN      CHAINSFD600A              99
C           *IN99     IFEQ *ON
C                     LEAVE
C                     END
C*
C                     ADD  1         @READS
C*
C           STATCD    IFNE *BLANKS
C           STATDS    ANDNE*BLANKS
C                     ADD  1         @RCDS
C                     END
C*
C                     END
C*
C                     EXFMTREADS
C                     EXSR @CMD
C                     ENDSR
C*
C*********************************************************
C*    * I N Z S R   S U B R O U T I N E                  *
C*                                                       *
C*    Retrieve the starting line and position for the    *
C*    window. Check to make sure that these values are   *
C*    valid. If not, put the closest valid number in.    *
C*********************************************************
C*
C           *INZSR    BEGSR
C*
C                     Z-ADD0         @RRN    50
C                     Z-ADD0         @READS
C                     Z-ADD0         @RCDS
C*
C                     ENDSR
```

Call SFR601. Your screen should look like this:

```
                          Display Program Messages

 Session or device error occurred (C G S D F).


 Type reply, press Enter.
   Reply . . .   ____________________________________________________________
_____________________________________________________________________________

 F3=Exit   F12=Cancel
```

What happened? You get this error message when your program attempts to display an empty subfile. This error is why the previous programs checked the relative record number to ensure it was greater than 0, and if it was, set on indicator 31.

The Subfile Initialize Keyword

But this program is not outputting records. How can you tell the system to display the subfile without getting an error, since there won't be any records until the user keys some in? Well, the system can never display an empty subfile. So the program must write some blank records out to the subfile, so the system will display the subfile. You probably could code your program to do a series of writes to the subfile format, but don't! You can use the Subfile Initialize (SFLINZ) keyword instead.

The SFLINZ keyword allows you to initialize the subfile with as many active records as the value specified in the SFLSIZ keyword. If you specify SFLINZ and your subfile size is set to 100, your subfile would contain 100 active, but blank, subfile records.

SFLINZ is the cousin of SFLCLR. They are both activated the same way. You set on the controlling indicator of the keyword, write the subfile control format, then set the indicator off. For example, if the DDS for the SFLINZ looks like this:

```
A  31                                   SFLINZ
```

Then you can initialize the subfile by issuing the following RPG statements.

```
C                     MOVE *ON       *IN31
C                     WRITESFD620B
C                     MOVE *OFF      *IN31
```

The difference between SFLINZ and SFLCLR is that SFLCLR removes all records from the subfile. The subfile will not contain any records after a clear, but will be empty. The SFLINZ will clear the subfile, but then initialize the number of records that is contained in the value of the SFLSIZ keyword.

Now add the SFLINZ keyword to the program so that you can display the subfile. Program SFR610 has the changes already made.

```
A*============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD610) SRCFILE(XXX/QDDSSRC)
A*
A*============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03)
A          R SFD610A                    SFL
A            STATCD    R        B  8  3REFFLD(RSTATES/STATCD *LIBL/STATES)
A                                       COLOR(WHT)
A            STATDS    R        B  8 13REFFLD(RSTATES/STATDS *LIBL/STATES)
A                                       COLOR(WHT)
A          R SFD610B                    SFLCTL(SFD610A)
A                                       OVERLAY
A  30                                   SFLDSP
A                                       SFLDSPCTL
A  31                                   SFLINZ
A                                       SFLSIZ(0100)
A                                       SFLPAG(0010)
A                                   1  4DATE
A                                       EDTCDE(Y)
A                                       COLOR(BLU)
A                                   2  4TIME
A                                       COLOR(BLU)
A                                   1 26'Input State Codes'
A                                       COLOR(WHT)
A                                   1 69USER
A                                       COLOR(BLU)
A                                   2 69'SFR610'
A                                       COLOR(BLU)
A                                   6  1'State Code'
A                                       COLOR(PNK)
A                                       DSPATR(UL)
A                                   6 13'Description    '
A                                       COLOR(PNK)
A                                       DSPATR(UL)
A          R TRAILER
A                                  23  2'F3-Exit'
```

```
A                                       COLOR(WHT)
A                                  21 44'Number Of Reads'
A                                  22 44'Number Of Records Entered'
A            @READS         5Y 00 21 73COLOR(PNK)
A                                       EDTCDE(Z)
A            @RCDS          5Y 00 22 73COLOR(PNK)
A                                       EDTCDE(Z)


F*==============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR610) SRCFILE(XXX/QRPGSRC)
F*
F*==============================================================
FSFD610  CF  E                     WORKSTN
F                                         @RRN  KSFILE SFD610A
C*
C                     WRITETRAILER
C*
C* Initialize the subfile
C*
C                     MOVE *ON       *IN31
C                     WRITESFD610B
C                     MOVE *OFF      *IN31
C                     MOVE *ON       *IN30
C*
C                     EXFMTSFD610B
C                     EXSR @CMD
C*
C                     EXSR @READ
C*
C*************************************************************
C*    @ C M D   S U B R O U T I N E                          *
C*                                                           *
C*    Executed after every EXFMT or display read op code.    *
C*    Used to process command keys.                          *
C*************************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, set on LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C                     ENDSR
C*
C*************************************************************
C*    @ R E A D   S U B R O U T I N E                        *
C*                                                           *
C*    Read the records inputted from the subfile             *
C*************************************************************
C*
```

```
C           @READ     BEGSR
C                     Z-ADD0         @READS
C                     Z-ADD0         @RCDS
C                     Z-ADD0         @RRN
C                     MOVE *OFF      *IN99
C*
C* Do until you reach the end of the subfile (100 records)
C*
C           *IN99     DOWEQ*OFF
C                     ADD  1         @RRN
C           @RRN      CHAINSFD610A              99
C           *IN99     IFEQ *ON
C                     LEAVE
C                     END
C*
C                     ADD  1         @READS
C*
C           STATCD    IFNE *BLANKS
C           STATDS    ANDNE*BLANKS
C                     ADD  1         @RCDS
C                     END
C*
C                     END
C*
C                     ENDSR
C*
C*************************************************************
C*    * I N Z S R   S U B R O U T I N E                      *
C*                                                           *
C*    Retrieve the starting line and position for the        *
C*    window. Check to make sure that these values are       *
C*    valid. If not, put the closest valid number in.        *
C*************************************************************
C*
C           *INZSR    BEGSR
C*
C                     Z-ADD0         @RRN    50
C*
C                     ENDSR
```

Call SFR610. Your display should look like this:

```
  xx/xx/xx              Input State Codes                        QPGMR
  xx:xx:xx                                                       SFR610


State Code  Description

  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________


                                        Number Of Reads
                                        Number Of Records Entered
 F3-Exit
```

No error this time! You have one page of input records. Since the SFLPAG value was set to ten, you have ten records to a page. Since the SFLSIZ value was set to 100, you can roll forward through 100 records. After that, you reach the end of the subfile.

This program is designed to show how to use certain subfile techniques to greatly increase the performance of your subfile. Key in the following data on the screen:

```
  xx/xx/xx              Input State Codes                        QPGMR
  xx:xx:xx                                                       SFR610


State Code  Description

  AK        ALASKA
  AL        ALABAMA
  AR        ARKANSAS
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________


                                        Number Of Reads
                                        Number Of Records Entered
 F3-Exit
```

Now press the Enter key. Your screen should look like this:

```
  xx/xx/xx              Input State Codes                        QPGMR
  xx:xx:xx                                                       SFR610


State Code  Description

  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________


                                        Number Of Reads                100
                                        Number Of Records Entered        3
 F3-Exit
```

Notice the counters in the bottom right hand corner of the screen. It says you entered three records, but processed 100. That is correct, you did enter three records, but the

program had to process all 100 to get those three records using the current design. In a minute, you will see how to make this change

You can also turn on the subfile end indicator for input subfiles. This change has been made in SFR620.

```
A*============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD620) SRCFILE(XXX/QDDSSRC)
A*
A*============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03)
A          R SFD620A                    SFL
A            STATCD    R        B  8  3REFFLD(RSTATES/STATCD *LIBL/STATES)
A                                       COLOR(WHT)
A            STATDS    R        B  8 13REFFLD(RSTATES/STATDS *LIBL/STATES)
A                                       COLOR(WHT)
A          R SFD620B                    SFLCTL(SFD620A)
A                                       OVERLAY
A  30                                   SFLDSP
A                                       SFLDSPCTL
A  31                                   SFLINZ
A  35                                   SFLEND(*MORE)
A                                       SFLSIZ(0100)
A                                       SFLPAG(0010)
A                                  1  4DATE
A                                       EDTCDE(Y)
A                                       COLOR(BLU)
A                                  2  4TIME
A                                       COLOR(BLU)
A                                  1 26'Input State Codes'
A                                       COLOR(WHT)
A                                  1 69USER
A                                       COLOR(BLU)
A                                  2 69'SFR620'
A                                       COLOR(BLU)
A                                  6  1'State Code'
A                                       COLOR(PNK)
A                                       DSPATR(UL)
A                                  6 13'Description    '
A                                       COLOR(PNK)
A                                       DSPATR(UL)
A          R TRAILER
A                                 23  2'F3-Exit'
A                                       COLOR(WHT)
A                                 21 44'Number Of Reads'
A                                 22 44'Number Of Records Entered'
A            @READS         5Y 00 21 73COLOR(PNK)
A                                       EDTCDE(Z)
A            @RCDS          5Y 00 22 73COLOR(PNK)
A                                       EDTCDE(Z)
```

```
F*==============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR620) SRCFILE(XXX/QRPGSRC)
F*
F*==============================================================
FSFD620  CF  E                    WORKSTN
F                                        @RRN  KSFILE SFD620A
C*
C*  Write the command key line
C*
C                     WRITETRAILER
C*
C*   Initialize the subfile with 100 active records
C*
C                     MOVE *ON       *IN31
C                     WRITESFD620B
C                     MOVE *OFF      *IN31
C*
C*   Since the subfile has been initialized, it will be ok
C*   to display it.
C*
C                     MOVE *ON       *IN30
C                     EXFMTSFD620B
C                     EXSR @CMD
C*
C*   Read through the subfile and see how many records
C*   were entered versus how many read operations it took.
C*
C                     EXSR @READ
C*
C*************************************************************
C*    @ C M D   S U B R O U T I N E                          *
C*                                                           *
C*    Executed after every EXFMT or display read op code.    *
C*    Used to process command keys.                          *
C*************************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, set on LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C                     ENDSR
C*
C*************************************************************
C*    @ R E A D   S U B R O U T I N E                        *
C*                                                           *
C*    Read the records inputted from the subfile             *
C*************************************************************
C*
C           @READ     BEGSR
C*
```

```
C*   Set the # of reads, # of records, and relative record
C*   number fields to 0. Ensure *IN99 is off.
C*
C                     Z-ADD0         @READS
C                     Z-ADD0         @RCDS
C                     Z-ADD0         @RRN
C                     MOVE *OFF      *IN99
C*
C* Do until you reach the end of the subfile (100 records)
C*
C           *IN99     DOWEQ*OFF
C                     ADD  1         @RRN
C           @RRN      CHAINSFD620A              99
C           *IN99     IFEQ *ON
C                     LEAVE
C                     END
C*
C*  If you read a record, increment the read counter.
C*
C                     ADD  1         @READS
C*
C*  If something was entered into either the state code
C*  or description field, increment the records counter.
C*
C           STATCD    IFNE *BLANKS
C           STATDS    ORNE *BLANKS
C                     ADD  1         @RCDS
C                     END
C*
C                     END
C*
C                     ENDSR
C*
C*************************************************************
C*    * I N Z S R   S U B R O U T I N E                      *
C*                                                           *
C*    Retrieve the starting line and position for the        *
C*    window. Check to make sure that these values are       *
C*    valid. If not, put the closest valid number in.        *
C*************************************************************
C*
C           *INZSR    BEGSR
C*
C*  Initialize the relative record number field for the subfile
C*  to 0. Turn on 35, so that the subfile will show the
C*  subfile end message.
C*
C                     Z-ADD0         @RRN    50
C                     MOVE *ON       *IN35
C*
C                     ENDSR
```

Call SFR620. Your display should look like this:

```
   xx/xx/xx              Input State Codes                          QPGMR
   xx:xx:xx                                                         SFR620


State Code  Description

  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
                                                                      More...

                                          Number Of Reads
                                          Number Of Records Entered
 F3-Exit
```

Now the screen has the “MORE...” and “Bottom” messages in the bottom right hand corner of the subfile. Go ahead and enter three states again like you did in SFR610 and press Enter.

```
  xx/xx/xx              Input State Codes                        QPGMR
  xx:xx:xx                                                       SFR620


State Code  Description

  AK        ALASKA
  AL        ALABAMA
  AR        ARKANSAS
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
                                                                 More...

                                        Number Of Reads               100
                                        Number Of Records Entered       3
 F3-Exit
```

Notice the counter fields in the bottom right hand corner of the screen. They say that you entered three records. This number is correct. The counter fields also say that the program read 100 records, which is not good. The program had to read every record to process only three. It spent a lot of wasted processor time to process all 100 records. Why? Because these records were initialized as being active. If the subfile record is active, it can be read. You might be asking, “what other kind of subfile record is there?”

There are two types of subfile records, active and inactive. An inactive record cannot be read by the system, but does allow the subfile to be displayed because the subfile is not empty.

Active and Inactive Subfile Records

A subfile record can be made active in three ways.

1. The record is written to the subfile with the write operation. When the programs wrote records to the load-all, page-at-a-time, and expanding subfiles, the records they loaded were made active because they wrote to them.

2. An inactive record can be made active if the user changes the record.

3. Specifying the SFLINZ without the SFLRNA keyword will initialize subfile records as active.

There is one way to make a subfile record inactive.

1. Specify the SFLINZ keyword with the SFLRNA keyword.

The Subfile Record Not Active Keyword

The SFLRNA keyword stands for Subfile Record Not Active. SFLRNA is used in conjunction with the SFLINZ keyword. When the two are specified together, the subfile is initialized with inactive records.

You want a subfile full of inactive records because it allows you to cut down on the amount of time used to process the subfile. The first example of the SFLRNA keyword is program SFR630.

```
A*============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD630) SRCFILE(XXX/QDDSSRC)
A*
A*============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03)
A          R SFD630A                    SFL
A            STATCD    R         B  8  3REFFLD(RSTATES/STATCD *LIBL/STATES)
A                                       COLOR(WHT)
A            STATDS    R         B  8 13REFFLD(RSTATES/STATDS *LIBL/STATES)
A                                       COLOR(WHT)
A          R SFD630B                    SFLCTL(SFD630A)
A                                       OVERLAY
A  30                                   SFLDSP
A                                       SFLDSPCTL
A  31                                   SFLINZ
A                                       SFLRNA
A  35                                   SFLEND(*MORE)
A                                       SFLSIZ(0100)
A                                       SFLPAG(0010)
A                                   1  4DATE
A                                       EDTCDE(Y)
A                                       COLOR(BLU)
A                                   2  4TIME
A                                       COLOR(BLU)
A                                   1 26'Input State Codes'
A                                       COLOR(WHT)
A                                   1 69USER
A                                       COLOR(BLU)
A                                   2 69'SFR630'
A                                       COLOR(BLU)
A                                   6  1'State Code'
```

```
A                                       COLOR(PNK)
A                                       DSPATR(UL)
A                                  6 13'Description    '
A                                       COLOR(PNK)
A                                       DSPATR(UL)
A          R TRAILER
A                                 23  2'F3-Exit'
A                                       COLOR(WHT)
A                                 21 44'Number Of Reads'
A                                 22 44'Number Of Records Entered'
A            @READS         5Y 00 21 73COLOR(PNK)
A                                       EDTCDE(Z)
A            @RCDS          5Y 00 22 73COLOR(PNK)
A                                       EDTCDE(Z)


F*============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR630) SRCFILE(XXX/QRPGSRC)
F*
F*============================================================
FSFD630  CF  E                    WORKSTN
F                                       @RRN  KSFILE SFD630A
C*
C*  Write the command key line
C*
C                     WRITETRAILER
C*
C*   Initialize the subfile with 100 active records
C*
C                     MOVE *ON       *IN31
C                     WRITESFD630B
C                     MOVE *OFF      *IN31
C*
C*   Since the subfile has been initialized, it will be ok
C*   to display it.
C*
C                     MOVE *ON       *IN30
C                     EXFMTSFD630B
C                     EXSR @CMD
C*
C*   Read through the subfile and see how many records
C*   were entered versus how many read operations it took.
C*
C                     EXSR @READ
C*
C*********************************************************
C*    @ C M D   S U B R O U T I N E                      *
C*                                                       *
C*    Executed after every EXFMT or display read op code.  *
C*    Used to process command keys.                      *
C*********************************************************
C*
C           @CMD      BEGSR
C*
```

```
C* If F3 was pressed, set on LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C                     ENDSR
C*
C*********************************************************
C*    @ R E A D   S U B R O U T I N E                    *
C*                                                       *
C*    Read the records inputted from the subfile         *
C*********************************************************
C*
C           @READ     BEGSR
C*
C*   Set the # of reads, # of records, and relative record
C*   number fields to 0. Ensure *IN99 is off.
C*
C                     Z-ADD0         @READS
C                     Z-ADD0         @RCDS
C                     Z-ADD0         @RRN
C                     MOVE *OFF      *IN99
C*
C* Do until you reach the end of the subfile (100 records)
C*
C           *IN99     DOWEQ*OFF
C                     ADD  1         @RRN
C           @RRN      CHAINSFD630A              99
C           *IN99     IFEQ *ON
C                     LEAVE
C                     END
C*
C*  If you read a record, increment the read counter.
C*
C                     ADD  1         @READS
C*
C*  If something was entered into either the state code
C*  or description field, increment the records counter.
C*
C           STATCD    IFNE *BLANKS
C           STATDS    ORNE *BLANKS
C                     ADD  1         @RCDS
C                     END
C*
C                     END
C*
C                     ENDSR
C*
C*********************************************************
C*    * I N Z S R   S U B R O U T I N E                  *
C*                                                       *
C*    Retrieve the starting line and position for the    *
C*    window. Check to make sure that these values are   *
C*    valid. If not, put the closest valid number in.    *
C*********************************************************
```

```
C*
C           *INZSR    BEGSR
C*
C*  Initialize the relative record number field for the subfile
C*  to 0. Turn on 35, so that the subfile will show the
C*  subfile end message.
C*
C                     Z-ADD0         @RRN    50
C                     MOVE *ON       *IN35
C*
C                     ENDSR
```

The following code will now behave differently with the SFLRNA keyword specified:

```
C           *IN99     DOWEQ*OFF
C                     ADD  1         @RRN
C           @RRN      CHAINSFD630A              99
C           *IN99     IFEQ *ON
C                     LEAVE
C                     END
```

What happens now in the DO loop is that as soon as the program reads the first inactive record, indicator 99 is turned on. An inactive record is something like a logical end of file. When the program chains to a subfile record, and the record is inactive, the program gets a miss on the CHAIN statement.

Now you can see how using the SFLRNA has affected the program. Call SFR630 and enter two states into the subfile. Your display should look like this:

```
  xx/xx/xx              Input State Codes                        QPGMR
  xx:xx:xx                                                       SFR630


State Code  Description

  AK        ALASKA
  AL        ALABAMA
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
                                                                 More...


                                        Number Of Reads
                                        Number Of Records Entered
 F3-Exit
```

Now press Enter to see the number of reads performed:

```
  xx/xx/xx              Input State Codes                        QPGMR
  xx:xx:xx                                                       SFR630


State Code  Description

  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
                                                                 More...

                                         Number Of Reads                 2
                                         Number Of Records Entered       2
 F3-Exit
```

You entered two records and the program read only two records. This change greatly improves the program's performance. It did not have to read all 100 records to process only two of them.

Now try something different in the same program. Enter a state on the first subfile record, press Field Exit through the next record, then enter another state on the third subfile record like this:

```
  xx/xx/xx              Input State Codes                        QPGMR
  xx:xx:xx                                                       SFR630


State Code  Description

  AK        ALASKA
  __        ______________
  AL        ALABAMA
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
                                                                 More...


                                        Number Of Reads
                                        Number Of Records Entered
 F3-Exit
```

Now press Enter and see what happens.

```
  xx/xx/xx              Input State Codes                        QPGMR
  xx:xx:xx                                                       SFR630


State Code  Description

  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
                                                                 More...


                                        Number Of Reads                  3
                                        Number Of Records Entered        2
 F3-Exit
```

Why did it say it read three records, when you only added two? Because, when you Field Exit through subfile record number two, the system assumes you changed the

record. If you change a record, the system turns that record into an active record. So when the program chained to the second subfile record, it retrieved a record because the record was active. Since the record did not contain a state code or description, it did not increment the number of added records field.

You really shouldn't use the CHAIN opcode to retrieve the input subfile records. It cannot handle active and inactive subfile records mixed in with each other. Call SFR630 again. On the first subfile record, enter a state. Now, tab down to the fourth subfile record and enter another state. Your display should look like this:

```
  xx/xx/xx              Input State Codes                        QPGMR
  xx:xx:xx                                                       SFR630


State Code  Description

  AK        ALASKA
  __        ______________
  __        ______________
  AL        ALABAMA
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
                                                                 More...


                                        Number Of Reads
                                        Number Of Records Entered
 F3-Exit
```

Now press the Enter key. Your display should look like this:

```
  xx/xx/xx              Input State Codes                        QPGMR
  xx:xx:xx                                                       SFR630


State Code  Description

  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
                                                                 More...


                                        Number Of Reads                1
                                        Number Of Records Entered      1
 F3-Exit
```

Why did the program only think you entered one record? And why did it only read one record when you entered two? The "bug" lies within the DO loop routine:

```
C            *IN99      DOWEQ*OFF
C                       ADD  1          @RRN
C            @RRN       CHAINSFD630A              99
C            *IN99      IFEQ *ON
C                       LEAVE
C                       END
```

The first subfile record the routine processed was number 1, which was an active record since you modified the subfile record contents. The next record the routine processed was number 2. But when it chained out to subfile record number 2, the CHAIN statement did not retrieve a record. Why not? Because record 2 was an inactive subfile record. It was inactive because you tabbed over it. By tabbing over it, you did not modify the contents of record 2. Therefore, the system did not flag it as an active record. It left it inactive. Since indicator 99 was on, the program left the DO loop. So the routine ended prematurely.

Also note that subfile record 3 would have been inactive because you tabbed over it also. Subfile record 4, however, would have been active because you modified the contents.

The Read Next Changed Record Opcode

So, how can your program read just the "active" records without ending the routine prematurely? It can use an opcode created just for subfiles. It is the READC (Read Next Change) opcode.

To get around this problem, you can modify the DO loop routine and use READC in place of the CHAIN opcode as shown in SFR640:

```
F*==============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR640) SRCFILE(XXX/QRPGSRC)
F*
F*==============================================================
FSFD630  CF  E                    WORKSTN
F                                        @RRN  KSFILE SFD630A
C*
C*  Write the command key line
C*
C                     WRITETRAILER
C*
C*   Initialize the subfile with 100 active records
C*
C                     MOVE *ON       *IN31
C                     WRITESFD630B
C                     MOVE *OFF      *IN31
C*
C*   Since the subfile has been initialized, it will be ok
C*   to display it.
C*
C                     MOVE *ON       *IN30
C                     EXFMTSFD630B
C                     EXSR @CMD
C*
C*   Read through the subfile and see how many records
C*   were entered versus how many read operations it took.
C*
C                     EXSR @READ
C*
C*********************************************************
C*    @ C M D   S U B R O U T I N E                      *
C*                                                       *
C*    Executed after every EXFMT or display read op code.  *
C*    Used to process command keys.                      *
C*********************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, set on LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
```

```
C*
C                     ENDSR
C*
C*************************************************************
C*    @ R E A D   S U B R O U T I N E                        *
C*                                                           *
C*    Read the records inputted from the subfile             *
C*************************************************************
C*
C           @READ     BEGSR
C*
C*   Set the # of reads, # of records, and relative record
C*   number fields to 0. Ensure *IN99 is off.
C*
C                     Z-ADD0         @READS
C                     Z-ADD0         @RCDS
C                     Z-ADD0         @RRN
C                     MOVE *OFF      *IN99
C*
C* Do until no more active records
C*
C           *IN99     DOWEQ*OFF
C                     READCSFD630A                  99
C           *IN99     IFEQ *ON
C                     LEAVE
C                     END
C*
C*  If you read a record, increment the read counter.
C*
C                     ADD  1         @READS
C*
C*  If something was entered into either the state code
C*  or description field, increment the records counter.
C*
C           STATCD    IFNE *BLANKS
C           STATDS    ORNE *BLANKS
C                     ADD  1         @RCDS
C                     END
C*
C                     END
C*
C                     ENDSR
C*
C*************************************************************
C*    * I N Z S R   S U B R O U T I N E                      *
C*                                                           *
C*    Retrieve the starting line and position for the        *
C*    window. Check to make sure that these values are       *
C*    valid. If not, put the closest valid number in.        *
C*************************************************************
C*
C           *INZSR    BEGSR
C*
C*  Initialize the relative record number field for the subfile
C*  to 0. Turn on 35, so that the subfile will show the
C*  subfile end message.
C*
```

```
C                       Z-ADD0          @RRN    50
C                       MOVE *ON        *IN35
C*
C                       ENDSR
```

The DO loop routine has been modified, so that it now looks like this:

```
C            *IN99      DOWEQ*OFF
C                       READCSFD630A                    99
C            *IN99      IFEQ *ON
C                       LEAVE
C                       END
```

This routine reads only the "active" records. When there are no more active records to process, indicator 99 is set on and the program exits the loop. To see if this routine really works, call SFR640. Enter a state on subfile record 1, TAB over subfile record 2, then enter a state into subfile record 3. Your display should look like this:

```
   xx/xx/xx              Input State Codes                        QPGMR
   xx:xx:xx                                                       SFR630


State Code  Description

  AK        ALASKA
  __        ______________
  AL        ALABAMA
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
                                                                 More...


                                         Number Of Reads
                                         Number Of Records Entered
 F3-Exit
```

Now press Enter:

```
  xx/xx/xx              Input State Codes                        QPGMR
  xx:xx:xx                                                       SFR630


State Code  Description

  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
  __        ______________
                                                                 More...

                                        Number Of Reads                2
                                        Number Of Records Entered      2
 F3-Exit
```

This time only two records were read and the routine did read every record you entered. Now, roll forward to the second input screen and enter the states just as you did before. Then press Enter and look at the results.

It doesn't matter where the active and inactive records are, the READC op code finds them. This feature allows you to read the file only if changes have been made. You don't have to read all 100 records, which would really make a difference if the subfile size were set to 1,000. This change would allow the user to enter 1,000 state codes. Then, instead of having to read 1,000 active subfile records, the program would read only in the number of subfile records that were made active by the user.

As you can tell, the user can enter only in as many records as specified by the subfile size keyword. If you set the subfile size to 100, the user can enter in 100 subfile records. If you set it to 2,000, they can enter in 2,000 subfile records. After they enter the maximum subfile records, the user must press Enter or a function key and have the current subfile records processed before he can enter any more records.

Update Subfiles

You can also use Subfiles to update many records at a time. However, of all the uses you can find for subfiles, updating information is a function that you need to question whether a subfile is the best solution.

If you update only a record at a time, then the user has a lock on the record and nobody else is allowed to update that record until the current user releases the lock.

However, subfiles cause a problem with this technique. Instead of one record, the user may be looking at 20 or more records if you are using a page-at-a-time subfile, or hundreds of records if you are using a load-all or expanding subfile.

Now you have a problem with data integrity. Your program must somehow lock all the records processed by the subfile when the subfile is first displayed. Or, it must lock the records after the user has maintained the records in the subfile and has requested that the system process them. Neither of these approaches is very appealing.

First, if you lock all the records when your program first displays the subfile, then one user now has a lock on many records in a file. Until the user releases all of the locks, no one else can update the records. You wouldn't want anyone else to update the records, but you also don't want one user to monopolize a file. Suppose you are working with an order entry file. A user could pull up an order to maintain all of the line items on a subfile update screen. While the user has this screen up, no other departments can pull up the order, or even line information, if the program specified to lock the records in the file. Again, this result is what you want, but the user at the subfile maintenance screen could potentially lock these people up for an extended amount of time. Even if the user is a fast keyer, you are working with multiple records.

The method other than attempting to lock all subfile records would be to not lock the records as they are originally displayed. As soon as the user is finished updating the subfile, you would lock each record that was modified, update it, then unlock it. This method has a problem in that once the records are displayed, another user could change the record. Once the subfile user pressed Enter, they would overlay the information.

For example, USER1 pulls up all line items for order 100. While USER1 is processing on the screen, USER2 pulls up a single line item for order 100, and changes the order quantity. When USER1 presses Enter to update order 100, it overlays the order quantity.

When designing an update subfile, you must take some things into consideration. First, will it hurt if I lock all the records processed by the subfile? In some cases, it may not. For instance, it may not hurt to lock all line items for an order because it would be rare that more than one person would ever be updating the same order

number's line items. In this case, a load-all subfile might be used, especially since most orders have a limited number of line items. If your orders did constantly go above 1,000 line items, an expanding subfile might be used as an alternative.

Suppose you are writing a subfile to update multiple order headers. A load-all subfile would be ruled out due to the number of possible orders that would be loaded. But an expanding subfile might also be ruled out. If you use an expanding subfile and lock all records before displaying the subfile, you could potentially lock a lot of users. It is not rare for more than one user to try to update two different orders. You could end up keeping the shipping department from being able to show a transaction against an order because another user has the record up on a subfile display. The subfile user may not even want to update the order that the shipping department needs, but in this situation it doesn't matter, the subfile user has a lock on the record. The expanding subfile makes matters worse. In this case, the subfile user could potentially lock a lot of order headers as they roll forward through the subfile loading additional records.

For this situation, the page-at-a-time subfile has a distinct advantage. The user would lock only one page worth of records at a time. Another alternative would be not to use a subfile at all.

When designing update subfiles, a lot of thought and planning has to be placed into the application. You want your users to be as productive as possible while maintaining data integrity. And you would like the applications to be written as simply as possible. A balance of all three points are needed.

Don't forget to look into the future and plan for what happens for when the business and this application matures. If you only have one order entry operator, you may not give much thought to locks. But if you did not design your application properly in the beginning, should your company ever hire additional order entry operators, your applications may not be able to support them.

Now look at some update subfiles. The next few example uses the STATES file. Here is the DDS code for the file:

```
A*==============================================================
A* To compile:
A*
A*      CRTPF      FILE(XXX/STATES) SRCFILE(XXX/QDDSSRC)
A*
A*==============================================================
A                                       UNIQUE
A          R RSTATES
A            STATCD          2
A            STATDS         15
A          K STATCD
```

Create the STATES file and enter some test data records into the file. Then take a look at program SFR650.

```
A*============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD650) SRCFILE(XXX/QDDSSRC)
A*
A*============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03)
A          R SFD650A                    SFL
A            STATCD    R         B  8  3REFFLD(RSTATES/STATCD *LIBL/STATES)
A                                       COLOR(WHT)
A            STATDS    R         B  8 13REFFLD(RSTATES/STATDS *LIBL/STATES)
A                                       COLOR(WHT)
A          R SFD650B                    SFLCTL(SFD650A)
A                                       SFLSIZ(0050)
A                                       SFLPAG(0010)
A                                       OVERLAY
A  30                                   SFLDSP
A                                       SFLDSPCTL
A  31                                   SFLCLR
A  35                                   SFLEND(*MORE)
A                                   1  4DATE
A                                       EDTCDE(Y)
A                                       COLOR(BLU)
A                                   2  4TIME
A                                       COLOR(BLU)
A                                   1 26'Update State Codes'
A                                       COLOR(WHT)
A                                   1 69USER
A                                       COLOR(BLU)
A                                   2 69'SFR650'
A                                       COLOR(BLU)
A                                   6  1'State Code'
A                                       COLOR(PNK)
A                                       DSPATR(UL)
A                                   6 13'Description    '
A                                       COLOR(PNK)
A                                       DSPATR(UL)
A          R TRAILER
A                                  23  2'F3-Exit'
A                                       COLOR(WHT)
A                                  21 44'Number Of Reads'
A                                  22 44'Number Of Records Entered'
A            @READS         5Y 00 21 73COLOR(PNK)
A                                       EDTCDE(Z)
A            @RCDS          5Y 00 22 73COLOR(PNK)
A                                       EDTCDE(Z)

F*============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR650) SRCFILE(XXX/QRPGSRC)
F*
F*============================================================
```

```
FSTATES  IF  E           K        DISK
FSFD650  CF  E                    WORKSTN
F                                        @RRN  KSFILE SFD650A
C*
C*  Write the command key line
C*
C                     WRITETRAILER
C*
C*   Clear The Subfile
C*
C                     MOVE *ON       *IN31
C                     WRITESFD650B
C                     MOVE *OFF      *IN31
C*
C*   Load records into the subfile
C*
C                     EXSR @LOAD
C*
C*   Display The Subfile
C*
C                     EXFMTSFD650B
C                     EXSR @CMD
C*
C*   Read through the subfile and see how many records
C*   were maintained versus how many read operations it took.
C*
C                     EXSR @READ
C*
C*************************************************************
C*    @ C M D   S U B R O U T I N E                          *
C*                                                           *
C*    Executed after every EXFMT or display read op code.    *
C*    Used to process command keys.                          *
C*************************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, set on LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C                     ENDSR
C*
C*************************************************************
C*    @ R E A D   S U B R O U T I N E                        *
C*                                                           *
C*    Read the records inputted from the subfile             *
C*************************************************************
C*
C           @READ     BEGSR
C*
C*   Set the # of reads, # of records, and relative record
C*   number fields to 0. Ensure *IN99 is off.
C*
```

```
C                     Z-ADD0         @READS
C                     Z-ADD0         @RCDS
C                     Z-ADD0         @RRN
C                     MOVE *OFF      *IN99
C*
C* Do until you reach the end of the subfile (100 records)
C*
C           *IN99     DOWEQ*OFF
C                     READCSFD650A                 99
C           *IN99     IFEQ *ON
C                     LEAVE
C                     END
C*
C*  If you read a record, increment the read counter.
C*
C                     ADD  1         @READS
C                     ADD  1         @RCDS
C*
C                     END
C*
C                     ENDSR
C*
C*********************************************************
C*    * I N Z S R   S U B R O U T I N E                  *
C*                                                       *
C*    Retrieve the starting line and position for the    *
C*    window. Check to make sure that these values are   *
C*    valid. If not, put the closest valid number in.    *
C*********************************************************
C*
C           *INZSR    BEGSR
C*
C*  Initialize the relative record number field for the subfile
C*  to 0. Turn on 35, so that the subfile will show the
C*  subfile end message.
C*
C                     Z-ADD0         @RRN    50
C                     MOVE *ON       *IN35
C*
C                     ENDSR
C*
C*********************************************************
C*    @ L O A D   S U B R O U T I N E                    *
C*                                                       *
C*    Load the subfile with all of the state codes       *
C*    and descriptions.                                  *
C*********************************************************
C*
C           @LOAD     BEGSR
C*
C* Do until the end of file has been reached
C*
C                     MOVE *OFF      *IN99
C                     Z-ADD0         @RRN
C           *LOVAL    SETLLSTATES
C*
C           *IN99     DOWEQ*OFF
```

```
C                       READ STATES                    99
C*
C             *IN99     IFEQ *ON
C             @RRN      OREQ 9999
C                       LEAVE
C                       END
C*
C                       ADD  1         @RRN
C                       WRITESFD650A
C*
C                       END
C*
C*  If there is at least one record in the subfile, it will
C*  be ok to display it, else do not display an empty
C*  subfile or you will get an error.
C*
C             @RRN      IFGT 0
C                       MOVE *ON       *IN30
C                       ELSE
C                       MOVE *OFF      *IN30
C                       END
C*
C                       ENDSR
```

Basically, this program is a load-all subfile program except that the subfile records are specified as input and output. Notice that this program does not do any real updating to a file. That change will be made later. Call SFR650. Your display should look like this:

```
   xx/xx/xx              Update State Codes                        QPGMR
   xx:xx:xx                                                        SFR650



State Code  Description

  AK        ALASKA
  AL        ALABAMA
  AR        ARKANSAS
  CA        CALIFORNIA
  CN        CANADA
  FL        FLORIDA
  GA        GEORGIA
  LA        LOUISIANA
  MS        MISSISSIPPI
  NC        NORTH CAROLINA
                                                                  More...


                                          Number Of Reads
                                          Number Of Records Entered
 F3-Exit
```

Now, blank out the first state code and press Enter. Your display should look like this:

```
   xx/xx/xx              Update State Codes                        QPGMR
   xx:xx:xx                                                        SFR650



State Code  Description

  AK        ALASKA
  AL        ALABAMA
  AR        ARKANSAS
  CA        CALIFORNIA
  CN        CANADA
  FL        FLORIDA
  GA        GEORGIA
  LA        LOUISIANA
  MS        MISSISSIPPI
  NC        NORTH CAROLINA
                                                                  More...


                                          Number Of Reads                1
                                          Number Of Records Entered      1
 F3-Exit
```

Notice how the “Entered” counter shows 1, and the"Read" counter also shows 1. You can see that the READC routine works. It reads any “changed” records. The program does not have to read through all the subfile records to process just one record.

Maintenance Subfile

Now, combine the update subfile and input subfile to create a single maintenance subfile program. You can see this technique in program SFR660.

```
A*============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD660) SRCFILE(XXX/QDDSSRC)
A*
A*============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03)
A          R SFD660A                    SFL
A            STATCD    R        B  8  3REFFLD(RSTATES/STATCD *LIBL/STATES)
A                                       COLOR(WHT)
A            STATDS    R        B  8 13REFFLD(RSTATES/STATDS *LIBL/STATES)
A                                       COLOR(WHT)
A            @KEY      R        H      REFFLD(RSTATES/STATCD *LIBL/STATES)
A          R SFD660B                    SFLCTL(SFD660A)
A                                       SFLSIZ(0500)
A                                       SFLPAG(0010)
A                                       OVERLAY
A  30                                   SFLDSP
A                                       SFLDSPCTL
A  31                                   SFLINZ
A                                       SFLRNA
A  35                                   SFLEND(*MORE)
A                                   1  4DATE
A                                       EDTCDE(Y)
A                                       COLOR(BLU)
A                                   2  4TIME
A                                       COLOR(BLU)
A                                   1 26'Update State Codes'
A                                       COLOR(WHT)
A                                   1 69USER
A                                       COLOR(BLU)
A                                   2 69'SFR660'
A                                       COLOR(BLU)
A                                   6  1'State Code'
A                                       COLOR(PNK)
A                                       DSPATR(UL)
A                                   6 13'Description    '
A                                       COLOR(PNK)
A                                       DSPATR(UL)
A          R TRAILER
A                                  23  2'F3-Exit'
A                                       COLOR(WHT)
```

```
A                                   21 44'Number Of Reads'
A                                   22 44'Number Of Records Added  '
A            @READS         5Y 00 21 73COLOR(PNK)
A                                     EDTCDE(Z)
A            @ADD           5Y 00 22 73COLOR(PNK)
A                                     EDTCDE(Z)
A            @UPD           5Y 00 23 73COLOR(PNK)
A                                     EDTCDE(Z)
A                                   23 44'Number Of Records Updated'


F*============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR660) SRCFILE(XXX/QRPGSRC)
F*
F*============================================================
FSTATES  IF  E           K        DISK
FSFD660  CF  E                    WORKSTN
F                                        @RRN  KSFILE SFD660A
C*
C*  Write the command key line
C*
C                     WRITETRAILER
C*
C*   Initialize the subfile with 500 non-active records
C*
C                     MOVE *ON       *IN31
C                     WRITESFD660B
C                     MOVE *OFF      *IN31
C*
C*   Load records into the subfile
C*
C                     EXSR @LOAD
C*
C*   Display The Subfile
C*
C                     EXFMTSFD660B
C                     EXSR @CMD
C*
C*   Read through the subfile and see how many records
C*   were maintained versus how many read operations it took.
C*
C                     EXSR @READ
C*
C*********************************************************
C*    @ C M D   S U B R O U T I N E                      *
C*                                                       *
C*    Executed after every EXFMT or display read op code.  *
C*    Used to process command keys.                      *
C*********************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, set on LR and exit.
C*
```

```
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C                     ENDSR
C*
C*************************************************************
C*    @ R E A D   S U B R O U T I N E                        *
C*                                                           *
C*    Read the records inputted from the subfile             *
C*************************************************************
C*
C           @READ     BEGSR
C*
C*   Set the # of reads, # of records, and relative record
C*   number fields to 0. Ensure *IN99 is off.
C*
C                     Z-ADD0         @READS
C                     Z-ADD0         @UPD
C                     Z-ADD0         @ADD
C                     Z-ADD0         @RRN
C                     MOVE *OFF      *IN99
C*
C* Process all changed records
C*
C           *IN99     DOWEQ*OFF
C                     READCSFD660A                  99
C           *IN99     IFEQ *ON
C                     LEAVE
C                     END
C*
C*  If you read a record, increment the read counter.
C*
C                     ADD  1         @READS
C*
C* If the hidden key field is greater than *blanks, it means
C* that this subfile record existed in the STATES file, and
C* was loaded into the subfile in the @LOAD subroutine. If
C* the @KEY field is blank, it meand the record did not
C* exist in the STATES file, but was added.
C*
C           @KEY      IFGT *BLANKS
C                     ADD  1         @UPD
C                     ELSE
C                     ADD  1         @ADD
C                     END
C*
C                     END
C*
C                     ENDSR
C*
C*************************************************************
C*    * I N Z S R   S U B R O U T I N E                      *
C*                                                           *
C*    Retrieve the starting line and position for the        *
C*    window. Check to make sure that these values are       *
```

```
C*    valid. If not, put the closest valid number in.       *
C*********************************************************
C*
C           *INZSR    BEGSR
C*
C*  Initialize the relative record number field for the subfile
C*  to 0. Turn on 35, so that the subfile will show the
C*  subfile end message.
C*
C                     Z-ADD0         @RRN    50
C                     MOVE *ON       *IN35
C*
C                     ENDSR
C*
C*********************************************************
C*    @ L O A D   S U B R O U T I N E                    *
C*                                                       *
C*    Load the subfile with all of the state codes       *
C*    and descriptions.                                  *
C*********************************************************
C*
C           @LOAD     BEGSR
C*
C* Do until the end of file has been reached
C*
C                     MOVE *OFF      *IN99
C                     Z-ADD0         @RRN
C           *LOVAL    SETLLSTATES
C*
C           *IN99     DOWEQ*OFF
C                     READ STATES                  99
C*
C           *IN99     IFEQ *ON
C           @RRN      OREQ 500
C                     LEAVE
C                     END
C*
C                     ADD  1         @RRN
C*
C*  Move the state code into the hidden field @KEY.
C*
C                     MOVELSTATCD    @KEY
C                     WRITESFD660A
C*
C                     END
C*
C*  Since you initialized the subfile, it will be ok to
C*  display it.
C*
C                     MOVE *ON       *IN30
C*
C                     ENDSR
```

Call SFR660. Your display should look like this:

```
  xx/xx/xx              Update State Codes                        QPGMR
  xx:xx:xx                                                        SFR660



State Code  Description

  AK        ALASKA
  AL        ALABAMA
  AR        ARKANSAS
  CA        CALIFORNIA
  CN        CANADA
  FL        FLORIDA
  GA        GEORGIA
  LA        LOUISIANA
  MS        MISSISSIPPI
  NC        NORTH CAROLINA
                                                                  More...


                                        Number Of Reads
                                        Number Of Records Added
 F3-Exit                                Number Of Records Updated
```

Now roll forward. You can see an input field at the bottom.

```
  xx/xx/xx              Update State Codes                        QPGMR
  xx:xx:xx                                                        SFR660



State Code  Description

  ND        NORTH DAKOTA
  NM        NEW MEXICO
  NV        NEVADA
  SC        SOUTH CAROLINA
  SD        SOUTH DAKOTA
  TN        TENNESSEE
  TX        TEXAS
  VT        VERMONT
  WA        WASHINGTON
  __        ________________
                                                                  More...


                                        Number Of Reads
                                        Number Of Records Added
 F3-Exit                                Number Of Records Updated
```

You can now roll forward again, but you will have nothing but input fields. Notice that the subfile size has been increased to 500. You could have a total of 500 subfile records, some of which would be update and the remainder input.

You can make this program actually update the STATES file as shown in program SFR670.

```
A*============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD670) SRCFILE(XXX/QDDSSRC)
A*
A*============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03)
A          R SFD670A                    SFL
A            STATCD    R        B  8  3REFFLD(RSTATES/STATCD *LIBL/STATES)
A                                       COLOR(WHT)
A            STATDS    R        B  8 13REFFLD(RSTATES/STATDS *LIBL/STATES)
A                                       COLOR(WHT)
A            @KEY      R        H      REFFLD(RSTATES/STATCD *LIBL/STATES)
A          R SFD670B                    SFLCTL(SFD670A)
A                                       SFLSIZ(0500)
A                                       SFLPAG(0010)
A                                       OVERLAY
A  30                                   SFLDSP
A                                       SFLDSPCTL
A  31                                   SFLINZ
A                                       SFLRNA
A  35                                   SFLEND(*MORE)
A                                  1  4DATE
A                                       EDTCDE(Y)
A                                       COLOR(BLU)
A                                  2  4TIME
A                                       COLOR(BLU)
A                                  1 26'Update State Codes'
A                                       COLOR(WHT)
A                                  1 69USER
A                                       COLOR(BLU)
A                                  2 69'SFR670'
A                                       COLOR(BLU)
A                                  6  1'State Code'
A                                       COLOR(PNK)
A                                       DSPATR(UL)
A                                  6 13'Description    '
A                                       COLOR(PNK)
A                                       DSPATR(UL)
A          R TRAILER
A                                 23  2'F3-Exit'
A                                       COLOR(WHT)
```

```
F*==============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR670) SRCFILE(XXX/QRPGSRC)
F*
F*==============================================================
FSTATES  UF  E           K        DISK                      A
FSFD670  CF  E                    WORKSTN
F                                        @RRN  KSFILE SFD670A
ISAVDS     E DSSTATES                    2
C*
C*  Write the command key line
C*
C                     WRITETRAILER
C*
C*   Initialize the subfile with 500 non-active records
C*
C                     MOVE *ON       *IN31
C                     WRITESFD670B
C                     MOVE *OFF      *IN31
C*
C*   Load records into the subfile
C*
C                     EXSR @LOAD
C*
C*   Display The Subfile
C*
C                     EXFMTSFD670B
C                     EXSR @CMD
C*
C*   Read through the subfile and see how many records
C*   were maintained versus how many read operations it took.
C*
C                     EXSR @UPDAT
C*
C*********************************************************
C*    @ C M D   S U B R O U T I N E                      *
C*                                                       *
C*    Executed after every EXFMT or display read op code.  *
C*    Used to process command keys.                      *
C*********************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, set on LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C                     ENDSR
C*
C*********************************************************
C*    @ U P D A R    S U B R O U T I N E                 *
C*                                                       *
C*    Read the records inputted from the subfile         *
```

```
C*    Add, Change, or Delete records to the STATES file     *
C*************************************************************
C*
C           @UPDAT    BEGSR
C*
C                     MOVE *OFF      *IN99
C*
C* Process all changed subfile records
C*
C           *IN99     DOWEQ*OFF
C                     READCSFD67OA                  99
C           *IN99     IFEQ *ON
C                     LEAVE
C                     END
C*
C* If the hidden key field is greater than *blanks, it means
C* that this subfile record existed in the STATES file, and
C* was loaded into the subfile in the @LOAD subroutine. If
C* the @KEY field is blank, it meand the record did not
C* exist in the STATES file, but was added.  If the hidden
C* key @KEY is not blanks, but the state code field STATCD
C* is blanks, it means the user wants to delete the record.
C*
C* Update/Delete Routine
C* Delete the record from STATES file first, then, if you are
C* updating a record, write the changed record back to the file.
C*
C           @KEY      IFGT *BLANKS
C*
C*  Switch to the 2nd occurrence of the data structure so that
C*  the chain will not overwrite the fields STATCD and STATDS
C*  that you will be updating from the subfile.
C*
C           2         OCUR SAVDS
C           @KEY      CHAINRSTATES              98
C           *IN98     IFEQ *OFF
C                     DELETRSTATES
C                     END
C*
C* Go back to the 1st occurrence of the data structure, which
C* is where the fields that you will be updating are stored.
C*
C           1         OCUR SAVDS
C           STATCD    IFGT *BLANKS
C                     WRITERSTATES
C                     END
C*
C*  Add Routine
C*  Ensure that the key field STATCD does not already exist,
C*  since the file STATES has a UNIQUE key.
C*
C                     ELSE
C*
C           STATCD    CHAINRSTATES              98
C           *IN98     IFEQ *ON
C                     WRITERSTATES
C                     END
```

```
C                     END
C*
C                     END
C*
C                     ENDSR
C*
C*************************************************************
C*    * I N Z S R   S U B R O U T I N E                      *
C*                                                           *
C*    Retrieve the starting line and position for the        *
C*    window. Check to make sure that these values are       *
C*    valid. If not, put the closest valid number in.        *
C*************************************************************
C*
C           *INZSR    BEGSR
C*
C*  Initialize the relative record number field for the subfile
C*  to 0. Turn on 35, so that the subfile will show the
C*  subfile end message.
C*
C                     Z-ADD0         @RRN    50
C                     MOVE *ON       *IN35
C*
C                     ENDSR
C*
C*************************************************************
C*    @ L O A D   S U B R O U T I N E                        *
C*                                                           *
C*    Load the subfile with all of the state codes           *
C*    and descriptions.                                      *
C*************************************************************
C*
C           @LOAD     BEGSR
C*
C* Do until the end of file has been reached
C*
C                     MOVE *OFF      *IN99
C                     Z-ADD0         @RRN
C           *LOVAL    SETLLSTATES
C*
C           *IN99     DOWEQ*OFF
C                     READ STATES                   99
C*
C           *IN99     IFEQ *ON
C           @RRN      OREQ 500
C                     LEAVE
C                     END
C*
C                     ADD  1         @RRN
C*
C*  Move the state code into the hidden field @KEY.
C*
C                     MOVELSTATCD    @KEY
C                     WRITESFD670A
C*
C                     END
C*
```

```
C*  Since you initialized the subfile, it will be ok to
C*  display it.
C*
C                     MOVE *ON       *IN30
C*
C                     ENDSR
```

You can add, delete, or change records in the STATES file. To add a record, just key in a new subfile code and description in the input section of the subfile. To delete a record, simply blank out an existing state code. To change a record, type over an existing subfile record.

Now look at the file update logic. In the program, you'll find the following code:

```
ISAVDS     E DSSTATES                    2
```

This line sets up a multiple occurrence external data structure. The name of the data structure is SAVDS. It is an external data structure where the data structure fields are defined like the record layout in the states file. The SAVDS data structure contains the fields STATCD and STATDS. The program specifies that the data structure has two occurrences, which yields the following:

```
SAVDS occurrence 1       STATCD  STATDS

SAVDS occurrence 2       STATCD  STATDS
```

Why would you want two occurences of the data structure? Well, the program is going to have to chain to the state file. When it does this, the fields STATCD and STATDS are updated with whatever the file STATES had in these fields. The subfile also has the fields defined. So when the program chains to the STATES file for the update process, it would overlay the information in the subfile record. In other words, the following would be happening:

```
Read Subfile Record (update value STATCD and STATDS)

Chain To STATES File (update value STATCD and STATDS)
```

So if the user wanted to update the state description, as soon as the program chained to the state file, the new description in the subfile record would be overwritten with the old description in the state file.

This is the reason you would use a multiple occurrence data structure. When you read the subfile record, the fields STATCD and STATDS in occurrence 1 are updated. Then the program switches to the second occurrence, and chains to the state file. When the program switches back to the first occurrence, the fields STATCD and STATDS contain the subfile record values.

This switching logic is contained in the @UPDAT subroutine. First, the program retrieves a changed record with the READC op code. If it retrieved a changed record, it processes the following routine:

```
C           @KEY      IFGT *BLANKS
C           2         OCUR SAVDS
C           @KEY      CHAINRSTATES              98
C           *IN98     IFEQ *OFF
C                     DELETRSTATES
C                     END
C           1         OCUR SAVDS
C           STATCD    IFGT *BLANKS
C                     WRITERSTATES
C                     END
```

This code does the add, change, and delete function. The following breaks it down to see how it works.

If the record was changed, and it already existed, then the program is going to update or delete the record. How can the program tell if a subfile record already existed in the states file? Because when it loaded the subfile, it loaded the STATCD field into a hidden field called @KEY. @KEY is part of the subfile record, but is not a displayable field. Do not confuse the attributes hidden field with a non-display field. A nondisplay field resides on the screen, it is just not displayed. A hidden field does not occupy a space on the display. It is, however, part of the subfile record. When the program reads the subfile, it is returned the value of this field when it was loaded.

Before the program loaded any subfile records from the states file, it initialized the subfile with 500 nonactive records. The subfile record contains three fields, STATCD, STATDS, and @KEY. Initially, all of these fields for all 500 records were blanks.

Before the program wrote the subfile record, it moved the value of STATCD into @KEY. So if @KEY is not blanks, then the subfile record exists in the states file. If the READC operation read this record, the user either wanted to update or delete it. The following logic controls the update and delete function.

```
C           @KEY      IFGT *BLANKS
C           2         OCUR SAVDS
C           @KEY      CHAINRSTATES              98
C           *IN98     IFEQ *OFF
C                     DELETRSTATES
C                     END
C           1         OCUR SAVDS
```

If @KEY is greater than blanks, the program changes to the second occurrence of the data structure in preparation to chain to the states file. This change will keep the values of STATCD and STATDS in the subfile record from being destroyed by the chain operation.

If the record exists in the states file (and it should) then the program deletes it. The program does this if it is updating or deleting the record. Then it switches the data structure back to the first occurrence, which means the fields STATCD and STATDS contain the values entered from the subfile. Then, you can process the next part of the routine.

```
C           STATCD    IFGT *BLANKS
C                     WRITERSTATES
C                     END
```

If the user wanted to delete the record, they would have done so by blanking out the state code field. If the record was deleted, then this routine does not get processed because STATCD would be equal to blanks.

If the user wanted to update the record, this routine would be processed. The program does this by simply writing the subfile record to the states file. It doesn't update the states file because it deleted the record first. This process allows a user to change the key of a record. If the user changed a state code from AK to AM, then the routine would take the value of @KEY, which would be AK, and delete the record from the file. Then it would add back the AM record.

If the record were flagged for update or delete, the routine would be finished. If it were flagged for addition, the next part of the routine would be processed. This routine is processed because the field @KEY is equal to blanks.

```
C                     ELSE
C           STATCD    CHAINRSTATES              98
C           *IN98     IFEQ *ON
C                     WRITERSTATES
C                     END
C                     END
```

The program is ensuring that the state code is not already in the states file. Since this file has a unique key, the program would receive an error if it attempted to write a duplicate key.

You can see this process work by calling program SFR670. Your display should look like this:

```
  xx/xx/xx              Update State Codes                        QPGMR
  xx:xx:xx                                                        SFR670


State Code  Description

  AK        ALASKA
  AL        ALABAMA
  AR        ARKANSAS
  CA        CALIFORNIA
  CN        CANADA
  FL        FLORIDA
  GA        GEORGIA
  LA        LOUISIANA
  MS        MISSISSIPPI
  NC        NORTH CAROLINA
                                                                  More...



 F3-Exit
```

Roll to the first input subfile record. Enter in a state and description. In this example, AZ - Arizona has been added. Press Enter. Your display should look like this:

```
  xx/xx/xx              Update State Codes                        QPGMR
  xx:xx:xx                                                        SFR670


State Code  Description

  AK        ALASKA
  AL        ALABAMA
  AR        ARKANSAS
  AZ        ARIZONA
  CA        CALIFORNIA
  CN        CANADA
  FL        FLORIDA
  GA        GEORGIA
  LA        LOUISIANA
  MS        MISSISSIPPI
                                                                  More...



 F3-Exit
```

Notice how when the subfile came back up, it added the record in alphabetical order. It added the record because the program rebuilds the subfile every time the Enter key is pressed. Since the program reads the states file by key, the added record shows up in the correct sequence.

Now, blank out Arizona's state code.

```
  xx/xx/xx              Update State Codes                        QPGMR
  xx:xx:xx                                                        SFR670


State Code  Description

  AK        ALASKA
  AL        ALABAMA
  AR        ARKANSAS
  __        ________
  CA        CALIFORNIA
  CN        CANADA
  FL        FLORIDA
  GA        GEORGIA
  LA        LOUISIANA
  MS        MISSISSIPPI
                                                                 More...



 F3-Exit
```

Press Enter. Your display should look like this:

```
  xx/xx/xx              Update State Codes                        QPGMR
  xx:xx:xx                                                        SFR670


State Code  Description

  AK        ALASKA
  AL        ALABAMA
  AR        ARKANSAS
  CA        CALIFORNIA
  CN        CANADA
  FL        FLORIDA
  GA        GEORGIA
  LA        LOUISIANA
  MS        MISSISSIPPI
  NC        NORTH CAROLINA
                                                                  More...



 F3-Exit
```

The program removes the record from the file because you specified the record was to be deleted by blanking out the state code.

Now you can tailor this subfile. Suppose the users want this subfile to come up and position the cursor on the first input-capable field. You can easily do this. Look at the code for program SFR680.

```
A*==============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD680) SRCFILE(XXX/QDDSSRC)
A*
A*==============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03)
A          R SFD680A                    SFL
A            STATCD    R         B  8  3REFFLD(RSTATES/STATCD *LIBL/STATES)
A                                       COLOR(WHT)
A            STATDS    R         B  8 13REFFLD(RSTATES/STATDS *LIBL/STATES)
A                                       COLOR(WHT)
A            @KEY      R         H      REFFLD(RSTATES/STATCD *LIBL/STATES)
A          R SFD680B                    SFLCTL(SFD680A)
A                                       SFLSIZ(0500)
A                                       SFLPAG(0010)
A                                       OVERLAY
A  30                                   SFLDSP
A                                       SFLDSPCTL
```

```
A  31                                   SFLINZ
A                                       SFLRNA
A  35                                   SFLEND(*MORE)
A            @REC           4  OH       SFLRCDNBR(CURSOR)
A                                 1  4DATE
A                                       EDTCDE(Y)
A                                       COLOR(BLU)
A                                 2  4TIME
A                                       COLOR(BLU)
A                                 1 26'Update State Codes'
A                                       COLOR(WHT)
A                                 1 69USER
A                                       COLOR(BLU)
A                                 2 69'SFR680'
A                                       COLOR(BLU)
A                                 6  1'State Code'
A                                       COLOR(PNK)
A                                       DSPATR(UL)
A                                 6 13'Description    '
A                                       COLOR(PNK)
A                                       DSPATR(UL)
A          R TRAILER
A                                23  2'F3-Exit'
A                                       COLOR(WHT)


F*==============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR680) SRCFILE(XXX/QRPGSRC)
F*
F*==============================================================
FSTATES  UF  E           K        DISK                      A
FSFD680  CF  E                    WORKSTN
F                                        @RRN  KSFILE SFD680A
ISAVDS     E DSSTATES                    2
C*
C*  Write the command key line
C*
C                     WRITETRAILER
C*
C*   Initialize the subfile with 500 non-active records
C*
C                     MOVE *ON       *IN31
C                     WRITESFD680B
C                     MOVE *OFF      *IN31
C*
C*   Load records into the subfile
C*
C                     EXSR @LOAD
C*
C*   Display The Subfile
C*
C                     EXFMTSFD680B
C                     EXSR @CMD
C*
```

```
C*   Read through the subfile and see how many records
C*   were maintained versus how many read operations it took.
C*
C                     EXSR @UPDAT
C*
C*************************************************************
C*    @ C M D   S U B R O U T I N E                          *
C*                                                           *
C*    Executed after every EXFMT or display read op code.    *
C*    Used to process command keys.                          *
C*************************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, set on LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C                     ENDSR
C*
C*************************************************************
C*    @ U P D A T    S U B R O U T I N E                     *
C*                                                           *
C*    Read the records inputted from the subfile             *
C*    Add, Change, or Delete records to the STATES file      *
C*************************************************************
C*
C           @UPDAT    BEGSR
C*
C                     MOVE *OFF      *IN99
C*
C* Process all changed subfile records
C*
C           *IN99     DOWEQ*OFF
C                     READCSFD680A                  99
C           *IN99     IFEQ *ON
C                     LEAVE
C                     END
C*
C* If the hidden key field is greater than *blanks, it means
C* that this subfile record existed in the STATES file, and
C* was loaded into the subfile in the @LOAD subroutine. If
C* the @KEY field is blank, it meand the record did not
C* exist in the STATES file, but was added.  If the hidden
C* key @KEY is not blanks, but the state code field STATCD
C* is blanks, it means the user wants to delete the record.
C*
C* Update/Delete Routine
C* Delete the record from STATES file first, then, if you are
C* updating a record, write the changed record back to the file.
C*
C           @KEY      IFGT *BLANKS
C*
C*  Switch to the 2nd occurrence of the data structure so that
```

```
C*  the chain will not overwrite the fields STATCD and STATDS
C*  that you will be updating from the subfile.
C*
C           2         OCUR SAVDS
C           @KEY      CHAINRSTATES              98
C           *IN98     IFEQ *OFF
C                     DELETRSTATES
C                     END
C*
C* Go back to the 1st occurrence of the data structure, which
C* is where the fields that you will be updating are stored.
C*
C           1         OCUR SAVDS
C           STATCD    IFGT *BLANKS
C                     WRITERSTATES
C                     END
C*
C*  Add Routine
C*  Ensure that the key field STATCD does not already exist,
C*  since the file STATES has a UNIQUE key.
C*
C                     ELSE
C*
C           STATCD    CHAINRSTATES              98
C           *IN98     IFEQ *ON
C                     WRITERSTATES
C                     END
C                     END
C*
C                     END
C*
C                     ENDSR
C*
C*********************************************************
C*    * I N Z S R   S U B R O U T I N E                  *
C*                                                       *
C*    Retrieve the starting line and position for the    *
C*    window. Check to make sure that these values are   *
C*    valid. If not, put the closest valid number in.    *
C*********************************************************
C*
C           *INZSR    BEGSR
C*
C*  Initialize the relative record number field for the subfile
C*  to 0. Turn on 35, so that the subfile will show the
C*  subfile end message.
C*
C                     Z-ADD0         @RRN    50
C                     MOVE *ON       *IN35
C*
C                     ENDSR
C*
C*********************************************************
C*    @ L O A D   S U B R O U T I N E                    *
C*                                                       *
C*    Load the subfile with all of the state codes       *
C*    and descriptions.                                  *
```

```
C*************************************************************
C*
C           @LOAD     BEGSR
C*
C* Do until the end of file has been reached
C*
C                     MOVE *OFF      *IN99
C                     Z-ADD0         @RRN
C           *LOVAL    SETLLSTATES
C*
C           *IN99     DOWEQ*OFF
C                     READ STATES                   99
C*
C           *IN99     IFEQ *ON
C           @RRN      OREQ 500
C                     LEAVE
C                     END
C*
C                     ADD  1         @RRN
C*
C*  Move the state code into the hidden field @KEY.
C*
C                     MOVELSTATCD    @KEY
C                     WRITESFD680A
C*
C                     END
C*
C*  Since you initialized the subfile, it will be ok to
C*  display it.
C*
C                     MOVE *ON       *IN30
C*
C*  Position the subfile to the last output record, which
C*  should be the page where the first input line exists.
C*
C                     Z-ADD@RRN      @REC
C*
C*  Now add 1 to the subfile @REC field. @REC should now
C*  be pointed to the first input subfile line. The cursor
C*  will be positioned here.
C*
C                     ADD  1         @REC
C                     ENDSR
```

All you need to do this time is use the SFLRCDNBR keyword in a new way. This keyword allows you to specify what page on which to display the subfile when the program issues the write or EXFMT screen. For instance, suppose the subfile has 10 records to a page. If the program places the value one in the SFLRCDNBR field, the first page of the subfile is displayed because subfile record 1 is on page one. If the program places 10 into the SFLRCDNBR field, then the first page is displayed because subfile records 1 through 10 are on the first page. If the program places the value 11 into the SFLRCDNBR keyword, then the second page is displayed initially because subfile record 11 is on page two. However, you could still roll back to the first page.

You can use SFLRCDNBR to control which page to initially display the subfile, but how do you get the cursor to position itself to the first input subfile record. First, you have to add the following to the display file:

```
A            @REC            4  0H      SFLRCDNBR(CURSOR)
```

This line tells the subfile to initially display the subfile on the page identified by the relative record number contained in the field @REC. It also tells the subfile to position the cursor on this record.

Now, to get the relative record number of the first input subfile record, add these statements to the RPG program in the subfile load subroutine:

```
C                       Z-ADD@RRN      @REC
C                       ADD  1         @REC
```

After the program finishes loading the subfile, it takes the last relative record number written to the subfile, @RRN, and moves it into the SFLRCDNBR field @REC. This action positions the cursor to the last subfile record written. Then the program adds 1 to @REC, and the cursor is positioned to the first input subfile record. To see this effect, call SFR680. Your display should look like this:

```
  xx/xx/xx            Update State Codes                        QPGMR
  xx:xx:xx                                                      SFR680


State Code  Description

  ND        NORTH DAKOTA
  NM        NEW MEXICO
  NV        NEVADA
  SC        SOUTH CAROLINA
  SD        SOUTH DAKOTA
  TN        TENNESSEE
  TX        TEXAS
  VT        VERMONT
  WA        WASHINGTON
  __        ______________
                                                              More...



 F3-Exit
```

Now you can take the input/update subfile program a step further. Suppose you do not want the user to key in any states that start with a “Q.” If the user keyed in a state code with a “Q,” you would want to reverse image the state code and redisplay the

entire subfile. You would let them correct the mistake before allowing the file to be updated.

Take a look at SFD681 and SFR681:

```
A*============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD681) SRCFILE(XXX/QDDSSRC)
A*
A*============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03)
A          R SFD681A                    SFL
A            STATCD    R        B  8  3REFFLD(RSTATES/STATCD *LIBL/STATES)
A                                       COLOR(WHT)
A  41                                   DSPATR(RI)
A            STATDS    R        B  8 13REFFLD(RSTATES/STATDS *LIBL/STATES)
A                                       COLOR(WHT)
A            @KEY      R        H      REFFLD(RSTATES/STATCD *LIBL/STATES)
A          R SFD681B                    SFLCTL(SFD681A)
A                                       SFLSIZ(0500)
A                                       SFLPAG(0010)
A                                       OVERLAY
A  30                                   SFLDSP
A                                       SFLDSPCTL
A  31                                   SFLINZ
A                                       SFLRNA
A  35                                   SFLEND(*MORE)
A            @REC           4  OH      SFLRCDNBR(CURSOR)
A                                   1  4DATE
A                                       EDTCDE(Y)
A                                       COLOR(BLU)
A                                   2  4TIME
A                                       COLOR(BLU)
A                                   1 26'Update State Codes'
A                                       COLOR(WHT)
A                                   1 69USER
A                                       COLOR(BLU)
A                                   2 69'SFR681'
A                                       COLOR(BLU)
A                                   6  1'State Code'
A                                       COLOR(PNK)
A                                       DSPATR(UL)
A                                   6 13'Description    '
A                                       COLOR(PNK)
A                                       DSPATR(UL)
A          R TRAILER
A                                  23  2'F3-Exit'
A                                       COLOR(WHT)


F*============================================================
F* To compile:
F*
```

```
F*      CRTRPGPGM  PGM(XXX/SFR681) SRCFILE(XXX/QRPGSRC)
F*
F*==============================================================
FSTATES  UF  E           K        DISK                      A
FSFD681  CF  E                    WORKSTN
F                                        @RRN  KSFILE SFD681A
ISAVDS     E DSSTATES                    2
C*
C*  Write the command key line
C*
C                     WRITETRAILER
C*
C*   Initialize the subfile with 500 non-active records
C*
C                     MOVE *ON       *IN31
C                     WRITESFD681B
C                     MOVE *OFF      *IN31
C*
C*   Load records into the subfile
C*
C                     EXSR @LOAD
C*
C*   Display The Subfile
C*
C           @DISP     TAG
C                     EXFMTSFD681B
C                     EXSR @CMD
C*
C*   Validate the state codes. Make sure none of them start
C*   with the letter  Q .
C*
C                     MOVE *OFF      @ERROR
C                     EXSR @VAL
C           @ERROR    IFEQ *ON
C                     GOTO @DISP
C                     END
C*
C*   Read through the subfile and see how many records
C*   were maintained versus how many read operations it took.
C*
C                     EXSR @UPDAT
C*
C*************************************************************
C*    @ C M D   S U B R O U T I N E                          *
C*                                                           *
C*    Executed after every EXFMT or display read op code.    *
C*    Used to process command keys.                          *
C*************************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, set on LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
```

```
C*
C                     ENDSR
C*
C*************************************************************
C*    @ U P D A T    S U B R O U T I N E                     *
C*                                                           *
C*    Read the records inputted from the subfile             *
C*    Add, Change, or Delete records to the STATES file      *
C*************************************************************
C*
C           @UPDAT    BEGSR
C*
C                     MOVE *OFF      *IN99
C*
C* Process all changed subfile records
C*
C           *IN99     DOWEQ*OFF
C                     READCSFD681A                  99
C           *IN99     IFEQ *ON
C                     LEAVE
C                     END
C*
C* If the hidden key field is greater than *blanks, it means
C* that this subfile record existed in the STATES file, and
C* was loaded into the subfile in the @LOAD subroutine. If
C* the @KEY field is blank, it meand the record did not
C* exist in the STATES file, but was added.  If the hidden
C* key @KEY is not blanks, but the state code field STATCD
C* is blanks, it means the user wants to delete the record.
C*
C* Update/Delete Routine
C* Delete the record from STATES file first, then, if you are
C* updating a record, write the changed record back to the file.
C*
C           @KEY      IFGT *BLANKS
C*
C*  Switch to the 2nd occurrence of the data structure so that
C*  the chain will not overwrite the fields STATCD and STATDS
C*  that you will be updating from the subfile.
C*
C           2         OCUR SAVDS
C           @KEY      CHAINRSTATES              98
C           *IN98     IFEQ *OFF
C                     DELETRSTATES
C                     END
C*
C* Go back to the 1st occurrence of the data structure, which
C* is where the fields that you will be updating are stored.
C*
C           1         OCUR SAVDS
C           STATCD    IFGT *BLANKS
C                     WRITERSTATES
C                     END
C*
C*  Add Routine
C*  Ensure that the key field STATCD does not already exist,
C*  since the file STATES has a UNIQUE key.
```

```
C*
C                       ELSE
C*
C            STATCD     CHAINRSTATES              98
C            *IN98      IFEQ *ON
C                       WRITERSTATES
C                       END
C                       END
C*
C                       END
C*
C                       ENDSR
C*
C*************************************************************
C*    * I N Z S R   S U B R O U T I N E                      *
C*                                                           *
C*    Retrieve the starting line and position for the        *
C*    window. Check to make sure that these values are       *
C*    valid. If not, put the closest valid number in.        *
C*************************************************************
C*
C            *INZSR     BEGSR
C*
C*  Initialize the relative record number field for the subfile
C*  to 0. Turn on 35, so that the subfile will show the
C*  subfile end message.
C*
C                       Z-ADD0          @RRN    50
C                       MOVE *ON        *IN35
C*
C                       ENDSR
C*
C*************************************************************
C*    @ L O A D   S U B R O U T I N E                        *
C*                                                           *
C*    Load the subfile with all of the state codes           *
C*    and descriptions.                                      *
C*************************************************************
C*
C            @LOAD      BEGSR
C*
C* Do until the end of file has been reached
C*
C                       MOVE *OFF       *IN99
C                       Z-ADD0          @RRN
C            *LOVAL     SETLLSTATES
C*
C            *IN99      DOWEQ*OFF
C                       READ STATES                   99
C*
C            *IN99      IFEQ *ON
C            @RRN       OREQ 500
C                       LEAVE
C                       END
C*
C                       ADD  1          @RRN
C*
```

```
C*  Move the state code into the hidden field @KEY.
C*
C                     MOVELSTATCD    @KEY
C                     WRITESFD681A
C*
C                     END
C*
C*  Since you initialized the subfile, it will be ok to
C*  display it.
C*
C                     MOVE *ON       *IN30
C*
C*  Position the subfile to the last output record, which
C*  should be the page where the first input line exists.
C*
C                     Z-ADD@RRN      @REC
C*
C*  Now add 1 to the subfile @REC field. @REC should now
C*  be pointed to the first input subfile line. The cursor
C*  will be positioned here.
C*
C                     ADD  1         @REC
C                     ENDSR
C*
C*************************************************************
C*    @ V A L    S U B R O U T I N E                         *
C*                                                           *
C*    Read the records inputted from the subfile             *
C*    Add, Change, or Delete records to the STATES file      *
C*************************************************************
C*
C           @VAL      BEGSR
C*
C                     MOVE *OFF      *IN99
C*
C* Process all changed subfile records
C*
C           *IN99     DOWEQ*OFF
C                     READCSFD681A                  99
C           *IN99     IFEQ *ON
C                     LEAVE
C                     END
C*
C           STATCD    IFGE 'QA'
C           STATCD    ANDLE'Q9'
C                     MOVE *ON       @ERROR  1
C                     END
C*
C                     END
C*
C                     ENDSR
```

The program has been modified to first go through a validation routine to ensure no state codes start with a “Q” before it performs the update routine. If any states do start with a “Q”, the program sets on an error flag that tells it to redisplay the subfile instead of updating it.

Notice what the validation routine does:

```
C           @VAL      BEGSR
C                     MOVE *OFF      *IN99
C           *IN99     DOWEQ*OFF
C                     READCSFD681A                  99
C           *IN99     IFEQ *ON
C                     LEAVE
C                     END
C           STATCD    IFGE 'QA'
C           STATCD    ANDLE'Q9'
C                     MOVE *ON       @ERROR  1
C                     END
C                     END
C                     ENDSR
```

This routine operates like the update routine. The program reads all changed subfile records. Should one of the state codes start with a "Q," then it sets on an error flag. If the error flag is on when the program exits this routine, it redisplays the subfile instead of performing the update subroutine.

However, this program won't work correctly. It validates the state codes, and if a state starts with a "Q," the program redisplays the subfile. But this is where the problem lays. Once the subfile is redisplayed, each record is now reset to the unchanged mode.

If the user keys in 20 new state codes, and one of them starts with a "Q," the validation routine reads in the 20 changed subfile records. It finds one that starts with a "Q," so it sets on the error flag. The subfile is redisplayed. If the user corrects the state code in error, the only state code that is maintained is the one the user changed, not all 20 of them. This record is the only one that has been flagged as changed since the last time the subfile was displayed.

If the user does not change the state code in error, then the validation routine would not read any records. Since no records would be in error, the routine would not have to read any, and the update subroutine would be executed. This routine also would read no records.

If there is an error, all the additions or changes the user makes would be lost. To demonstrate this process, call SFR681. Now add the state code QA and some sort of description. Your display would look like this:

```
  xx/xx/xx              Update State Codes                        QPGMR
  xx:xx:xx                                                        SFR681



State Code  Description

  ND        NORTH DAKOTA
  NM        NEW MEXICO
  NV        NEVADA
  SC        SOUTH CAROLINA
  SD        SOUTH DAKOTA
  TN        TENNESSEE
  TX        TEXAS
  VT        VERMONT
  WA        WASHINGTON
  QA        BAD STATE CODE
                                                                  More...



 F3-Exit
```

Now press Enter. The subfile is redisplayed. Press Enter again without keying anything into the fields. Your display should look something like this:

```
  xx/xx/xx              Update State Codes                        QPGMR
  xx:xx:xx                                                        SFR681



State Code  Description

  ND        NORTH DAKOTA
  NM        NEW MEXICO
  NV        NEVADA
  SC        SOUTH CAROLINA
  SD        SOUTH DAKOTA
  TN        TENNESSEE
  TX        TEXAS
  VT        VERMONT
  WA        WASHINGTON
  __        ______________
                                                                  More...



 F3-Exit
```

You lost what you keyed in. This program is not working correctly. The reason QA was not shown to be in error the second time you pressed Enter is because it was not flagged as changed. It was flagged the first time the subfile was displayed, but when the program displayed the subfile again, this "changed record" information was lost.

The Subfile Next Changed Record

To correct this problem, you need to flag a subfile record as changed, and if the program displays it again on the screen, not lose that attribute.

You can correct the problem with a keyword called the Subfile Next Change (SFLNXTCHG). This keyword is specified on the subfile format, not the subfile control format because the SFLNXTCHG information will be unique to each subfile record. It is actually stored in the record, but cannot be viewed on the screen, similar to a hidden field.

The SFLNXTCHG keyword has been added to program SFR682:

```
A*============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD682) SRCFILE(XXX/QDDSSRC)
A*
A*============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03)
A          R SFD682A                    SFL
A  39                                   SFLNXTCHG
A            STATCD    R        B  8  3REFFLD(RSTATES/STATCD *LIBL/STATES)
A                                       COLOR(WHT)
A  41                                   DSPATR(RI)
A            STATDS    R        B  8 13REFFLD(RSTATES/STATDS *LIBL/STATES)
A                                       COLOR(WHT)
A            @KEY      R        H      REFFLD(RSTATES/STATCD *LIBL/STATES)
A          R SFD682B                    SFLCTL(SFD682A)
A                                       SFLSIZ(0500)
A                                       SFLPAG(0010)
A                                       OVERLAY
A  30                                   SFLDSP
A                                       SFLDSPCTL
A  31                                   SFLINZ
A                                       SFLRNA
A  35                                   SFLEND(*MORE)
A            @REC            4  OH      SFLRCDNBR(CURSOR)
A                                   1  4DATE
A                                       EDTCDE(Y)
A                                       COLOR(BLU)
A                                   2  4TIME
A                                       COLOR(BLU)
```

```
A                                   1 26'Update State Codes'
A                                       COLOR(WHT)
A                                   1 69USER
A                                       COLOR(BLU)
A                                   2 69'SFR682'
A                                       COLOR(BLU)
A                                   6  1'State Code'
A                                       COLOR(PNK)
A                                       DSPATR(UL)
A                                   6 13'Description    '
A                                       COLOR(PNK)
A                                       DSPATR(UL)
A          R TRAILER
A                                  23  2'F3-Exit'
A                                       COLOR(WHT)


F*=============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR682) SRCFILE(XXX/QRPGSRC)
F*
F*=============================================================
FSTATES  UF  E           K        DISK                      A
FSFD682  CF  E                    WORKSTN
F                                        @RRN  KSFILE SFD682A
ISAVDS     E DSSTATES                    2
C*
C*  Write the command key line
C*
C                     WRITETRAILER
C*
C*   Initialize the subfile with 500 non-active records
C*
C                     MOVE *ON       *IN31
C                     WRITESFD682B
C                     MOVE *OFF      *IN31
C*
C*   Load records into the subfile
C*
C                     EXSR @LOAD
C*
C*   Display The Subfile
C*
C           @DISP     TAG
C                     EXFMTSFD682B
C                     EXSR @CMD
C*
C*   Validate the state codes. Make sure none of them start
C*   with the letter  Q .
C*
C                     MOVE *OFF      @ERROR
C                     EXSR @VAL
C           @ERROR    IFEQ *ON
C                     GOTO @DISP
C                     END
```

```
C*
C*   Read through the subfile and see how many records
C*   were maintained versus how many read operations it took.
C*
C                     EXSR @UPDAT
C*
C*********************************************************
C*    @ C M D   S U B R O U T I N E                      *
C*                                                       *
C*    Executed after every EXFMT or display read op code.  *
C*    Used to process command keys.                      *
C*********************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, set on LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C                     ENDSR
C*
C*********************************************************
C*    @ U P D A T    S U B R O U T I N E                 *
C*                                                       *
C*    Read the records inputted from the subfile         *
C*    Add, Change, or Delete records to the STATES file  *
C*********************************************************
C*
C           @UPDAT    BEGSR
C*
C                     MOVE *OFF      *IN99
C*
C* Process all changed subfile records
C*
C           *IN99     DOWEQ*OFF
C                     READCSFD682A                 99
C           *IN99     IFEQ *ON
C                     LEAVE
C                     END
C*
C* If the hidden key field is greater than *blanks, it means
C* that this subfile record existed in the STATES file, and
C* was loaded into the subfile in the @LOAD subroutine. If
C* the @KEY field is blank, it meand the record did not
C* exist in the STATES file, but was added.  If the hidden
C* key @KEY is not blanks, but the state code field STATCD
C* is blanks, it means the user wants to delete the record.
C*
C* Update/Delete Routine
C* Delete the record from STATES file first, then, if you are
C* updating a record, write the changed record back to the file.
C*
C           @KEY      IFGT *BLANKS
C*
```

```
C*  Switch to the 2nd occurrence of the data structure so that
C*  the chain will not overwrite the fields STATCD and STATDS
C*  that you will be updating from the subfile.
C*
C           2         OCUR SAVDS
C           @KEY      CHAINRSTATES              98
C           *IN98     IFEQ *OFF
C                     DELETRSTATES
C                     END
C*
C* Go back to the 1st occurrence of the data structure, which
C* is where the fields that you will be updating are stored.
C*
C           1         OCUR SAVDS
C           STATCD    IFGT *BLANKS
C                     WRITERSTATES
C                     END
C*
C*  Add Routine
C*  Ensure that the key field STATCD does not already exist,
C*  since the file STATES has a UNIQUE key.
C*
C                     ELSE
C*
C           STATCD    CHAINRSTATES              98
C           *IN98     IFEQ *ON
C                     WRITERSTATES
C                     END
C                     END
C*
C                     END
C*
C                     ENDSR
C*
C*********************************************************
C*    * I N Z S R   S U B R O U T I N E                  *
C*                                                       *
C*    Retrieve the starting line and position for the    *
C*    window. Check to make sure that these values are   *
C*    valid. If not, put the closest valid number in.    *
C*********************************************************
C*
C           *INZSR    BEGSR
C*
C*  Initialize the relative record number field for the subfile
C*  to 0. Turn on 35, so that the subfile will show the
C*  subfile end message.
C*
C                     Z-ADD0         @RRN    50
C                     MOVE *ON       *IN35
C*
C                     ENDSR
C*
C*********************************************************
C*    @ L O A D   S U B R O U T I N E                    *
C*                                                       *
C*    Load the subfile with all of the state codes       *
```

```
C*    and descriptions.                                          *
C*************************************************************
C*
C           @LOAD     BEGSR
C*
C* Do until the end of file has been reached
C*
C                     MOVE *OFF      *IN99
C                     Z-ADD0         @RRN
C           *LOVAL    SETLLSTATES
C*
C           *IN99     DOWEQ*OFF
C                     READ STATES                  99
C*
C           *IN99     IFEQ *ON
C           @RRN      OREQ 500
C                     LEAVE
C                     END
C*
C                     ADD  1         @RRN
C*
C*  Move the state code into the hidden field @KEY.
C*
C                     MOVELSTATCD    @KEY
C                     WRITESFD682A
C*
C                     END
C*
C*  Since you initialized the subfile, it will be ok to
C*  display it.
C*
C                     MOVE *ON       *IN30
C*
C*  Position the subfile to the last output record, which
C*  should be the page where the first input line exists.
C*
C                     Z-ADD@RRN      @REC
C*
C*  Now add 1 to the subfile @REC field. @REC should now
C*  be pointed to the first input subfile line. The cursor
C*  will be positioned here.
C*
C                     ADD  1         @REC
C                     ENDSR
C*
C*************************************************************
C*    @ V A L    S U B R O U T I N E                          *
C*                                                            *
C*    Read the records inputted from the subfile              *
C*    Add, Change, or Delete records to the STATES file       *
C*************************************************************
C*
C           @VAL      BEGSR
C*
C                     MOVE *OFF      *IN99
C*
C* Process all changed subfile records
```

```
C*
C           *IN99     DOWEQ*OFF
C                     READCSFD682A                  99
C           *IN99     IFEQ *ON
C                     LEAVE
C                     END
C*
C           STATCD    IFGE 'QA'
C           STATCD    ANDLE'Q9'
C                     MOVE *ON       @ERROR  1
C                     MOVE *ON       *IN41
C                     END
C*
C                     MOVE *ON       *IN39
C                     UPDATSFD682A
C                     MOVE *OFF      *IN41
C                     END
C*
C                     MOVE *OFF      *IN39
C*
C                     ENDSR
```

The DDS changes were easy.

```
A  39                                  SFLNXTCHG
```

Notice that indicator 39 has been specified to activate the SFLNXTCHG keyword. The program has control over when this keyword is put into effect.

The program changes were just as easy as the DDS changes.

```
C           @VAL      BEGSR
C                     MOVE *OFF      *IN99
C           *IN99     DOWEQ*OFF
C                     READCSFD682A                  99
C           *IN99     IFEQ *ON
C                     LEAVE
C                     END
C           STATCD    IFGE 'QA'
C           STATCD    ANDLE'Q9'
C                     MOVE *ON       @ERROR  1
C                     MOVE *ON       *IN41
C                     END
C                     MOVE *ON       *IN39
C                     UPDATSFD682A
C                     MOVE *OFF      *IN41
C                     END
C                     MOVE *OFF      *IN39
C                     ENDSR
```

In the validation routine, the program is reading all changed records. Then it tests for a bad state code. If the state code is invalid, the program sets on indicator 41, which will reverse image the state code. Next it turns on the SFLNXTCHG indicator and updates all the changed subfile records, not just the ones in error. After the update,

the program turns off indicator 41 so that a good state code will not be in reverse image.

The reason the program updates all changed records with the SFLNXTCHG keyword is that if one record is bad, it redisplays all of them. If the program redisplays a changed subfile record, and does not specify SFLNXTCHG, it would not be in the change mode the next time it reads the subfile. Therefore, the program updates every changed record in the subfile with the SFLNXTCHG record. Even if the user does not key into this record again, it will be picked up on a subsequent READC operation.

To test this program out, call SFR682. Attempt to add an invalid state code and press Enter. Your display will look like this:

```
   xx/xx/xx              Update State Codes                        QPGMR
   xx:xx:xx                                                        SFR682


State Code  Description

  ND        NORTH DAKOTA
  NM        NEW MEXICO
  NV        NEVADA
  SC        SOUTH CAROLINA
  SD        SOUTH DAKOTA
  TN        TENNESSEE
  TX        TEXAS
  VT        VERMONT
  WA        WASHINGTON
  QA        BAD STATE CODE
                                                                More...



 F3-Exit
```

This time only the record that is not in error is displayed in reverse image. Keep pressing the Enter key. Notice how you do not lose the information like you did before. It is redisplayed every time. Now change the state code QA to OK. Press Enter. Notice that state OK was added to the file.

You should use the SFLNXTCHG keyword for this reason. If your program ever has to redisplay a subfile, but you do not want to lose the changed record status, you can use the SFLNXTCHG keyword to update each subfile record.

This chapter previously mentioned that a maintenance program may not be the best alternative because it holds a lock on the last database record read. Since the STATES file in this program was described as an update file, whenever the program loaded

the subfile records, it locked each record. When the program is through loading the subfile records, it continues to have a lock on the last record read in from the STATES file. This lock would keep another user from pulling up the display at the same time. His display would "hang" until the program released the lock unless you coded a routine to check for a lock condition first.

You can keep a lock from being placed on the records when your program is loading them by placing an "N" in position 53 of the READ statement in the load routine. You can see this process in program SFR683:

```
F*=============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR683) SRCFILE(XXX/QRPGSRC)
F*
F*=============================================================
FSTATES  UF  E           K        DISK                      A
FSFD682  CF  E                    WORKSTN
F                                        @RRN  KSFILE SFD682A
ISAVDS     E DSSTATES                    2
C*
C*  Write the command key line
C*
C                     WRITETRAILER
C*
C*   Initialize the subfile with 500 non-active records
C*
C                     MOVE *ON       *IN31
C                     WRITESFD682B
C                     MOVE *OFF      *IN31
C*
C*   Load records into the subfile
C*
C                     EXSR @LOAD
C*
C*   Display The Subfile
C*
C           @DISP     TAG
C                     EXFMTSFD682B
C                     EXSR @CMD
C*
C*   Validate the state codes. Make sure none of them start
C*   with the letter  Q .
C*
C                     MOVE *OFF      @ERROR
C                     EXSR @VAL
C           @ERROR    IFEQ *ON
C                     GOTO @DISP
C                     END
C*
C*   Read through the subfile and see how many records
C*   were maintained versus how many read operations it took.
C*
C                     EXSR @UPDAT
C*
```

```
C*********************************************************
C*    @ C M D   S U B R O U T I N E                      *
C*                                                       *
C*    Executed after every EXFMT or display read op code.  *
C*    Used to process command keys.                      *
C*********************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, set on LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C                     ENDSR
C*
C*********************************************************
C*    @ U P D A T    S U B R O U T I N E                 *
C*                                                       *
C*    Read the records inputted from the subfile         *
C*    Add, Change, or Delete records to the STATES file  *
C*********************************************************
C*
C           @UPDAT    BEGSR
C*
C                     MOVE *OFF      *IN99
C*
C* Process all changed subfile records
C*
C           *IN99     DOWEQ*OFF
C                     READCSFD682A                  99
C           *IN99     IFEQ *ON
C                     LEAVE
C                     END
C*
C* If the hidden key field is greater than *blanks, it means
C* that this subfile record existed in the STATES file, and
C* was loaded into the subfile in the @LOAD subroutine. If
C* the @KEY field is blank, it meand the record did not
C* exist in the STATES file, but was added.  If the hidden
C* key @KEY is not blanks, but the state code field STATCD
C* is blanks, it means the user wants to delete the record.
C*
C* Update/Delete Routine
C* Delete the record from STATES file first, then, if you are
C* updating a record, write the changed record back to the file.
C*
C           @KEY      IFGT *BLANKS
C*
C*  Switch to the 2nd occurrence of the data structure so that
C*  the chain will not overwrite the fields STATCD and STATDS
C*  that you will be updating from the subfile.
C*
C           2         OCUR SAVDS
C           @KEY      CHAINRSTATES              98
```

```
C           *IN98     IFEQ *OFF
C                     DELETRSTATES
C                     END
C*
C* Go back to the 1st occurrence of the data structure, which
C* is where the fields that you will be updating are stored.
C*
C           1         OCUR SAVDS
C           STATCD    IFGT *BLANKS
C                     WRITERSTATES
C                     END
C*
C*  Add Routine
C*  Ensure that the key field STATCD does not already exist,
C*  since the file STATES has a UNIQUE key.
C*
C                     ELSE
C*
C           STATCD    CHAINRSTATES              98
C           *IN98     IFEQ *ON
C                     WRITERSTATES
C                     END
C                     END
C*
C                     END
C*
C                     ENDSR
C*
C*************************************************************
C*    * I N Z S R   S U B R O U T I N E                      *
C*                                                           *
C*    Retrieve the starting line and position for the        *
C*    window. Check to make sure that these values are       *
C*    valid. If not, put the closest valid number in.        *
C*************************************************************
C*
C           *INZSR    BEGSR
C*
C*  Initialize the relative record number field for the subfile
C*  to 0. Turn on 35, so that the subfile will show the
C*  subfile end message.
C*
C                     Z-ADD0         @RRN    50
C                     MOVE *ON       *IN35
C*
C                     ENDSR
C*
C*************************************************************
C*    @ L O A D   S U B R O U T I N E                        *
C*                                                           *
C*    Load the subfile with all of the state codes           *
C*    and descriptions.                                      *
C*************************************************************
C*
C           @LOAD     BEGSR
C*
C* Do until the end of file has been reached
```

```
C*
C                     MOVE *OFF      *IN99
C                     Z-ADD0         @RRN
C           *LOVAL    SETLLSTATES
C*
C           *IN99     DOWEQ*OFF
C                     READ STATES              N    99
C*
C           *IN99     IFEQ *ON
C           @RRN      OREQ 500
C                     LEAVE
C                     END
C*
C                     ADD  1         @RRN
C*
C*  Move the state code into the hidden field @KEY.
C*
C                     MOVELSTATCD    @KEY
C                     WRITESFD682A
C*
C                     END
C*
C*  Since you initialized the subfile, it will be ok to
C*  display it.
C*
C                     MOVE *ON       *IN30
C*
C*  Position the subfile to the last output record, which
C*  should be the page where the first input line exists.
C*
C                     Z-ADD@RRN      @REC
C*
C*  Now add 1 to the subfile @REC field. @REC should now
C*  be pointed to the first input subfile line. The cursor
C*  will be positioned here.
C*
C                     ADD  1         @REC
C                     ENDSR
C*
C*************************************************************
C*    @ V A L    S U B R O U T I N E                         *
C*                                                           *
C*    Read the records inputted from the subfile             *
C*    Add, Change, or Delete records to the STATES file      *
C*************************************************************
C*
C           @VAL      BEGSR
C*
C                     MOVE *OFF      *IN99
C*
C* Process all changed subfile records
C*
C           *IN99     DOWEQ*OFF
C                     READCSFD682A                     99
C           *IN99     IFEQ *ON
C                     LEAVE
C                     END
```

```
C*
C           STATCD    IFGE 'QA'
C           STATCD    ANDLE'Q9'
C                     MOVE *ON       @ERROR  1
C                     MOVE *ON       *IN41
C                     END
C*
C                     MOVE *ON       *IN39
C                     UPDATSFD682A
C                     MOVE *OFF      *IN41
C                     END
C*
C                     MOVE *OFF      *IN39
C*
C                     ENDSR
```

This situation brings up another problem that was mentioned previously. In this problem scenario, your program loads the records into a subfile. At the same time, another user loads the same records into their subfile. The other user then updates a record. Then you decide to update the same record. You would have just overlaid the information in that record with old data.

For this reason, update subfiles should be page-at-a-time subfiles. If a user has too many records in their subfile to play around with, they may have them on their screen an extended amount of time. Other users may have already made quite a few changes to these records while that user was making changes. This user would then overlay all the work performed by the other users. Designing for locks in your subfile program should not be taken lightly.

In the previous examples, you used the load-all technique for loading an input/update subfile. The method required that you pay special attention to the subfile size (SFLSIZ) value, since you could only add or display the number of subfile records as determined by the value of the subfile size.

If you want more flexibility in the number of subfile records, you can display and/or add in an input/update subfile, you can also use the expanding load technique. Instead of using the SFLINZ to initialize the subfile with blank subfile records, you will revert back to using the SFLCLR. If the subfile is an input-only subfile, you need to write enough blank subfile records to fill up one screen. If your SFLPAG were set to 10, you would write 10 blank subfile records to the subfile. You can also set the SFLSIZ value to 11 in this case. If the user rolls forward to enter more subfile records, you can simply write 10 more blank records to the subfile and present page two of the subfile to the user.

If you are using an update subfile, you would load the subfile with one page of displayable/updatable subfile records. If the user presses the roll key, you load another screen of displayable/updatable records. The user can change any records

on the screen, and if he needs to add a new record, he can roll until he reaches a screen that has no data in the fields.

You could also expand on this technique by always making the first subfile record on a page an input only, and load the displayable/updatable records after this one. Doing this only requires a minor change to the load process and allows the user to have at least one line on each display for adding a new record.

Let's look at a function used once before and see how it can help you design user-friendly input and update screens. The function was the Subfile Drop (SFLDROP) keyword, which allows you to initially specify that only the first line of the subfile is to be displayed. You saw how this function affected output fields, but you can also control input/update fields the same way. Suppose you have an order entry system in which almost every time the quantity of an item would be 1. In this case, you can default the quantities to 1. But this change doesn't keep the user from having to tab over the quantity field.

To keep the user from having to see the quantity field on the display, you can design your subfile records so that the quantity is on the second line of a subfile record. You can then use the SFLDROP keyword to specify that the second (or more) lines of the subfile should not be displayed. All the user would have to do is to fast key item numbers. You can see how this basic program design can be accomplished in SFD690 and SFR690.

```
A*============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD690) SRCFILE(XXX/QDDSSRC)
A*
A*============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03)
A          R SFD690A                    SFL
A            @KEY      R         H      REFFLD(RSTATES/STATCD *LIBL/STATES)
A            FLD001        15A  B  6  2COLOR(WHT)
A                                   7 24'QTY:'
A                                       COLOR(BLU)
A            FLD002        11Y 2B  7 29EDTWRD('         .  ')
A                                       COLOR(WHT)
A          R SFD690B                    SFLCTL(SFD690A)
A                                       OVERLAY
A  30                                   SFLDSP
A                                       SFLDSPCTL
A  31                                   SFLINZ
A  35                                   SFLEND(*MORE)
A                                       SFLRNA
A                                       SFLDROP(CF10)
A                                       SFLSIZ(0500)
A                                       SFLPAG(0007)
A                                   1  4DATE
```

```
A                                       EDTCDE(Y)
A                                       COLOR(BLU)
A                                  2  4TIME
A                                       COLOR(BLU)
A                                  1 69USER
A                                       COLOR(BLU)
A                                  2 69'SFR690'
A                                       COLOR(BLU)
A                                  5 24'Item Description                  -
A                                        '
A                                       COLOR(PNK)
A                                       DSPATR(UL)
A                                  5  2'Item Number'
A                                       COLOR(PNK)
A                                       DSPATR(UL)
A                                  1 27'Order Entry Item Entry'
A                                       COLOR(WHT)
A          R TRAILER
A                                 23  1' F3-Exit     F10-Toggle Quantity F-
A                                       ield'
A                                       COLOR(WHT)

F*=============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR690) SRCFILE(XXX/QRPGSRC)
F*
F*=============================================================
FSFD690  CF  E                    WORKSTN
F                                        @RRN  KSFILE SFD690A
C*
C*   Initialize the subfile with 500 non-active records
C*
C                     MOVE *ON       *IN31
C                     WRITESFD690B
C                     MOVE *OFF      *IN31
C                     MOVE *ON       *IN30
C*
C*  Write the command key line
C*
C                     WRITETRAILER
C*
C*   Display The Subfile
C*
C                     EXFMTSFD690B
C                     EXSR @CMD
C*
C*********************************************************
C*    @ C M D   S U B R O U T I N E                      *
C*                                                       *
C*    Executed after every EXFMT or display read op code.  *
C*    Used to process command keys.                      *
C*********************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, set on LR and exit.
```

```
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C                     ENDSR
C*
C*************************************************************
C*    * I N Z S R   S U B R O U T I N E                      *
C*                                                           *
C*    Retrieve the starting line and position for the        *
C*    window. Check to make sure that these values are       *
C*    valid. If not, put the closest valid number in.        *
C*************************************************************
C*
C           *INZSR    BEGSR
C*
C*  Initialize the relative record number field for the subfile
C*  to 0. Turn on 35, so that the subfile will show the
C*  subfile end message.
C*
C                     Z-ADD0         @RRN    50
C                     MOVE *ON       *IN35
C*
C                     ENDSR
```

Call SFR690. Your display should look like this:

```
  xx/xx/xx              Order Entry Item Entry                  QPGMR
  xx:xx:xx                                                      SFR690


Item Number           Item Description
_______________
_______________
_______________
_______________
_______________
_______________
_______________
_______________
_______________
_______________
_______________
_______________
_______________
_______________
                                                                 More...


F3-Exit     F10-Toggle Quantity Field
```

The user only sees, and therefore only has to key, the item number. They don't have to worry about Tabbing or Field Exiting over a quantity field.

If the user need to enter a quantity other than 1, they can simply press the F10 key, which controls the toggle. The subfile now changes to folded mode.

```
  xx/xx/xx              Order Entry Item Entry                  QPGMR
  xx:xx:xx                                                      SFR690

Item Number          Item Description
_______________
                     QTY: ___________
_______________
                     QTY: ___________
_______________
                     QTY: ___________
_______________
                     QTY: ___________
_______________
                     QTY: ___________
_______________
                     QTY: ___________
_______________
                     QTY: ___________
                                                                More...

F3-Exit     F10-Toggle Quantity Field
```

Now the user can enter the quantity for just one item, then toggle the mode back to drop by pressing the F10 key. Your data entry operators are happy because they only have to contend with fields that they must enter. Any optional fields can be accessed by changing the subfile to the folded mode.

If you notice that when you are in the folded mode and you press the Enter key, the subfile jumps back into the dropped mode, which was the initial mode. This problem is inherent to using the SFLDROP and SFLFOLD keywords. If the Enter key is pressed, the subfile returns to the original mode. This situation can be confusing to users because the subfile appears to be jumping to a different mode at random. But there is a way you can control it.

The Subfile Mode Keyword

Look at the changes in SFD691 and SFR691.

```
A*============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD691) SRCFILE(XXX/QDDSSRC)
A*
A*============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03)
A          R SFD691A                    SFL
A            @KEY      R        H       REFFLD(RSTATES/STATCD *LIBL/STATES)
A            FLD001        15A  B  6  2COLOR(WHT)
A                                   7 24'QTY:'
A                                       COLOR(BLU)
A            FLD002        11Y 2B  7 29EDTWRD('         .  ')
A                                       COLOR(WHT)
A          R SFD691B                    SFLCTL(SFD691A)
A                                       SFLSIZ(0500)
A                                       SFLPAG(0007)
A                                       OVERLAY
A                                       SFLMODE(&@MODE)
A  30                                   SFLDSP
A                                       SFLDSPCTL
A  31                                   SFLINZ
A  35                                   SFLEND(*MORE)
A                                       SFLRNA
A  36                                   SFLDROP(CF10)
A N36                                   SFLFOLD(CF10)
A                                   1  4DATE
A                                       EDTCDE(Y)
A                                       COLOR(BLU)
A                                   2  4TIME
A                                       COLOR(BLU)
A                                   1 69USER
A                                       COLOR(BLU)
A                                   2 69'SFR691'
A                                       COLOR(BLU)
A                                   5 24'Item Description                 -
A                                         '
A                                       COLOR(PNK)
A                                       DSPATR(UL)
A                                   5  2'Item Number'
A                                       COLOR(PNK)
A                                       DSPATR(UL)
A                                   1 27'Order Entry Item Entry'
A                                       COLOR(WHT)
A            @MODE          1A  H
A          R TRAILER
A                                 23  1' F3-Exit     F10-Toggle Quantity F-
A                                       ield'
A                                       COLOR(WHT)

F*============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR691) SRCFILE(XXX/QRPGSRC)
F*
F*============================================================
```

```
FSFD691  CF  E                    WORKSTN
F                                        @RRN  KSFILE SFD691A
C*
C*  Write the command key line
C*
C                     WRITETRAILER
C*
C*   Initialize the subfile with 500 non-active records
C*
C                     MOVE *ON       *IN31
C                     WRITESFD691B
C                     MOVE *OFF      *IN31
C                     MOVE *ON       *IN30
C*
C*   Get the subfile mode to set up for either subfile drop
C*   or subfile fold.
C*
C           @MODE     IFEQ *OFF
C                     MOVE *OFF      *IN36
C                     ELSE
C                     MOVE *ON       *IN36
C                     END
C*
C*   Display The Subfile
C*
C                     EXFMTSFD691B
C                     EXSR @CMD
C*
C*************************************************************
C*    @ C M D   S U B R O U T I N E                          *
C*                                                           *
C*    Executed after every EXFMT or display read op code.    *
C*    Used to process command keys.                          *
C*************************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, set on LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C                     ENDSR
C*
C*************************************************************
C*    * I N Z S R   S U B R O U T I N E                      *
C*                                                           *
C*    Retrieve the starting line and position for the        *
C*    window. Check to make sure that these values are       *
C*    valid. If not, put the closest valid number in.        *
C*************************************************************
C*
C           *INZSR    BEGSR
C*
C*  Initialize the relative record number field for the subfile
```

```
C*  to 0. Turn on 35, so that the subfile will show the
C*  subfile end message.
C*
C                     Z-ADD0         @RRN    50
C                     MOVE *ON       *IN35
C*
C                     ENDSR
```

Notice that the display file has the subfile mode keyword:

```
A                                      SFLMODE(&@MODE)
```

In the field descriptions for the record format, the field @mode is described.

```
A            @MODE          1A  H
```

@MODE is described as a one position, alphanumeric, hidden field. Essentially, @MODE is a logical field that contains either a "0" or a "1". If the subfile is in the folded mode when control is returned to the program, @MODE will contain a "0", and if it was in the drop mode, @MODE will contain a "1". Look at how the SFLDROP and SFLFOLD keywords are described.

```
A  36                                  SFLDROP(CF10)
A N36                                  SFLFOLD(CF10)
```

Both keywords are controlled by F10. This is a requirement when you specify both the fold and drop keywords on the same format. They must both be controlled by the same function key.

Notice that the SFLDROP and SFLFOLD keywords are conditioned with indicator 36. If 36 is on, the subfile is displayed in a dropped mode. If 36 is off, the subfile is displayed in a folded mode.

Inside the RPG program, you now have a way to determine what mode the subfile was in when control was passed to the program. You also have a way to ensure that when your program outputs the subfile, it sets it to the mode it was in when the program received control. The program performs this check in the following routine:

```
C           @MODE     IFEQ *OFF
C                     MOVE *OFF      *IN36
C                     ELSE
C                     MOVE *ON       *IN36
C                     END
```

When the subfile is initially displayed, it is in the dropped mode because indicator 36 is off. If the user pressed the F10 key to toggle to the folded mode, @MODE would contain a "1", or the logical condition *ON. In this routine, which is executed right before the program outputs the subfile, indicator 36 is set on if @MODE is *ON. Now when the subfile is displayed, it will be in the folded mode, which was the mode it was in when the program received control.

To see an example of this process, call SFR691. Your display should look like this:

```
  xx/xx/xx              Order Entry Item Entry                    QPGMR
  xx:xx:xx                                                        SFR691

Item Number           Item Description
_______________
_______________
_______________
_______________
_______________
_______________
_______________
_______________
_______________
_______________
_______________
_______________
_______________
_______________
                                                                   More...

F3-Exit     F10-Toggle Quantity Field
```

Now press the F10 key, which will toggle you into the folded mode. Your display will look like this:

```
  xx/xx/xx              Order Entry Item Entry                    QPGMR
  xx:xx:xx                                                        SFR691

Item Number           Item Description
_______________
                      QTY: _____________
_______________
                      QTY: _____________
_______________
                      QTY: _____________
_______________
                      QTY: _____________
_______________
                      QTY: _____________
_______________
                      QTY: _____________
_______________
                      QTY: _____________
                                                                   More...

F3-Exit     F10-Toggle Quantity Field
```

Now press the Enter key. Notice how the display now stays in the folded mode. This situation shows one of the few reasons left to use the MODE keyword. It was formerly used for subfile records number location in conjunction with the cursor position, but the SFLCSRRRN keyword has now easily replaced this function. You may still see the subfile mode keyword used for this purpose in programs you are maintaining. If so, it may be worth your while to convert the program to use the SFLCSRRRN keyword, which will remove quite a bit of code, and make the program more readable.

The Subfile Cursor Progression Keyword

In V2R3 of OS/400, you have a new subfile keyword. It is called Subfile Cursor Progression (SFLCSRPRG). It is used to allow you to control what field the cursor is to be positioned to.

Program SFR692 has been set up to demonstrate this keyword.

```
A*==============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD692) SRCFILE(XXX/QDDSSRC)
A*
A*==============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03)
A          R SFD692A                    SFL
A            @KEY      R         H      REFFLD(RSTATES/STATCD *LIBL/STATES)
A            FLD001        15A  B  6  2COLOR(WHT)
A            FLD002        11Y 2B  6 22EDTWRD('         .  ')
A                                       COLOR(WHT)
A                                       SFLCSRPRG
A            FLD003        11Y 2B  6 41EDTWRD('         .  ')
A                                       COLOR(WHT)
A          R SFD692B                    SFLCTL(SFD692A)
A                                       SFLSIZ(0500)
A                                       SFLPAG(0007)
A                                       OVERLAY
A  30                                   SFLDSP
A                                       SFLDSPCTL
A  31                                   SFLINZ
A  35                                   SFLEND(*MORE)
A                                       SFLRNA
A                                   1  4DATE
A                                       EDTCDE(Y)
A                                       COLOR(BLU)
A                                   2  4TIME
A                                       COLOR(BLU)
A                                   1 69USER
A                                       COLOR(BLU)
A                                   2 69'SFR692'
A                                       COLOR(BLU)
```

```
A                                   5  2'Item Number    '
A                                       COLOR(PNK)
A                                       DSPATR(UL)
A                                   1 27'Order Entry Item Entry'
A                                       COLOR(WHT)
A                                   5 22'Quantity    '
A                                       COLOR(PNK)
A                                       DSPATR(UL)
A                                   5 41'Price       '
A                                       COLOR(PNK)
A                                       DSPATR(UL)
A          R TRAILER
A                                  23  1' F3-Exit'
A                                       COLOR(WHT)


F*=============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR692) SRCFILE(XXX/QRPGSRC)
F*
F*=============================================================
FSFD692  CF  E                     WORKSTN
F                                         @RRN  KSFILE SFD692A
C*
C*  Write the command key line
C*
C                     WRITETRAILER
C*
C*   Initialize the subfile with 500 non-active records
C*
C                     MOVE *ON       *IN31
C                     WRITESFD692B
C                     MOVE *OFF      *IN31
C                     MOVE *ON       *IN30
C*
C*   Display The Subfile
C*
C                     EXFMTSFD692B
C                     EXSR @CMD
C*
C*********************************************************
C*    @ C M D   S U B R O U T I N E                      *
C*                                                       *
C*    Executed after every EXFMT or display read op code.  *
C*    Used to process command keys.                      *
C*********************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, set on LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
```

```
C*
C                       ENDSR
C*
C*************************************************************
C*    * I N Z S R   S U B R O U T I N E                      *
C*                                                           *
C*    Retrieve the starting line and position for the        *
C*    window. Check to make sure that these values are       *
C*    valid. If not, put the closest valid number in.        *
C*************************************************************
C*
C            *INZSR    BEGSR
C*
C*  Initialize the relative record number field for the subfile
C*  to 0. Turn on 35, so that the subfile will show the
C*  subfile end message.
C*
C                       Z-ADD0          @RRN    50
C                       MOVE *ON        *IN35
C*
C                       ENDSR
```

As you can see, all you have to do is to place the SFLCSRPRG keyword on the field that you want the cursor to progress to after a Field Exit or Tab key is pressed. In this example, the SFLCSRPRG keyword has been placed on the quantity field. When the screen comes up, the first field to which the cursor is positioned is the item number field. In the following examples "X" represents the cursor.

```
  xx/xx/xx              Order Entry Item Entry                  QPGMR
  xx:xx:xx                                                      SFR692

Item Number      Quantity        Price
X
                                                                More...

F3-Exit
```

The cursor progression keyword does not affect where the cursor is placed until the cursor lands on a field with the cursor progression keyword. So the cursor will be positioned on the item number field. Once you press Tab or Field Exit, the cursor progresses as normal, in this case from left to right. So the next field that the cursor is placed on is the quantity field.

```
  xx/xx/xx              Order Entry Item Entry                    QPGMR
  xx:xx:xx                                                        SFR692

Item Number      Quantity      Price
                 X
                                                                 More...

F3-Exit
```

Since the quantity field has the cursor progression keyword on it, the cursor is told that it should progress to the next quantity field. So if you use the Field Exit or Tab key now, the cursor will drop to the next quantity field.

```
  xx/xx/xx              Order Entry Item Entry                    QPGMR
  xx:xx:xx                                                        SFR692

Item Number      Quantity        Price
_______________  _____________   _____________
_______________  X____________   _____________
_______________  _____________   _____________
_______________  _____________   _____________
_______________  _____________   _____________
_______________  _____________   _____________
_______________  _____________   _____________
                                                                   More...

F3-Exit
```

It will now progress all the way down the quantity field whenever a Tab or Field Exit key is pressed. If you had wanted the cursor to start and progress all the way down the screen on the quantity field, you could have specified the position cursor keyword, as well as the cursor progression keyword, on the quantity field.

You can use the cursor progression keyword to cause the cursor to progress up and down on the screen, instead of left to right. Using the screen from the last example, you could cause the cursor to drop all the way through the item number field, then cause the cursor to go to the first quantity field and drop all the way down, then cause the cursor to position itself to the price field and drop all the way down. You can do this by placing the subfile cursor progression keyword on all three input fields, as in SFR693.

```
A*==============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD693) SRCFILE(XXX/QDDSSRC)
A*
A*==============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03)
A          R SFD693A                    SFL
A            @KEY      R         H      REFFLD(RSTATES/STATCD *LIBL/STATES)
A            FLD001        15A  B  6  2COLOR(WHT)
A                                       SFLCSRPRG
A            FLD002        11Y 2B  6 22EDTWRD('         .  ')
A                                       COLOR(WHT)
A                                       SFLCSRPRG
```

```
A            FLD003        11Y 2B  6 41EDTWRD('         .  ')
A                                      COLOR(WHT)
A                                      SFLCSRPRG
A          R SFD693B                   SFLCTL(SFD693A)
A                                      SFLSIZ(0500)
A                                      SFLPAG(0007)
A                                      OVERLAY
A  30                                  SFLDSP
A                                      SFLDSPCTL
A  31                                  SFLINZ
A  35                                  SFLEND(*MORE)
A                                      SFLRNA
A                                  1  4DATE
A                                      EDTCDE(Y)
A                                      COLOR(BLU)
A                                  2  4TIME
A                                      COLOR(BLU)
A                                  1 69USER
A                                      COLOR(BLU)
A                                  2 69'SFR693'
A                                      COLOR(BLU)
A                                  5  2'Item Number    '
A                                      COLOR(PNK)
A                                      DSPATR(UL)
A                                  1 27'Order Entry Item Entry'
A                                      COLOR(WHT)
A                                  5 22'Quantity    '
A                                      COLOR(PNK)
A                                      DSPATR(UL)
A                                  5 41'Price       '
A                                      COLOR(PNK)
A                                      DSPATR(UL)
A          R TRAILER
A                                 23  1' F3-Exit'
A                                      COLOR(WHT)


F*==============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR693) SRCFILE(XXX/QRPGSRC)
F*
F*==============================================================
FSFD693  CF  E                     WORKSTN
F                                         @RRN  KSFILE SFD693A
C*
C*  Write the command key line
C*
C                     WRITETRAILER
C*
C*   Initialize the subfile with 500 non-active records
C*
C                     MOVE *ON       *IN31
C                     WRITESFD693B
C                     MOVE *OFF      *IN31
C                     MOVE *ON       *IN30
```

```
C*
C*   Display The Subfile
C*
C                     EXFMTSFD693B
C                     EXSR @CMD
C*
C*********************************************************
C*    @ C M D   S U B R O U T I N E                      *
C*                                                       *
C*    Executed after every EXFMT or display read op code.  *
C*    Used to process command keys.                      *
C*********************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, set on LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C                     ENDSR
C*
C*********************************************************
C*    * I N Z S R   S U B R O U T I N E                  *
C*                                                       *
C*    Retrieve the starting line and position for the    *
C*    window. Check to make sure that these values are   *
C*    valid. If not, put the closest valid number in.    *
C*********************************************************
C*
C           *INZSR    BEGSR
C*
C*  Initialize the relative record number field for the subfile
C*  to 0. Turn on 35, so that the subfile will show the
C*  subfile end message.
C*
C                     Z-ADD0         @RRN    50
C                     MOVE *ON       *IN35
C*
C                     ENDSR
```

When you call SFR693, the cursor is positioned on the item number field.

```
  xx/xx/xx              Order Entry Item Entry                  QPGMR
  xx:xx:xx                                                      SFR693

Item Number      Quantity       Price
X
                                                                 More...

F3-Exit
```

Since the item number field has the subfile cursor progression keyword specified on it, the cursor will progress to the next item number field.

```
  xx/xx/xx              Order Entry Item Entry                  QPGMR
  xx:xx:xx                                                      SFR693

Item Number      Quantity       Price

X
                                                                 More...

F3-Exit
```

The cursor will continue this way until the last item number field on the screen.

```
  xx/xx/xx                Order Entry Item Entry                    QPGMR
  xx:xx:xx                                                          SFR693

Item Number      Quantity       Price
                 ____________   ____________
                 ____________   ____________
                 ____________   ____________
                 ____________   ____________
                 ____________   ____________
                 ____________   ____________
X                ____________   ____________
                                                                More...

F3-Exit
```

Once the cursor reaches the last item number field on the screen, it will progress to the first quantity field.

```
  xx/xx/xx              Order Entry Item Entry                    QPGMR
  xx:xx:xx                                                        SFR693

Item Number      Quantity      Price
                 X
                                                                   More...

F3-Exit
```

Since the quantity field has the cursor progression keyword specified, the cursor will progress down the screen on the quantity field. Once the cursor reaches the last quantity field, it will progress to the first price field. Since the price field also has the cursor progression keyword specified, the cursor will progress down the screen on the price field. This keyword gives a columnar progression of the cursor, instead of the standard right to left.

7

Message Subfiles

What Are Message Subfiles?

If you have never used message subfiles, you are missing out on one of the most powerful, yet simple, forms of displaying multiple detail messages.

Message subfiles allow you to easily display multiple error conditions to a display station operator so that all error conditions can be corrected the first time. Contrast this situation to displaying only one message at a time, correcting the error, pressing Enter, and receiving a different error condition, which can be very aggravating.

Look at SFD700 and SFR700 to see how to define the message subfile.

```
A*==============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD700) SRCFILE(XXX/QDDSSRC)
A*
A*==============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03)
A          R SFD700A                    SFL
A                                       SFLMSGRCD(24)
A            @KEY                       SFLMSGKEY
A            @PGMQ                      SFLPGMQ
A          R SFD700B                    SFLCTL(SFD700A)
```

```
A                                       SFLDSP
A                                       SFLDSPCTL
A                                       SFLINZ
A N03                                   SFLEND
A                                       SFLSIZ(0002)
A                                       SFLPAG(0001)
A            @PGMQ                      SFLPGMQ
A          R SFD700C
A                                       BLINK
A                                       OVERLAY
A                                  1  4DATE
A                                       EDTCDE(Y)
A                                       COLOR(BLU)
A                                  1 70USER
A                                       COLOR(BLU)
A                                  2  4TIME
A                                       COLOR(BLU)
A                                  2 70'SFR700'
A                                       COLOR(BLU)
A                                  1 25'Example Of A Message Subfile'
A                                       COLOR(WHT)
A                                 23  2'F3-Exit'
A                                       COLOR(WHT)
A                                  9  1'Enter Your Last Name........:'
A                                       COLOR(PNK)
A                                  7  1'Enter Your First Name.......:'
A                                       COLOR(PNK)
A                                 11  1'Enter Your Social Security #:'
A                                       COLOR(PNK)
A                                 13  1'Enter The State If Residence:'
A                                       COLOR(PNK)
A            @FNAME        30   B  7 32COLOR(WHT)
A            @LNAME        30   B  9 32COLOR(WHT)
A            @SSN           9Y 0B 11 32COLOR(WHT)
A                                       EDTWRD('   -  -    ')
A            @STATE         2   B 13 32COLOR(WHT)


F*=============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR700) SRCFILE(XXX/QRPGSRC)
F*
F*=============================================================
FSFD700  CF  E                     WORKSTN
FSTATES  IF  E           K         DISK
I*
I* This data structure contains the binary field descriptions
I* for the message API's
I*
I           IDS
I                                     B   1   40@STK
I                                     B   5   80@LEN
I                                     B   9  120@ERR
C*
C* Write the message subfile
```

```
C*
C                     WRITESFD700B
C*
C* Write and read the display file
C*
C                     EXFMTSFD700C
C                     EXSR @CMD
C*
C* Clear the message subfile
C*
C                     MOVEL*BLANKS   @KEY
C                     CALL 'QMHRMVPM'@PARM1
C*
C*  Validate First Name
C*
C           @FNAME    IFEQ *BLANKS
C                     MOVE 'MSG0001' @MSGID
C                     CALL 'QMHSNDPM'@PARM2
C                     ENDIF
C*
C*  Validate Last Name
C*
C           @LNAME    IFEQ *BLANKS
C                     MOVE 'MSG0002' @MSGID
C                     CALL 'QMHSNDPM'@PARM2
C                     ENDIF
C*
C* Validate Social Security Number
C*
C           @SSN      IFEQ 0
C                     MOVE 'MSG0003' @MSGID
C                     CALL 'QMHSNDPM'@PARM2
C                     ENDIF
C*
C* Validate State Code
C*
C           @STATE    CHAINSTATES               90
C           *IN90     IFEQ *ON
C                     MOVE 'MSG0004' @MSGID
C                     CALL 'QMHSNDPM'@PARM2
C                     ENDIF
C*
C*************************************************************
C*    * I N Z S R   S U B R O U T I N E                      *
C*                                                           *
C*    Retrieve the starting line and position for the        *
C*    window. Check to make sure that these values are       *
C*    valid. If not, put the closest valid number in.        *
C*************************************************************
C*
C           *INZSR    BEGSR
C*
C* Movel  *  into the program queue field, which will tell
C* the system to use this program's message queue.
C*
C                     MOVEL'*'       @PGMQ
C*
```

```
C*  Specify that the message type should be diagnostic
C*
C                     MOVEL'*DIAG'   @TYPE
C*
C* Set up the message file and library into a 20 byte field
C*
C                     MOVEL'*LIBL'   @LIB   10
C                     MOVE @LIB      @MSGF  20
C                     MOVEL'MSGF'    @MSGF
C*
C* This field will be used to tell the message API to remove
C* all messages from the program queue, similar to a SFLCLR.
C*
C                     MOVEL'*ALL'    @RMV   10
C*
C                     MOVE *BLANKS   @KEY    4
C*
C* This parameter list is used to clear the program queue
C*
C           @PARM1    PLIST
C                     PARM           @PGMQ
C                     PARM           @STK
C                     PARM           @KEY
C                     PARM           @RMV
C                     PARM           @ERR
C*
C* This parameter list is used to send a message to the
C* program queue
C*
C           @PARM2    PLIST
C                     PARM           @MSGID  7
C                     PARM           @MSGF
C                     PARM           @DTA   80
C                     PARM           @LEN
C                     PARM           @TYPE  10
C                     PARM           @PGMQ
C                     PARM           @STK
C                     PARM           @KEY
C                     PARM           @ERR
C*
C                     ENDSR
C*
C*************************************************************
C*    @ C M D   S U B R O U T I N E                          *
C*                                                           *
C*    Executed after every EXFMT or display read op code.    *
C*    Used to process command keys.                          *
C*************************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, seton LR and exit.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
```

```
C*
C                         ENDSR
```

To see an example of a message subfile, call program SFR700.

Your display should look like this:

```
   xx/xx/xx            Example Of A Message Subfile                QPGMR
   xx:xx:xx                                                        SFR700



Enter Your First Name.......:  ____________________________

Enter Your Last Name........:  ____________________________

Enter Your Social Security #:  __________

Enter The State If Residence:  __



 F3-Exit
```

This program performs a check to ensure that a first name, last name, and social security number have been keyed. The program also validates that the state code entered is in the STATES master file.

Press the Enter key without keying in any data. Your display should look like this:

```
  xx/xx/xx            Example Of A Message Subfile                QPGMR
  xx:xx:xx                                                        SFR700



Enter Your First Name.......:  ____________________________

Enter Your Last Name........:  ____________________________

Enter Your Social Security #:  __________

Enter The State If Residence:  __

 F3-Exit
 The First Name Field Is Blank                                          +
```

You'll receive an error that states that the first name field is blank. But that is not the only error message you'll receive. Use the roll forward key. Your display should look like this:

```
  xx/xx/xx            Example Of A Message Subfile                QPGMR
  xx:xx:xx                                                        SFR700



Enter Your First Name.......:  ____________________________

Enter Your Last Name........:  ____________________________

Enter Your Social Security #:  __________

Enter The State If Residence:  __



 F3-Exit
 The Last Name Field Is Blank                                          +
```

Now the error message says that the last name field is blank. Keep rolling forward. You will get a message about an invalid social security number, and another message about an invalid state. A message subfile is essentially a way to display multiple messages on a single display screen.

Message subfiles are similar to regular subfiles in that they make it much easier to display multiple messages, just like how regular subfiles make it easier to display multiple lines of data. However, message subfiles are defined differently in DDS and are structurally different from the regular subfiles.

Differences Between Message Subfiles and Regular Subfiles

The biggest difference between message subfiles versus the other subfiles that you have looked at is that you won't clear and load the message subfile in the typical manner. Message subfiles utilize a program message queue. A program message queue is a queue to which you can send messages. Clearing a message subfile is accomplished by removing all the messages out of a program message queue to which the subfile is linked. Loading a message subfile can be performed automatically. If there are messages in the program message queue when you display the subfile, they are loaded. You do not have to have a "LOAD" routine coded in the program for a message subfile like you do for the regular subfiles. When you display a message

subfile, it can know to automatically load itself with the records from the program message queue to which it is linked.

You will also not see a "subfile clear" routine like you did in the regular subfiles. Clearing a message subfile is accomplished by removing all the messages from the program message queue to which the subfile is linked.

The Message Subfile Format

The message subfile needs a few different keywords specified than the ones used for the regular subfiles. First, on the subfile format, the Subfile Message Record (SFLMSGRCD), Subfile Message Key (SFLMSGKEY), and Subfile Program Queue (SFLPGMQ) are required keywords.

The SFLMSGRCD keyword allows you to specify on which line the subfile is to start. Usually, you would specify line 24, which is the standard most people use. If you specified line 24 as the start line number, then the subfile page on a standard 24 line display would have to be one. You could not specify a subfile page higher than one because the subfile would attempt to display the second record at line 25, which would result in an error.

The SFLMSGKEY allows you to specify a message key that is used to catalog the message. It can be used as a message key to retrieve the message data from a message file. You could also leave it blank if you wanted to hard code your messages, and not use a message file from which to extract your messages.

In the example, you actually use a message file from which you extract the messages, so you will be placing the key to the message that you want to display in this field.

The SFLPGMQ keyword allows you to specify the field which contains the program message queue name. The program message queue is where the subfile messages are initially written, and is where the message subfile is going retrieve the message to display when it is called upon. When you "clear" the message subfile, you will actually be removing the messages from the program message queue. Using this keyword is how you link the message subfile to its respective program message queue. In the example, you specify the program queue name to be an "*", which tells the system to use the program message queue of the program that is currently running.

☛ Note on the subfile program queue keyword and V2R3 of OS/400

The subfile program queue keyword has been modified under V2R3 of OS/400 to allow its use with the Integrated Language Environment (ILE). The length of the program queue can either be the default of value of 10, or 276 bytes. Using 276 bytes

allows the program queue to be used with the ILE. If you specify a program queue name of 276 bytes, the first 256 bytes must contain the qualified name of the ILE program queue name. Bytes 257 through 266 can optionally contain the ILE module name. Bytes 267 through 276 can optionally contain the bound program name.

If you specify a 276-byte program queue, you are allowed to use a none ILE program queue as long as bytes 257 through 276 are blank.

If you use the program queue keyword on both the subfile format and subfile control format, the program queue definition must be identical between the two.

The Message Subfile Control Format

The subfile control format has the following requirements:

- The SFLDSP, SFLDSPCTL, and SFLINZ keywords must be specified without conditioning indicators.
- The SFLPGMQ may or may not be specified. If you do specify the SFLPGMQ keyword on both the subfile format and the subfile control format, then the message subfile will automatically load itself whenever you display the subfile. If you specify the SFLPGMQ keyword on the subfile format only, then you are responsible for loading the subfile inside the program.

Since there is no reason why you must load the message subfile inside the program, you can code the SFLPGMQ keyword on both the subfile format and the subfile control format. Doing this causes the subfile to automatically load itself and keeps you from having to add a chunk of code into the RPG program to handle the loading of the message subfile. The program is simpler, shorter and easier to maintain.

Look at the subfile and subfile control formats in detail. The subfile format looks like this:

```
A          R SFD700A                    SFL
A                                       SFLMSGRCD(24)
A            @KEY                       SFLMSGKEY
A            @PGMQ                      SFLPGMQ
```

The subfile control format looks like this:

```
A          R SFD700B                    SFLCTL(SFD700A)
A                                       SFLDSP
A                                       SFLDSPCTL
A                                       SFLINZ
A N03                                   SFLEND
A                                       SFLSIZ(0002)
```

```
A                                        SFLPAG(0001)
A            @PGMQ                       SFLPGMQ
```

This code comprises the entire subfile! Notice the mandatory SFLMSGRCD, SFLMSGKEY, and SFLPGMQ keywords on the subfile format. These keywords must be in this sequence, or else you will receive an error at compile time.

The starting message line for the subfile has been set to be line 24, as described on the SFLMSGRCD keyword. The message key field will be referenced inside the RPG program as @KEY, and the program message queue will be referenced as @PGMQ.

Notice how you do not have to describe any output fields for the message text itself. The message subfile handles this for you automatically.

Now look at the subfile control format and you'll see the SFLDSP, SFLDSPCTL, and SFLINZ keywords, which are mandatory. Notice the SFLEND keyword. The subfile displays the "Bottom" or " " messages whenever indicator 03 is off. Indicator 03 is the F3 indicator. If F3 is pressed, 03 is turned on, and you exit the program. As long as the program is running and an end of subfile is reached, the program will display the " " or "Bottom" message, which is the same way the load-all subfiles were handled previously. In essence, message subfiles are load-all subfiles.

Notice how the (*more) option was not used for the SFLEND keyword. Since the starting line is 24, you cannot specify the *more option. The *more option places the "MORE..." and "Bottom" text on the line right after the last subfile line on the display. Since the first and last subfile line is line 24, specifying *more would cause the system to attempt to display the text on line 25, resulting in an error. If you moved the starting line up to 23, and specified that only one message subfile record was to be displayed, then you could specify the *more option.

It is mandatory that the subfile size be at least one more than the subfile page. A message subfile is essentially a load-all subfile.

Notice that the SFLPGMQ keyword was specified on the subfile control format, in addition to specifying it on the subfile format. Here, you are telling the system to automatically load the subfile records for you.

Now look at some of the program code in detail. Notice the statement in the code:

```
C                     WRITESFD700B
```

This statement is where the message subfile is displayed. At this point, the system checks the program message queue linked to this subfile. If there are any messages in this queue, the system will load them automatically into the subfile before it displays the subfile. The first time you execute this code, the program queue should not have any messages.

Execute the display format SFD700C, which is the entry screen. If the user presses F3, then the program ends. But if the user presses Enter, the message subfile is cleared. This process is coded in the following lines:

```
C                     CALL 'QMHRMVPM'@PARM1
```

Program QMHRMVPM is an Application Programming Interface (API). It allows you to remove messages from a program message queue. Remember, clearing the message subfile is done by removing messages in the program message queue to which the subfile is linked. You will need to pass a series of parameters to this program that are described in the parameter list @PARM1. @PARM1 is described in the *INZSR subroutine:

```
C           @PARM1    PLIST
C                     PARM            @PGMQ
C                     PARM            @STK
C                     PARM            @KEY
C                     PARM            @RMV
C                     PARM            @ERR
```

You will set up the values for these fields in the *INZSR.

@PGMQ has the value "*". When you pass this value to the QMHRMVPM program, it knows to clear the messages from the program message queue of this program.

@STK and @ERR are four-byte binary fields. These fields must be passed to the QMHRMVPM program, but they only have to have the value of zero. You can do this in the data structure. Specify that the data structure should be initialized when the program is started. ("I" in position 18 of the data structure specification).

@KEY is a four-byte alphanumeric field. It is initialized with blanks and that is the way to pass it to QMHRMVPM.

The only other field is one called @RMV. @RMV has the value *ALL, which tells the QMHRMVPM program to remove all messages from the program message queue.

In essence, you only need to specify to QMHRMVPM the program queue name and which messages to clear. The other required parameters can be zeros and blanks.

Before this API was available, you would have to clear the program message queue by calling a CL program to perform a RMVMSG command on the program message queue. The RMVMSG command cannot be run called from an RPG program because it is not allowed in this setting. This API is also much more powerful than what it is being used for here. You can find more information on it in the *Systems Programmers Interface Reference.*

After you have cleared the subfile, execute a validation routine to ensure that the first and last name fields are not blank, the social security number is not 0, and the state

code keyed is in the STATES file. In the following example, you can see what happens if one of these error conditions exists.

```
C           @FNAME    IFEQ *BLANKS
C                     MOVE 'MSG0001' @MSGID
C                     CALL 'QMHSNDPM'@PARM2
C                     ENDIF
```

If the first name field is blanks, then "MSG0001" is moved into a field called @MSGID. MSG001 is the message ID of a message record in a message file. Then the program QMHSNDPM is called. This API is similar to QMHRMVPM, except that this API will send a message to a program message queue instead of removing messages. This API is sent a series of parameters that are described in @PARM2 parameter list.

```
C           @PARM2    PLIST
C                     PARM           @MSGID  7
C                     PARM           @MSGF
C                     PARM           @DTA   80
C                     PARM           @LEN
C                     PARM           @TYPE  10
C                     PARM           @PGMQ
C                     PARM           @STK
C                     PARM           @KEY
C                     PARM           @ERR
```

The first field passed, QMHSNDPM, is the message ID field. If the first name field were left blank, this field would contain "MSG0001." It is a seven-byte alphanumeric field. This field is cleared in the same routine that cleared the subfile. This field should be blank when you enter the validation routine because you need to check it after the validation routine to see if any errors occurred. If @MSGID is blank after the validation routine, it means that no errors were encountered. If it is not blank, then there was at least one error.

The next field is @MSGF, which contains the message file name and library name. It is a 20-byte alphanumeric field. The message file name is placed in the first 10 bytes of this field, and the library name is placed in the rightmost 10 bytes. In this example, the message file name is "MSGF." The library name is set to *LIBL, which will cause the program to search the library list for the message file. The field @MSGF in this example would look like this:

```
MSGF      *LIBL
```

The next field is called @DTA. It is an 80-byte field. If you did not want to use a message file, you could hard code the message text in the program, place it in this field, and send it to the QMHSNDPM program. You would leave the @MSGF field blank if you did not want to use a message file. In this example, a message file is used, so this field is left blank and the message text data is extracted from the message file itself.

The next field is @LEN. Since this field is a message file, it can be left 0. It is a four-byte binary field, described by a data structure in the "I" specifications of the program. If this file were not a message file, but message text hard coded into the field @DTA, you would set the value of @LEN to 80. This value is the length of the @DTA field. This field is only used to specify the length of data in the @DTA field, as the QMHSNDPM program will not know that @DTA is 80 bytes long. Again, you need only to put a value into this field if you do not use a message file. Since this example is using a message file, @LEN is set to 0.

Field @TYPE is used to specify what type of message to send. You can send several different types, such as informational, diagnostic, and so on. Here, the message type has been set to *DIAG, which is a diagnostic message.

Field @PGMQ contains the name of the program queue to send the message. As seen earlier, this field contains an "*", which says to send the message to this program's message queue.

The last three fields, @STK, @KEY, and @ERR, can be left zeros and blanks like they were when QMHRMVPM was called

Look at the message send function again.

```
C                     CALL 'QMHSNDPM'@PARM2
```

In this call, the QMHSNDPM program is called to send a message. By specifying a value for the field @MSGF, you indicate that a message file is being used and that the message text can be found in this file. You also indicate which message to retrieve and place in the program message queue. You can do this by specifying the message ID and placing this ID in the field @MSGID. You must also specify that the message should be sent as a diagnostic message because @TYPE field had a value *DIAG.

In this case, QMHSNDPM went out to the message file *LIBL/MSGF and extracted the message text for message ID "MSG0001." It then sent this message as a diagnostic message to the program queue "*", which the system interprets as being this program's message queue.

The validation routine now continues to check the last name field, social security number field, and the state code field.

The program now returns to the top where the subfile is displayed. Before the subfile is displayed, it will be loaded.

Now that you see how the message subfile is functioning, it's time to make a change to it. Now specify that four message lines can be displayed at the same time. The starting message line number will have to be modified. These changes have been made in SFD710 and SFR710.

```
A*============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD710) SRCFILE(XXX/QDDSSRC)
A*
A*============================================================
A                                       DSPSIZ(24 80 *DS3)
A                                       CF03(03)
A          R SFD710A                    SFL
A                                       SFLMSGRCD(20)
A            @KEY                       SFLMSGKEY
A            @PGMQ                      SFLPGMQ
A          R SFD710B                    SFLCTL(SFD710A)
A                                       SFLDSP
A                                       SFLDSPCTL
A                                       SFLINZ
A N03                                   SFLEND
A                                       SFLSIZ(0005)
A                                       SFLPAG(0004)
A            @PGMQ                      SFLPGMQ
A          R SFD710C
A                                       BLINK
A                                       OVERLAY
A                                  1  4DATE
A                                       EDTCDE(Y)
A                                       COLOR(BLU)
A                                  1 70USER
A                                       COLOR(BLU)
A                                  2  4TIME
A                                       COLOR(BLU)
A                                  2 70'SFR710'
A                                       COLOR(BLU)
A                                  1 25'Example Of A Message Subfile'
A                                       COLOR(WHT)
A                                 15  2'F3-Exit'
A                                       COLOR(WHT)
A                                  9  1'Enter Your Last Name........:'
A                                       COLOR(PNK)
A                                  7  1'Enter Your First Name.......:'
A                                       COLOR(PNK)
A                                 11  1'Enter Your Social Security #:'
A                                       COLOR(PNK)
A                                 13  1'Enter The State If Residence:'
A                                       COLOR(PNK)
A            @FNAME        30A  B  7 32COLOR(WHT)
A            @LNAME        30A  B  9 32COLOR(WHT)
A            @SSN           9Y 0B 11 32COLOR(WHT)
A                                       EDTWRD('   -  -    ')
A            @STATE         2A  B 13 32COLOR(WHT)
A                                 18  2'***  Error Messages  ***'
A                                       COLOR(BLU)
```

```
F*==============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR710) SRCFILE(XXX/QRPGSRC)
F*
F*==============================================================
FSFD710  CF  E                    WORKSTN
FSTATES  IF  E           K        DISK
I*
I* This data structure contains the binary field descriptions
I* for the message API's
I*
I           IDS
I                                   B   1   40@STK
I                                   B   5   80@LEN
I                                   B   9  120@ERR
C*
C* Write the message subfile
C*
C                     WRITESFD710B
C*
C* Write and read the display file
C*
C                     EXFMTSFD710C
C                     EXSR @CMD
C*
C* Clear the message subfile
C*
C                     MOVEL*BLANKS   @KEY
C                     CALL 'QMHRMVPM'@PARM1
C*
C*  Validate First Name
C*
C           @FNAME    IFEQ *BLANKS
C                     MOVE 'MSG0001' @MSGID
C                     CALL 'QMHSNDPM'@PARM2
C                     ENDIF
C*
C*  Validate Last Name
C*
C           @LNAME    IFEQ *BLANKS
C                     MOVE 'MSG0002' @MSGID
C                     CALL 'QMHSNDPM'@PARM2
C                     ENDIF
C*
C* Validate Social Security Number
C*
C           @SSN      IFEQ 0
C                     MOVE 'MSG0003' @MSGID
C                     CALL 'QMHSNDPM'@PARM2
C                     ENDIF
C*
C* Validate State Code
C*
C           @STATE    CHAINSTATES               90
C           *IN90     IFEQ *ON
C                     MOVE 'MSG0004' @MSGID
```

```
C                     CALL 'QMHSNDPM'@PARM2
C                     ENDIF
C*
C*********************************************************
C*    * I N Z S R   S U B R O U T I N E                  *
C*                                                       *
C*    Retrieve the starting line and position for the    *
C*    window. Check to make sure that these values are   *
C*    valid. If not, put the closest valid number in.    *
C*********************************************************
C*
C           *INZSR    BEGSR
C*
C* Movel  *  into the program queue field, which will tell
C* the system to use this program's message queue.
C*
C                     MOVEL'*'       @PGMQ
C*
C*  Specify that the message type should be diagnostic
C*
C                     MOVEL'*DIAG'   @TYPE
C*
C* Set up the message file and library into a 20 byte field
C*
C                     MOVEL'*LIBL'   @LIB   10
C                     MOVE @LIB      @MSGF  20
C                     MOVEL'MSGF'    @MSGF
C*
C* This field will be used to tell the message API to remove
C* all messages from the program queue, similar to a SFLCLR.
C*
C                     MOVEL'*ALL'    @RMV   10
C*
C                     MOVE *BLANKS   @KEY    4
C*
C* This parameter list is used to clear the program queue
C*
C           @PARM1    PLIST
C                     PARM           @PGMQ
C                     PARM           @STK
C                     PARM           @KEY
C                     PARM           @RMV
C                     PARM           @ERR
C*
C* This parameter list is used to send a message to the
C* program queue
C*
C           @PARM2    PLIST
C                     PARM           @MSGID  7
C                     PARM           @MSGF
C                     PARM           @DTA   80
C                     PARM           @LEN
C                     PARM           @TYPE  10
C                     PARM           @PGMQ
C                     PARM           @STK
C                     PARM           @KEY
C                     PARM           @ERR
```

```
C*
C                       ENDSR
C*
C*************************************************************
C*    @ C M D   S U B R O U T I N E                          *
C*                                                           *
C*    Executed after every EXFMT or display read op code.    *
C*    Used to process command keys.                          *
C*************************************************************
C*
C           @CMD        BEGSR
C*
C* If F3 was pressed, seton LR and exit.
C*
C           *IN03       IFEQ *ON
C                       MOVE *ON        *INLR
C                       RETRN
C                       END
C*
C                       ENDSR
```

In this example, you first specify that the starting subfile message line is line 20:

```
A                                       SFLMSGRCD(20)
```

Then, you specify that the subfile page should be four. You must also change the value of subfile size to at least 5.

```
A                                       SFLSIZ(0005)
A                                       SFLPAG(0004)
```

To see the changes to this program, call SFR710. Your display should look like this:

```
  xx/xx/xx            Example Of A Message Subfile                QPGMR
  xx:xx:xx                                                        SFR710



Enter Your First Name.......:  ____________________________

Enter Your Last Name........:  ____________________________

Enter Your Social Security #:  ___________

Enter The State If Residence:  __

 F3-Exit


 ***  Error Messages  ***
```

Now press the Enter key. Your display should look like this:

```
  xx/xx/xx            Example Of A Message Subfile                QPGMR
  xx:xx:xx                                                        SFR710



Enter Your First Name.......:  ____________________________

Enter Your Last Name........:  ____________________________

Enter Your Social Security #:  ___________

Enter The State If Residence:  __

 F3-Exit


 ***  Error Messages  ***
 The First Name Field Is Blank
 The Last Name Field Is Blank
 The Social Security Number Field Is Zero
 You Have Keyed An Invalid State Code
```

Now move the cursor down to the message about the invalid social security number and press the Help key. Your screen should look like this:

```
                        Additional Message Information

Message ID . . . . . . :   MSG0003       Severity . . . . . . . :   00
Message type . . . . . :   Diagnostic
Date sent  . . . . . . :   07/23/94      Time sent  . . . . . . :   11:14:59

Message . . . . :   The Social Security Number Field Is Zero
If you do not have a social security number because you are an illegal alien,
  please apply for a green card. Once you have a green card, you will be
  assigned a number that you can key into this field. Until then, you may not
  use this program.









                                                                        Bottom
Press Enter to continue.

F3=Exit   F6=Print   F10=Display messages in job log
F11=Display message details   F12=Cancel   F21=Select assistance level
```

Combining message files and message subfiles is powerful. Although you do not have to use a message file with a message subfile, utilizing a message file can allow you to describe a problem and the steps needed to rectify the problem with incredible detail. Think of what you could really do with a message file. Suppose you have a new user who is using a program and receives an error message. Suppose she keyed an invalid state code. The message cannot only tell her that it is invalid, but give directions as to how to rectify the problem. For instance, you could tell the user that a state code is invalid if it does not reside in the STATES file. If the state code needs to be set up in the STATES file, please contact the Data Base Administration Department, and they will set the state up for you. This message would be better than displaying text on the bottom of the display that says “INVALID STATE,” which would result in a phone call from the user to the I/S department.

Now, key something into the first and last name fields, and enter a valid state and press Enter. The display should look like this.

```
  xx/xx/xx            Example Of A Message Subfile                QPGMR
  xx:xx:xx                                                        SFR710



Enter Your First Name.......:  JOHN                          

Enter Your Last Name........:  DOE                           

Enter Your Social Security #:  __________

Enter The State If Residence:  CA

 F3-Exit


 ***  Error Messages  ***

 The Social Security Number Field Is Zero
```

Now the only message you get is an invalid social security number message. Every time you redisplay this screen, all of the previous messages are removed from this program's message queue, which in essence clears the subfile. Then you can revalidate the data on the screen, writing out only the errors.

Hopefully, this section has opened your eyes to the power of message subfiles. They can very easily display multiple messages. Combine this power with that of a message file, and you can display multiple messages with complex instructions very easily.

8

Subfiles and Open Query File

The Open Query File (OPNQRYF) command is a very powerful way to select and sort records from a file. You can design very potent inquiry programs using the subfile/OPNQRYF combination. Suppose you have a one million record customer master file and you are creating a customer master inquiry subfile program to inquire over the file. In this program, you give the user the option of keying in a phone number, or a zip code, or a state code. The user can select customer master records based on the criteria they keyed in. Output of the subfile must be sorted in customer number sequence.

OPNQRYF is the command you should use. You wouldn't have to key any of these select/sort criteria in your subfile program. Your program would be set up to read the customer master file as if you were starting with the first record and reading every record into the subfile. You must trick the program into reading only the selected/sorted records that were generated through the OPNQRYF by first issuing an OVRDBF share(*YES). This trick would point the subfile program not to the customer master file, but the customer master file that was opened with the OPNQRYF.

If you used the OPNQRYF for selecting the records, you could actually make the initial subfile screen be displayed nearly instantaneously (even a one million-record file). And if you wanted the subfile sorted, the initial subfile screen may still be displayed nearly instantaneously. But the OPNQRYF command may not be able to do this without some help.

The process of selecting/sorting records using OPNQRYF is different for a load-all subfile than it is for an expanding subfile. You have to tell the OPNQRYF command which one you will be using by utilizing the OPTIMIZE parameter on the OPNQRYF command.

You can also greatly speed the process of selecting/sorting if you supply OPNQRYF with an access path that matches the select/omit criteria, or the key field. But don't just start creating logical files. First, get to know the query optimizer, and see how it goes about selecting and sorting a file based on existing access paths. Otherwise, you may be creating logical files that will never be used.

The Query Optimizer

Whenever the query function is executed, such as with an OPNQRYF command, SQL/400, and Query/400, it will do more than just process your command. It will attempt to optimize itself to process faster and increase performance. The query component responsible for this task of increasing performance is known as the query optimizer. The optimizer has the responsibility for good database performance during the execution of a query function. The optimizer's objective is to come up with the best access method to the data being requested.

The query optimizer has a challenging task, It must find out what the query is being requested to do, attempt to devise a plan to execute the current query request, and process the request with the plan. All of these tasks must be performed in a very restricted time frame. The optimizer must have a quick interactive response, and utilize machine resources efficiently. Essentially, the optimizer is a tradeoff. The query function spends some additional time coming up with an implementation plan, but this time is offset by the benefit of greatly reducing the amount of time it takes to process the request as a whole.

To increase the processing speed, the query optimizer will determine the best possible access to the data. The possible accesses include:

1. Reading records directly from a file.

This access is desirable especially when there is no required sorting of data, and the optimizer detects that most of the records in the file will be processed. This type of access is also known as dynamic processing. The query optimizer can "block" a good number of records to the requesting program since it is reading the physical file top down. Blocking allows a group of records to be brought into main memory at the same time, so that the selection process can be greatly reduced.

2. Reading records through an existing access path.

This access plan is desirable in a couple of situations. First, if you are selecting only one or two records out of the file, the optimizer can essentially chain to an access path built over the selection field and bring in the records. Also, if you are selecting on a field using the RANGE or selecting if a field is equal to specific data, the optimizer can perform a "read equal" on the access path to make the selections. Another condition would be if you were not doing any selection, and you specified a key. If you already had an access path with the key built, the optimizer could use the existing key.

3. Create a temporary access path, then perform the query selection off of the temporary access path.

This method would be desirable if you were selecting on a field and wanted the output sorted off of that field also. In this case, no logical files would have been found with a key structure based on the select/key field on the OPNQRYF command.

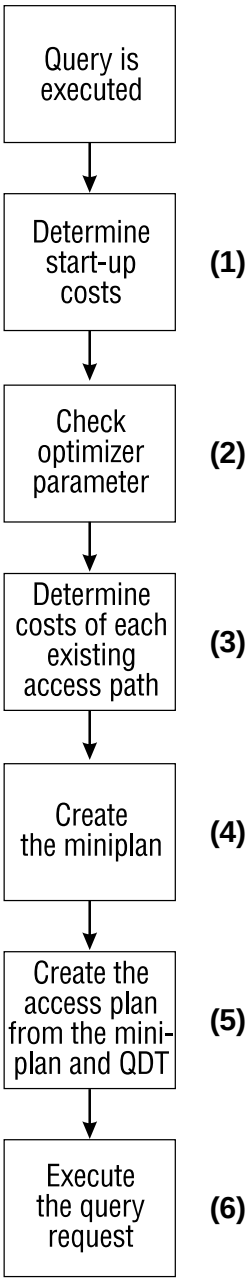
4. Create an access path from an existing access path.

This method could be desirable if you were sorting a file based on more than two fields. If you had an access path already built on the first key field or fields, the optimizer could use this path and cut down on the sort time since most of it is already sorted. It might also be used if you were doing a selection and a sort. If there were an access path already created on the selection fields, the query optimizer could select this existing path and create a temporary path sorted in the order specified on the query command.

5. Use the query sort routine.

This method is desirable when you will be sorting a large number of records. When the sort is completed, you have a copy of the entire file in sorted sequence, not just an access path. This allows the sequential reading of the file in the high-level program, which is much faster when you are working with a large number of records.

The following is a diagram of what happens when you execute a query function.



The Optimizer Strategy

The above chart shows the optimizer strategy, which is performed in the following steps.

Step 1: Calculating Startup Costs

Calculate the File Size

At run time, the query optimizer calculates the startup costs for the request. Essentially, it is finding out how large the file is. It retrieves attributes on the file, such as file size, from internally stored information. If the file size is small, the optimizer will not to spend too much time, if any, on optimization. The query function may be able to select and sort a small file faster than the optimizer could find the fastest access method. On the other hand, if the file is large, the optimizer will spend a good amount of time determining a good implementation plan.

Calculate the Filter Factor

The optimizer also comes up with a weighted measure of expected records to retrieve in the selection, which is known as the filter factor. The optimizer calculates the filter factor by scanning the query select field to see what operators are being used.

If the query select has an equal operation, the optimizer will anticipate that 10 percent of the records will be selected. It calculates 10 percent per equal operation. Look at the following example:

```
QRYSLT (FLD1 *eq 24 *or FLD1 *eq 25)
```

In this example, the optimizer estimates a filter factor of 20 percent, which means the optimizer anticipates 20% of the records in the file will be selected, since there are two equal operations. If we were working with a 1,000-record file, the optimizer would anticipate 200 records to be selected. If it were a million-record file, it would estimate 200,000 records to be selected. The differences in file size plays a big role in which access plan the optimizer is going to look for. If the optimizer anticipated only 200 records to be selected, it's possible that the optimizer would exit the optimization process and scratch build a quick index.

If the query select statement contains a < or > (less than or greater than) the optimizer will anticipate selecting 33 percent of the records in the file. If the select contains a *NE (not equal) operation, the optimizer will anticipate that 90 percent of the records in the file will be selected. The optimizer will anticipate selecting 10 percent of the records in the file for each values field. The optimizer will anticipate selecting 25 percent of the records in the file for a range operation.

The optimizer now has some idea as to how many records it will be selecting. This number is very important in the selection of an implementation plan.

Step 2: Check For Optimizer Biases

You can use an OPNQRYF parameter to bias the optimizer. The possible biases are:

- ***ALLIO**: the *ALLIO parameter specifies that the optimizer should find the access path that would yield all of the selected records in the shortest amount of time.

- ***FIRSTIO**: the *FIRSTIO parameter specifies that the optimizer should attempt to find the access path that would yield the first buffer of records in the shortest amount of time.

- *MINWAIT: the *MINWAIT parameter tells the optimizer to attempt to come up with the access path that would cause the smallest amount of wait time between multiple retrievals.

Why Would You Want to Specify an Optimizer Bias?

Suppose you had an OPNQRYF command performing a select and sort on a million-record open accounts receivable file to be used with a subfile inquiry program. Say you want to select if the invoice due date is less than today, so you would be pulling up past due invoices. Then, you want to sort by customer number. From this sort you will load the subfile, so all the past invoices for each customer show up together.

Assume that the user probably will not need to roll more than three pages. Therefore, you will create an expanding subfile. Now, assume the following access paths are available. There is a logical by customer number and a logical by invoice due date.

On the OPNQRYF optimizer keyword, you would want to either specify *FIRSTIO or *MINWAIT. In this case you will go with *FIRSTIO since the user isn't going to have to roll very much. By doing so, you would bias the optimizer to use the logical by customer number. If the OPNQRYF used the logical by due date, it could also set a lower limit and read on the date, greatly speeding the selection process. But it would have to read all records with a due date less than today's date, so that it could sort by customer number.

By using the customer number logical, all the optimizer has to do is read the already sorted date logical until it finds enough records with past due invoices to fill up the program buffer. Then it will allow the program to display the first page of the subfile. If you rolled forward, the OPNQRYF would select another buffer full of records. By selecting this logical, the OPNQRYF could return the first screen of records in the minimum amount of time, and it will keep you from having to read the entire file every time.

Suppose the customers are good about paying on time, and you would probably never have more than a few hundred past due invoices out of the one million that you have open. In this case, you might want to look at the subfile program performing a load all. Since you will be loading all selected records, you would want the optimizer to bias itself in selecting and sorting all of the records in the shortest amount of time. In this case, the optimizer would probably choose the logical by due date. It could perform a set lower limits and read a couple of hundred records that matched the selection criteria. Then the optimizer could create a temporary access path and sort those couple of hundred records by customer number. After this has been done, control would pass to the subfile program and you would load all of the selected and sorted records.

In this situation, the OPNQRYF definitely would not want to select the customer number logical. By selecting this approach, you would read one million records every time you loaded the subfile. Even though the records are already sorted, it would still take a lot longer to perform the process if it were to read a by the logical based on customer number. In this case, the OPNQRYF would be reading one million records every time.

Once step one and two have been performed, the optimizer has determined what it is looking for as far as an ideal implementation plan. It will know if it is better to find a logical file built over the select/omit criteria, or the key field. Now it must search and see if the ideal implementation plan already exists in the form of a logical file, and if not, select the best plan available.

Step 3: Determine the Cost of Each Access Path

The optimizer will begin checking any existing logical files, starting with the most recently created. It compares the logical file's select/omit criteria, as well as key structure. The optimizer knows what the ideal logical should look like, but it may not find the ideal logical. Therefore, it compares the logical file to the ideal plan and comes up with a cost of using it. After each logical is compared, the cost to execute the implementation plan with that logical is compared to the best logical file found so far. If the execution cost is better with this logical, this logical becomes the current

best cost method and the previous best is scrapped. If no sorting is required in the query command, the cost of this logical is also compared to the cost of dynamic processing. If an index is needed, it will compare the cost of using this index versus the cost of building a temporary one from scratch.

After each logical is checked for a cost to implement, the optimizer checks to see how much time it has spent looking for a good implementation plan. If it has not spent too much time, and if a good implementation plan has not been found yet, it may continue searching through the logical files. At some point, the optimizer will time out and may not get to check all logical files.

Normally, the optimizer will check about seven logicals before timing out. However, if the physical file is small, or if the selection codes suggest that a minimum of records will be returned, the optimizer will time out earlier. It may also time out if an ideal, or near-ideal access plan is found in one of the logical files.

If the file or expected number of selected records is large, and an ideal access plan has not been found, it may attempt to look at more than seven logicals. If a physical file has 10 or more logicals, the optimizer may not ever select the best logical file for this application, because the optimizer may time out before looking at all the attached logicals. It will analyze logical files in the order of the most recently created. If you know that you have a good logical file that the OPNQRYF should be using, but you suspect it is not, start the debug command (STRDBG), and then run the program. Then, display the joblog (DSPJOBLOG). The joblog will tell you which logical was chosen, and whether or not the optimizer timed out. If the optimizer timed out, and it did not select the logical you thought it should have, you can force the optimizer not to time out by specifying OPTALLAP (*YES) on the OPNQRYF command. Doing so will force the optimizer to look at all access paths built against the physical file.

If you do specify OPTALLAP (*YES), or if the optimizer did not time out, and it still did not choose the logical that you thought it should have, don't worry. There was probably a good reason why the optimizer chose the access plan it did. The optimizer is not a fixed science, and can be very unpredictable at times, but it is very good at what it does.

Step 4: Create the Miniplan

After step three is completed, the optimizer comes up with a miniplan. The miniplan is a place where the access plan method is stored. The optimizer determined what the best access method should have been, but that method may not have existed. So, it will calculate the best method based on the best logical file found, the size of the physical file, and the expected number of selected records. It will then decide to:

- Read the records directly from the file.
- Read the records through an existing access path.
- Create a temporary access path from scratch, and read the records through this access path.
- Create a temporary access path from an existing access path, then read the records from this access path.
- Sort the physical file into a temporary physical file, and read the records from the sorted file.

This information about the access method is stored in the miniplan.

Step 5: Create Access Plan From Miniplan and QDT

The combination of the miniplan and query definition template (QDT) is used by the query optimizer to perform the query function. The miniplan tells the optimizer what access method to use. The query definition plan tells the optimizer how to perform the selection and sort.

If the query function was executed from Query/400 or SQL/400, the access plan (miniplan + QDT) are saved. The next time this process executes this query, it will not have to go through the this process again.

If the query function was executed from the OPNQRYF command, the miniplan is scrapped upon completion of the command. Therefore, the optimizer performs its algorithm to find the best access method every time the OPNQRYF is executed.

Step 6: Execute the Access Plan

The optimizer then takes the miniplan and QDT and begins to execute it.

You can do this in two ways. If *FIRSTIO or *MINWAIT was specified on the query optimizer parameter, the query function will select only enough records to fill up the program buffer. At that time, the query definition will not select any more records unless the high-level program attempts to read past the records in the buffer. If it tries to do so, the query function will select another buffer's worth of records.

If default optimizer parameter *ALLIO was specified, the query function will completely process the access plan and then pass control to the requesting program.

The Optimizer Strategy in a Nutshell

To summarize the optimizer strategy, the optimizer attempts to find out what it is working with, determine how much time should be devoted to optimization, check for any optimizer biases, find out what is already available based on the sort/select and any biases, create the miniplan, combine the miniplan with the query definition template, perform the query function, and (if it was invoked by SQL/400 or Query/400) save the miniplan and QDT.

Tailoring OPNQRYF For Subfiles

The ALL I/O Bias

The *ALLIO option biases the query optimizer to find the selection method to bring in all selected records on the shortest amount of time. You want to specify this option on a subfile load all. *ALLIO is the default parameter.

The First I/O Bias

The OPNQRYF (Open Query File) command has a *FIRSTIO option to optimize the length of time the OPNQRYF has to run before displaying records on a display. This option is especially effective for subfiles which do not perform a load all. Suppose you were using an OPNQRYF to select certain records for loading into a subfile. Without the optimization option, you would have to allow the OPNQRYF to run over the whole file before you even started loading records into the subfile. For a large file, the wait can be substantial.

By using the *FIRSTIO option, the OPNQRYF command does not process until an I/O is requested against the file, Then, it only processes enough records to fill the buffer with the needed number of records. Therefore, if you specified *FIRSTIO, and you need to display 10 records from a million-record file, once 10 records have been selected, they will be displayed without waiting. If you roll forward, the OPNQRYF will select another 10 records from the file. If you rolled backward, then the display file would handle the request since the records are already in the subfile.

The Minimum Wait Bias

The *MINWAIT option is another optimizer selection you can use to bias the OPNQRYF command. With *MINWAIT, you are telling the query optimizer to make an attempt at minimizing the delay before bringing in records. You want to do use this option if you are not loading all of the subfile. But, the user will probably need to roll several times before finding the information needed.

In summary, the *FIRSTIO option is the choice for page-at-a-time subfiles and expanding subfiles. It is not a good choice for load-all subfiles.

The *MINWAIT option is a good choice for expanding subfiles. It is not a good choice for page-at-a-time or load-all subfiles.

The *ALLIO is a good choice for load-all subfiles. It is not a good choice for page-at-a-time or expanding subfiles.

When you set the optimizer option, you are attempting to bias the query optimizer to find the selection path that best suits your needs. Here is an example.

Suppose you had an OPNQRYF performing a select and sort on a million-record open accounts receivable file to be used with a subfile inquiry program. Say you want to select if the invoice due date is less than today, so you would be pulling up past due invoices. Then, you want to sort by customer number. From this point, you will load the subfile so that all the past invoices for each customer show up together.

Assume that the user probably wouldn't need to roll more than three pages. Therefore, you will create an expanding subfile. Now assume the following access paths are available.

- There is a logical by customer number.
- There is a logical by invoice due date.

On the OPNQRYF optimizer keyword, you would want to either specify *FIRSTIO or *MINWAIT. Probably in this case you would use *FIRSTIO, since the user isn't going to have to roll very much. By using *FIRSTIO, you would bias the optimizer to use the logical by customer number. If the OPNQRYF used the logical by due date, it could perform a set lower limit and read on the logical, which would greatly speed the process. But it would have to read all records with a due date less than today's date so that it could sort by customer number. By using the customer number logical, all it has to do is read the already sorted logical until it finds enough records with past due invoices to fill up the buffer. Then it would allow the program to display the first page of the subfile. If you rolled forward, the OPNQRYF would select another buffer full of records. By selecting this logical, the OPNQRYF could return the first

screen of records in the minimum amount of time and will keep you from having to read the entire file every time.

Suppose your customers are good about paying on time, and you probably would never have more than a few hundred past due invoices out of the one million that you have open. In this case, you might want to look at the subfile program performing a load all. You would want the optimizer to bias itself in selecting and sorting all of the records in the shortest amount of time. The optimizer would probably choose the logical by due date. This way, it could perform a set lower limit and read 200 records, not a million. Then it creates a temporary access path and sorts those 200 records. Then you could load the entire subfile. In this situation, the OPNQRYF definitely would not want to select the customer number logical, because it would have to read one million records every time it loaded the subfile. Even though the records are already sorted, it would still take longer because the OPNQRYF would be reading one million records every time.

One note about the query optimizer. Normally, it will analyze the access paths of about seven logicals per physical file on the OPNQRYF command. If a physical file has ten or more logicals, the optimizer may not ever select the best logical file for the application, because the optimizer may time out before looking at all the attached logicals. The time out is used by the optimizer so that it doesn't spend too much time looking for a good existing access path to use in the OPNQRYF selection. It will analyze logical files in the order of the most recently created and look backward. If you know that you have a good logical file that the OPNQRYF should be using, but you suspect it is not, start debug on the program (STRDBG), and then run the program. Then, display the joblog. The joblog will tell you which logical was chosen, and whether or not the optimizer timed out. If the optimizer timed out, and it did not select the logical you thought it should have, you can force the optimizer not to time out by specifying OPTALLAP(*YES) on the OPNQRYF command. Doing so will force the optimizer to look at all access paths built against the physical. If you specify OPTALLAP(*YES), or if the optimizer did not time out, and it still did not choose the logical that you thought it should have, don't worry. The optimizer probably chose the logical it did for a good reason.

Using Logical Files to Speed the Loading Process

Of course, if this program was going to be heavily utilized by a lot of people every day, you could build a logical by customer number, that selects only the past due invoices. However, you want to be careful not to create too many logical files. Too many logicals can increase the overhead on the system (by updating many logical files whenever the physical is added/updated), and can cause ordinary read programs

to take longer to process due to the higher and more common seize times (the time it takes the system to update all logicals when the physical has been updated).

When loading subfile data, you must be careful in how data is selected. For instance, suppose you are loading all customer invoices that are past due into a subfile. If your open invoice file is relatively small, or if there are only a few orders per customer, then selecting out only the past due records for a customer is not a big performance drain. But if a customer has many open orders, then the impact on performance to filter out the past due invoices could be substantial. At this point, a decision needs to be made. First, how often will this program be used. Second, how many records could the subfile load have to filter out to find the past due records. If this program is heavily used, or is reading thousands of records to select only the past due invoices, then building a logical file that selects only the past due invoices may be a good idea. There will be a tradeoff in performance. You are trading off a slight decrease in performance whenever you add, delete, and possibly maintain records in the open invoice file. But you would be gaining much faster response times, lower disk activity, and possibly a lot less CPU utilization loading a subfile and having the program filter out the invoices that are not past due.

Query Optimizer Demo with Subfiles

There are six demo CL programs to show the OPNQRYF command and subfile relationship. Programs SFC900A, SFC900B, SFC900C, and SFC900D call the load-all subfile program SFR900. SFC901C and SFC901D are call the expanding subfile program SFR901. Each CL program performs an OPNQRYF command.

OPNQRYF/Load-All Subfile/Small Load

Programs SFC900A and SFC900B, perform an OPNQRYF over file SMALLF.

```
/*==============================================================*/
/* To compile:                                                  */
/*                                                              */
/*           CRTCLPGM   PGM(XXX/SFC900A) SRCFILE(XXX/QCLSRC)    */
/*                                                              */
/*==============================================================*/
             PGM

/* Override the file to share the access path.  Select    */
/* only the records in which the name field is greater    */
/* than blanks.  Sort by the number field.                */

             OVRDBF     FILE(SMALLF)  SHARE(*YES)
```

```
            OPNQRYF    FILE((SMALLF)) QRYSLT('DATDU *LE 931201') +
                       KEYFLD((CUSNO))

/* Call the subfile program.                                  */

            CALL       PGM(SFR900)

            CLOF       OPNID(SMALLF)
            DLTOVR     FILE(SMALLF)

            ENDPGM


/*==============================================================*/
/* To compile:                                                  */
/*                                                              */
/*          CRTCLPGM   PGM(XXX/SFC900B) SRCFILE(XXX/QCLSRC)     */
/*                                                              */
/*==============================================================*/
            PGM

/* Override the file to share the access path.  Select        */
/* only the records in which the name field is greater        */
/* than blanks.  Sort by the number field.                    */

            OVRDBF     FILE(SMALLF)  SHARE(*YES)
            OPNQRYF    FILE((SMALLF)) QRYSLT('DATDU *LE 931201') +
                         KEYFLD((CUSNO)) OPTIMIZE(*FIRSTIO)


/* Call the subfile program.                                  */

            CALL       PGM(SFR900)

            CLOF       OPNID(SMALLF)
            DLTOVR     FILE(SMALLF)

            ENDPGM
```

These CL programs call RPG program SFR900.

```
A*============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD900) SRCFILE(XXX/QDDSSRC)
A*
A*============================================================
A                                     DSPSIZ(24 80 *DS3)
A                                     CF03(03)
A          R SFD900A                  SFL
A            CUSNO     R        O  7  2REFFLD(RFILE/CUSNO *LIBL/SMALLF)
A                                     COLOR(PNK)
A            DATDU     R        O  7 42REFFLD(RFILE/DATDU *LIBL/SMALLF)
A                                     EDTWRD('  /  /  ')
A                                     COLOR(PNK)
A            AMTDU     R        O  7 51REFFLD(RFILE/AMTDU *LIBL/SMALLF)
```

```
A                                       EDTCDE(M)
A                                       COLOR(PNK)
A            CUSNM     R        O  7 1OREFFLD(RCUST/CUSNM *LIBL/CUSTOMER)
A                                       COLOR(PNK)
A          R SFD900B                    SFLCTL(SFD900A)
A                                       SFLSIZ(0016)
A                                       SFLPAG(0015)
A                                       OVERLAY
A  30                                   SFLDSP
A                                       SFLDSPCTL
A N03                                   SFLEND(*MORE)
A                                  1  2DATE
A                                       EDTCDE(Y)
A                                       COLOR(BLU)
A                                  2  2TIME
A                                       COLOR(BLU)
A                                  1 70'SFR900'
A                                       COLOR(BLU)
A                                  2 70USER
A                                       COLOR(BLU)
A                                  5  2'Number'
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A                                  5 10'Customer Name                  '
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A                                  5 42'Date Due'
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A                                  5 51'Amount Due'
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A                                  1 28'Accounts Past Due Inquiry'
A                                       COLOR(WHT)
A          R SFD900C
A                                 23  2'F3-Exit'
A                                       COLOR(WHT)


F*==============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR900) SRCFILE(XXX/QRPGSRC)
F*
F*==============================================================
FSMALLF  IF  E                     DISK
FCUSTOMERIF  E           K         DISK
FSFD900  CF  E                     WORKSTN
F                                        @RRN  KSFILE SFD900A
C*
C* Write the command key format, then display the subfile
C* control format.
C*
C                     WRITESFD900C
C                     EXFMTSFD900B
C*
```

```
C* When control returns to the program, check to see if a
C* command key was pressed.
C*
C                     EXSR @CMD
C*
C*****************************************************
C*                                                   *
C* * I N Z S R   S U B R O U T I N E                 *
C*                                                   *
C*****************************************************
C*
C           *INZSR    BEGSR
C*
C* Load the subfile.
C*
C           *IN99     DOWEQ*OFF
C                     READ SMALLF                  99
C*
C           *IN99     IFEQ *ON
C                     LEAVE
C                     END
C*
C* Get the customer name. If the customer record isn't found,
C* blank out the customer name field.
C*
C           CUSNO     CHAINCUSTOMER             98
C           *IN98     IFEQ *ON
C                     MOVEL*BLANKS   CUSNM
C                     END
C*
C* Write the subfile record.
C*
C                     ADD  1         @RRN    50
C                     WRITESFD900A
C                     END
C*
C* If you have written at least one subfile record, then it will
C* be ok to display the subfile format.
C*
C           @RRN      IFGT 0
C                     MOVE *ON       *IN30
C                     END
C                     ENDSR
C*
C*********************************************************
C*    @ C M D   S U B R O U T I N E                      *
C*                                                       *
C*    Executed after every EXFMT or display read op code.  *
C*    Used to process command keys.                      *
C*********************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, exit the program.
C*
C           *IN03     IFEQ *ON
C                     MOVE *ON       *INLR
```

```
C                       RETRN
C                       END
C*
C                       ENDSR
```

These program use the file is SMALLF. SMALLF contains three fields, CUSNO, DATDU, and AMTDU. After you create this file you'll need to write a program to load it with about 350 records worth of test data.

```
A*============================================================
A* To compile:
A*
A*      CRTPF      FILE(XXX/SMALLF) SRCFILE(XXX/QDDSSRC)
A*
A*============================================================
A          R RFILE
A            CUSNO          5  0
A            DATDU          6  0
A            AMTDU          9  2
```

There are two logical files the query optimizer will analyze, SMALLFL1 and SMALLFL2.

```
A*============================================================
A* To compile:
A*
A*      CRTLF      FILE(XXX/SMALLFL1) SRCFILE(XXX/QDDSSRC)
A*
A*============================================================
A          R RFILE                     PFILE(SMALLF)
A          K CUSNO

A*============================================================
A* To compile:
A*
A*      CRTLF      FILE(XXX/SMALLFL2) SRCFILE(XXX/QDDSSRC)
A*
A*============================================================
A          R RFILE                     PFILE(SMALLF)
A          K DATDU
```

The SMALLFL1 logical is built with the CUSNO field as the key, whereas SMALLFL2 is built with DATDU as the key field. Remember, the query optimizer will analyze these logicals starting with the most recently created, which would be SMALLFL2. Since there are only two logicals over SMALLF file, both logicals should be analyzed before the optimizer times out.

In each CL program, you are selecting if the DATDU field is greater than 93/12/01. In a normal production program, this date would not be hard coded, but would be a variable that contains today's date. You may want to hard code the date so that these programs will run the same regardless of when you run them. In the example, the OPNQRYF has CUSNO as the key field.

The only difference in each CL is that the OPNQRYF in SFC900A is set to optimize (*ALLIO), whereas SFC900B is set to optimize (*FIRSTIO).

Start debug by issuing the command STRDBG. Then run SFC900A and SFC900B. You can exit both programs with an F3. Then issue the Display Job Log (DSPJOBLOG) command. Press F10 to see the detail, then F18 to go to the end of the job log.

In the joblog you can see the following:

```
call SFC900A
All access paths were considered for file SMALLF
Access path built for file SMALLF
call SFC900B
All access paths were considered for file SMALLF
Access path of file SMALLFL1 was used by query
```

You may have noticed when you ran the two programs that they took about the same amount of time to bring up the initial screen even though the query optimizer chose a different access plan for each method. Because the file was small, it didn't matter much which one it chose. Normally, in a load-all subfile, you would not want to specify *FIRSTIO. Although the *FIRSTIO method usually returns the first buffer of records sooner, it generally takes much longer to process the query as a whole.

In the example above, in the second program (SFC900B), the query optimizer acted as anticipated. Since it was biased for *FIRSTIO, it chose to read the logical that was already sorted and filter out the select criteria from this file.

But the first program chose to build an access path from scratch, rather than read the logical built by due date. If it had read the logical by due date, the query optimizer could have read the logical until the due date exceeded the select criteria, and then sorted the records. But since the operator was a greater than (QRYSLT('DATDU *GT 93/12/01')), the query optimizer expected to have to select 30 percent of the records, or about 100. The optimizer determined that, because the file as a whole is small, it could read the file dynamically, performing the selection, then build an access path over the selected records. Again, the optimizer may not seem to choose what you think is a good choice in an access plan, but in this case, it didn't matter much which way it went.

OPNQRYF/Load-All Subfile/Large File

Now you can see what the difference would be if the query optimizer were working with a much larger file. The programs SFC900C and SFC900D call the same load-all subfile program.

```
/*==============================================================*/
/* To compile:                                                  */
/*                                                              */
/*           CRTCLPGM   PGM(XXX/SFC900C) SRCFILE(XXX/QCLSRC)    */
/*                                                              */
/*==============================================================*/
            PGM

/* Override the file to share the access path.  Select    */
/* only the records in which the name field is greater    */
/* than blanks.  Sort by the number field.                */

            OVRDBF     FILE(SMALLF) TOFILE(LARGEF) SHARE(*YES)
            OPNQRYF    FILE((SMALLF)) QRYSLT('DATDU *LE 931201') +
                       KEYFLD((CUSNO))

/* Call the subfile program.                              */

            CALL       PGM(SFR900)

            CLOF       OPNID(SMALLF)
            DLTOVR     FILE(SMALLF)

            ENDPGM


/*==============================================================*/
/* To compile:                                                  */
/*                                                              */
/*           CRTCLPGM   PGM(XXX/SFC900D) SRCFILE(XXX/QCLSRC)    */
/*                                                              */
/*==============================================================*/
            PGM

/* Override the file to share the access path.  Select    */
/* only the records in which the name field is greater    */
/* than blanks.  Sort by the number field.                */

            OVRDBF     FILE(SMALLF) TOFILE(LARGEF) SHARE(*YES)
            OPNQRYF    FILE((SMALLF)) QRYSLT('DATDU *LE 931201') +
                          OPTIMIZE(*FIRSTIO) KEYFLD((CUSNO))


/* Call the subfile program.                              */

            CALL       PGM(SFR900)

            CLOF       OPNID(SMALLF)
            DLTOVR     FILE(SMALLF)

            ENDPGM
```

The only difference between these programs and SFC900A and SFC900B is that you have overridden the file SMALLF to file LARGEF. After you create this file, you'll need to write a program to load it with about 10,000 records worth of test data.

```
A*============================================================
A* To compile:
A*
A*      CRTPF      FILE(XXX/LARGEF) SRCFILE(XXX/QDDSSRC)
A*
A*============================================================
A          R RFILE
A            CUSNO          5  0
A            DATDU          6  0
A            AMTDU          9  2
```

There are two logical files the query optimizer will analyze, LARGEFL1 and LARGEFL2.

```
A*============================================================
A* To compile:
A*
A*      CRTLF      FILE(XXX/LARGEFL1) SRCFILE(XXX/QDDSSRC)
A*
A*============================================================
A          R RFILE                     PFILE(LARGEF)
A          K CUSNO
```

```
A*============================================================
A* To compile:
A*
A*      CRTLF      FILE(XXX/LARGEFL2) SRCFILE(XXX/QDDSSRC)
A*
A*============================================================
A          R RFILE                     PFILE(LARGEF)
A          K DATDU
```

This file, by most standards, is not very large. But it is very large when working with subfiles, as you will be loading nearly 10,000 records into the subfile. These programs will take a couple of minutes to run, regardless of your AS/400 model.

Ensure that you have DEBUG started and run SFC900C and SFC900D. Your joblog should look like this:

```
call SFC900C
All access paths were considered for file LARGEF
Access path built for file LARGEF
call SFC900D
All access paths were considered for file LARGEF
Access path of file LARGEFL1 was used by query
```

In this example, the *ALLIO beats out the *FIRSTIO by a small margin. By looking at the job log, you can see that the access plans created for SMALLF were identical to the ones created for LARGEF. The reason *ALLIO was faster than *FIRSTIO is that part of the processing against the file was done dynamically, whereas the *ALLIO process was working strictly with keys. For this reason, you should not specify *FIRSTIO for

load-all subfiles. But from this example, you can also see why you should not be using the load-all subfile for processing thousands of records either.

One other thing was noticeable about the way the programs SFC900C and SFC900D ran. SFC900C had a message that the access path was being built for file LARGEF on the bottom of the screen, whereas SFC900D had a message updated constantly as to how many records were selected and processed. What happened in SFC900C was that the access plan was totally created for all 8,500 or so selected records, then control passed to the RPG program to load the subfile. SFC900D, on the other hand, selected 200 sorted records, then passed control to the RPG program. The RPG program loaded 200 records into the subfile and control then passed back to the OPNQRYF, where another 200 records were selected. This process continued back and forth until all 8,500 records had been processed. because *FIRSTIO was specified, and the query process was selecting just enough records to fill the program's buffer before it stopped processing. This type of processing can yield very high performance gains when working with large files, and an expanding or page-at-a-time subfile.

OPNQRYF/Expanding Subfile/Large File

Let's take a look at our last two programs, SFC901C and SFC901D:

```
/*==============================================================*/
/* To compile:                                                  */
/*                                                              */
/*           CRTCLPGM   PGM(XXX/SFC901C) SRCFILE(XXX/QCLSRC)    */
/*                                                              */
/*==============================================================*/
             PGM

/* Override the file to share the access path.  Select    */
/* only the records in which the name field is greater    */
/* than blanks.  Sort by the number field.                */

             OVRDBF     FILE(SMALLF) TOFILE(LARGEF) SHARE(*YES)
             OPNQRYF    FILE((SMALLF)) QRYSLT('DATDU *LE 931201') +
                        KEYFLD((CUSNO))

/* Call the subfile program.                              */

             CALL       PGM(SFR901)

             CLOF       OPNID(SMALLF)
             DLTOVR     FILE(SMALLF)

             ENDPGM
```

```
/*==============================================================*/
/* To compile:                                                  */
/*                                                              */
/*           CRTCLPGM   PGM(XXX/SFC901D) SRCFILE(XXX/QCLSRC)    */
/*                                                              */
/*==============================================================*/
            PGM

/* Override the file to share the access path.  Select    */
/* only the records in which the name field is greater    */
/* than blanks.  Sort by the number field.                */

            OVRDBF     FILE(SMALLF) TOFILE(LARGEF) SHARE(*YES)
            OPNQRYF    FILE((SMALLF)) QRYSLT('DATDU *LE 931201') +
                         KEYFLD((CUSNO)) OPTIMIZE(*FIRSTIO 30)


/* Call the subfile program.                              */

            CALL       PGM(SFR901)

            CLOF       OPNID(SMALLF)
            DLTOVR     FILE(SMALLF)

            ENDPGM
```

Now change the subfile program to an expanding subfile instead of a load-all subfile. You will call this program SFR901:

```
A*==============================================================
A* To compile:
A*
A*      CRTDSPF    FILE(XXX/SFD901) SRCFILE(XXX/QDDSSRC)
A*
A*==============================================================
A                                     DSPSIZ(24 80 *DS3)
A                                     CF03(03)
A          R SFD901A                  SFL
A            CUSNO     R        O  7  2REFFLD(RFILE/CUSNO *LIBL/SMALLF)
A                                     COLOR(PNK)
A            CUSNM     R        O  7 10REFFLD(RCUST/CUSNM *LIBL/CUSTOMER)
A                                     COLOR(PNK)
A            DATDU     R        O  7 41REFFLD(RFILE/DATDU *LIBL/SMALLF)
A                                     EDTWRD('  /  /  ')
A                                     COLOR(PNK)
A            AMTDU     R        O  7 50REFFLD(RFILE/AMTDU *LIBL/SMALLF)
A                                     EDTCDE(M)
A                                     COLOR(PNK)
A          R SFD901B                  SFLCTL(SFD901A)
A N99                                 ROLLUP(25)
A                                     OVERLAY
A  30                                 SFLDSP
A                                     SFLDSPCTL
A  99                                 SFLEND(*MORE)
A                                     SFLSIZ(0016)
A                                     SFLPAG(0015)
A            @PAG           4S 0H     SFLRCDNBR
```

```
A                                   1  2DATE
A                                       EDTCDE(Y)
A                                       COLOR(BLU)
A                                   2  2TIME
A                                       COLOR(BLU)
A                                   1 70'SFR901'
A                                       COLOR(BLU)
A                                   2 70USER
A                                       COLOR(BLU)
A                                   5  2'Number'
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A                                   5 10'Name                          '
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A                                   5 41'Date Due'
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A                                   5 50'Amount Due'
A                                       COLOR(YLW)
A                                       DSPATR(UL)
A                                   1 31'Accounts Past Due Inquiry'
A                                       COLOR(WHT)
A          R SFD901C
A                                  23  2'F3-Exit'
A                                       COLOR(WHT)


F*=============================================================
F* To compile:
F*
F*      CRTRPGPGM  PGM(XXX/SFR901) SRCFILE(XXX/QRPGSRC)
F*
F*=============================================================
FSMALLF  IF  E                     DISK
FCUSTOMERIF  E           K         DISK
FSFD901  CF  E                     WORKSTN
F                                        @RRN  KSFILE SFD901A
C*
C* Write the command key format, then display the subfile
C* control format.
C*
C                     WRITESFD901C
C                     EXFMTSFD901B
C*
C* When control returns to the program, check to see if a
C* command key was pressed.
C*
C                     EXSR @CMD
C*
C*****************************************************
C*                                                   *
C* * I N Z S R   S U B R O U T I N E                 *
C*                                                   *
C*****************************************************
C*
```

```
C           *INZSR    BEGSR
C*
C                     EXSR @LOAD
C*
C                     ENDSR
C*
C*************************************************************
C*    @ C M D   S U B R O U T I N E                          *
C*                                                           *
C*    Executed after every EXFMT or display read op code.    *
C*    Used to process command keys.                          *
C*************************************************************
C*
C           @CMD      BEGSR
C*
C* If F3 was pressed, exit the program.
C*
C           *IN03     IFEQ '1'
C                     MOVE *ON       *INLR
C                     RETRN
C                     END
C*
C* If the ROLLUP key was pressed and control was passed back
C* to the program, load 15 more records into the subfile.
C*
C           *IN25     IFEQ '1'
C                     MOVE *OFF      *IN25
C                     EXSR @LOAD
C                     END
C*
C                     ENDSR
C*
C*******************************************************
C*                                                      *
C* @ L O A D   S U B R O U T I N E                      *
C*                                                      *
C*******************************************************
C*
C           @LOAD     BEGSR
C*
C* Set the first subfile page to display
C*
C                     Z-ADD@RRN      @PAG
C*
C* Load the subfile. Load 1 page (15 records) at a time.
C*
C                     DO   15
C                     READ SMALLF                   99
C*
C           *IN99     IFEQ '1'
C                     LEAVE
C                     END
C*
C* Get the customer name. If the customer record isn't found,
C* blank out the customer name field.
C*
C           CUSNO     CHAINCUSTOMER             98
```

```
C           *IN98     IFEQ '1'
C                     MOVEL*BLANKS   CUSNM
C                     END
C*
C* Write the subfile record.
C*
C                     ADD  1         @RRN    40
C                     WRITESFD901A
C                     END
C*
C* If the subfile has any records in it, it will be ok to display
C* the subfile format.
C*
C           @RRN      IFGT 0
C                     MOVE *ON       *IN30
C                     END
C*
C*
C*  If you added at least one record during the load routine,
C*  then cause the subfile to display the next page. If you did
C*  not add any records, display the last subfile page that was
C*  on the display.
C*
C           @PAG      IFNE @RRN
C                     ADD  1         @PAG
C                     END
C                     ENDSR
```

The two programs that will be used to run this demo are SFC901C and SFC901D. Start debug and run these two programs. Instantly, you should see a difference between the two programs in the time it took to bring up the first screen. When you told the optimizer to bias towards *FIRSTIO, the process was much quicker. The default, which is *ALLIO, took much longer. Why did the SFC901D program, which is biased *FIRSTIO, run so much faster? Because the query optimizer used the LARGEFL1 logical file, read in the first buffer full of records (about 150), and then paused its execution until more records were needed in the buffer. Control then passed to the program. If you run program SFC900D, you will see a query message on the bottom of the screen momentarily, then the initial subfile screen comes up. Now the program has control. Fifteen records were loaded into the subfile and displayed. If you press the roll-up key, the subfile will load 15 more records from the buffer. Control does not yet pass back to the OPNQRYF command because you still have enough records already selected for your use. After rolling forward about ten more times, you will notice that the OPNQRYF message quickly returns. Since all of the initial buffer records had been used, the OPNQRYF took control again and loaded in another buffer's worth of records.

Program SFC901C did not run very fast. It ran faster than the load-all program SFC900C because the entire subfile was not loaded. It still, however, processed the entire OPNQRYF. This reason is why SFC901D outperformed SFC900C. SFC900D's

OPNQRYF only had to process a very few records, whereas SFC900C's OPNQRYF processed them all.

You can see in this case how biasing the optimizer can make a big improvement in the response time for your subfile program. Think of the difference in time if you were working with a million record file. SFC900C could possibly take an hour or longer to bring up the first screen, whereas SFC900D would still only need a second or two.

Hopefully, this section has given you some insight as to what the OPNQRYF optimizer is attempting to do, and what you can do to influence its judgments by using the optimize parameter, or by creating logical files the optimizer can use. The OPNQRYF is a very powerful command, for both interactive and batch work. Couple it with the power of subfiles, and you can turn what would otherwise be an extremely complex search or inquiry program into a few simple lines of code that execute extremely fast and efficiently.

9

Subfile Design Summary

Subfile Loading Techniques

When designing a subfile, you must choose a loading method. There are three different ways to load a subfile:

1. Load-All Method: every record that is to be displayed is loaded into the subfile first, then the subfile is displayed.

2. Expanding Method: a reasonable number of records is loaded into the subfile, and then the subfile is displayed. If the user attempts to roll past the last loaded subfile record, more records have to be loaded into the subfile.

3. Single-Page Load Method: the subfile is only loaded with the number of records that can be displayed on the screen at one time.

The loading method you choose determines the subfile page and size combination.

When to Use a Load-All Subfile

Ideally, you would use the load-all subfile whenever the number of records that you would want to load into a file is relatively small. "Small" usually means less than 100 to 200 records. Using a load-all subfile to load more than this number of records can cause the user to wait an increasing amount of time before the initial screen is returned to them.

Using a load-all subfile reduces the programming effort. It also allows, once the subfile records have been initially loaded and displayed, for the user to quickly roll back and forth through the subfile because the subfile records are kept in memory.

Remember, you can clear a load-all subfile of its records by using the SFLCLR keyword, and load in another set of records. But once you clear the subfile, you can no longer display the records that were in the subfile prior to the clear.

The load-all subfile is the preferred method if you are sure that the number of records that could be loaded will be low, now and in the future. The roll keys are handled by the subfile program, and you don't have to reference them in the RPG programs or DDS.

Different Ways to Perform the Load-All

You have several methods in which to load records into a load-all subfile.

Method 1: Load All Records

The first is to perform a read loop to read until you have loaded all the records that you want into the subfile. The problem with this method is that you do not check to ensure that more than 9,999 records are loaded. If for some reason a user ever attempted to load more than 9,999 records, he would receive an error. You don't want users to be faced with these errors. So you should always, at a minimum, want to load using the second method.

Method 2: Load Up To 9,999 Subfile Records

The second method is to trap to ensure that the subfile does not load more than 9,999 records. If the relative record number field ever reaches 9,999, then you want to exit the load loop. You should also send the user a message letting him know that the

maximum number of loaded records has been reached, and that all records have not been loaded. You may also want to add in a message that lets the user know he needs to contact I/S as soon as this situation occurs. You can easily add the message using the SFLMSGID keyword and a message file. The user should notify you if the load-all subfile program he was running ever reached the 9,999 limit because a load-all subfile should only be used in applications where the number of records to load is low. So if there is ever an instance where the maximum of 9,999 was attempted to be loaded, you will want to know about it. You might just have a bug in the program. Or, the files that you are loading from have grown since you originally wrote the application. You may need to change the loading method to an expanding subfile.

Method 3: Restricted Load All

Another loading method is to use the combination of the subfile size and the write operation indicator. By using this method, you can assure yourself that the load-all subfile will never load more than the subfile size. You could set your subfile size to 100, place a write operation indicator in your program, and trap for this error. If you load 100 records, send a message to the user that more records exist that are not loaded in the subfile. The user could then press a function key, and the existing subfile records would be cleared and another 100 would be loaded. However, this technique is not a preferred method. In this case, it would be much wiser to use an expanding subfile.

Load-all subfiles are used in input and update subfiles.

When to Use an Expanding Subfile

Expanding subfiles are the answer when load-all subfiles would be nice, but impractical. They allow you to limit the loading of the subfile to a meaningful number of records. At the same time you can allow the user to roll backward to see the previously loaded data. You do not need to program for the rollback key, but you do have to program for the roll forward key. Rolling forward is not always as fast as it is with the load-all subfile. Occasionally, during a roll forward you will be loading more records into the subfile. However, rolling backward through a expanding subfile is as fast as a load-all subfile.

Loading Methods For an Expanding Subfile

For an expanding subfile, there are several methods of loading the records.

Method 1: Load One Page of Records

First, you might only load one page worth of records at a time. Every time the roll forward key is pressed, you would load another page worth of records into the subfile. In this case, if the subfile page were set to 10, and subfile size was 20, in the RPG program you would load 10 records into the subfile.

Method 2: Load More Than One Page of Records

Another way to load records is to load more than one page worth of records at a time. If the subfile page was 10, and you wanted to load 5 pages of records at a time, you could set the subfile size to 50. Then your processing loop in the RPG program could load 50 records into the subfile. You would use this process if you were certain that the user almost always would want to roll forward a couple of times. By loading more pages of records, each load process requires more time to load, but each subsequent roll forward is very fast. Every fifth roll forward, the program would require more records to be loaded.

For an expanding subfile, you should always load in an increment of the subfile page. For example, if your subfile page is set to 6, then load 6, 12, 18, 24 (and so on) records into the subfile at the same time. If you were to load seven, it would not display properly. The first subfile page would contain six records. The next subfile page would contain one record. When you pressed roll forward again, another seven records would be loaded into the subfile. Page three would then contain six records and page four would contain one. So, to keep the number of records displayed on a page the same, you need to load an even increment of the subfile page into the subfile.

You must be certain to use some sort of trapping, so you don't attempt to extend the subfile past 9,999 records. By using an expanding subfile, you are somewhat admitting that the number of records could be high. Therefore, the user could, after rolling forward a number of times, cause the program to load more than 9,999 records if you did not limit it.

The expanding subfile is ideal for inquiry programs in which you cannot be certain as to the highest number of loaded records.

Expanding subfiles are not usually good for update programs because a user could load a considerable number of records into the subfile before updating them. This situation could cause data between two users to get overlaid.

Here is a good hint to use to keep you from having to modify the RPG program if you ever want to increase an expanding subfile page or size values. So far, if you have wanted to use an expanding subfile, you would code into the RPG program a do loop in which you would specify the number of iterations needed to load subfile records. But this solution is not a good practice. If you wanted to change the values of the subfile size or page, you would have to modify both the display file and the program. However, if you use a technique that you learned early on in the book, you can eliminate the need. Remember how the load of a subfile could be limited by using the write conditioning indicator? You turn on the write conditioning indicator if you load the number of records into the subfile equal to the subfile size parameter. Using this technique, you can find out how many records to load. In your initial subroutine, you could code the following:

```
C           *IN99     DOUEQ*ON
C                     ADD  1         @RRN
C                     WRITESFLFMT                    99
C                     END
C                     Z-ADD@RRN      @COUNT
C                     SUB  1         @COUNT
```

In this routine, you are writing blank records to the subfile. You continue to do write blank records until indicator 99 is on, which means you have loaded the subfile with the number of records equal to the subfile size. Then you subtract one from this number and you can get the value of the subfile page, assuming that the subfile size is set to one more than the subfile page. Then, the load loop will load the number of records controlled by the variable @count. If you ever want to change the subfile page or size, the RPG program will adjust accordingly.

When to Use a Single-Page Subfile?

The page-at-a-time subfile is usually the least desirable subfile implementation because you must program for the roll-up and roll-back key. Programming for them doesn't just mean making reference to them in the program. You must know how to reset the file pointers when you roll backward, so the screen contains the correct data. If you are working a uniquely keyed file, this process is not hard to code. All you have to do is to take the key data in the subfile record number 1, set lower limits in your program, and read backward through the file loading the records in reverse order. If you had ten records per page, you would start loading the subfile records

at relative record number 10, and do this loop ten times subtracting one from the subfile relative record number each time.

But if the file is not uniquely keyed, you cannot set the pointer in the file and read backward. You must, instead, start with the last record you loaded, read backward the number of records on that page, which may not be the subfile page value, and then read backward another subfile page value. Of course, you could also read backward after reading past the current screen of records. All of this reading requires additional system overhead. For this reason, if there is a good chance that the user would need to read backward, then the expanding subfile is almost always the better implementation choice.

There is only one loading method for a single-page subfile, which is to load one page of records. Loading more than one page of records results in a error.

You can use the same loading hint here as you did in the expanding subfile. Instead of hard coding the number of subfile records to load, use the subfile condition write indicator. Since in a single-page subfile load technique the subfile page is equal to the subfile size, the load loop should use the write indicator. This way you can change the subfile page and size in the display file, and not have to change the RPG programs.

When Not to Use a Subfile

It is important to note when it would be better to write a non-subfile program instead of a subfile program. Any application that updates data is of potential concern. This is especially true when you are woking with highly volatile data. Suppose you are wanting to display the quantities on hand for your inventory, and you want to allow the user to update the quantities on the screen. If you load the inventory records into a subfile for the user to modify, the changes won't take affect until the user presses the Enter key. If you give them a lot of records to change, the user may take a substantial amount of time before they press the Enter key. During that time, other transactions may have been posted to the file. Once you update from the subfile, you essentially wipe out the transactions that occurred while the user was keying in their changes. If you are writing an application to modify highly volatile data, you should very carefully consider only updating one record at a time. The user can lock the record, change it, update it, and then release the lock. This way, you would keep other users from making any changes to this same record.

If your application is a subfile candidate, then writing a subfile program is much easier than coding a non-subfile program to get the same effect. Remember, the reason you would want a subfile is to display multiple lines of related data.

If an application needs to be written this way, use a subfile. Initially, subfiles are much easier to program than a regular program. For instance, a load-all subfile to display 18 lines on a display, with six fields per line, and the ability to roll forward and backward, could be done in less than a dozen lines of RPG code. It could easily take more than a hundred to do the same thing in a standard RPG program. And don't think that the page-at-a-time and expanding subfiles are much more difficult to program than the load all. The load all is the easiest, but the other two are not far behind.

Subfile programs are much easier to maintain than regular programs. Most of the functionality of a subfile resides within the DDS. To make a change in functionality, you will most likely be changing DDS, not RPG code. A lot of functionality you added to the subfiles in this book did not require one line of RPG code to be changed, added, or deleted.

Subfile Page and Subfile Size

Once you have chosen a subfile loading method, you must correctly set the subfile page and size values.

For a load-all and expanding subfile, the subfile size must be at least one more than the subfile page. The subfile size should be set to a reasonable number, usually the average number of records that could reside in the subfile at any one time. If you attempt to write more records to the subfile than what is specified in the subfile size, the subfile will extend itself to handle these records, up to 9,999 records. The extended records in a subfile are not kept together with the initial records. For example, if you specified a subfile size of 100, the first 100 loaded subfile records are kept together in main memory, whereas any records loaded beyond this point are located in memory separate from these. If you specify a subfile size that is always too small, (the subfile is always extending itself), performance of the program may suffer very slightly as memory locations for the extended records must be allocated.

For a single-page subfile, the subfile page and subfile size values must be equal. Attempting to load more records than the subfile size will cause an error.

The subfile size value must be the number of subfile records you want to see on the display *in folded mode.* If each subfile record takes up three lines, and you have fifteen lines in which to view subfile records, the subfile page would be five. (15 display lines/3 lines per subfile record). If you bring the subfile up in dropped mode, the subfile knows to display 15 subfiles records automatically. (15 lines/1 line per subfile record).

It doesn't matter which mode you specify, the subfile page must be equal to the number of subfile records you want to display on the screen in folded mode.

Subfile Display Type

You must also choose which subfile display type to use—full display or window display.

Subfile windows are ideal for use in output (inquiry) style subfiles. They allow a user to view a partial portion of the screen in which they are operating, and the user can view an inquiry window on another part of the screen. If the amount of information to be displayed on the subfile is low, window subfiles are ideal. If a lot of information is needed, you could use a folded subfile in your window. A subfile record can then reside on two or more display lines. But this arrangement clutters the display and can look confusing to the user if too much data is displayed in a small area of the screen.

Window subfiles are ideal for an inquiry program that will be called from many different programs. If you call this program only from a menu, though, you probably need to make the subfile a full screen subfile. It may not feel right to the user to see the menu displayed, and an application in a window on top of it.

If the subfile application has the possibility of being called from both a menu, as well as from inside many other programs, then you can design the subfile for both methods. If you remember back to the inquiry subfile program, you passed the subfile program the customer number. Well if you call this program from a menu, you wouldn't pass it the customer number field. If the program detected that a parameter was passed, it would load and display a window subfile. If it did not detect a parameter, it could load and display the full screen subfile.

Multiple Display Sizes

At times, you may want one subfile program to work across a variety of display devices. For example, some devices allow a display size of only 24 x 80. Other devices allow a display size of 27 X 132. You may want the subfile to display more data on the larger displays, and truncate some of this data on the smaller displays. But you would like to have just one subfile program to maintain. You can do so by describing your subfile twice in the display file, one at 24 X 80 and the other at 27 X 132. When the application is called, the POST op code is performed, so the display device can be found. If it is a device capable of the 27 X 132 display, then load and display the

subfile associated with the 27 X 132 display size. Otherwise, load and display the subfile that is associated with the 24 X 80 display size.

If a lot of information needs to be displayed in a subfile, then the full screen subfile should be used. However, too much information in any subfile can adversely affect performance. Remember, when you are working with subfiles and their loading methods, you are dealing with a looping program.

Performance Problems to Watch For

Subfiles can be potentially dangerous to performance because they are highly repetitious. This danger comes from the fact that you cause the program to loop through many iterations when loading the subfile. If you are loading a single page, then the iteration loop may be executed a few times. If you perform a load all, then the loop could be executed hundreds or thousands of times.

Always be concerned with any program logic within the subfile load iteration. Suppose you need to move *blanks into a field. If you can do this before you enter the load iteration, do it. That one line of code may look harmless inside the iteration, but remember, it may be executed hundreds of thousands of times a day when it only needs to be executed a couple of hundred times.

In general, keep the code tight within a subfile load iteration. For instance if you are performing record selection within the loop, ensure that this method is the best way to select records. In other words, you may be reading a file and if the delete code is not a "D," then you would load the record. This program might perform better if this record selection were done by reading a logical file that has already selected out the deleted records.

Look into the future for maturing files and the impact they will have on future performance. If you are developing the application using test data, remember that these files usually contain a limited set of data. A load-all subfile may run fine in this environment, but migrate the application into production, and you may have a severe problem on your hands.

Specifically, keep the iteration loop "top-down." Minimize or eliminate any subroutine calls or external calls inside the iteration.

Also, do not add any functionality that the user does not need. For example, suppose you were writing an accounts receivable aging subfile inquiry. The user requested that the customer number, customer name, and date due be placed on the subfile display. You might say that it wouldn't be any problem to add the number of days that the invoice is past due.

If the user doesn't want it, don't add it. Not only are you cluttering their display with another field, but you are about to cause this program to be a system resource hog. Suppose you loaded 20 records into the subfile at a time. You would have to perform the routine to calculate the number of days past due *20 times*. This routine may operate in a split second when you are dealing with one record, but now you are talking about 20 split seconds. Think of how this routine may affect you if this subfile were a load-all subfile, with the possibility of loading a hundred records.

You may not want to do any date formatting inside the subfile load loop, or at least ensure that the date format routine is one that does not require a lot of CPU time to process. Take the accounts receivable aging inquiry example above. Suppose the user wants the invoice date, discount date, and past due date on the display. That is three date fields. You would store these fields in YY/MM/DD format in the files. If the user does not mind them coming up in YY/MM/DD format, then do not automatically format the dates into MM/DD/YY. If you perform date formatting by multiplying the date by 10000.01 or 100.0001, then you are about to spend a lot of CPU time to do nothing more than to flip the dates around. If you load 20 records into this subfile at a time, you would be performing *sixty* of these expensive date calculation routines every time you loaded records into the subfile. Even if you use the more economical method of formatting dates by utilizing data structures, you would still be performing this routine 60 times. If you are changing the format of the date by calling a CL program, you would be performing this call 60 times per load. And think of what would happen if you were loading a hundred records at a time.

In essence, you must be very careful of what functionality you give a user if it requires some calculation statements inside the load loop. If you must perform a call to a program inside the load loop, and the subfile program is heavily utilized or you are calling a CL program, then set the called program up into a datapool. This way, the program will already reside in memory when it is called. Otherwise, the program may have to be retrieved from the disk system many times during each loading process.

High Non-DB Pool Fault Rates

If you utilize subfiles heavily on your system, and they do a lot of external calls from within the load loop, you may experience high non-DB faulting in your system memory pools.

This high non-DB faulting is not a sign that you need more memory. It is caused by the memory page stealing algorithm removing a recently called program from memory, only to have to call it again right after it removes it. To see if this problem is happening in your system, use the WRKSYSSTS command to display your pool information. If the non-DB fault rate is much higher than the DB fault rate over a

time frame of at least five minutes, with no jobs becoming ineligible, then external calls are being made inside a program that loops. Subfile programs probably make up the largest number of looping programs on your system.

If your system is suffering from high non-DB fault rates, then you will benefit from a new option available in V2R3. You can change the paging option for a shared pool so that the memory-stealing algorithm is changed. This change will keep programs that have been brought in from the disk system many times in memory longer, reducing your non-DB fault rate. You can change the pool paging option using the CHGSHRPOOL command. With this command, change the pool paging option to *CALC.

If you don't have V2R3, you can use datapools to keep the heavily called programs in memory. You can do this using the SETOBJACC command.

Spend Time on the Subfile Design

It is very important to keep in mind that subfile applications and support files will mature. You don't want an efficiently running program to turn into a slug in six to 12 months. Spend a little extra time on these programs in "wise development" up front, so that you won't be modifying them because of performance implications later.

Glossary

Access path The order in which records in a database file are organized for processing by a program. Access paths can be kept on the system permanently (such as a physical or logical file) or temporarily. The OPNQRYF command may create a temporary access path for use one time, then discard the access path.

Active record format If a record format is on the display, it is considered an active format, except in the case of windows. More than one window format can be on the display, but only one can be active at a time. If the format is not a window and it is on the display, it is considered active. Contrast with inactive record format.

Active subfile A subfile in which a write operation was issued to the subfile record format or to the subfile control record format with the DDS keyword SFLINZ in effect. A subfile can be made inactive by deleting it with the SFLDLT (subfile delete) keyword.

Active subfile record A subfile record is made active if it is written, changed, or initialized. Contrast with inactive subfile record.

Attribute character A character associated with a field in a display file record format that defines how the field is displayed (i.e., reverse image, blinking, color, and so on).

Binary The numbering system with a base of two. Contrast with decimal and hexadecimal.

Changed subfile record A subfile record into which a user has entered data, or a subfile record for which a write or change operation was issued with the DDS keyword SFLNXTCHG or DSPATR(MDT) in effect.

Command attention (CA) key This is a specification in DDS which allows you to define which function key to use. Data is not returned to the system. Contrast with command function (CF) key.

Command function (CF) key This specification in DDS allows you to define a function key. Data is returned to the system. Contrast with command attention (CA) key.

Decimal The numbering system that has a base of ten. Contrast to hexadecimal and binary.

Display file A device file to support a display station.

DROP When used with subfiles, this is a way you tell the system to display only the first line of each multiple line subfile record. This is specified with the subfile drop (SFLDROP) keyword. Contrast with FOLD.

Dynamic select/omit Selection and omission of logical file records performed during processing, instead of when the access path (if any) is maintained. The select/omit processing is performed when the file is accessed. Dynamic select/omit may also be used when no keyed access path exists.

Field-level specifications In DDS, specifications coded on the same line as a field name or on lines immediately following a field name. In any case, these specifications directly affect individual fields, such as the field's color or other display attributes. See also file-level specifications, record-level specifications.

File-level specifications In DDS, specifications coded on the lines before the first record format name. These specifications affect the entire file. See also field-level specifications, record-level specifications.

Fold When used with subfiles, fold means to continue a subfile record on a second line, or more, on the display. This allows a subfile record to occupy many lines on the display, and when you specify that the subfile is to be folded (SFLFOLD), you

tell the system you want to see all display lines for that subfile record. Contrast with drop.

Hexadecimal The numbering system with a base of 16. Contrast to decimal and binary.

Hidden field A field in a display file that is passed to and from the program but is not sent to the display. If used in a subfile record, the field can be written to and read from, but is never present on the display.

Inactive record format An inactive subfile record or any record format that is not currently shown on a display. See also inactive subfile record. Contrast with active record.

Inactive subfile record When the subfile is initialized with records not active SFLINZ and SFLRNA keyword combinations, the subfile records are inactive. Contrast with active subfile record.

Key field A field used to sort the records of a file.

Keyword A name that identifies a function within DDS.

Message file A file object with a type *MSGF that contains messages and their descriptions.

Message identifier A seven-character code that identifies a predefined message, and is used to get the message description from a message file.

Message line An area on the display where messages are displayed. Message lines are usually line 24 on a normal display, and the last line on a window display. With message subfiles, you can define one or more message lines, and start them anywhere on the display.

MSGCTL (Message Subfile Control Format) Use this record-level field to specify that this format is a message subfile control format. Message subfiles are similar to regular subfiles in that they require a subfile format and a control format. Therefore, this keyword is similar to the SFLCTL keyword for regular subfiles.

MSGSFL (Message Subfile Format) Use this record-level field to specify that this record format is a message subfile format. Message subfiles are similar to regular

subfiles in that they require a subfile format and a control format. Therefore, this keyword is similar to the SFL keyword for regular subfiles.

Message subfile A type of subfile where the subfile records are messages from a program message queue.

Page Each group of records in a subfile that are displayed at the same time. The number of subfile records that can be displayed at one time is controlled by the SFLPAG (subfile page) keyword.

Page down This is also known as roll forward, in which the user is wanting to scroll through subfile records towards the end of the subfile. Contrast with page up.

Page up This is also known as roll back, in which the user is wanting to scroll towards the front of the subfile. Contrast with page down.

Read operation An input operation that obtains data from a file or device and passes it to the program. Subfiles have a read operation unique to themselves known as the READC (read next changed subfile record).

READC Opcode An RPG op code for use strictly with subfiles. It will return the next changed subfile record to the program.

Record A set of related data treated and accessed as one unit.

Record format A section of a file that identifies records associated with that format. Subfiles must have at least two formats—the subfile format and the subfile control format.

Record-level specifications These specifications affect an entire record format, such as whether the format is a subfile, a window, and so on. They may be coded on the same line as the record format name, or on lines immediately afterward. They must be coded before the first field defined on the record format. See also field-level specifications and file-level specifications.

Relative record number A number that associates a file record to the location in the file where the record resides. For example, the first record in a database file or subfile has a relative record number of 1.

SFL (Subfile) This record-level keyword is used to specify that this format is a subfile format. The subfile format must be placed immediately before the subfile control

format, which is identified with the keyword SFLCTL. This keyword has no parameters.

At least one displayable field must be specified in the subfile record format unless the subfile is a message subfile. The locations specified for fields in the subfile format must be displayed after the last displayable subfile format line.

Option indicators are not valid for this keyword.

Besides SFL, the following keywords are also valid on the subfile record format:

Message Subfile Only

- SFLMSGRCD (required at the record level)
- SFLMSGKEY (required at the field level)
- SFLPGMQ

All other Subfiles

- CHANGE
- LOGINP
- CHECK(AB)
- LOGOUT
- CHECK(RL)
- SETOF
- CHGINPDFT
- SETOFF
- INDTXT
- SFLNXTCHG
- KEEPTEXT

The following keywords are not valid at the field level when specified on the subfile record format:

- DATE
- ERRMSGID

- DFTVAL
- MSGID
- ERRMSG
- TIME
- SFLCLR (Subfile Clear)

Specify this record-level format keyword if you would like to be able to clear the subfile of its records inside your program. After a clear, the subfile remains active, but contains no records. The SFLCLR differs from SFLDLT in that the subfile delete keyword actually deletes the subfile, and it can no longer have input/output operations performed against it.

The SFLCLR differs from the SFLINZ keyword in that after being cleared, the subfile contains no data.

This keyword has no parameters.

If SFLCLR is attempted on a subfile that contains no records, then the clear operation is ignored.

This optional keyword is valid only for the subfile control record format. An option keyword is required. If this keyword does not exist, then OS/400 will clear the subfile every time it outputs the control record format.

SFLCSRPRG (Subfile Cursor Progression) This keyword is used to control the progression of the cursor through a screen of subfile records. Whenever the cursor is positioned to a field that specifies the SFLCSRPRG keyword, the cursor will be advanced to the same field on the next subfile record if a field exit or tab key is pressed.

SFLCSRRRN (Subfile Cursor Relative Record Number) Keyword This keyword will return the relative record number of the subfile record on which the cursor was placed.

The format of the keyword is:

```
SFLCSRRRN(&FIELDNAME)
```

Notice that the field used as the parameter must be preceded with the "&" symbol.

The parameter is required. It must also be defined on the same record format as a hidden field, signed numeric, a length of 5 with 0 decimal places. The current subfile

relative record number that the cursor is on is placed in this field automatically whenever the format is read.

The field will contain the value 0 if the cursor is not located in the subfile associated with this subfile control record, or if the cursor is located within the subfile, but is not in an active record within the subfile.

SFLCTL (Subfile Control) This record-level keyword specifies that this record format is to be a subfile control record format. This record format must immediately follow the subfile record format.

The format of the keyword is:

```
SFLCTL(subfile-record-format-name)
```

The subfile control record format can contain field descriptions as well as subfile control keywords.

Your program can display subfile records only by issuing an output. The subfile record format (SFL keyword) defines the layout of the records in the subfile as opposed to the subfile control record format (SFLCTL keyword), which defines how the subfile can be displayed, cleared, deleted, and initialized.

The subfile control format works upon the subfile as an entity, whereas the subfile format works with individual subfile records.

The program sends output operations to the subfile control record format to display, clear delete, or initialize the subfile.

The following is a summary of subfile keywords used with the SFLCTL.

Required:

- SFLCTL
- SFLDSP
- SFLPAG
- SFLSIZ
- SFLDSPCTL

Optional:

- SFLCLR
- SFLLIN

- SFLDLT
- SFLMSG
- SFLDROP
- SFLMSGID
- SFLDSPCTL
- SFLPGMQ
- SFLEND
- SFLRCDNBR
- SFLENTER
- SFLRNA
- SFLFOLD
- SFLROLVAL
- SFLEND
- SFLRCDNBR
- SFLENTER
- SFLRNA
- SFLFOLD
- SFLROLVAL
- SFLINZ

The SFLDSPCTL keyword is required if your program sends an input operation to the subfile control record format.

If subfile size equals subfile page, the following keywords are ignored.

- SFLDROP
- SFLFOLD
- SFLROLVAL

The USRDFN keyword is not valid for the subfile control record format.

Option indicators are not valid for this keyword.

SFLDLT (Subfile Delete) Use this record-level keyword with an option indicator on the subfile control record format to enable your program to delete a subfile. You would want to do this when the maximum number of subfiles in a display file are already active (12), and another subfile needs to be made active. In this case, your program must delete one of the active subfiles before making another active. Otherwise an error will be issued.

This keyword has no parameters.

If your program issues a subfile delete to a subfile that is not currently active, the subfile delete command is ignored.

Option indicators are required for this keyword.

Contrast this keyword to subfile clear (SFLCLR).

SFLDROP (Subfile Drop) Use this record-level keyword on the subfile control record format to assign a function key that the display station user can press to fold or drop subfile records that require more than one display line. The first line of a subfile record is always displayed. Using subfile drop allows you to toggle between the first line of a subfile being displayed, and all lines of the subfile record being displayed.

The format of the keyword is:

```
SFLDROP(Function Key)
```

Without SFLDROP, if the subfile record was contained on more than one line on the display, all display lines for the subfile record would be displayed.

By pressing the specified function key, the subfile will toggle from dropped to folded modes.

A warning message is sent at compilation time if SFLDROP has been specified, but the subfile record only occupies a single line on the display.

If subfile size equals subfile page, SFLDROP is ignored.

If the subfile contains input-capable fields, you should specify a CF key rather than a CA key, or data will be lost when the CA key is pressed.

SFLDROP can be specified on the same subfile control record format as the SFLFOLD keyword, but both keywords must use the same function key. You can use indicators to tell the system which mode should be used initially when the subfile is displayed.

SFLDSP (Subfile Display) This record-level format keyword specifies if the subfile is to be displayed or not when an output operation is performed on the subfile. You usually use an indicator with this keyword. Before outputting the subfile, check within the program to see if there are any records in the subfile. If there are, set the indicator so the SFLDSP keyword is activated. If the program sees that there are no records in the subfile, you should toggle the indicator, so the SFLDSP keyword is not activated. If you specified to activate the SFLDSP on an empty subfile, then you would be issued an error message upon outputting the subfile.

This keyword has no parameters.

This keyword is required and is valid only for the subfile control record format.

Option indicators are valid for this keyword.

SFLDSPCTL (Subfile Display Control) You must use this record-format keyword if you are going to perform an input operation to your subfile.

This keyword has no parameters.

Option indicators are valid for this keyword.

SFLEND (Subfile End) This record-level keyword will show whether or not there are more displayable records in a subfile. You can either have a "+" character or the "MORE..." phrase to show more records.

The format of the keyword is:

```
SFLEND{(*PLUS | *MORE)}
```

The parameter values *PLUS and *MORE are optional. If no parameter is specified, *PLUS is used.

*PLUS tells the system to use the plus sign to indicate that you can use the page up key to see more records.

*MORE tells the system to use the More... text to indicate that you can use the page up key to see more records. *MORE also tells the system to use the "Bottom" phrase if the end of the subfile has been reached.

When *MORE is specified, the subfile takes up one more line on the screen, because the "More..." phrase is placed one line below the last subfile record. The "+" character, on the other hand, is displayed on the last subfile line.

An option indicator must be specified for this keyword. This indicator conditions whether or not the end of subfile message ("Bottom" or " ") can be displayed. If the

end of the subfile has been reached, the subfile will only display the end of subfile message if the conditioning indicator is on. Otherwise, the subfile will display the more records phrase ("+" or "More..."). The indicator does not control if the "MORE..." phrase or the "Bottom" phrase is displayed, it can only keep the "Bottom" message from being displayed if you are at the end of a subfile. This is useful in expanding and page-at-a-time subfiles. For example, page-at-a-time subfiles are always at the end of the subfile, since only the records displayed are in the subfile. So, if you specify SFLEND for a page-at-a-time subfile, it will always read "Bottom" or " ", unless you do not turn on the conditioning indicator. What you would do to prevent this is to keep the conditioning indicator off, which would cause the message to display "More..." or "+", until you loaded all of the subfile records. Once you have loaded all the records, you turn the conditioning indicator on. At this point, on a page-at-a-time subfile, the message would read bottom. If you do this on an expanding subfile, however, the message would still read "More..." or "+", until you reach the end of the subfile AND the conditioning indicator is on.

SFLENTER (Subfile Enter) This record-level keyword allows you to substitute a function key for the Enter key. This allows the Enter key to be used as the rollup key, which has some advantages when using a mouse-controlled display station, in which the Enter key is one of the mouse buttons. A user can point and click where a subfile was to pop up at, then with the press of a mouse key, allow the subfile to be paged forward.

The format of the keyword is:

```
SFLENTER(Function Key)
```

This optional keyword is valid only for the subfile control record format.

Option indicators are not valid for this keyword.

☛ **Note:** This keyword is in effect only until the next output operation. At the next output operation, the specifications for that record apply.

If more than one subfile using SFLENTER is displayed at the same time, the only CA or CF key in effect as an Enter key is the CA or CF key specified for SFLENTER on the most recently displayed subfile. The cursor position at the time the Enter key is pressed determines which subfile is affected.

SFLFOLD (Subfile Fold) This keyword is the opposite of the SFLDROP keyword. Whereas on a multiple line subfile the SFLDROP only displayed the first subfile displayable line, the subfile fold specifies that all subfile displayable lines will be initially displayed. A function key is specified on this keyword, which when pressed, will cause the mode of the subfile to toggle between folded and dropped

The format of the keyword is:

```
SFLFOLD(CAnn | CFnn)
```

A warning message is sent at compilation time if the SFLFOLD keyword is specified, but the subfile occupies only one displayable line.

If subfile size equals subfile page, an error message is issued and SFLFOLD is ignored.

It is recommended that you specify a command function key (CF) instead of a command attention key (CA), since the CA key would cause you to lose the data on the display.

SFLFOLD can be specified on the same subfile control record format as the SFLDROP keyword. The same function key must be specified. You can condition each with an indicator so the program can control which mode the subfile will initially be displayed. If both keywords are active, SFLFOLD is used.

Option indicators are valid for this keyword.

When the subfile is displayed, the workstation user can press the CF03 key to change the subfile from folded to truncated form and from truncated to folded form.

SFLINZ (Subfile Initialize) This record-level keyword is used to initialize a subfile with records. This keyword is mainly used in input and update subfiles. You must initialize a subfile, or write records to a subfile, before displaying it. If you are writing an input subfile, you would not be writing any records to it. Therefore, you can use the SFLINZ keyword to initialize the subfile with as many subfile records as specified by the subfile size. You can use SFLRNA keyword in conjunction with this keyword to specify that the subfile should be initialized with inactive records. Without SFLRNA, the SFLINZ keyword will initialize a subfile with active records.

This keyword has no parameters.

If any display attributes have been specified for the subfile records, and they are conditioned with indicators, these indicators are initialized in the *OFF state. This will affect the way your subfile is initially displayed.

After your program sends an output operation to the subfile control record with SFLINZ in effect, all records in the subfile are considered active but not changed. They are considered changed only when the workstation user changes them or when your program sends an output operation to the subfile record format with the SFLNXTCHG keyword in effect.

Specify SFLINZ with the SFLRNA keyword so that your program can initialize a subfile, then add records to that subfile without having the initialized records considered active.

Specify SFLINZ with the SFLPGMQ keyword so that your program can build a message subfile with a single output operation.

Option indicators are valid for this keyword. Display size condition names are not valid.

SFLLIN (Subfile Line) This record-level keyword is used to define how many spaces are to be inserted between subfile records. The default is for the system to insert enough spaces so that each subfile record fits on one display line. Using this keyword, you can cause a subfile to be displayed horizontally, in which more than one subfile record occupies the same display line.

The format of the keyword is:

```
SFLLIN(spaces)
```

If you had a subfile in which each line was 35-bytes long, and you specified 1 space between subfile records, (SFLLIN(1)), then the subfile would appear like this.

```
REC1    REC3
REC2    REC4
```

If you do not specify the SFLLIN keyword, the system would display the same subfile like this:

```
REC1
REC2
REC3
REC4
```

SFLMODE (Subfile Mode) This record-level keyword returns to the program what mode the subfile was in when control was returned to the program. Mode indicates whether the subfile in the drop mode or fold mode.

The format of the keyword is:

```
SFLMODE(&mode)
```

The mode parameter is required. It specifies the name of a hidden field that, on input, will contain the subfile mode.

The field must be defined in the subfile control record format as a character field with a length of 1, hidden.

The mode field will contain the value 0 if the subfile is in folded mode and it will contain the value 1 if the subfile is in dropped mode.

IF a SFLDROP or SFLFOLD keyword is not specified on the subfile control record, the mode value returned is 0.

Option indicators are not valid for this keyword.

SFLMSG (Subfile Message) This keyword allows you to display a message on the message line of a subfile. You can hard code a message text. The next time the format is written, the message will be displayed on the message line.

The format of the keyword is:

```
SFLMSG('message-text' <response-indicator>)
```

SFLMSG Keyword: Specify SFLMSG is similar to the ERRMSG keyword. The parameters specify a message text and, optionally, a response indicator. The message text is the message to be displayed.

If you specify a response indicator, it should be the same as the option indicator used to condition SFLMSG. The next input operation to the format will turn off the option indicator, so it will automatically be turning off the response indicator also.

SFLMSGID (Subfile Message Identifier) The SFLMSGID keyword is similar to the SFLMSG keyword, except that instead of hard coding the message text, you specify the message ID and the message file that the message is contained in. This will, in turn, display the appropriate message on the message line of a subfile as soon as a write to the subfile is performed.

The format of the keyword is:

```
SFLMSGID(msgid <library-name/>msg-file <response-indicator>
      <&msg-data>)
```

The SFLMSGID Keyword is similar to the ERRMSGID keyword.

For SFLMSGID, the parameters specify:

- The mandatory message identifier.
- The mandatory message file and optionally, the message library.
- The optional response indicator.
- The optional message data field name.

The response indicator, if specified, should be the same as the option indicator used to condition the SFLMSGID keyword. This allows OS/400 to, upon the next input to the format, set off the conditioning and response indicators, since they are one in the same.

The msg-data field, if specified, contains the replacement text for the specified message. The field must exist in the record format, and the field must be defined as a character field (data type A) with usage P.

SFLMSGKEY (Subfile Message Key) This record-level keyword is used to specify the field used as the message key. This keyword is only valid for message subfiles, and must be specified on the subfile format.

This keyword has no parameters.

This parameter field is predefined as a four-position, alphanumeric, hidden field.

This parameter field must always be the first field defined in the subfile record format. The field name and SFLMSGKEY are the only DDS you can specify for this field.

Option indicators are not valid for this keyword or with the associated field.

SFLMSGRCD (Subfile Message Record) This record-level keyword is used to specify what line number to use as the starting line for the message line on the display. This keyword is only valid for message subfiles, and must be specified on the subfile format.

The format of the keyword is:

```
SFLMSGRCD(line-number)
```

The parameter value specified with SFLMSGRCD specifies the first line on the display on which messages are displayed. The value specified must not be greater than the maximum line number for the display size being used. The number of messages displayed is controlled by the SFLPAG. If you set SFLPAG to 4, and the SFLMSGRCD to 20, you will get a message subfile that consists of four displayable lines starting at line 20.

SFLNXTCHG (Subfile Next Changed) This record-level keyword allows you to “mark” subfile records as being changed. This is needed if your subfile is returned to the program, and upon validity checking, is determined to contain errors. After the errors are corrected, only the corrected subfile entries would be marked as changed. You would want all of the original subfile changes, as well as the corrected ones, to be passed back to the program. SFLNXTCHG keyword allows this to happen.

This keyword has no parameters.

The following is an example of why to use the SFLNXTCHG keyword.

A workstation user changes some records in a displayed subfile (this could be for a data-entry application or a data-update application). After changing some records, the work station user presses the Enter key, and the program reads only the changed records with get-next-changed operations. (For example, READC in RPG III and READ-SUBFILE-NEXT-MODIFIED in COBOL.)

If the program detects keying errors in the changed records, it can send update operations (UPDAT in RPG III, REWRITE SUBFILE in COBOL) to the subfile records in error, setting indicators so that SFLNXTCHG is in effect during the update operations. These update operations are sent to the subfile record format.

After all the records in error have been updated, the program sends an output/input operation to the subfile control record format to display the subfile again.

With the subfile displayed again, the workstation user types the data again and presses the Enter key. If the data is correct, the program does not display the subfile again.

The records in error (and any other records changed by the workstation user) are returned to the program on the next get-next-changed operation. This is because SFLNXTCHG caused the subfile records to be considered changed even though the work station user did not change them. This allows the program to prohibit the workstation user from ignoring program-detected keying errors in subfile records.

Subfile Operations without SFLNXTCHG

If SFLNXTCHG is not specified, or is specified but not selected on the update operations to the subfile records, then the workstation user can simply press the Enter key instead of correcting the program-detected errors. The program then reads no records because the get-next-changed operations find no changed records the second time the Enter key is pressed.

Option indicators are valid for this keyword.

You cannot specify SFLNXTCHG with the SFLMSGRCD keyword.

The following example shows how to specify the SFLNXTCHG keyword.

SFLPAG (Subfile Page) This record-level keyword is specified on the subfile control format, and is used to determine how many subfile records to show per display screen.

The format of the keyword is:

```
SFLPAG(number-of-records-to-be-displayed)
```

When you specify the same parameter values for SFLPAG and SFLSIZ, the maximum number of records that can be contained in the subfile equals the maximum number of subfile records that can appear on the display at one time. This is what happens in a page-at-a-time subfile.

If subfile size equals subfile page, the following keywords are not allowed:

- SFLDROP
- SFLFOLD
- SFLROLVAL

SFLPGMQ (Subfile Program Message Queue) This keyword specifies the program message queue that contains the subfile records for a message subfile. This keyword is located on the subfile format and the subfile control format (SFLINZ must be specified) of a message subfile.

This keyword has no parameters.

This field is predefined as a 10-position, alphanumeric, hidden field.

The program message queue name that is contained in the parameter field must be left justified and padded with blanks.

This field is required on the subfile record format, and will build the subfile one message at a time through multiple output operations to the subfile record format.

You can also specify this field on the subfile control record format, which will build the subfile all at once through a single write operation.

The SFLPGMQ field can contain a special value, * (asterisk), instead of a program message queue name. This will cause the program its own message queue as the program message queue. The asterisk must be left justified, padded with blanks on the right.

☛ **Note on the subfile program queue keyword and V2R3 of OS/400:**
The subfile program queue keyword has been modified under V2R3 of OS/400 to allow its use with the ILE (integrated language environment). The length of the program queue can either be the default of value of 10, or 276 bytes. Using 276 bytes allows the program queue to be used with the ILE. If you specify a program queue name of 276 bytes, the first 256 bytes must contain the qualified

name of the ILE program queue name. Bytes 257 through 266 can optionally contain the ILE module name. Bytes 267 through 276 can optionally contain the bound program name.

If you specify a 276-byte program queue, you are allowed to use a none ILE program queue as long as bytes 257 through 276 are blank.

If you use the program queue keyword on both the subfile format and subfile control format, the program queue definition must be identical between the two.

SFLRCDNBR (Subfile Record Number) This field-level keyword is used to specify what page of the subfile to display on the next write operation. The system will display the subfile page on which the relative record number is located.

The format of the keyword is:

```
SFLRCDNBR<(CURSOR)>
```

If CURSOR is specified, the cursor is placed in the subfile record of the relative record number identified by the contents of this field. The cursor is positioned at the first input-capable field in the subfile record. If there is no input-capable field, the cursor is positioned at the first output field.

This field must be a zoned decimal field with zero decimal positions. It must have the keyboard shift attribute of signed numeric, and it can be up to 4 digits in length. It must be defined as an output-only, an output/input, or a hidden field, with hidden being the preferred method.

If a value less than 1 or a value greater than the number of records in the subfile is contained in this field on an output operation to the subfile control record format, an error is returned to your program.

You cannot specify both SFLRCDNBR and SFLROLVAL for the same field.

Option indicators are not valid for this keyword.

SFLRNA (Subfile Records Not Active) This record-level keyword is used in conjunction with the SFLINZ (initialize subfile records) to specify that the initialized subfile records should be set initially to the inactive state.

This keyword has no parameters.

SFLRNA is normally used so that a program can write some records to the subfile before displaying it, and the work station user can then add records to the subfile.

When your program displays a subfile initialized with the SFLINZ and SFLRNA keywords in effect, fields in inactive records have the following values:

- Character fields are blank.
- Numeric fields are zero-filled.
- An input-only field with a constant value specified has the constant value.
- The SFLINZ keyword is required when SFLRNA is specified.

SFLRNA cannot be specified for a message subfile (identified by the SFLMSGRCD keyword on the subfile record format).

Option indicators are not valid for this keyword.

SFLROLVAL (Subfile Roll Value) This field-level keyword is used to tell the system how many subfile records to roll whenever a roll-up or roll-down key was pressed. The default is to roll one full page worth of subfile records.

This field must have the keyboard shift attribute of signed numeric with zero decimal positions. It can be up to 4 digits in length, and it must be defined as an output/input or input-only field.

This keyword has no parameters.

The SFLROLVAL value will remain the same until it is changed by the user.

If subfile size equals subfile page, SFLROLVAL is ignored.

This field is returned to your program as part of the input for this subfile control record.

If pressing the page down key pages beyond the first page of records of the subfile, one of the following conditions occurs. If the first page of records is not currently displayed, paging down will display it. If the first page of records is currently displayed, paging down will cause a message to be displayed.

If pressing the page up key pages beyond the last active record of the subfile, one of the following conditions occurs.

If the last full page of records is not already displayed, keying page up will display it. If the last full page of records is already displayed, keying page up will cause a message to be displayed.

The SFLEND keyword notifies the work station user when the last subfile record is displayed, therefore it should be used whenever the SFLROLVAL keyword is used.

SFLSIZ (Subfile Size) This record-level keyword is used to set the initial size of the subfile.

The format of the keyword is:

```
SFLSIZ(number-of-records-in-subfile)
```

If subfile size equals subfile page, then the maximum number of records that can be written to the subfile is equal to the subfile size. Attempting to write more than this number of subfile records will cause an error.

When the subfile size does not equal the subfile page, the subfile is initially set up to handle the number of subfile records by the value of subfile size. Two things can now happen. If you attempt to write more records to the subfile than the size, and the write operation is conditioned with an indicator, the indicator is set on. This allows you to condition our load loop to only fill the subfile up with the number of records equal to the subfile size.

If the write operation is not conditioned, the subfile is extended. You can write more records than the subfile size (up to 9999). But, these additional records are not in a contiguous space as the initial subfile records. Therefore, a careful number should be applied to the subfile size, because setting it too low can cause performance degradation due to the continual processing of subfile records outside of the contiguous space of the initial subfile records. In contrast, you do not want to set this value too high because the subfile will be initialized to handle this many records, and memory space for this many records must be set up in memory, regardless of whether or not they are ever used. If you set the size to 9,999, but loaded an average of only 100 records, the job's process access group would be large enough to handle 9,999 records, but actually only contain 100. Sizing the process access group this high could cause performance to degrade, because this job must occupy enough memory for all 9,999 subfile records.

Subfile A set of records from the same display record format that can be displayed at the same time at a workstation. The record layout for a single record is defined, then the subfile is told how many of these records to size the subfile at, and how many of these records can be viewed on the display screen at the same time.

Subfile control record format One of the record formats required to define a subfile. The subfile control record format describes the size of the subfile and the size of the subfile page, and is used by the program to write the subfile to and read the subfile from the display. The subfile control format is used to act upon the subfile as a whole. See also subfile record format.

Subfile record format One of the record formats required to define a subfile. The subfile record format defines the fields in a subfile record. This format is used to access each record, for input or output. See also subfile control record format.

Write operation An output operation which sends data to a file or to an output device, such as a display file.

Index

!

*ALLIO, 444, 447, 448, 458
*ENTRY Parameter List, 126
*FIRSTIO, 444, 447, 448, 458
*INZSR, 4, 103, 122, 429
*MINWAIT, 444, 447, 449

A

Access Path, 440, 441, 445, 456, 459, 477
Access Plan, 447
Active Record Format, 477
Active Subfile, 477
Active Subfile Records, 298, 341, 477
Active Window, 83, 91
API, 429
Application Programming Interface, 429
See also API
ASSUME, 33, 73, 77, 81, 111
Attribute Character, 477
Attributes, 88

B

Binary, 103, 429, 478
Binary Fields, 125
Border Attributes, 92, 312
Border Characters, 92
Border Color, 92

C

CHAIN, 345, 348, 349, 350
CHANGE, 481
Changed Subfile Record, 386, 393, 394, 478
CHECK(AB), 481
CHECK(RL), 481
CHGINPDFT, 481
CHGSHRPOOL, 475
CLRL, 65, 67
Communications, 84

D

DATE, 481
DDS, 1, 9, 15, 24, 27, 34, 37, 44, 65, 83, 108, 124, 139, 154, 230, 258, 287, 306, 331, 355, 393, 425, 466, 471
DDS Keywords, 83, 84, 258
    Remove Window, 84
    Restore Display, 84
    Window Border, 84
Decimal, 103, 125, 478
DFTVAL, 482
Display File, 5, 7, 8, 9, 15, 24, 27, 41, 44, 56, 57, 63, 72, 77, 81, 85, 89, 98, 107, 108, 111, 122, 161, 162, 200, 210, 218, 278, 380, 448, 469, 470, 472, 478
Display Format, 4, 29, 67, 72, 81, 321
Dynamic Processing, 440, 446

E

ERRMSG, 91, 482
ERRMSGID, 91, 481
Error Messages, 91
EXFMT, 4, 35, 67, 73, 108, 123, 124, 145, 180, 379
Expanding Method, 14, 107, 268, 465
Expanding Subfile, 199, 459, 467

F

Field Exit, 303, 304, 346, 347, 402, 410, 411, 412
Field-Level Specifications, 478
File Information Data Structure, 103, 320, 321
File Size, 443
File-Level Specifications, 478

H

Help, 82
Hexadecimal, 103, 125, 479
Hidden Field, 26, 27, 184, 290, 371, 388, 406, 479
Horizontal Subfile, 187, 197

I

Inactive Record Format, 479
Inactive Subfile Record, 341, 479
INDTXT, 481

Input Subfiles, 325

K

KEEPTEXT, 481
Key Field, 440, 441, 445, 455, 479
Keywords, 20, 30, 31, 32, 33, 37, 43, 65, 81, 83, 89, 91, 169, 403, 406, 426, 427, 428, 479

L

Load-All Method, 107, 268, 465
Load-All Subfile, 15, 38, 43, 44, 48, 51, 52, 55, 56, 63, 107, 120, 124, 129, 130, 131, 132, 199, 200, 201, 207, 268, 269, 277, 355, 359, 428, 440, 451, 456, 459, 466, 467, 471
    Coding, 129
    Roll Key Consideration, 108
Loading, 11, 267
Loading Method, 267, 465, 467, 468
    Expanding Subfile, 465, 468
    Load-All Method, 465
    Single-Page Load Method, 465
Loading Process, 11, 128, 199, 200, 450, 474
Local Area Network, 85
Locks, 354, 355, 399
Logical Files, 450
LOGINP, 481
LOGOUT, 481

M

Maximum Subfile Records, 129, 353
Message File, 286, 290, 426, 430, 431, 437, 438, 467, 479
Message Identifier, 286, 479
Message Line, 82, 286, 290, 428, 431, 435, 479
Message Subfile, 3, 286, 419, 425, 426, 427, 428, 429, 437, 438, 480
    Format, 426
Message Subfile Control Format, 427, 479
Message Subfile Format, 479
Messages, 72, 82, 90, 91, 92, 154, 278, 286, 290, 419, 425, 426, 428, 429, 438
    Error, 91
Miniplan, 446, 447, 448
MSGCTL, 479
MSGID, 482
MSGLIN, 82
MSGSFL, 479
Multiple Occurrence Data Structures, 37, 305, 370
Multiple Subfiles, 3, 293, 323
Multiple Subfiles Displays, 304

N

Non-DB Faulting, 474

O

Open Query File, 3, 439, 448
OPNQRYF, 3, 439, 440, 444, 445, 446, 447, 448, 449, 450, 451, 455, 456, 459, 463
    OPTIMIZE parameter, 440
OPTALLAP, 446, 450
OPTIMIZE, 440
Optimizer Biases, 444
Optimizer Strategy, 442, 448
Over/Under Subfile Method, 294
OVRDBF, 439

P

PAG, 129
Page, 480
Page Down, 480
Page Up, 480
Page-at-a-Time Subfile, 469
Performance, 3, 5, 84, 85, 129, 131, 269, 290, 335, 346, 440, 451, 459, 471, 473, 475
Point and Click, 82, 99, 186, 251
Pop-Up Window, 82, 103, 108, 112
Process Access Group, 129
Program Message Queue, 425
Program to System Fields, 90, 98, 125

Q

QDT, 447
QMHRMVPM Program, 429
QMHSNDPM Program, 430
Query Definition Template, 447
Query Optimizer, 440, 441, 443, 448, 449, 451, 455, 456, 463
Query/400, 440, 447, 448

R

Read, 73, 480
Read Next Change, 5, 350
Read Next Changed Subfile Record, 480
READC, 5, 350, 353, 371, 394, 480
Record, 480
Record Format, 10, 28, 83, 88, 92, 111, 184, 303, 406, 480
Record Layout, 7, 13, 25, 26, 370
Record-Level Specifications, 480
Relative Record Number, 29, 39, 41, 42, 62, 121, 122, 123, 155, 176, 183, 217, 222, 230, 287, 290, 331, 380, 466, 480
RMVMSG, 429
RMVWDW, 84, 87
Roll Back, 14, 15, 480
Roll Down, 27, 44, 123, 259, 287
Roll Forward, 14, 15, 480
Roll Keys, 9, 15, 24, 37, 56, 57, 107, 108, 124, 200, 268, 278, 466
Roll Up, 27, 44, 123, 259, 267, 287
RUMBA/400, 99

S

SETOBJACC, 475
SETOF, 481
SETOFF, 481
SFILE, 121
SFL, 10, 30, 480
SFLCLR, 32, 61, 240, 247, 331, 332, 466, 482, 483, 485
SFLCSRPRG, 408, 482
SFLCSRRRN, 176, 482
SFLCTL, 10, 31, 36, 479, 481, 483
SFLDLT, 32, 477, 482, 484, 485
SFLDROP, 32, 33, 169, 399, 403, 406, 478, 484, 485, 493
SFLDSP, 20, 31, 33, 34, 37, 43, 123, 230, 250, 427, 428, 483, 486
SFLDSPCTL, 20, 31, 32, 33, 34, 37, 43, 250, 427, 428, 483, 484, 486
SFLEND, 32, 145, 154, 210, 216, 267, 277, 278, 428, 484, 486, 495
SFLEND(*MORE), 286
SFLENTER, 32, 251, 258, 484, 487
SFLFOLD, 32, 33, 169, 403, 406, 478, 484, 487, 493
SFLINZ, 32, 33, 331, 332, 342, 427, 428, 477, 479, 482, 484, 488, 495
SFLLIN, 32, 33, 186, 187, 197, 199, 483, 489
SFLMODE, 489
SFLMSG, 32, 91, 286, 484, 490
SFLMSGID, 32, 91, 286, 290, 467, 484, 490, 491
SFLMSGKEY, 426, 428, 481, 491
SFLMSGRCD, 426, 428, 481, 491, 492, 495
SFLNXTCHG, 388, 393, 394, 478, 481, 488, 491
SFLPAG, 11, 12, 27, 31, 37, 38, 130, 268, 335, 480, 483, 492
SFLPGMQ, 32, 426, 427, 428, 481, 484, 489, 493
SFLRCDNBR, 32, 33, 221, 222, 230, 231, 267, 320, 322, 379, 380, 484, 494
SFLRCDNBR(CURSOR), 380
SFLRNA, 32, 33, 342, 345, 479, 484, 488, 494
SFLROLVAL, 32, 33, 259, 266, 484, 493, 495
SFLSIZ, 31, 37, 38, 129, 207, 268, 331, 335, 483, 496
Side-By-Side Subfiles, 304
    Windowed, 306
Single-Page Load Method, 14, 18, 38, 107, 465
Single-Page Method, 268
Single-Page Subfile, 267, 469
SQL/400, 440, 447, 448
Subfile, 496
Subfile Clear, 240
Subfile Control Format, 10, 11, 18, 19, 20, 27, 28, 30, 31, 33, 36, 43, 55, 62, 108, 124, 127, 167, 299, 427, 428
Subfile Control Record Format, 30, 31, 32, 496

Subfile Cursor Progression, 408, 415, 482
Subfile Cursor Relative Record Number, 176, 482
Subfile Display Format, 29
Subfile Display Type, 472
Subfile Drop, 32, 169, 399
Subfile End, 32, 216, 286
Subfile Enter, 32, 251
Subfile Fold, 169
Subfile Format, 10, 14, 15, 30, 230, 427, 480
Subfile Initialize Keyword, 331
Subfile Keywords, 31
Subfile Line, 32, 186
Subfile Message, 32, 286
Subfile Message Identifier, 32, 286
Subfile Message Key, 426
Subfile Message Line, 435
Subfile Message Record, 426
Subfile Mode Keyword, 403
Subfile Next Change, 388
Subfile Next Changed, 491
Subfile Page, 11, 130, 201, 267, 480
Subfile Program Queue, 426
Subfile Record Format, 497
Subfile Record Not Active, 342
Subfile Record Number, 32, 221
Subfile Roll Value, 32, 259
Subfile Size, 128, 129, 201, 267
Subfiles, 1, 7
And Open Query File, 439
Potential Performance Problems, 473
Basic Concepts, 9
Controlling, 33
Design, 475
End Message, 139
End Option, 146
Expanding, 52, 199
Formats, 19
Functionality, 107, 108
Input, 325
Keywords, 31
Load-All, 39
Loading Techniques, 465
Multiple Line, 161
Page And Size Values, 471
Screen Domains, 27
Side-by-Side, 304
Single-Page, 57, 267
Types, 13
Update, 325
What They Are, 7
Why Use Them, 9

T

Temporary Access Path, 441, 445, 447
TIME, 482
Trailer Format, 28, 29, 45
Twinax, 85
Twisted Pair, 85

U

Update Subfiles, 290, 325, 354, 355, 399, 467
USRDFN, 33
USRRSTDSP, 85, 87

V

Vertical Subfile, 197

W

WDWBORDER, 84, 89, 92
WINDOW, 77, 81, 83, 84, 88, 90, 98, 111
Window Border, 83
Window Definition Format, 126
Window Definition Record, 83, 88
Window Reference Format, 127
Window Reference Record, 83, 89
Window Size, 90
Windows
    Defining Borders, 92
    Displaying, 88
    Functions, 81
    Keywords, 84
    Terminology, 82
Write Operation, 497
WRKSYSSTS, 474

NOTICE

This electronic book carries the same copyright protection as its printed counterpart. You may not distribute this book, in whole or in part, either electronically or in printed form, to any other party.

Copyright © 1995
Midrange Computing
5650 El Camino Real, Suite 225
Carlsbad, CA 92008-7128

Welcome to Midrange Computing's Electronic Library!

The following information will help you navigate through this online book using a mouse. If you do not have a mouse, Acrobat Reader follows the Windows convention of pressing the Alt key and the underlined menu letter simultaneously (for example, pressing the Alt key and the letter F simultaneously opens the File menu).

If you need to return to this page at any time, press the Ctrl key and the End key simultaneously, then the Page Up key twice (these instructions constitute the last three pages of the book). Or click the mouse on the first bookmark, "HOW TO USE ACROBAT READER.")

The screen has two parts. On the left is a list of topics or headings. These are Acrobat bookmarks created from the Table of Contents. The right side of the screen displays a page from the book. Arrows to the left of the bookmarks indicate additional information is available. Click on the arrow to collapse or expand the list. A menu bar and a tool bar run across the top of the screen. To move from page to page: click on a bookmark, use the tool bar across (select the View menu or the arrow buttons), or click on the corresponding thumbnail page (select the View menu or the Thumbnail button). Scroll bars are

located at the bottom of the screen and to the right of each section (i.e., bookmarks and book pages). The scroll bars allow you to move through the bookmark list or position the visible area of the page on your screen. There is also a movable partition between the bookmarks and the page allowing you to adjust the amount of screen area used by each section.

To move from page to page in the book:

- Use the mouse to click on any heading in the list of bookmarks (left side of the screen). The right side of the screen displays the corresponding page automatically.

  -OR-

- Click on the View menu and choose a menu option.

  -OR-

- Click on the View menu and select Thumbnails and Page. A miniature of each page appears in place of the bookmarks. You can scroll through these thumbnails until you see the page you want, then click on that thumbnail to display the page on the right side of the screen.

To adjust the size of the area on the screen used for bookmarks and pages:

Move the cursor along the screen until it changes to look like two arrows going in opposite directions. Hold down the left mouse button and drag the dividing line left or right. Release the mouse button. (The same instructions apply to thumbnails and pages.)

To change the magnification of the page being viewed:

- Use the View menu and select an option.

  -OR-

- Press the Ctrl key and the letter L simultaneously.

To search for a word or text string:

Click on the Find button on the tool bar. Type the string. Make any other selections, as appropriate, and click the Find button.

There are many ways to move through the book. Experiment with the menus and buttons to find the best method.

From left to right, the buttons on the tool bar are:

Page Only (closes bookmarks/thumbnail area to display the page only)

Bookmarks and Page (displays bookmarks and the page simultaneously)

Thumbnails and Page (displays thumbnails and the page simultaneously)

Hand (scrolls through the window as you move the hand while holding down the left mouse button)

Zoom In (magnifies the page view)

Zoom Out (reduces the page view)

Select Text (selects text)

First Page (displays the first page of the book)

Previous Page (displays the previous page of the book)

Next Page (displays the next page of the book)

Last Page (displays the last page of the book)

Go Back (displays previous page view)

Go Forward (returns from Go Back)

Actual Size (sets page view to 100%)

Fit Page (displays the whole page)

Fit Width (fits the page view to the width of the window)

Find (searches for words or character strings)